

StyleParameterSet.__init__(name, add_intensity=True, doc=None) [Function]

Create a new style group based on *name*, which is a string. For example if *name* is ‘my-style’, then GDB will create the prefix commands ‘set style my-style’ and ‘show style my-style’. Within these prefix commands will be ‘foreground’, ‘background’, and ‘intensity’ parameters with the appropriate types.

It is also possible for *name* to consist of multiple words, so long as each prefix command (except the last one) already exists. For example, it is valid to use a *name* value of ‘disassembler my-style’, as the ‘disassembler’ prefix command already exists. GDB would then create ‘set style disassembler my-style’ and ‘show style disassembler my-style’, and within the ‘my-style’ prefixes will be the ‘foreground’, ‘background’, and ‘intensity’ parameters with the appropriate types.

Every style requires a ‘foreground’ and ‘background’, but not every style needs an ‘intensity’. If *add_intensity* is *True* (the default), then the ‘intensity’ parameter will be created. If *add_intensity* is *False*, then the ‘intensity’ parameter will not be created.

If the ‘intensity’ parameter is not created, then the `gdb.Style` (see Section 23.3.2.37 [Styles In Python], page 510) created from this `gdb.StyleParameterSet` will have `gdb.INTENSITY_NORMAL`.

The *doc* should be a string which will be used as the help text for the *name* prefix command. This text is used as the `Command.__doc__` value for the `gdb.Command` object that is the prefix command object (see Section 23.3.2.21 [CLI Commands In Python], page 471). If *doc* is *None* (the default) then a basic default help text is used.

StyleParameterSet.apply(string) [Function]

Equivalent to `StyleParameterSet.style.apply(string)`. Returns a copy of *string* with escape sequences added to the start and end. The escape sequence at the start applies this style, and the escape sequence at the end restores the terminal default.

If styling is disabled (i.e. ‘set style enabled off’), then no escape sequences are added and this method returns a copy of *string*.

StyleParameterSet.style [Variable]

This read/write attribute holds a `gdb.Style` object (see Section 23.3.2.37 [Styles In Python], page 510), that is a named style associated with this style parameter group.

StyleParameterSet.value [Variable]

This is an alias for `StyleParameterSet.style`, see above.

23.3.2.25 Writing new convenience functions

You can implement new convenience functions (see Section 10.12 [Convenience Vars], page 173) in Python. A convenience function is an instance of a subclass of the class `gdb.Function`.

Function.__init__(name) [Function]

The initializer for `Function` registers the new function with GDB. The argument *name* is the name of the function, a string. The function will be visible to the user

as a convenience variable of type **internal function**, whose name is the same as the given *name*.

The documentation for the new function is taken from the documentation string for the new class.

Function.invoke (*args) [Function]

When a convenience function is evaluated, its arguments are converted to instances of **gdb.Value**, and then the function's **invoke** method is called. Note that GDB does not predetermine the arity of convenience functions. Instead, all available arguments are passed to **invoke**, following the standard Python calling convention. In particular, a convenience function can have default values for parameters without ill effect.

The return value of this method is used as its value in the enclosing expression. If an ordinary Python value is returned, it is converted to a **gdb.Value** following the usual rules.

The following code snippet shows how a trivial convenience function can be implemented in Python:

```
class Greet (gdb.Function):
    """Return string to greet someone.
    Takes a name as argument."""

    def __init__ (self):
        super (Greet, self).__init__ ("greet")

    def invoke (self, name):
        return "Hello, %s!" % name.string ()

Greet ()
```

The last line instantiates the class, and is necessary to trigger the registration of the function with GDB. Depending on how the Python code is read into GDB, you may need to import the **gdb** module explicitly.

Now you can use the function in an expression:

```
(gdb) print $greet("Bob")
$1 = "Hello, Bob!"
```

23.3.2.26 Program Spaces In Python

A program space, or *progspace*, represents a symbolic view of an address space. It consists of all of the objfiles of the program. See Section 23.3.2.27 [Objfiles In Python], page 487. See Section 4.9 [Inferiors Connections and Programs], page 43, for more details about program spaces.

The following progspace-related functions are available in the **gdb** module:

gdb.current_progspace () [Function]

This function returns the program space of the currently selected inferior. See Section 4.9 [Inferiors Connections and Programs], page 43. This is identical to **gdb.selected_inferior().progspace** (see Section 23.3.2.17 [Inferiors In Python], page 455) and is included for historical compatibility.

gdb.progspace () [Function]

Return a sequence of all the progspace currently known to GDB.

Each `progspace` is represented by an instance of the `gdb.Progspace` class.

Progspace.filename [Variable]

The file name, as a string, of the main symbol file (from which debug symbols have been loaded) for the `progspace`, e.g. the argument to the *symbol-file* or *file* commands.

If there is no main symbol table currently loaded, then this attribute will be `None`.

Progspace.symbol_file [Variable]

The `gdb.Objfile` representing the main symbol file (from which debug symbols have been loaded) for the `gdb.Progspace`. This is the symbol file set by the *symbol-file* or *file* commands.

This will be the `gdb.Objfile` representing `Progspace.filename` when `Progspace.filename` is not `None`.

If there is no main symbol table currently loaded, then this attribute will be `None`.

If the `Progspace` is invalid, i.e., when `Progspace.is_valid()` returns `False`, then attempting to access this attribute will raise a `RuntimeError` exception.

Progspace.executable_filename [Variable]

The file name, as a string, of the executable file in use by this program space. The executable file is the file that GDB will invoke in order to start an inferior when using a native target. The file name within this attribute is updated by the *exec-file* and *file* commands.

If no executable is currently set within this `Progspace` then this attribute contains `None`.

If the `Progspace` is invalid, i.e., when `Progspace.is_valid()` returns `False`, then attempting to access this attribute will raise a `RuntimeError` exception.

Progspace.pretty_printers [Variable]

The `pretty_printers` attribute is a list of functions. It is used to look up pretty-printers. A `Value` is passed to each function in order; if the function returns `None`, then the search continues. Otherwise, the return value should be an object which is used to format the value. See Section 23.3.2.6 [Pretty Printing API], page 430, for more information.

Progspace.type_printers [Variable]

The `type_printers` attribute is a list of type printer objects. See Section 23.3.2.9 [Type Printing API], page 435, for more information.

Progspace.frame_filters [Variable]

The `frame_filters` attribute is a dictionary of frame filter objects. See Section 23.3.2.10 [Frame Filter API], page 436, for more information.

Progspace.missing_file_handlers [Variable]

The `missing_file_handlers` attribute is a list of tuples. Each tuple holds a missing-file handler object for this program space. For more information, see Section 23.3.2.43 [Missing Debug Info In Python], page 526, and Section 23.3.2.44 [Missing Objfiles In Python], page 528.

A program space has the following methods:

`Progspace.block_for_pc (pc)` [Function]

Return the innermost `gdb.Block` containing the given *pc* value. If the block cannot be found for the *pc* value specified, the function will return `None`.

`Progspace.find_pc_line (pc)` [Function]

Return the `gdb.Symtab_and_line` object corresponding to the *pc* value. See Section 23.3.2.31 [Symbol Tables In Python], page 500. If an invalid value of *pc* is passed as an argument, then the `symtab` and `line` attributes of the returned `gdb.Symtab_and_line` object will be `None` and 0 respectively.

`Progspace.is_valid ()` [Function]

Returns `True` if the `gdb.Progspace` object is valid, `False` if not. A `gdb.Progspace` object can become invalid if the program space file it refers to is not referenced by any inferior. All other `gdb.Progspace` methods will throw an exception if it is invalid at the time the method is called.

`Progspace.objfiles ()` [Function]

Return a sequence of all the objfiles referenced by this program space. See Section 23.3.2.27 [Objfiles In Python], page 487.

`Progspace.solib_name (address)` [Function]

Return the name of the shared library holding the given *address* as a string, or `None`.

`Progspace.objfile_for_address (address)` [Function]

Return the `gdb.Objfile` holding the given address, or `None` if no objfile covers it.

One may add arbitrary attributes to `gdb.Progspace` objects in the usual Python way. This is useful if, for example, one needs to do some extra record keeping associated with the program space.

See [choosing attribute names], page 457, for guidance on selecting a suitable name for new attributes.

In this contrived example, we want to perform some processing when an objfile with a certain symbol is loaded, but we only want to do this once because it is expensive. To achieve this we record the results with the program space because we can't predict when the desired objfile will be loaded.

```
(gdb) python
def clear_objfiles_handler(event):
    event.progspace.expensive_computation = None
def expensive(symbol):
    """A mock routine to perform an "expensive" computation on symbol."""
    print ("Computing the answer to the ultimate question ...")
    return 42
```

```

def new_objfile_handler(event):
    objfile = event.new_objfile
    progspace = objfile.progspace
    if not hasattr(progspace, 'expensive_computation') or \
        progspace.expensive_computation is None:
        # We use 'main' for the symbol to keep the example simple.
        # Note: There's no current way to constrain the lookup
        # to one objfile.
        symbol = gdb.lookup_global_symbol('main')
        if symbol is not None:
            progspace.expensive_computation = expensive(symbol)
gdb.events.clear_objfiles.connect(clear_objfiles_handler)
gdb.events.new_objfile.connect(new_objfile_handler)
end
(gdb) file /tmp/hello
Reading symbols from /tmp/hello...
Computing the answer to the ultimate question ...
(gdb) python print(gdb.current_progspace().expensive_computation)
42
(gdb) run
Starting program: /tmp/hello
Hello.
[Inferior 1 (process 4242) exited normally]

```

23.3.2.27 Objfiles In Python

GDB loads symbols for an inferior from various symbol-containing files (see Section 18.1 [Files], page 293). These include the primary executable file, any shared libraries used by the inferior, and any separate debug info files (see Section 18.3 [Separate Debug Files], page 303). GDB calls these symbol-containing files *objfiles*.

The following objfile-related functions are available in the `gdb` module:

`gdb.current_objfile ()` [Function]

When auto-loading a Python script (see Section 23.3.3 [Python Auto-loading], page 532), GDB sets the “current objfile” to the corresponding objfile. This function returns the current objfile. If there is no current objfile, this function returns `None`.

`gdb.objfiles ()` [Function]

Return a sequence of objfiles referenced by the current program space. See Section 23.3.2.27 [Objfiles In Python], page 487, and Section 23.3.2.26 [Progspace In Python], page 484. This is identical to `gdb.selected_inferior().progspace.objfiles()` and is included for historical compatibility.

`gdb.lookup_objfile (name [, by_build_id])` [Function]

Look up *name*, a file name or build ID, in the list of objfiles for the current program space (see Section 23.3.2.26 [Progspace In Python], page 484). If the objfile is not found throw the Python `ValueError` exception.

If *name* is a relative file name, then it will match any source file name with the same trailing components. For example, if *name* is `'gcc/expr.c'`, then it will match source file name of `/build/trunk/gcc/expr.c`, but not `/build/trunk/libcpp/expr.c` or `/build/trunk/gcc/x-expr.c`.

If *by_build_id* is provided and is `True` then *name* is the build ID of the objfile. Otherwise, *name* is a file name. This is supported only on some operating systems, notably

those which use the ELF format for binary files and the GNU Binutils. For more details about this feature, see the description of the `--build-id` command-line option in Section “Command Line Options” in *The GNU Linker*.

Each objfile is represented by an instance of the `gdb.Objfile` class.

Objfile.filename [Variable]

The file name of the objfile as a string, with symbolic links resolved.

The value is `None` if the objfile is no longer valid. See the `gdb.Objfile.is_valid` method, described below.

Objfile.username [Variable]

The file name of the objfile as specified by the user as a string.

The value is `None` if the objfile is no longer valid. See the `gdb.Objfile.is_valid` method, described below.

Objfile.is_file [Variable]

An objfile often comes from an ordinary file, but in some cases it may be constructed from the contents of memory. This attribute is `True` for file-backed objfiles, and `False` for other kinds.

Objfile.owner [Variable]

For separate debug info objfiles this is the corresponding `gdb.Objfile` object that debug info is being provided for. Otherwise this is `None`. Separate debug info objfiles are added with the `gdb.Objfile.add_separate_debug_file` method, described below.

Objfile.build_id [Variable]

The build ID of the objfile as a string. If the objfile does not have a build ID then the value is `None`.

This is supported only on some operating systems, notably those which use the ELF format for binary files and the GNU Binutils. For more details about this feature, see the description of the `--build-id` command-line option in Section “Command Line Options” in *The GNU Linker*.

Objfile.progspace [Variable]

The containing program space of the objfile as a `gdb.Progspace` object. See Section 23.3.2.26 [Progspace In Python], page 484.

Objfile.pretty_printers [Variable]

The `pretty_printers` attribute is a list of functions. It is used to look up pretty-printers. A `Value` is passed to each function in order; if the function returns `None`, then the search continues. Otherwise, the return value should be an object which is used to format the value. See Section 23.3.2.6 [Pretty Printing API], page 430, for more information.

Objfile.type_printers [Variable]

The `type_printers` attribute is a list of type printer objects. See Section 23.3.2.9 [Type Printing API], page 435, for more information.

Objfile.frame_filters [Variable]

The `frame_filters` attribute is a dictionary of frame filter objects. See Section 23.3.2.10 [Frame Filter API], page 436, for more information.

One may add arbitrary attributes to `gdb.Objfile` objects in the usual Python way. This is useful if, for example, one needs to do some extra record keeping associated with the objfile.

See [choosing attribute names], page 457, for guidance on selecting a suitable name for new attributes.

In this contrived example we record the time when GDB loaded the objfile.

```
(gdb) python
import datetime
def new_objfile_handler(event):
    # Set the time_loaded attribute of the new objfile.
    event.new_objfile.time_loaded = datetime.datetime.today()
gdb.events.new_objfile.connect(new_objfile_handler)
end
(gdb) file ./hello
Reading symbols from ./hello...
(gdb) python print(gdb.objfiles()[0].time_loaded)
2014-10-09 11:41:36.770345
```

A `gdb.Objfile` object has the following methods:

Objfile.is_valid () [Function]

Returns True if the `gdb.Objfile` object is valid, False if not. A `gdb.Objfile` object can become invalid if the object file it refers to is not loaded in GDB any longer. All other `gdb.Objfile` methods will throw an exception if it is invalid at the time the method is called.

Objfile.add_separate_debug_file (file) [Function]

Add *file* to the list of files that GDB will search for debug information for the objfile. This is useful when the debug info has been removed from the program and stored in a separate file. GDB has built-in support for finding separate debug info files (see Section 18.3 [Separate Debug Files], page 303), but if the file doesn't live in one of the standard places that GDB searches then this function can be used to add a debug info file from a different place.

Objfile.lookup_global_symbol (name [, domain]) [Function]

Search for a global symbol named *name* in this objfile. Optionally, the search scope can be restricted with the *domain* argument. The *domain* argument must be a domain constant defined in the `gdb` module and described in Section 23.3.2.30 [Symbols In Python], page 495. This function is similar to `gdb.lookup_global_symbol`, except that the search is limited to this objfile.

The result is a `gdb.Symbol` object or None if the symbol is not found.

Objfile.lookup_static_symbol (name [, domain]) [Function]

Like `Objfile.lookup_global_symbol`, but searches for a global symbol with static linkage named *name* in this objfile.

23.3.2.28 Accessing inferior stack frames from Python

When the debugged program stops, GDB is able to analyze its call stack (see Section 8.1 [Stack frames], page 115). The `gdb.Frame` class represents a frame in the stack. A `gdb.Frame` object is only valid while its corresponding frame exists in the inferior's stack. If you try to use an invalid frame object, GDB will throw a `gdb.error` exception (see Section 23.3.2.3 [Exception Handling], page 416).

Two `gdb.Frame` objects can be compared for equality with the `==` operator, like:

```
(gdb) python print(gdb.newest_frame() == gdb.selected_frame ())
True
```

The following frame-related functions are available in the `gdb` module:

`gdb.selected_frame ()` [Function]
Return the selected frame object. (see Section 8.3 [Selecting a Frame], page 119).

`gdb.newest_frame ()` [Function]
Return the newest frame object for the selected thread.

`gdb.frame_stop_reason_string (reason)` [Function]
Return a string explaining the reason why GDB stopped unwinding frames, as expressed by the given *reason* code (an integer, see the `unwind_stop_reason` method further down in this section).

`gdb.invalidate_cached_frames` [Function]
GDB internally keeps a cache of the frames that have been unwound. This function invalidates this cache.
This function should not generally be called by ordinary Python code. It is documented for the sake of completeness.

A `gdb.Frame` object has the following methods:

`Frame.is_valid ()` [Function]
Returns true if the `gdb.Frame` object is valid, false if not. A frame object can become invalid if the frame it refers to doesn't exist anymore in the inferior. All `gdb.Frame` methods will throw an exception if it is invalid at the time the method is called.

`Frame.name ()` [Function]
Returns the function name of the frame, or `None` if it can't be obtained.

`Frame.architecture ()` [Function]
Returns the `gdb.Architecture` object corresponding to the frame's architecture. See Section 23.3.2.38 [Architectures In Python], page 512.

`Frame.type ()` [Function]
Returns the type of the frame. The value can be one of:

`gdb.NORMAL_FRAME`

An ordinary stack frame.

`gdb.DUMMY_FRAME`

A fake stack frame that was created by GDB when performing an inferior function call.

`gdb.INLINE_FRAME`

A frame representing an inlined function. The function was inlined into a `gdb.NORMAL_FRAME` that is older than this one.

`gdb.TAILCALL_FRAME`

A frame representing a tail call. See Section 11.2 [Tail Call Frames], page 198.

`gdb.SIGTRAMP_FRAME`

A signal trampoline frame. This is the frame created by the OS when it calls into a signal handler.

`gdb.ARCH_FRAME`

A fake stack frame representing a cross-architecture call.

`gdb.SENTINEL_FRAME`

This is like `gdb.NORMAL_FRAME`, but it is only used for the newest frame.

`Frame.unwind_stop_reason ()` [Function]

Return an integer representing the reason why it's not possible to find more frames toward the outermost frame. Use `gdb.frame_stop_reason_string` to convert the value returned by this function to a string. The value can be one of:

`gdb.FRAME_UNWIND_NO_REASON`

No particular reason (older frames should be available).

`gdb.FRAME_UNWIND_NULL_ID`

The previous frame's analyzer returns an invalid result. This is no longer used by GDB, and is kept only for backward compatibility.

`gdb.FRAME_UNWIND_OUTERMOST`

This frame is the outermost.

`gdb.FRAME_UNWIND_UNAVAILABLE`

Cannot unwind further, because that would require knowing the values of registers or memory that have not been collected.

`gdb.FRAME_UNWIND_INNER_ID`

This frame ID looks like it ought to belong to a NEXT frame, but we got it for a PREV frame. Normally, this is a sign of unwinder failure. It could also indicate stack corruption.

`gdb.FRAME_UNWIND_SAME_ID`

This frame has the same ID as the previous one. That means that unwinding further would almost certainly give us another frame with exactly the same ID, so break the chain. Normally, this is a sign of unwinder failure. It could also indicate stack corruption.

`gdb.FRAME_UNWIND_NO_SAVED_PC`

The frame unwinder did not find any saved PC, but we needed one to unwind further.

`gdb.FRAME_UNWIND_MEMORY_ERROR`

The frame unwinder caused an error while trying to access memory.

`gdb.FRAME_UNWIND_FIRST_ERROR`

Any stop reason greater or equal to this value indicates some kind of error. This special value facilitates writing code that tests for errors in unwinding in a way that will work correctly even if the list of the other values is modified in future GDB versions. Using it, you could write:

```
reason = gdb.selected_frame().unwind_stop_reason ()
reason_str = gdb.frame_stop_reason_string (reason)
if reason >= gdb.FRAME_UNWIND_FIRST_ERROR:
    print ("An error occurred: %s" % reason_str)
```

Frame.pc () [Function]

Returns the frame's resume address.

Frame.block () [Function]

Return the frame's code block. See Section 23.3.2.29 [Blocks In Python], page 493. If the frame does not have a block – for example, if there is no debugging information for the code in question – then this will throw an exception.

Frame.function () [Function]

Return the symbol for the function corresponding to this frame. See Section 23.3.2.30 [Symbols In Python], page 495.

Frame.older () [Function]

Return the frame that called this frame. If this is the oldest frame, return `None`.

Frame.newer () [Function]

Return the frame called by this frame. If this is the newest frame, return `None`.

Frame.find_sal () [Function]

Return the frame's symtab and line object. See Section 23.3.2.31 [Symbol Tables In Python], page 500.

Frame.read_register (*register*) [Function]

Return the value of *register* in this frame. Returns a `Gdb.Value` object. Throws an exception if *register* does not exist. The *register* argument must be one of the following:

1. A string that is the name of a valid register (e.g., `'sp'` or `'rax'`).
2. A `gdb.RegisterDescriptor` object (see Section 23.3.2.39 [Registers In Python], page 513).
3. A GDB internal, platform specific number. Using these numbers is supported for historic reasons, but is not recommended as future changes to GDB could change the mapping between numbers and the registers they represent, breaking any Python code that uses the platform-specific numbers. The numbers are usually found in the corresponding `platform-tdep.h` file in the GDB source tree.

Using a string to access registers will be slightly slower than the other two methods as GDB must look up the mapping between name and internal register number. If performance is critical consider looking up and caching a `gdb.RegisterDescriptor` object.

Frame.read_var (*variable* [, *block*]) [Function]

Return the value of *variable* in this frame. If the optional argument *block* is provided, search for the variable from that block; otherwise start at the frame's current block (which is determined by the frame's current program counter). The *variable* argument must be a string or a `gdb.Symbol` object; *block* must be a `gdb.Block` object.

Frame.select () [Function]

Set this frame to be the selected frame. See Chapter 8 [Examining the Stack], page 115.

Frame.static_link () [Function]

In some languages (e.g., Ada, but also a GNU C extension), a nested function can access the variables in the outer scope. This is done via a “static link”, which is a reference from the nested frame to the appropriate outer frame.

This method returns this frame's static link frame, if one exists. If there is no static link, this method returns `None`.

Frame.level () [Function]

Return an integer, the stack frame level for this frame. See Section 8.1 [Stack Frames], page 115.

Frame.language () [Function]

Return a string, the source language for this frame.

23.3.2.29 Accessing blocks from Python

In GDB, symbols are stored in blocks. A block corresponds roughly to a scope in the source code. Blocks are organized hierarchically, and are represented individually in Python as a `gdb.Block`. Blocks rely on debugging information being available.

A frame has a block. Please see Section 23.3.2.28 [Frames In Python], page 490, for a more in-depth discussion of frames.

The outermost block is known as the *global block*. The global block typically holds public global variables and functions.

The block nested just inside the global block is the *static block*. The static block typically holds file-scoped variables and functions.

GDB provides a method to get a block's superblock, but there is currently no way to examine the sub-blocks of a block, or to iterate over all the blocks in a symbol table (see Section 23.3.2.31 [Symbol Tables In Python], page 500).

Here is a short example that should help explain blocks:

```
/* This is in the global block. */
int global;

/* This is in the static block. */
static int file_scope;

/* 'function' is in the global block, and 'argument' is
   in a block nested inside of 'function'. */
int function (int argument)
{
    /* 'local' is in a block inside 'function'. It may or may
```

```

    not be in the same block as 'argument'.  */
int local;

{
    /* 'inner' is in a block whose superblock is the one holding
       'local'.  */
    int inner;

    /* If this call is expanded by the compiler, you may see
       a nested block here whose function is 'inline_function'
       and whose superblock is the one holding 'inner'.  */
    inline_function ();
}
}

```

A `gdb.Block` is iterable. The iterator returns the symbols (see Section 23.3.2.30 [Symbols In Python], page 495) local to the block. Python programs should not assume that a specific block object will always contain a given symbol, since changes in GDB features and infrastructure may cause symbols move across blocks in a symbol table. You can also use Python's *dictionary syntax* to access variables in this block, e.g.:

```
symbol = some_block['variable'] # symbol is of type gdb.Symbol
```

The following block-related functions are available in the `gdb` module:

`gdb.block_for_pc (pc)` [Function]
 Return the innermost `gdb.Block` containing the given *pc* value. If the block cannot be found for the *pc* value specified, the function will return `None`. This is identical to `gdb.current_progspace().block_for_pc(pc)` and is included for historical compatibility.

A `gdb.Block` object has the following methods:

`Block.is_valid ()` [Function]
 Returns `True` if the `gdb.Block` object is valid, `False` if not. A block object can become invalid if the block it refers to doesn't exist anymore in the inferior. All other `gdb.Block` methods will throw an exception if it is invalid at the time the method is called. The block's validity is also checked during iteration over symbols of the block.

A `gdb.Block` object has the following attributes:

`Block.start` [Variable]
 The start address of the block. This attribute is not writable.

`Block.end` [Variable]
 One past the last address that appears in the block. This attribute is not writable.

`Block.ranges` [Variable]
 A tuple representing address ranges of the block. Each range is represented as a pair (two-element tuple) where the first element is the start of the range and the second element is one past the last address that appears in the range. The order of ranges is unspecified. Contiguous blocks have only one range. This attribute is not writable.

Block.function [Variable]

The name of the block represented as a `gdb.Symbol`. If the block is not named, then this attribute holds `None`. This attribute is not writable.

For ordinary function blocks, the superblock is the static block. However, you should note that it is possible for a function block to have a superblock that is not the static block – for instance this happens for an inlined function.

Block.superblock [Variable]

The block containing this block. If this parent block does not exist, this attribute holds `None`. This attribute is not writable.

Block.subblocks [Variable]

A list of blocks nested in this block. If there are no blocks nested, this attribute holds an empty list. This attribute is not writable.

Block.global_block [Variable]

The global block associated with this block. This attribute is not writable.

Block.static_block [Variable]

The static block associated with this block. This attribute is not writable.

Block.is_global [Variable]

`True` if the `gdb.Block` object is a global block, `False` if not. This attribute is not writable.

Block.is_static [Variable]

`True` if the `gdb.Block` object is a static block, `False` if not. This attribute is not writable.

23.3.2.30 Python representation of Symbols

GDB represents every variable, function and type as an entry in a symbol table. See Chapter 16 [Examining the Symbol Table], page 265. Similarly, Python represents these symbols in GDB with the `gdb.Symbol` object.

The following symbol-related functions are available in the `gdb` module:

`gdb.lookup_symbol (name [, block [, domain]])` [Function]

This function searches for a symbol by name. The search scope can be restricted to the parameters defined in the optional domain and block arguments.

name is the name of the symbol. It must be a string. The optional *block* argument restricts the search to symbols visible in that *block*. The *block* argument must be a `gdb.Block` object. If omitted, the block for the current frame is used. The optional *domain* argument restricts the search to the domain type. The *domain* argument must be a domain constant defined in the `gdb` module and described later in this chapter.

The result is a tuple of two elements. The first element is a `gdb.Symbol` object or `None` if the symbol is not found. If the symbol is found, the second element is `True` if the symbol is a field of a method's object (e.g., `this` in C++), otherwise it is `False`. If the symbol is not found, the second element is `False`.

`gdb.lookup_global_symbol (name [, domain])` [Function]

This function searches for a global symbol by name. The search scope can be restricted to by the domain argument.

name is the name of the symbol. It must be a string. The optional *domain* argument restricts the search to the domain type. The *domain* argument must be a domain constant defined in the `gdb` module and described later in this chapter.

The result is a `gdb.Symbol` object or `None` if the symbol is not found.

`gdb.lookup_static_symbol (name [, domain])` [Function]

This function searches for a global symbol with static linkage by name. The search scope can be restricted to by the domain argument.

name is the name of the symbol. It must be a string. The optional *domain* argument restricts the search to the domain type. The *domain* argument must be a domain constant defined in the `gdb` module and described later in this chapter.

The result is a `gdb.Symbol` object or `None` if the symbol is not found.

Note that this function will not find function-scoped static variables. To look up such variables, iterate over the variables of the function's `gdb.Block` and check that `block.addr_class` is `gdb.SYMBOL_LOC_STATIC`.

There can be multiple global symbols with static linkage with the same name. This function will only return the first matching symbol that it finds. Which symbol is found depends on where the program is currently stopped, as GDB will first search for matching symbols in the current compilation unit. If it finds no match, or if there is no current compilation unit (perhaps because the program is not running), then `gdb.lookup_static_symbol` will return an unspecified matching symbol. To safely handle the case of multiple static symbols with the same name, an application can use `gdb.lookup_static_symbols`.

`gdb.lookup_static_symbols (name [, domain])` [Function]

Similar to `gdb.lookup_static_symbol`, this function searches for global symbols with static linkage by name, and optionally restricted by the domain argument. However, this function returns a list of all matching symbols found, not just the first one.

name is the name of the symbol. It must be a string. The optional *domain* argument restricts the search to the domain type. The *domain* argument must be a domain constant defined in the `gdb` module and described later in this chapter.

The result is a list of `gdb.Symbol` objects which could be empty if no matching symbols were found.

Note that this function will not find function-scoped static variables. To look up such variables, iterate over the variables of the function's `gdb.Block` and check that `block.addr_class` is `gdb.SYMBOL_LOC_STATIC`.

A `gdb.Symbol` object has the following attributes:

`Symbol.type` [Variable]

The type of the symbol or `None` if no type is recorded. This attribute is represented as a `gdb.Type` object. See Section 23.3.2.5 [Types In Python], page 425. This attribute is not writable.

- Symbol.symtab** [Variable]
The symbol table in which the symbol appears. This attribute is represented as a `gdb.Symtab` object. See Section 23.3.2.31 [Symbol Tables In Python], page 500. This attribute is not writable.
- Symbol.line** [Variable]
The line number in the source code at which the symbol was defined. This is an integer.
- Symbol.name** [Variable]
The name of the symbol as a string. This attribute is not writable.
- Symbol.linkage_name** [Variable]
The name of the symbol, as used by the linker (i.e., may be mangled). This attribute is not writable.
- Symbol.print_name** [Variable]
The name of the symbol in a form suitable for output. This is either **name** or **linkage_name**, depending on whether the user asked GDB to display demangled or mangled names.
- Symbol.addr_class** [Variable]
The address class of the symbol. This classifies how to find the value of a symbol. Each address class is a constant defined in the `gdb` module and described later in this chapter.
- Symbol.domain** [Variable]
The domain of the symbol. Each domain is a constant defined in the `gdb` module and described later in this chapter.
- Symbol.needs_frame** [Variable]
This is **True** if evaluating this symbol's value requires a frame (see Section 23.3.2.28 [Frames In Python], page 490) and **False** otherwise. Typically, local variables will require a frame, but other symbols will not.
- Symbol.is_argument** [Variable]
True if the symbol is an argument of a function.
- Symbol.is_artificial** [Variable]
True if the symbol is artificial. An artificial symbol is one that is introduced by the compiler.
- Symbol.is_constant** [Variable]
True if the symbol is a constant.
- Symbol.is_function** [Variable]
True if the symbol is a function or a method.
- Symbol.is_variable** [Variable]
True if the symbol is a variable, as opposed to something like a function or type. Note that this also returns **False** for arguments.

A `gdb.Symbol` object has the following methods:

`Symbol.is_valid ()` [Function]

Returns `True` if the `gdb.Symbol` object is valid, `False` if not. A `gdb.Symbol` object can become invalid if the symbol it refers to does not exist in GDB any longer. All other `gdb.Symbol` methods will throw an exception if it is invalid at the time the method is called.

`Symbol.value ([frame])` [Function]

Compute the value of the symbol, as a `gdb.Value`. For functions, this computes the address of the function, cast to the appropriate type. If the symbol requires a frame in order to compute its value, then *frame* must be given. If *frame* is not given, or if *frame* is invalid, then this method will throw an exception.

The available domain categories in `gdb.Symbol` are represented as constants in the `gdb` module:

`gdb.SYMBOL_UNDEF_DOMAIN`

This is used when a domain has not been discovered or none of the following domains apply. This usually indicates an error either in the symbol information or in GDB's handling of symbols.

`gdb.SYMBOL_VAR_DOMAIN`

This domain contains variables.

`gdb.SYMBOL_FUNCTION_DOMAIN`

This domain contains functions.

`gdb.SYMBOL_TYPE_DOMAIN`

This domain contains types. In a C-like language, types using a tag (the name appearing after a `struct`, `union`, or `enum` keyword) will not appear here; in other languages, all types are in this domain.

`gdb.SYMBOL_STRUCT_DOMAIN`

This domain holds struct, union and enum tag names. This domain is only used for C-like languages. For example, in this code:

```
struct type_one { int x; };
typedef struct type_one type_two;
```

Here `type_one` will be in `SYMBOL_STRUCT_DOMAIN`, but `type_two` will be in `SYMBOL_TYPE_DOMAIN`.

`gdb.SYMBOL_LABEL_DOMAIN`

This domain contains names of labels (for `gotos`).

`gdb.SYMBOL_MODULE_DOMAIN`

This domain contains names of Fortran module types.

`gdb.SYMBOL_COMMON_BLOCK_DOMAIN`

This domain contains names of Fortran common blocks.

When searching for a symbol, the desired domain constant can be passed verbatim to the `lookup` function. For example:

```
symbol = gdb.lookup_symbol ("name", domain=gdb.SYMBOL_VAR_DOMAIN)
```


For more complex searches, there is a corresponding set of constants, each named after one of the preceding constants, but with the ‘SEARCH’ prefix replacing the ‘SYMBOL’ prefix; for example, `SEARCH_LABEL_DOMAIN`. These may be or’d together to form a search constant, e.g.:

```
symbol = gdb.lookup_symbol ("name",
                           domain=gdb.SEARCH_VAR_DOMAIN | gdb.SEARCH_TYPE_DOMAIN)
```

The available address class categories in `gdb.Symbol` are represented as constants in the `gdb` module:

`gdb.SYMBOL_LOC_UNDEF`

If this is returned by address class, it indicates an error either in the symbol information or in GDB’s handling of symbols.

`gdb.SYMBOL_LOC_CONST`

Value is constant int.

`gdb.SYMBOL_LOC_STATIC`

Value is at a fixed address.

`gdb.SYMBOL_LOC_REGISTER`

Value is in a register.

`gdb.SYMBOL_LOC_ARG`

Value is an argument. This value is at the offset stored within the symbol inside the frame’s argument list.

`gdb.SYMBOL_LOC_REF_ARG`

Value address is stored in the frame’s argument list. Just like `LOC_ARG` except that the value’s address is stored at the offset, not the value itself.

`gdb.SYMBOL_LOC_REGPARAM_ADDR`

Value is a specified register. Just like `LOC_REGISTER` except the register holds the address of the argument instead of the argument itself.

`gdb.SYMBOL_LOC_LOCAL`

Value is a local variable.

`gdb.SYMBOL_LOC_TYPEDEF`

Value not used. Symbols in the domain `SYMBOL_STRUCT_DOMAIN` all have this class.

`gdb.SYMBOL_LOC_LABEL`

Value is a label.

`gdb.SYMBOL_LOC_BLOCK`

Value is a block.

`gdb.SYMBOL_LOC_CONST_BYTES`

Value is a byte-sequence.

`gdb.SYMBOL_LOC_UNRESOLVED`

Value is at a fixed address, but the address of the variable has to be determined from the minimal symbol table whenever the variable is referenced.

`gdb.SYMBOL_LOC_OPTIMIZED_OUT`

The value does not actually exist in the program.

`gdb.SYMBOL_LOC_COMPUTED`

The value's address is a computed location.

`gdb.SYMBOL_LOC_COMMON_BLOCK`

The value's address is a symbol. This is only used for Fortran common blocks.

23.3.2.31 Symbol table representation in Python

Access to symbol table data maintained by GDB on the inferior is exposed to Python via two objects: `gdb.Symtab_and_line` and `gdb.Symtab`. Symbol table and line data for a frame is returned from the `find_sal` method in `gdb.Frame` object. See Section 23.3.2.28 [Frames In Python], page 490.

For more information on GDB's symbol table management, see Chapter 16 [Examining the Symbol Table], page 265, for more information.

A `gdb.Symtab_and_line` object has the following attributes:

`Symtab_and_line.symtab` [Variable]

The symbol table object (`gdb.Symtab`) for this frame. This attribute is not writable.

`Symtab_and_line.pc` [Variable]

Indicates the start of the address range occupied by code for the current source line. This attribute is not writable.

`Symtab_and_line.last` [Variable]

Indicates the end of the address range occupied by code for the current source line. This attribute is not writable.

`Symtab_and_line.line` [Variable]

Indicates the current line number for this object. This attribute is not writable.

A `gdb.Symtab_and_line` object has the following methods:

`Symtab_and_line.is_valid ()` [Function]

Returns `True` if the `gdb.Symtab_and_line` object is valid, `False` if not. A `gdb.Symtab_and_line` object can become invalid if the Symbol table and line object it refers to does not exist in GDB any longer. All other `gdb.Symtab_and_line` methods will throw an exception if it is invalid at the time the method is called.

A `gdb.Symtab` object has the following attributes:

`Symtab.filename` [Variable]

The symbol table's source filename. This attribute is not writable.

`Symtab.objfile` [Variable]

The symbol table's backing object file. See Section 23.3.2.27 [Objfiles In Python], page 487. This attribute is not writable.

Symtab.producer [Variable]

The name and possibly version number of the program that compiled the code in the symbol table. The contents of this string is up to the compiler. If no producer information is available then **None** is returned. This attribute is not writable.

A `gdb.Symtab` object has the following methods:

Symtab.is_valid () [Function]

Returns **True** if the `gdb.Symtab` object is valid, **False** if not. A `gdb.Symtab` object can become invalid if the symbol table it refers to does not exist in GDB any longer. All other `gdb.Symtab` methods will throw an exception if it is invalid at the time the method is called.

Symtab.fullname () [Function]

Return the symbol table's source absolute file name.

Symtab.global_block () [Function]

Return the global block of the underlying symbol table. See Section 23.3.2.29 [Blocks In Python], page 493.

Symtab.static_block () [Function]

Return the static block of the underlying symbol table. See Section 23.3.2.29 [Blocks In Python], page 493.

Symtab.linetable () [Function]

Return the line table associated with the symbol table. See Section 23.3.2.32 [Line Tables In Python], page 502.

Symtab.source_lines (*first* = 1, *last* = 0, *unstyled* = **False)** [Function]

Read source lines from the file associated with this `Symtab`, this will be the filename pointed to by `Symtab.fullname`. The source lines are returned as a tuple of strings, each string represents a single source line.

The *first* and *last* arguments can be used to only return some lines from the file, *first* will be the first line returned, and *last* will be the last line returned, as such *first* must be less than, or equal to, *last*.

The default for *first* is 1, which is the first line of the file. The default for *last* is 0, which is a special value meaning the last line of the file.

When *unstyled* is **True** then the strings returned will have no style information included. When *unstyled* is **False**, the default, then the returned strings will include style information if styling is enabled (see Section 22.5 [Output Styling], page 375), and GDB knows how to style the source file. Styling information will take the form of ANSI terminal escape sequences embedded within the strings.

If the source file cannot be read, for example, if the source file is missing, then this method returns **None**.

23.3.2.32 Manipulating line tables using Python

Python code can request and inspect line table information from a symbol table that is loaded in GDB. A line table is a mapping of source lines to their executable locations in memory. To acquire the line table information for a particular symbol table, use the `linetable` function (see Section 23.3.2.31 [Symbol Tables In Python], page 500).

A `gdb.LineTable` is iterable. The iterator returns `LineTableEntry` objects that correspond to the source line and address for each line table entry. `LineTableEntry` objects have the following attributes:

LineTableEntry.line [Variable]

The source line number for this line table entry. This number corresponds to the actual line of source. This attribute is not writable.

LineTableEntry.pc [Variable]

The address that is associated with the line table entry where the executable code for that source line resides in memory. This attribute is not writable.

As there can be multiple addresses for a single source line, you may receive multiple `LineTableEntry` objects with matching `line` attributes, but with different `pc` attributes. The iterator is sorted in ascending `pc` order. Here is a small example illustrating iterating over a line table.

```
syntab = gdb.selected_frame().find_sal().syntab
linetable = syntab.linetable()
for line in linetable:
    print ("Line: "+str(line.line)+" Address: "+hex(line.pc))
```

This will have the following output:

```
Line: 33 Address: 0x4005c8L
Line: 37 Address: 0x4005caL
Line: 39 Address: 0x4005d2L
Line: 40 Address: 0x4005f8L
Line: 42 Address: 0x4005ffL
Line: 44 Address: 0x400608L
Line: 42 Address: 0x40060cL
Line: 45 Address: 0x400615L
```

In addition to being able to iterate over a `LineTable`, it also has the following direct access methods:

LineTable.line (*line*) [Function]

Return a Python `Tuple` of `LineTableEntry` objects for any entries in the line table for the given *line*, which specifies the source code line. If there are no entries for that source code *line*, the Python `None` is returned.

LineTable.has_line (*line*) [Function]

Return a Python `Boolean` indicating whether there is an entry in the line table for this source line. Return `True` if an entry is found, or `False` if not.

LineTable.source_lines () [Function]

Return a Python `List` of the source line numbers in the symbol table. Only lines with executable code locations are returned. The contents of the `List` will just be the source line entries represented as Python `Long` values.

23.3.2.33 Manipulating breakpoints using Python

Python code can manipulate breakpoints via the `gdb.Breakpoint` class.

A breakpoint can be created using one of the two forms of the `gdb.Breakpoint` constructor. The first one accepts a string like one would pass to the **break** (see Section 5.1.1 [Setting Breakpoints], page 60) and **watch** (see Section 5.1.2 [Setting Watchpoints], page 68) commands, and can be used to create both breakpoints and watchpoints. The second accepts separate Python arguments similar to Section 9.2.2 [Explicit Locations], page 131, and can only be used to create breakpoints.

Breakpoint.__init__ (*spec* [, *type*][, *wp-class*][, *internal*][, *temporary*][, *qualified*]) [Function]

Create a new breakpoint according to *spec*, which is a string naming the location of a breakpoint, or an expression that defines a watchpoint. The string should describe a location in a format recognized by the **break** command (see Section 5.1.1 [Setting Breakpoints], page 60) or, in the case of a watchpoint, by the **watch** command (see Section 5.1.2 [Setting Watchpoints], page 68).

The optional *type* argument specifies the type of the breakpoint to create, as defined below.

The optional *wp-class* argument defines the class of watchpoint to create, if *type* is `gdb.BP_WATCHPOINT`. If *wp-class* is omitted, it defaults to `gdb.WP_WRITE`.

The optional *internal* argument allows the breakpoint to become invisible to the user. The breakpoint will neither be reported when created, nor will it be listed in the output from **info breakpoints** (but will be listed with the **maint info breakpoints** command).

The optional *temporary* argument makes the breakpoint a temporary breakpoint. Temporary breakpoints are deleted after they have been hit. Any further access to the Python breakpoint after it has been hit will result in a runtime error (as that breakpoint has now been automatically deleted).

The optional *qualified* argument is a boolean that allows interpreting the function passed in *spec* as a fully-qualified name. It is equivalent to **break**'s **-qualified** flag (see Section 9.2.1 [Linespec Locations], page 129, and Section 9.2.2 [Explicit Locations], page 131).

Breakpoint.__init__ ([*source*][, *function*][, *label*][, *line*], [*internal*][, *temporary*][, *qualified*]) [Function]

This second form of creating a new breakpoint specifies the explicit location (see Section 9.2.2 [Explicit Locations], page 131) using keywords. The new breakpoint will be created in the specified source file *source*, at the specified *function*, *label* and *line*.

internal, *temporary* and *qualified* have the same usage as explained previously.

The available types are represented by constants defined in the `gdb` module:

`gdb.BP_BREAKPOINT`

Normal code breakpoint.

`gdb.BP_HARDWARE_BREAKPOINT`

Hardware assisted code breakpoint.

```

gdb.BP_WATCHPOINT
    Watchpoint breakpoint.

gdb.BP_HARDWARE_WATCHPOINT
    Hardware assisted watchpoint.

gdb.BP_READ_WATCHPOINT
    Hardware assisted read watchpoint.

gdb.BP_ACCESS_WATCHPOINT
    Hardware assisted access watchpoint.

gdb.BP_CATCHPOINT
    Catchpoint. Currently, this type can't be used when creating gdb.Breakpoint
    objects, but will be present in gdb.Breakpoint objects reported from
    gdb.BreakpointEvents (see Section 23.3.2.18 [Events In Python], page 458).

```

The available watchpoint types are represented by constants defined in the `gdb` module:

```

gdb.WP_READ
    Read only watchpoint.

gdb.WP_WRITE
    Write only watchpoint.

gdb.WP_ACCESS
    Read/Write watchpoint.

```

`Breakpoint.stop (self)` [Function]

The `gdb.Breakpoint` class can be sub-classed and, in particular, you may choose to implement the `stop` method. If this method is defined in a sub-class of `gdb.Breakpoint`, it will be called when the inferior reaches any location of a breakpoint which instantiates that sub-class. If the method returns `True`, the inferior will be stopped at the location of the breakpoint, otherwise the inferior will continue.

If there are multiple breakpoints at the same location with a `stop` method, each one will be called regardless of the return status of the previous. This ensures that all `stop` methods have a chance to execute at that location. In this scenario if one of the methods returns `True` but the others return `False`, the inferior will still be stopped. You should not alter the execution state of the inferior (i.e., `step`, `next`, etc.), alter the current frame context (i.e., change the current active frame), or alter, add or delete any breakpoint. As a general rule, you should not alter any data within GDB or the inferior at this time.

Example `stop` implementation:

```

class MyBreakpoint (gdb.Breakpoint):
    def stop (self):
        inf_val = gdb.parse_and_eval("foo")
        if inf_val == 3:
            return True
        return False

```

`Breakpoint.is_valid ()` [Function]

Return `True` if this `Breakpoint` object is valid, `False` otherwise. A `Breakpoint` object can become invalid if the user deletes the breakpoint. In this case, the object

still exists, but the underlying breakpoint does not. In the cases of watchpoint scope, the watchpoint remains valid even if execution of the inferior leaves the scope of that watchpoint.

Breakpoint.delete () [Function]

Permanently deletes the GDB breakpoint. This also invalidates the Python **Breakpoint** object. Any further access to this object's attributes or methods will raise an error.

Breakpoint.enabled [Variable]

This attribute is **True** if the breakpoint is enabled, and **False** otherwise. This attribute is writable. You can use it to enable or disable the breakpoint.

Breakpoint.silent [Variable]

This attribute is **True** if the breakpoint is silent, and **False** otherwise. This attribute is writable.

Note that a breakpoint can also be silent if it has commands and the first command is **silent**. This is not reported by the **silent** attribute.

Breakpoint.pending [Variable]

This attribute is **True** if the breakpoint is pending, and **False** otherwise. See Section 5.1.1 [Set Breaks], page 60. This attribute is read-only.

Breakpoint.thread [Variable]

If the breakpoint is thread-specific (see Section 5.5.4 [Thread-Specific Breakpoints], page 99), this attribute holds the thread's global id. If the breakpoint is not thread-specific, this attribute is **None**. This attribute is writable.

Only one of **Breakpoint.thread** or **Breakpoint.inferior** can be set to a valid id at any time, that is, a breakpoint can be thread specific, or inferior specific, but not both.

Breakpoint.inferior [Variable]

If the breakpoint is inferior-specific (see Section 4.9.1 [Inferior-Specific Breakpoints], page 47), this attribute holds the inferior's id. If the breakpoint is not inferior-specific, this attribute is **None**.

This attribute can be written for breakpoints of type **gdb.BP_BREAKPOINT** and **gdb.BP_HARDWARE_BREAKPOINT**.

Breakpoint.task [Variable]

If the breakpoint is Ada task-specific, this attribute holds the Ada task id. If the breakpoint is not task-specific (or the underlying language is not Ada), this attribute is **None**. This attribute is writable.

Breakpoint.ignore_count [Variable]

This attribute holds the ignore count for the breakpoint, an integer. This attribute is writable.

Breakpoint.number [Variable]

This attribute holds the breakpoint's number — the identifier used by the user to manipulate the breakpoint. This attribute is not writable.

Breakpoint.type [Variable]

This attribute holds the breakpoint's type — the identifier used to determine the actual breakpoint type or use-case. This attribute is not writable.

Breakpoint.visible [Variable]

This attribute tells whether the breakpoint is visible to the user when set, or when the `'info breakpoints'` command is run. This attribute is not writable.

Breakpoint.temporary [Variable]

This attribute indicates whether the breakpoint was created as a temporary breakpoint. Temporary breakpoints are automatically deleted after that breakpoint has been hit. Access to this attribute, and all other attributes and functions other than the `is_valid` function, will result in an error after the breakpoint has been hit (as it has been automatically deleted). This attribute is not writable.

Breakpoint.hit_count [Variable]

This attribute holds the hit count for the breakpoint, an integer. This attribute is writable, but currently it can only be set to zero.

Breakpoint.location [Variable]

This attribute holds the location of the breakpoint, as specified by the user. It is a string. If the breakpoint does not have a location (that is, it is a watchpoint) the attribute's value is `None`. This attribute is not writable.

Breakpoint.locations [Variable]

Get the most current list of breakpoint locations that are inserted for this breakpoint, with elements of type `gdb.BreakpointLocation` (described below). This functionality matches that of the `info breakpoint` command (see Section 5.1.1 [Set Breaks], page 60), in that it only retrieves the most current list of locations, thus the list itself when returned is not updated behind the scenes. This attribute is not writable.

Breakpoint.expression [Variable]

This attribute holds a breakpoint expression, as specified by the user. It is a string. If the breakpoint does not have an expression (the breakpoint is not a watchpoint) the attribute's value is `None`. This attribute is not writable.

Breakpoint.condition [Variable]

This attribute holds the condition of the breakpoint, as specified by the user. It is a string. If there is no condition, this attribute's value is `None`. This attribute is writable.

Breakpoint.commands [Variable]

This attribute holds the commands attached to the breakpoint. If there are commands, this attribute's value is a string holding all the commands, separated by newlines. If there are no commands, this attribute is `None`. This attribute is writable.

Breakpoint Locations

A breakpoint location is one of the actual places where a breakpoint has been set, represented in the Python API by the `gdb.BreakpointLocation` type. This type is never

instantiated by the user directly, but is retrieved from `Breakpoint.locations` which returns a list of breakpoint locations where it is currently set. Breakpoint locations can become invalid if new symbol files are loaded or dynamically loaded libraries are closed. Accessing the attributes of an invalidated breakpoint location will throw a `RuntimeError` exception. Access the `Breakpoint.locations` attribute again to retrieve the new and valid breakpoints location list.

BreakpointLocation.source [Variable]

This attribute returns the source file path and line number where this location was set. The type of the attribute is a tuple of *string* and *long*. If the breakpoint location doesn't have a source location, it returns `None`, which is the case for watchpoints and catchpoints. This will throw a `RuntimeError` exception if the location has been invalidated. This attribute is not writable.

BreakpointLocation.address [Variable]

This attribute returns the address where this location was set. This attribute is of type *long*. This will throw a `RuntimeError` exception if the location has been invalidated. This attribute is not writable.

BreakpointLocation.enabled [Variable]

This attribute holds the value for whether or not this location is enabled. This attribute is writable (boolean). This will throw a `RuntimeError` exception if the location has been invalidated.

BreakpointLocation.owner [Variable]

This attribute holds a reference to the `gdb.Breakpoint` owner object, from which this `gdb.BreakpointLocation` was retrieved from. This will throw a `RuntimeError` exception if the location has been invalidated. This attribute is not writable.

BreakpointLocation.function [Variable]

This attribute gets the name of the function where this location was set. If no function could be found this attribute returns `None`. This will throw a `RuntimeError` exception if the location has been invalidated. This attribute is not writable.

BreakpointLocation.fullname [Variable]

This attribute gets the full name of where this location was set. If no full name could be found, this attribute returns `None`. This will throw a `RuntimeError` exception if the location has been invalidated. This attribute is not writable.

BreakpointLocation.thread_groups [Variable]

This attribute gets the thread groups it was set in. It returns a `List` of the thread group ID's. This will throw a `RuntimeError` exception if the location has been invalidated. This attribute is not writable.

23.3.2.34 Finish Breakpoints

A finish breakpoint is a temporary breakpoint set at the return address of a frame, based on the `finish` command. `gdb.FinishBreakpoint` extends `gdb.Breakpoint`. The underlying breakpoint will be disabled and deleted when the execution will run out of the breakpoint scope (i.e. `Breakpoint.stop` or `FinishBreakpoint.out_of_scope` triggered). Finish breakpoints are thread specific and must be create with the right thread selected.

It is not possible to create a `gdb.FinishBreakpoint` for an inline frame (see Section 11.1 [Inline Functions], page 197).

FinishBreakpoint.__init__ (*[frame]* [, *internal*]) [Function]

Create a finish breakpoint at the return address of the `gdb.Frame` object *frame*. If *frame* is not provided, this defaults to the newest frame. The optional *internal* argument allows the breakpoint to become invisible to the user. See Section 23.3.2.33 [Breakpoints In Python], page 503, for further details about this argument.

FinishBreakpoint.out_of_scope (*self*) [Function]

In some circumstances (e.g. `longjmp`, C++ exceptions, GDB `return` command, . . .), a function may not properly terminate, and thus never hit the finish breakpoint. When GDB notices such a situation, the `out_of_scope` callback will be triggered.

You may want to sub-class `gdb.FinishBreakpoint` and override this method:

```
class MyFinishBreakpoint (gdb.FinishBreakpoint)
    def stop (self):
        print ("normal finish")
        return True

    def out_of_scope ():
        print ("abnormal finish")
```

FinishBreakpoint.return_value [Variable]

When GDB is stopped at a finish breakpoint and the frame used to build the `gdb.FinishBreakpoint` object had debug symbols, this attribute will contain a `gdb.Value` object corresponding to the return value of the function. The value will be `None` if the function return type is `void` or if the return value was not computable. This attribute is not writable.

23.3.2.35 Python representation of lazy strings

A *lazy string* is a string whose contents is not retrieved or encoded until it is needed.

A `gdb.LazyString` is represented in GDB as an **address** that points to a region of memory, an **encoding** that will be used to encode that region of memory, and a **length** to delimit the region of memory that represents the string. The difference between a `gdb.LazyString` and a string wrapped within a `gdb.Value` is that a `gdb.LazyString` will only be retrieved and encoded during printing, while a `gdb.Value` wrapping a string is immediately retrieved and encoded on creation.

A `gdb.LazyString` can be created using the `gdb.Value.lazy_string` method (see Section 23.3.2.4 [Values From Inferior], page 417).

A `gdb.LazyString` object has the following functions:

LazyString.value () [Function]

Convert the `gdb.LazyString` to a `gdb.Value`. This value will point to the string in memory, but will lose all the delayed retrieval, encoding and handling that GDB applies to a `gdb.LazyString`.

LazyString.address [Variable]

This attribute holds the address of the string. This attribute is not writable.

LazyString.length [Variable]

This attribute holds the length of the string in characters. If the length is -1, then the string will be fetched and encoded up to the first null of appropriate width. This attribute is not writable.

LazyString.encoding [Variable]

This attribute holds the encoding that will be applied to the string when the string is printed by GDB. If the encoding is not set, or contains an empty string, then GDB will select the most appropriate encoding when the string is printed. This attribute is not writable.

LazyString.type [Variable]

This attribute holds the type that is represented by the lazy string's type. For a lazy string this is a pointer or array type. To resolve this to the lazy string's character type, use the type's `target` method. See Section 23.3.2.5 [Types In Python], page 425. This attribute is not writable.

23.3.2.36 Python representation of colors

You can assign instance of `gdb.Color` to the value of a `Parameter` instance created with `PARAM_COLOR`.

`gdb.Color` may refer to an index from a color palette or contain components of a color from some color space.

Color.__init__ ([*value* [, *color_space*]]) [Function]

value is `None` (meaning the terminal's default color), an integer index of a color in palette, tuple with color components or a string. String can be a hex RGB triplet in `'#RRGGBB'` format or one of the following color names: `'none'` (meaning the terminal's default color), `'black'`, `'red'`, `'green'`, `'yellow'`, `'blue'`, `'magenta'`, `'cyan'`, or `'white'`. *color_space* should be one of the `'COLORSPACE_'` constants listed below. This argument tells GDB which color space *value* belongs to.

Color.is_none [Variable]

This attribute is boolean. If its value is `True` then color is terminal's default.

Color.is_indexed [Variable]

This attribute is boolean. If its value is `True` then color is indexed, i.e. belongs to some palette.

Color.is_direct [Variable]

This attribute is boolean. If its value is `True` then this object describes direct colour in the sense of ISO/IEC 8613-6.

Color.index [Variable]

This attribute exist if `is_indexed` is `True`. Its integer value is index of a color in a palette.

Color.components [Variable]

This attribute exist if `is_direct` is `True`. Its value is tuple with integer components of a color.

Color.escape_sequence (*is_foreground*) [Function]

Returns string to change terminal's color to this.

If *is_foreground* is **True**, then the returned sequence will change foreground color. Otherwise, the returned sequence will change background color.

If styling is currently disabled (see Section 22.5 [**set style enabled**], page 375), then this method will return an empty string.

When color is initialized, its color space must be specified. The available color spaces are represented by constants defined in the **gdb** module:

gdb.COLORSPACE_MONOCHROME

Palette with only terminal's default color.

gdb.COLORSPACE_ANSI_8COLOR

Palette with eight standard colors of ISO/IEC 6429 "black", "red", "green", etc.

gdb.COLORSPACE_AIXTERM_16COLOR

Palette with 16 colors. First eight are standard colors of ISO/IEC 6429 "black", "red", "green", etc. Next eight are their bright version.

gdb.COLORSPACE_XTERM_256COLOR

Palette with 256 colors. First 16 are from **COLORSPACE_AIXTERM_16COLOR**. Next 216 colors are 6x6x6 RGB cube. And last 24 colors form grayscale ramp.

gdb.COLORSPACE_RGB_24BIT

Direct 24-bit RGB colors.

It is not possible to sub-class the **Color** class.

23.3.2.37 Python representation of styles

A style object contains the foreground and background colors (see Section 23.3.2.36 [Colors In Python], page 509), along with an intensity, and can be used to apply this styling to output produced from Python.

GDB has many styles builtin (see Section 22.5 [Output Styling], page 375), and style objects can be created that apply these builtin styles to Python output. It is also possible to create new styles which can be used to style Python output (see [Creating Style Parameters], page 482).

The style class is called **gdb.Style**, and has the following methods and attributes:

Style.__init__ (*style_name*) [Function]

Create a **gdb.Style** that represents a builtin named style. The *style_name* must be a non-empty string that names a style that exists as a setting in GDB within '**set style ...**'. For example, the string "**filename**" can be used to create a **gdb.Style** object representing the '**set style filename**' style.

If *style_name* names an unknown style then a **RuntimeError** exception is raised.

Style.__init__ (*foreground=None, background=None, intensity=gdb.INTENSITY_NORMAL*) [Function]

Create a custom **gdb.Style**, manually specifying the three individual components. All of the arguments are optional. By default, if no arguments are given, then the

default style will be created, this will produce output with the default terminal foreground and background colors, along with the normal level of intensity (i.e. neither bold, nor dim).

The *foreground* and *background* arguments should either be `None`, in which case the terminal default colors are used, or a `gdb.Color` object (see Section 23.3.2.36 [Colors In Python], page 509). Any other object type will result in a `TypeError` exception being raised.

The *intensity* argument should be one of the intensity constants defined below (see [Style Intensities], page 512). Passing a none integer value results in a `TypeError` exception, and passing an invalid constant results in a `ValueError` exception.

Style.escape_sequence () [Function]

If styling is enabled (see Section 22.5 [Output Styling], page 375), then this method returns the string which is the terminal escape sequence necessary to apply this `gdb.Style`.

If styling is disabled then this method returns the empty string.

If this is a named style, which for some reason can no longer be read, then a `RuntimeError` exception is raised.

Style.apply (string) [Function]

This method returns *string*, which must be a `str` object, with an escape sequence at both the start, and at the end. The escape sequence at the start applies this style, while the escape sequence at the end applies the terminal's default style.

The effect of this is that, printing the result of this method, will print *string* with this style applied. Future output will be printed with the terminal's default style.

If styling is currently disabled (see Section 22.5 [Output Styling], page 375), then `Style.apply` just returns a copy of *string* with no escape sequences added.

If this is a named style, which for some reason can no longer be read, then a `RuntimeError` exception is raised.

Style.foreground [Variable]

This read/write attribute contains the `gdb.Color` object representing this style's foreground color. Writing to this attribute updates the style's foreground color.

If the `gdb.Style` object was created using a named style (i.e. using the single argument *style_name* `__init__` method), then the underlying setting will be updated.

For unnamed styles, only the `gdb.Style` object will change.

Style.background [Variable]

This read/write attribute contains the `gdb.Color` object representing this style's background color. Writing to this attribute updates the style's background color.

If the `gdb.Style` object was created using a named style (i.e. using the single argument *style_name* `__init__` method), then the underlying setting will be updated.

For unnamed styles, only the `gdb.Style` object will change.

Style.intensity [Variable]

This read/write attribute contains the intensity for this style, and will be one of the constants defined below (see [Style Intensities], page 512). Writing to this attribute updates the style's intensity.

It is also possible to write `None` to this attribute, this is equivalent of writing `gdb.INTENSITY_NORMAL`. This attribute will never read as `None`.

If the `gdb.Style` object was created using a named style (i.e. using the single argument `style_name __init__` method), then the underlying setting will be updated.

For unnamed styles, only the `gdb.Style` object will change.

The following constants are defined to represent the different style intensities:

`gdb.INTENSITY_NORMAL`

This is the terminal's default intensity.

`gdb.INTENSITY_BOLD`

This is for bold text.

`gdb.INTENSITY_DIM`

This is for dim text.

23.3.2.38 Python representation of architectures

GDB uses architecture specific parameters and artifacts in a number of its various computations. An architecture is represented by an instance of the `gdb.Architecture` class.

A `gdb.Architecture` class has the following methods:

Architecture.name () [Function]

Return the name (string value) of the architecture.

Architecture.disassemble (*start_pc* [, *end_pc* [, *count*]] [, *styling* = False]) [Function]

Return a list of disassembled instructions starting from the memory address *start_pc*. The optional arguments *end_pc* and *count* determine the number of instructions in the returned list. If both the optional arguments *end_pc* and *count* are specified, then a list of at most *count* disassembled instructions whose start address falls in the closed memory address interval from *start_pc* to *end_pc* are returned. If *end_pc* is not specified, but *count* is specified, then *count* number of instructions starting from the address *start_pc* are returned. If *count* is not specified but *end_pc* is specified, then all instructions whose start address falls in the closed memory address interval from *start_pc* to *end_pc* are returned. If neither *end_pc* nor *count* are specified, then a single instruction at *start_pc* is returned. For all of these cases, each element of the returned list is a Python `dict` with the following string keys:

addr The value corresponding to this key is a Python long integer capturing the memory address of the instruction.

asm The value corresponding to this key is a string value which represents the instruction with assembly language mnemonics. The assembly language flavor used is the same as that specified by the current CLI variable `disassembly-flavor`. See Section 9.6 [Machine Code], page 137.

When the optional argument *styling* is **True** the ‘asm’ string can contain ANSI terminal escape sequences if styling is enabled (see Section 22.5 [Output Styling], page 375). When *styling* is **False**, which is the default, then no styling will be applied to ‘asm’.

length The value corresponding to this key is the length (integer value) of the instruction in bytes.

Architecture.integer_type (*size* [, *signed*]) [Function]

This function looks up an integer type by its *size*, and optionally whether or not it is signed.

size is the size, in bits, of the desired integer type. Only certain sizes are currently supported: 0, 8, 16, 24, 32, 64, and 128.

If *signed* is not specified, it defaults to **True**. If *signed* is **False**, the returned type will be unsigned.

If the indicated type cannot be found, this function will throw a **ValueError** exception.

Architecture.void_type () [Function]

This function returns a void type.

Architecture.registers ([*reggroup*]) [Function]

Return a **gdb.RegisterDescriptorIterator** (see Section 23.3.2.39 [Registers In Python], page 513) for all of the registers in *reggroup*, a string that is the name of a register group. If *reggroup* is omitted, or is the empty string, then the register group ‘all’ is assumed.

Architecture.register_groups () [Function]

Return a **gdb.RegisterGroupsIterator** (see Section 23.3.2.39 [Registers In Python], page 513) for all of the register groups available for the **gdb.Architecture**.

23.3.2.39 Registers In Python

Python code can request from a **gdb.Architecture** information about the set of registers available (see [Architecture.registers], page 513). The register information is returned as a **gdb.RegisterDescriptorIterator**, which is an iterator that in turn returns **gdb.RegisterDescriptor** objects.

A **gdb.RegisterDescriptor** does not provide the value of a register (see [Frame.read_register], page 492, for reading a register’s value), instead the **RegisterDescriptor** is a way to discover which registers are available for a particular architecture.

A **gdb.RegisterDescriptor** has the following read-only properties:

RegisterDescriptor.name [Variable]

The name of this register.

It is also possible to lookup a register descriptor based on its name using the following **gdb.RegisterDescriptorIterator** function:

RegisterDescriptorIterator.find (*name*) [Function]

Takes *name* as an argument, which must be a string, and returns a `gdb.RegisterDescriptor` for the register with that name, or `None` if there is no register with that name.

Python code can also request from a `gdb.Architecture` information about the set of register groups available on a given architecture (see `[Architecture.register_groups]`, page 513).

Every register can be a member of zero or more register groups. Some register groups are used internally within GDB to control things like which registers must be saved when calling into the program being debugged (see Section 17.5 `[Calling Program Functions]`, page 283). Other register groups exist to allow users to easily see related sets of registers in commands like `info registers` (see `[info registers reggroup]`, page 181).

The register groups information is returned as a `gdb.RegisterGroupsIterator`, which is an iterator that in turn returns `gdb.RegisterGroup` objects.

A `gdb.RegisterGroup` object has the following read-only properties:

RegisterGroup.name [Variable]

A string that is the name of this register group.

23.3.2.40 Connections In Python

GDB lets you run and debug multiple programs in a single session. Each program being debugged has a connection, the connection describes how GDB controls the program being debugged. Examples of different connection types are ‘native’ and ‘remote’. See Section 4.9 `[Inferiors Connections and Programs]`, page 43.

Connections in GDB are represented as instances of `gdb.TargetConnection`, or as one of its sub-classes. To get a list of all connections use `gdb.connections` (see `[gdb.connections]`, page 414).

To get the connection for a single `gdb.Inferior` read its `gdb.Inferior.connection` attribute (see `[gdb.Inferior.connection]`, page 455).

Currently there is only a single sub-class of `gdb.TargetConnection`, `gdb.RemoteTargetConnection`, however, additional sub-classes may be added in future releases of GDB. As a result you should avoid writing code like:

```
conn = gdb.selected_inferior().connection
if type(conn) is gdb.RemoteTargetConnection:
    print("This is a remote target connection")
```

as this may fail when more connection types are added. Instead, you should write:

```
conn = gdb.selected_inferior().connection
if isinstance(conn, gdb.RemoteTargetConnection):
    print("This is a remote target connection")
```

A `gdb.TargetConnection` has the following method:

TargetConnection.is_valid () [Function]

Return `True` if the `gdb.TargetConnection` object is valid, `False` if not. A `gdb.TargetConnection` will become invalid if the connection no longer exists within GDB, this might happen when no inferiors are using the connection, but could be delayed until the user replaces the current target.

Reading any of the `gdb.TargetConnection` properties will throw an exception if the connection is invalid.

A `gdb.TargetConnection` has the following read-only properties:

TargetConnection.num [Variable]

An integer assigned by GDB to uniquely identify this connection. This is the same value as displayed in the ‘Num’ column of the `info connections` command output (see Section 4.9 [info connections], page 43).

TargetConnection.type [Variable]

A string that describes what type of connection this is. This string will be one of the valid names that can be passed to the `target` command (see Section 19.2 [target command], page 313).

TargetConnection.description [Variable]

A string that gives a short description of this target type. This is the same string that is displayed in the ‘Description’ column of the `info connection` command output (see Section 4.9 [info connections], page 43).

TargetConnection.details [Variable]

An optional string that gives additional information about this connection. This attribute can be `None` if there are no additional details for this connection.

An example of a connection type that might have additional details is the ‘remote’ connection, in this case the details string can contain the ‘*hostname:port*’ that was used to connect to the remote target.

The `gdb.RemoteTargetConnection` class is a sub-class of `gdb.TargetConnection`, and is used to represent ‘remote’ and ‘extended-remote’ connections. In addition to the attributes and methods available from the `gdb.TargetConnection` base class, a `gdb.RemoteTargetConnection` has the following method:

RemoteTargetConnection.send_packet (*packet*) [Function]

This method sends *packet* to the remote target and returns the response. The *packet* should either be a `bytes` object, or a `Unicode` string.

If *packet* is a `Unicode` string, then the string is encoded to a `bytes` object using the ASCII codec. If the string can’t be encoded then an `UnicodeError` is raised.

If *packet* is not a `bytes` object, or a `Unicode` string, then a `TypeError` is raised. If *packet* is empty then a `ValueError` is raised.

The response is returned as a `bytes` object. If it is known that the response can be represented as a string then this can be decoded from the buffer. For example, if it is known that the response is an ASCII string:

```
remote_connection.send_packet("some_packet").decode("ascii")
```

The prefix, suffix, and checksum (as required by the remote serial protocol) are automatically added to the outgoing packet, and removed from the incoming packet before the contents of the reply are returned.

This is equivalent to the `maintenance packet` command (see [maint packet], page 781).

23.3.2.41 Implementing new TUI windows

New TUI (see Chapter 25 [TUI], page 599) windows can be implemented in Python.

`gdb.register_window_type (name, factory)` [Function]

Because TUI windows are created and destroyed depending on the layout the user chooses, new window types are implemented by registering a factory function with GDB.

name is the name of the new window. It's an error to try to replace one of the built-in windows, but other window types can be replaced. The *name* should match the regular expression `[a-zA-Z][-_.a-zA-Z0-9]*`, it is an error to try and create a window with an invalid name.

function is a factory function that is called to create the TUI window. This is called with a single argument of type `gdb.TuiWindow`, described below. It should return an object that implements the TUI window protocol, also described below.

As mentioned above, when a factory function is called, it is passed an object of type `gdb.TuiWindow`. This object has these methods and attributes:

`TuiWindow.is_valid ()` [Function]

This method returns `True` when this window is valid. When the user changes the TUI layout, windows no longer visible in the new layout will be destroyed. At this point, the `gdb.TuiWindow` will no longer be valid, and methods (and attributes) other than `is_valid` will throw an exception.

When the TUI is disabled using `tui disable` (see Section 25.5 [tui disable], page 602) the window is hidden rather than destroyed, but `is_valid` will still return `False` and other methods (and attributes) will still throw an exception.

`TuiWindow.width` [Variable]

This attribute holds the width of the window. It is not writable.

`TuiWindow.height` [Variable]

This attribute holds the height of the window. It is not writable.

`TuiWindow.title` [Variable]

This attribute holds the window's title, a string. This is normally displayed above the window. This attribute can be modified.

`TuiWindow.erase ()` [Function]

Remove all the contents of the window.

`TuiWindow.write (string [, full_window])` [Function]

Write *string* to the window. *string* can contain ANSI terminal escape styling sequences; GDB will translate these as appropriate for the terminal.

If the *full_window* parameter is `True`, then *string* contains the full contents of the window. This is similar to calling `erase` before `write`, but avoids the flickering.

The factory function that you supply should return an object conforming to the TUI window protocol. These are the methods that can be called on this object, which is referred to below as the “window object”. The methods documented below are optional; if the object

does not implement one of these methods, GDB will not attempt to call it. Additional new methods may be added to the window protocol in the future. GDB guarantees that they will begin with a lower-case letter, so you can start implementation methods with upper-case letters or underscore to avoid any future conflicts.

Window.close () [Function]

When the TUI window is closed, the `gdb.TuiWindow` object will be put into an invalid state. At this time, GDB will call `close` method on the window object.

After this method is called, GDB will discard any references it holds on this window object, and will no longer call methods on this object.

Window.render () [Function]

In some situations, a TUI window can change size. For example, this can happen if the user resizes the terminal, or changes the layout. When this happens, GDB will call the `render` method on the window object.

If your window is intended to update in response to changes in the inferior, you will probably also want to register event listeners and send output to the `gdb.TuiWindow`.

Window.hscroll (num) [Function]

This is a request to scroll the window horizontally. *num* is the amount by which to scroll, with negative numbers meaning to scroll right. In the TUI model, it is the viewport that moves, not the contents. A positive argument should cause the viewport to move right, and so the content should appear to move to the left.

Window.vscroll (num) [Function]

This is a request to scroll the window vertically. *num* is the amount by which to scroll, with negative numbers meaning to scroll backward. In the TUI model, it is the viewport that moves, not the contents. A positive argument should cause the viewport to move down, and so the content should appear to move up.

Window.click (x, y, button) [Function]

This is called on a mouse click in this window. *x* and *y* are the mouse coordinates inside the window (0-based, from the top left corner), and *button* specifies which mouse button was used, whose values can be 1 (left), 2 (middle), or 3 (right).

When TUI mouse events are disabled by turning off the `tui mouse-events` setting (see [set tui mouse-events], page 606), then `click` will not be called.

23.3.2.42 Instruction Disassembly In Python

GDB's builtin disassembler can be extended, or even replaced, using the Python API. The disassembler related features are contained within the `gdb.disassembler` module:

gdb.disassembler.DisassembleInfo [class]

Disassembly is driven by instances of this class. Each time GDB needs to disassemble an instruction, an instance of this class is created and passed to a registered disassembler. The disassembler is then responsible for disassembling an instruction and returning a result.

Instances of this type are usually created within GDB, however, it is possible to create a copy of an instance of this type, see the description of `__init__` for more details.

This class has the following properties and methods:

DisassembleInfo.address [Variable]
 A read-only integer containing the address at which GDB wishes to disassemble a single instruction.

DisassembleInfo.architecture [Variable]
 The `gdb.Architecture` (see Section 23.3.2.38 [Architectures In Python], page 512) for which GDB is currently disassembling, this property is read-only.

DisassembleInfo.progspace [Variable]
 The `gdb.Progspace` (see Section 23.3.2.26 [Program Spaces In Python], page 484) for which GDB is currently disassembling, this property is read-only.

DisassembleInfo.is_valid () [Function]
 Returns `True` if the `DisassembleInfo` object is valid, `False` if not. A `DisassembleInfo` object will become invalid once the disassembly call for which the `DisassembleInfo` was created, has returned. Calling other `DisassembleInfo` methods, or accessing `DisassembleInfo` properties, will raise a `RuntimeError` exception if it is invalid.

DisassembleInfo.__init__ (*info*) [Function]
 This can be used to create a new `DisassembleInfo` object that is a copy of *info*. The copy will have the same `address`, `architecture`, and `progspace` values as *info*, and will become invalid at the same time as *info*.
 This method exists so that sub-classes of `DisassembleInfo` can be created, these sub-classes must be initialized as copies of an existing `DisassembleInfo` object, but sub-classes might choose to override the `read_memory` method, and so control what GDB sees when reading from memory (see [builtin_disassemble], page 525).

DisassembleInfo.read_memory (*length*, *offset*) [Function]
 This method allows the disassembler to read the bytes of the instruction to be disassembled. The method reads *length* bytes, starting at *offset* from `DisassembleInfo.address`.

It is important that the disassembler read the instruction bytes using this method, rather than reading inferior memory directly, as in some cases GDB disassembles from an internal buffer rather than directly from inferior memory, calling this method handles this detail.

Returns a buffer object, which behaves much like an array or a string, just as `Inferior.read_memory` does (see [Inferior.read_memory], page 456). The length of the returned buffer will always be exactly *length*.

If GDB is unable to read the required memory then a `gdb.MemoryError` exception is raised (see Section 23.3.2.3 [Exception Handling], page 416).

This method can be overridden by a sub-class in order to control what GDB sees when reading from memory (see [builtin_disassemble], page 525). When overriding this method it is important to understand how `builtin_disassemble` makes use of this method.

While disassembling a single instruction there could be multiple calls to this method, and the same bytes might be read multiple times. Any single call might only read a subset of the total instruction bytes.

If an implementation of `read_memory` is unable to read the requested memory contents, for example, if there's a request to read from an invalid memory address, then a `gdb.MemoryError` should be raised.

Raising a `MemoryError` inside `read_memory` does not automatically mean a `MemoryError` will be raised by `builtin_disassemble`. It is possible the GDB's builtin disassembler is probing to see how many bytes are available. When `read_memory` raises the `MemoryError` the builtin disassembler might be able to perform a complete disassembly with the bytes it has available, in this case `builtin_disassemble` will not itself raise a `MemoryError`.

Any other exception type raised in `read_memory` will propagate back and be re-raised by `builtin_disassemble`.

DisassembleInfo.text_part (*style*, *string*) [Function]

Create a new `DisassemblerTextPart` representing a piece of a disassembled instruction. *string* should be a non-empty string, and *style* should be an appropriate style constant (see [Disassembler Style Constants], page 522).

Disassembler parts are used when creating a `DisassemblerResult` in order to represent the styling within an instruction (see [DisassemblerResult Class], page 520).

DisassembleInfo.address_part (*address*) [Function]

Create a new `DisassemblerAddressPart`. *address* is the value of the absolute address this part represents. A `DisassemblerAddressPart` is displayed as an absolute address and an associated symbol, the address and symbol are styled appropriately.

gdb.disassembler.Disassembler [class]

This is a base class from which all user implemented disassemblers must inherit.

Disassembler.__init__ (*name*) [Function]

The constructor takes *name*, a string, which should be a short name for this disassembler.

Disassembler.__call__ (*info*) [Function]

The `__call__` method must be overridden by sub-classes to perform disassembly. Calling `__call__` on this base class will raise a `NotImplementedError` exception.

The *info* argument is an instance of `DisassembleInfo`, and describes the instruction that GDB wants disassembling.

If this function returns `None`, this indicates to GDB that this sub-class doesn't wish to disassemble the requested instruction. GDB will then use its builtin disassembler to perform the disassembly.

Alternatively, this function can return a `DisassemblerResult` that represents the disassembled instruction, this type is described in more detail below.

The `__call__` method can raise a `gdb.MemoryError` exception (see Section 23.3.2.3 [Exception Handling], page 416) to indicate to GDB that there was a problem accessing the required memory, this will then be displayed by GDB within the disassembler output.

Ideally, the only three outcomes from invoking `__call__` would be a return of `None`, a successful disassembly returned in a `DisassemblerResult`, or a `MemoryError` indicating that there was a problem reading memory.

However, as an implementation of `__call__` could fail due to other reasons, e.g. some external resource required to perform disassembly is temporarily unavailable, then, if `__call__` raises a `GdbError`, the exception will be converted to a string and printed at the end of the disassembly output, the disassembly request will then stop.

Any other exception type raised by the `__call__` method is considered an error in the user code, the exception will be printed to the error stream according to the `set python print-stack` setting (see [set python print-stack], page 408).

`gdb.disassembler.DisassemblerResult` [class]

This class represents the result of disassembling a single instruction. An instance of this class will be returned from `builtin_disassemble` (see [builtin_disassemble], page 525), and an instance of this class should be returned from `Disassembler.__call__` (see [Disassembler Class], page 519) if an instruction was successfully disassembled.

It is not possible to sub-class the `DisassemblerResult` class.

The `DisassemblerResult` class has the following properties and methods:

`DisassemblerResult.__init__` (*length*, *string*, *parts*) [Function]

Initialize an instance of this class, *length* is the length of the disassembled instruction in bytes, which must be greater than zero.

Only one of *string* or *parts* should be used to initialize a new `DisassemblerResult`; the other one should be passed the value `None`. Alternatively, the arguments can be passed by name, and the unused argument can be ignored.

The *string* argument, if not `None`, is a non-empty string that represents the entire disassembled instruction. Building a result object using the *string* argument does not allow for any styling information to be included in the result. GDB will style the result as a single `DisassemblerTextPart` with `STYLE_TEXT` style (see [Disassembler Styling Parts], page 521).

The *parts* argument, if not `None`, is a non-empty sequence of `DisassemblerPart` objects. Each part represents a small part of the disassembled instruction along with associated styling information. A result object built using *parts* can be displayed by GDB with full styling information (see [set style disassembler enabled], page 376).

`DisassemblerResult.length` [Variable]

A read-only property containing the length of the disassembled instruction in bytes, this will always be greater than zero.

DisassemblerResult.string [Variable]

A read-only property containing a non-empty string representing the disassembled instruction. The *string* is a representation of the disassembled instruction without any styling information. To see how the instruction will be styled use the *parts* property.

If this instance was initialized using separate **DisassemblerPart** objects, the *string* property will still be valid. The *string* value is created by concatenating the **DisassemblerPart.string** values of each component part (see [Disassembler Styling Parts], page 521).

DisassemblerResult.parts [Variable]

A read-only property containing a non-empty sequence of **DisassemblerPart** objects. Each **DisassemblerPart** object contains a small part of the instruction along with information about how that part should be styled. GDB uses this information to create styled disassembler output (see [*set style disassembler enabled*], page 376).

If this instance was initialized using a single string rather than with a sequence of **DisassemblerPart** objects, the *parts* property will still be valid. In this case the *parts* property will hold a sequence containing a single **DisassemblerTextPart** object, the string of which will represent the entire instruction, and the style of which will be **STYLE_TEXT**.

gdb.disassembler.DisassemblerPart [class]

This is a parent class from which the different part sub-classes inherit. Only instances of the sub-classes detailed below will be returned by the Python API.

It is not possible to directly create instances of either this parent class, or any of the sub-classes listed below. Instances of the sub-classes listed below are created by calling **builtin_disassemble** (see [builtin_disassemble], page 525) and are returned within the **DisassemblerResult** object, or can be created by calling the **text_part** and **address_part** methods on the **DisassembleInfo** class (see [DisassembleInfo Class], page 517).

The **DisassemblerPart** class has a single property:

DisassemblerPart.string [Variable]

A read-only property that contains a non-empty string representing this part of the disassembled instruction. The string within this property doesn't include any styling information.

gdb.disassembler.DisassemblerTextPart [class]

The **DisassemblerTextPart** class represents a piece of the disassembled instruction and the associated style for that piece. Instances of this class can't be created directly, instead call **DisassembleInfo.text_part** to create a new instance of this class (see [DisassembleInfo Class], page 517).

As well as the properties of its parent class, the **DisassemblerTextPart** has the following additional property:

DisassemblerTextPart.style [Variable]

A read-only property that contains one of the defined style constants. GDB will use this style when styling this part of the disassembled instruction (see [Disassembler Style Constants], page 522).

`gdb.disassembler.DisassemblerAddressPart` [class]

The `DisassemblerAddressPart` class represents an absolute address within a disassembled instruction. Using a `DisassemblerAddressPart` instead of a `DisassemblerTextPart` with `STYLE_ADDRESS` is preferred, GDB will display the address as both an absolute address, and will look up a suitable symbol to display next to the address. Using `DisassemblerAddressPart` also ensures that user settings such as `set print max-symbolic-offset` are respected.

Here is an example of an x86-64 instruction:

```
call    0x401136 <foo>
```

In this instruction the `0x401136 <foo>` was generated from a single `DisassemblerAddressPart`. The `0x401136` will be styled with `STYLE_ADDRESS`, and `foo` will be styled with `STYLE_SYMBOL`. The `<` and `>` will be styled as `STYLE_TEXT`.

If the inclusion of the symbol name is not required then a `DisassemblerTextPart` with style `STYLE_ADDRESS` can be used instead.

Instances of this class can't be created directly, instead call `DisassembleInfo.address_part` to create a new instance of this class (see [DisassembleInfo Class], page 517).

As well as the properties of its parent class, the `DisassemblerAddressPart` has the following additional property:

`DisassemblerAddressPart.address` [Variable]

A read-only property that contains the `address` passed to this object's `__init__` method.

The following table lists all of the disassembler styles that are available. GDB maps these style constants onto its style settings (see Section 22.5 [Output Styling], page 375). In some cases, several style constants produce the same style settings, and thus will produce the same visual effect on the screen. This could change in future releases of GDB, so care should be taken to select the correct style constant to ensure correct output styling in future releases of GDB.

`gdb.disassembler.STYLE_TEXT`

This is the default style used by GDB when styling disassembler output. This style should be used for any parts of the instruction that don't fit any of the other styles listed below. GDB styles text with this style using its default style.

`gdb.disassembler.STYLE_MNEMONIC`

This style is used for styling the primary instruction mnemonic, which usually appears at, or near, the start of the disassembled instruction string.

GDB styles text with this style using the `disassembler mnemonic` style setting.

`gdb.disassembler.STYLE_SUB_MNEMONIC`

This style is used for styling any sub-mnemonics within a disassembled instruction. A sub-mnemonic is any text within the instruction that controls the

function of the instruction, but which is disjoint from the primary mnemonic (which will have styled `STYLE_MNEMONIC`).

As an example, consider this AArch64 instruction:

```
add w16, w7, w1, lsl #1
```

The `add` is the primary instruction mnemonic, and would be given style `STYLE_MNEMONIC`, while `lsl` is the sub-mnemonic, and would be given the style `STYLE_SUB_MNEMONIC`.

GDB styles text with this style using the `disassembler mnemonic` style setting.

`gdb.disassembler.STYLE_ASSEMBLER_DIRECTIVE`

Sometimes a series of bytes doesn't decode to a valid instruction. In this case the disassembler may choose to represent the result of disassembling using an assembler directive, for example:

```
.word 0x1234
```

In this case, the `.word` would be given the `STYLE_ASSEMBLER_DIRECTIVE` style. An assembler directive is similar to a mnemonic in many ways but is something that is not part of the architecture's instruction set.

GDB styles text with this style using the `disassembler mnemonic` style setting.

`gdb.disassembler.STYLE_REGISTER`

This style is used for styling any text that represents a register name, or register number, within a disassembled instruction.

GDB styles text with this style using the `disassembler register` style setting.

`gdb.disassembler.STYLE_ADDRESS`

This style is used for styling numerical values that represent absolute addresses within the disassembled instruction.

When creating a `DisassemblerTextPart` with this style, you should consider if a `DisassemblerAddressPart` would be more appropriate. See [Disassembler Styling Parts], page 521, for a description of what each part offers.

GDB styles text with this style using the `disassembler address` style setting.

`gdb.disassembler.STYLE_ADDRESS_OFFSET`

This style is used for styling numerical values that represent offsets to addresses within the disassembled instruction. A value is considered an address offset when the instruction itself is going to access memory, and the value is being used to offset which address is accessed.

For example, an architecture might have an instruction that loads from memory using an address within a register. If that instruction also allowed for an immediate offset to be encoded into the instruction, this would be an address offset. Similarly, a branch instruction might jump to an address in a register plus an address offset that is encoded into the instruction.

GDB styles text with this style using the `disassembler immediate` style setting.

`gdb.disassembler.STYLE_IMMEDIATE`

Use `STYLE_IMMEDIATE` for any numerical values within a disassembled instruction when those values are not addresses, address offsets, or register numbers

(The styles `STYLE_ADDRESS`, `STYLE_ADDRESS_OFFSET`, or `STYLE_REGISTER` can be used in those cases).

GDB styles text with this style using the `disassembler immediate` style setting.

`gdb.disassembler.STYLE_SYMBOL`

This style is used for styling the textual name of a symbol that is included within a disassembled instruction. A symbol name is often included next to an absolute address within a disassembled instruction to make it easier for the user to understand what the address is referring too. For example:

```
call 0x401136 <foo>
```

Here `foo` is the name of a symbol, and should be given the `STYLE_SYMBOL` style.

Adding symbols next to absolute addresses like this is handled automatically by the `DisassemblerAddressPart` class (see [Disassembler Styling Parts], page 521).

GDB styles text with this style using the `disassembler symbol` style setting.

`gdb.disassembler.STYLE_COMMENT_START`

This style is used to start a line comment in the disassembly output. Unlike other styles, which only apply to the single `DisassemblerTextPiece` to which they are applied, the comment style is sticky, and overrides the style of any further pieces within this instruction.

This means that, after a `STYLE_COMMENT_START` piece has been seen, GDB will apply the comment style until the end of the line, ignoring the specific style within a piece.

GDB styles text with this style using the `disassembler comment` style setting.

The following functions are also contained in the `gdb.disassembler` module:

`register_disassembler (disassembler, architecture)` [Function]

The *disassembler* must be a sub-class of `gdb.disassembler.Disassembler` or `None`.

The optional *architecture* is either a string, or the value `None`. If it is a string, then it should be the name of an architecture known to GDB, as returned either from `gdb.Architecture.name` (see [gdb.Architecture.name], page 512), or from `gdb.architecture_names` (see [gdb.architecture_names], page 414).

The *disassembler* will be installed for the architecture named by *architecture*, or if *architecture* is `None`, then *disassembler* will be installed as a global disassembler for use by all architectures.

GDB only records a single disassembler for each architecture, and a single global disassembler. Calling `register_disassembler` for an architecture, or for the global disassembler, will replace any existing disassembler registered for that *architecture* value. The previous disassembler is returned.

If *disassembler* is `None` then any disassembler currently registered for *architecture* is deregistered and returned.

When GDB is looking for a disassembler to use, GDB first looks for an architecture specific disassembler. If none has been registered then GDB looks for a global disassembler (one registered with *architecture* set to `None`). Only one disassembler is called

to perform disassembly, so, if there is both an architecture specific disassembler, and a global disassembler registered, it is the architecture specific disassembler that will be used.

GDB tracks the architecture specific, and global disassemblers separately, so it doesn't matter in which order disassemblers are created or registered; an architecture specific disassembler, if present, will always be used in preference to a global disassembler.

You can use the `maint info python-disassemblers` command (see [maint info python-disassemblers], page 778) to see which disassemblers have been registered.

`builtin_disassemble (info)` [Function]

This function calls back into GDB's builtin disassembler to disassemble the instruction identified by *info*, an instance, or sub-class, of `DisassembleInfo`.

When the builtin disassembler needs to read memory the `read_memory` method on *info* will be called. By sub-classing `DisassembleInfo` and overriding the `read_memory` method, it is possible to intercept calls to `read_memory` from the builtin disassembler, and to modify the values returned.

It is important to understand that, even when `DisassembleInfo.read_memory` raises a `gdb.MemoryError`, it is the internal disassembler itself that reports the memory error to GDB. The reason for this is that the disassembler might probe memory to see if a byte is readable or not; if the byte can't be read then the disassembler may choose not to report an error, but instead to disassemble the bytes that it does have available.

If the builtin disassembler is successful then an instance of `DisassemblerResult` is returned from `builtin_disassemble`, alternatively, if something goes wrong, an exception will be raised.

A `MemoryError` will be raised if `builtin_disassemble` is unable to read some memory that is required in order to perform disassembly correctly.

Any exception that is not a `MemoryError`, that is raised in a call to `read_memory`, will pass through `builtin_disassemble`, and be visible to the caller.

Finally, there are a few cases where GDB's builtin disassembler can fail for reasons that are not covered by `MemoryError`. In these cases, a `GdbError` will be raised. The contents of the exception will be a string describing the problem the disassembler encountered.

Here is an example that registers a global disassembler. The new disassembler invokes the builtin disassembler, and then adds a comment, `## Comment`, to each line of disassembly output:

```
class ExampleDisassembler(gdb.disassembler.Disassembler):
    def __init__(self):
        super().__init__("ExampleDisassembler")

    def __call__(self, info):
        result = gdb.disassembler.builtin_disassemble(info)
        length = result.length
        text = result.string + "\t## Comment"
        return gdb.disassembler.DisassemblerResult(length, text)

gdb.disassembler.register_disassembler(ExampleDisassembler())
```

The following example creates a sub-class of `DisassembleInfo` in order to intercept the `read_memory` calls, within `read_memory` any bytes read from memory have the two 4-bit nibbles swapped around. This isn't a very useful adjustment, but serves as an example.

```
class MyInfo(gdb.disassembler.DisassembleInfo):
    def __init__(self, info):
        super().__init__(info)

    def read_memory(self, length, offset):
        buffer = super().read_memory(length, offset)
        result = bytearray()
        for b in buffer:
            v = int.from_bytes(b, 'little')
            v = (v << 4) & 0xf0 | (v >> 4)
            result.append(v)
        return memoryview(result)

class NibbleSwapDisassembler(gdb.disassembler.Disassembler):
    def __init__(self):
        super().__init__("NibbleSwapDisassembler")

    def __call__(self, info):
        info = MyInfo(info)
        return gdb.disassembler.builtin_disassemble(info)

gdb.disassembler.register_disassembler(NibbleSwapDisassembler())
```

23.3.2.43 Missing Debug Info In Python

When GDB encounters a new objfile (see Section 23.3.2.27 [Objfiles In Python], page 487), e.g. the primary executable, or any shared libraries used by the inferior, GDB will attempt to load the corresponding debug information for that objfile. The debug information might be found within the objfile itself, or within a separate objfile which GDB will automatically locate and load.

Sometimes though, GDB might not find any debug information for an objfile, in this case the debugging experience will be restricted.

If GDB fails to locate any debug information for a particular objfile, there is an opportunity for a Python extension to step in. A Python extension can potentially locate the missing debug information using some platform- or project-specific steps, and inform GDB of its location. Or a Python extension might provide some platform- or project-specific advice to the user about how to obtain the missing debug information.

A missing debug information Python extension consists of a handler object which has the `name` and `enabled` attributes, and implements the `__call__` method. When GDB encounters an objfile for which it is unable to find any debug information, it invokes the `__call__` method. Full details of how handlers are written can be found below.

The `gdb.missing_debug` Module

GDB comes with a `gdb.missing_debug` module which contains the following class and global function:

```
gdb.missing_debug.MissingDebugHandler [class]
MissingDebugHandler is a base class from which user-created handlers can derive,
though it is not required that handlers derive from this class, so long as any user
```

created handler has the `name` and `enabled` attributes, and implements the `__call__` method.

MissingDebugHandler.__init__ (*name*) [Function]

The *name* is a string used to reference this missing debug handler within some GDB commands. Valid names consist of the characters `[-_a-zA-Z0-9]`, creating a handler with an invalid name raises a `ValueError` exception.

MissingDebugHandler.__call__ (*objfile*) [Function]

Sub-classes must override the `__call__` method. The *objfile* argument will be a `gdb.Objfile`, this is the objfile for which GDB was unable to find any debug information.

The return value from the `__call__` method indicates what GDB should do next. The possible return values are:

- **None**

This indicates that this handler could not help with *objfile*, GDB should call any other registered handlers.

- **True**

This indicates that this handler has installed the debug information into a location where GDB would normally expect to find it when looking for separate debug information files (see Section 18.3 [Separate Debug Files], page 303). GDB will repeat the normal lookup process, which should now find the separate debug file.

If GDB still doesn't find the separate debug information file after this second attempt, then the Python missing debug information handlers are not invoked a second time, this prevents a badly behaved handler causing GDB to get stuck in a loop. GDB will continue without any debug information for *objfile*.

- **False**

This indicates that this handler has done everything that it intends to do with *objfile*, but no separate debug information can be found. GDB will not call any other registered handlers for *objfile*. GDB will continue without debugging information for *objfile*.

- **A string**

The returned string should contain a filename. GDB will not call any further registered handlers, and will instead load the debug information from the file identified by the returned filename.

Invoking the `__call__` method from this base class will raise a `NotImplementedError` exception.

MissingDebugHandler.name [Variable]

A read-only attribute which is a string, the name of this handler passed to the `__init__` method.

MissingDebugHandler.enabled [Variable]

A modifiable attribute containing a boolean; when **True**, the handler is enabled, and will be used by GDB. When **False**, the handler has been disabled, and will not be used.

`gdb.missing_debug.register_handler` (*locus*, *handler*, *replace=False*) [Function]

Register a new missing debug handler with GDB.

handler is an instance of a sub-class of **MissingDebugHandler**, or at least an instance of an object that has the same attributes and methods as **MissingDebugHandler**.

locus specifies to which handler list to prepend *handler*. It can be either a **`gdb.Progspace`** (see Section 23.3.2.26 [Progspace In Python], page 484) or **None**, in which case the handler is registered globally. The newly registered *handler* will be called before any other handler from the same locus. Two handlers in the same locus cannot have the same name, an attempt to add a handler with an already existing name raises an exception unless *replace* is **True**, in which case the old handler is deleted and the new handler is prepended to the selected handler list.

GDB first calls the handlers for the current program space, and then the globally registered handlers. As soon as a handler returns a value other than **None**, no further handlers are called for this objfile.

23.3.2.44 Missing Objfiles In Python

When GDB opens a core file, for example with the **`core-file`** command (see [core-file command], page 294), GDB will attempt to load the corresponding executable and shared libraries. Often these files can be found on the local machine, but sometimes these files cannot be found, in which case the debugging experience will be restricted.

If GDB fails to locate a particular file then there is an opportunity for a Python extension to step in. A Python extension can potentially locate the missing file using some platform- or project-specific steps, and inform GDB of its location. Or a Python extension might provide some platform- or project-specific advice to the user about how to obtain the missing file.

A missing-objfile Python extension consists of a handler object which has the **`name`** and **`enabled`** attributes, and implements the **`__call__`** method. When GDB encounters a situation where a file cannot be found, but the build-id (see [build ID], page 303) for the missing file is known, then the **`__call__`** method is invoked to try and find the file. Full details of how handlers are written can be found below.

The **`gdb.missing_objfile`** Module

GDB comes with a **`gdb.missing_objfile`** module which contains the following class and global function:

`gdb.missing_objfile.MissingObjfileHandler` [class]

MissingObjfileHandler is a base class from which user-created handlers can derive, though it is not required that handlers derive from this class, so long as any user created handler has the **`name`** and **`enabled`** attributes, and implements the **`__call__`** method.

MissingObjfileHandler.__init__ (*name*) [Function]

The *name* is a string used to reference this missing-objfile handler within some GDB commands. Valid names consist of the characters ‘[-_a-zA-Z0-9]’, creating a handler with an invalid name raises a `ValueError` exception.

MissingObjfileHandler.__call__ (*pspace*, *build_id*, *filename*) [Function]

Sub-classes must override the `__call__` method. The *pspace* argument will be a `gdb.Progspace` (see Section 23.3.2.26 [Progspace In Python], page 484), this is the program space in which GDB is looking for the missing file.

The *build_id* argument is a string containing the build-id of the file that is missing, this will be in the same format as returned by `Objfile.build_id` (see [Objfile.build_id], page 488).

The *filename* argument contains the name of the file that GDB is looking for. This information is provided to allow handlers to generate informative messages for the user. A handler is not required to place the missing file at this location. There might already be a file present at this location, but it might not match the required build-id, in which case GDB will have ignored it. In some limited cases GDB might not be able to establish the *filename* of the file it is searching for, in this case GDB will use a string ‘with build-id *build_id*’ as a replacement.

The return value from the `__call__` method indicates what GDB should do next. The possible return values are:

- **None**

This indicates that this handler could not locate the missing file and GDB should call any other registered handlers.

- **True**

This indicates that this handler has installed the missing file into a location where GDB would normally expect to find it. The only location in which GDB will look is within the `.build-id` sub-directory within the *debug-file-directory* (see [debug-file-directory], page 304).

GDB will repeat the normal lookup process, which should now find the previously missing file.

If GDB still doesn’t find file after this second attempt, then the Python missing-objfile handlers are not invoked a second time, this prevents a badly behaved handler causing GDB to get stuck in a loop. GDB will continue without the missing file, though this will degrade the debugging experience.

- **False**

This indicates that this handler has done everything that it intends to do but the missing file could not be found. GDB will not call any other registered handlers to look for the missing file. GDB will continue without the missing file, though this will degrade the debugging experience.

- **A string**

The returned string should contain a filename. GDB will not call any further registered handlers, and will instead use the returned filename as the missing file.

Invoking the `__call__` method from this base class will raise a `NotImplementedError` exception.

MissingObjfileHandler.name [Variable]

A read-only attribute which is a string, the name of this handler passed to the `__init__` method.

MissingObjfileHandler.enabled [Variable]

A modifiable attribute containing a boolean; when `True`, the handler is enabled, and will be used by GDB. When `False`, the handler has been disabled, and will not be used.

`gdb.missing_objfile.register_handler (locus, handler, replace=False)` [Function]

Register a new missing-objfile handler with GDB.

handler is an instance of a sub-class of `MissingObjfileHandler`, or at least an instance of an object that has the same attributes and methods as `MissingObjfileHandler`.

locus specifies to which handler list to prepend *handler*. It can be either a `gdb.Progspace` (see Section 23.3.2.26 [Progspace In Python], page 484) or `None`, in which case the handler is registered globally. The newly registered *handler* will be called before any other handler from the same locus. Two handlers in the same locus cannot have the same name, an attempt to add a handler with an already existing name raises an exception unless *replace* is `True`, in which case the old handler is deleted and the new handler is prepended to the selected handler list.

GDB first calls the handlers for the current program space, and then the globally registered handlers. As soon as a handler returns a value other than `None`, no further handlers are called.

Managing Missing-Objfile Handlers

GDB defines the following commands to manage registered missing-objfile handlers:

`info missing-objfile-handlers [ locus [ name-regexp ] ]`

Lists all registered missing-objfile handlers. Arguments *locus* and *name-regexp* are both optional and can be used to filter which handlers are listed.

The *locus* argument should be either *global*, *progspace*, or the name of an object file. Only handlers registered for the specified locus will be listed.

The *name-regexp* is a regular expression used to match against handler names.

`disable missing-objfile-handler [ locus [ name-regexp ] ]`

The *locus* and *name-regexp* are interpreted as in *info missing-objfile-handlers* above, but instead of listing the matching handlers, all of the matching handlers are disabled. The `enabled` field of each matching handler is set to `False`.

`enable missing-objfile-handler [ locus [ name-regexp ] ]`

The *locus* and *name-regexp* are interpreted as in *info missing-objfile-handlers* above, but instead of listing the matching handlers, all of the

matching handlers are enabled. The `enabled` field of each matching handler is set to `True`.

23.3.2.45 Core Files In Python

When a core file is loaded into an inferior (see Section 23.3.2.17 [Inferiors In Python], page 455) for examination (see [core-file command], page 294), information about the core file is contained in a `gdb.Corefile` object.

The `gdb.Corefile` for an inferior can be accessed using the `Inferior.corefile` attribute. This will be `None` if no core file is loaded.

A `gdb.Corefile` object has the following attributes:

Corefile.filename [Variable]
This read only attribute contains a non-empty string, the file name of the core file. Attempting to access this attribute on an invalid `gdb.Corefile` object will raise a `RuntimeError` exception.

A `gdb.Corefile` object has the following methods:

Corefile.is_valid () [Function]
Returns `True` if the `gdb.Corefile` object is valid, `False` if not. A `gdb.Corefile` object will become invalid when the core file is unloaded from the inferior using the `core-file` command (see [core-file command], page 294), or if the inferior in which the core file is loaded is deleted. All other `gdb.Corefile` methods and attributes will throw an exception if it is invalid at the time the method is called, or the attribute accessed.

Corefile.mapped_files () [Function]
Return a list of `gdb.CorefileMappedFile` (see below) objects representing files that were mapped into the process when the core file was created. This information is read from the 'NT_FILE' core file note on Linux. Not every target supports accessing this information, for targets without support, an empty list will be returned.

One may add arbitrary attributes to `gdb.Corefile` objects in the usual Python way. This is useful if, for example, one needs to do some extra record keeping associated with the corefile. See [choosing attribute names], page 457, for guidance on selecting a suitable name for new attributes.

The `Corefile.mapped_files ()` method returns a list of `gdb.CorefileMappedFile` objects. Each of these objects represents a file that was fully, or partially, mapped into the processes address space when the core file was created.

A `gdb.CorefileMappedFile` object has the following attributes:

CorefileMappedFile.filename [Variable]
This read only attribute contains a string, the file name of the mapped file. This string can be empty if GDB was unable to find the name of the mapped file. If this string is empty then `CorefileMappedFile.build_id` will not be `None`.

CorefileMappedFile.build_id [Variable]
This read only attribute contains a non-empty string or `None`. This is the build-id of the mapped file extracted from the core file, or `None` if there was no build-id, or GDB was unable to extract the build-id.

CorefileMappedFile.is_main_executable [Variable]

This read only attribute is **True** if GDB believes this mapping represents the main executable for which this core file was created. This will be **False** for all other mappings.

CorefileMappedFile.regions [Variable]

This read only attribute contains a list of `gdb.CorefileMappedFileRegion` objects. Each of these objects describes a region of the file that was mapped into the process when the core file was created, further details are given below.

The `gdb.CorefileMappedFileRegion` object describes which part of a file that was mapped into a process when the core file was created.

A `gdb.CorefileMappedFile` object has the following attributes:

CorefileMappedFileRegion.start [Variable]

This read only attribute contains the start address of this mapping within the inferior.

CorefileMappedFileRegion.end [Variable]

This read only attribute contains end address of this mapping within the inferior.

CorefileMappedFileRegion.file_offset [Variable]

This read only attribute contains the offset within the mapped file for this mapping. This attribute will be 0 if the containing `CorefileMappedFile.filename` is empty, in which case this attribute has no meaning.

23.3.3 Python Auto-loading

When a new object file is read (for example, due to the `file` command, or because the inferior has loaded a shared library), GDB will look for Python support scripts in several ways: `objfile-gdb.py` and `.debug_gdb_scripts` section. See Section 23.5 [Auto-loading extensions], page 592.

The auto-loading feature is useful for supplying application-specific debugging commands and scripts.

Auto-loading can be enabled or disabled, and the list of auto-loaded scripts can be printed.

set auto-load python-scripts [on|off]

Enable or disable the auto-loading of Python scripts.

show auto-load python-scripts

Show whether auto-loading of Python scripts is enabled or disabled.

info auto-load python-scripts [regexp]

Print the list of all Python scripts that GDB auto-loaded.

Also printed is the list of Python scripts that were mentioned in the `.debug_gdb_scripts` section and were either not found (see Section 23.5.2 [dotdebug_gdb_scripts section], page 593) or were not auto-loaded due to `auto-load safe-path` rejection (see Section 22.8 [Auto-loading], page 381). This is useful because their names are not printed when GDB tries to load them and fails. There may be many of them, and printing an error message for each one is problematic.

If *regex* is supplied only Python scripts with matching names are printed.

Example:

```
(gdb) info auto-load python-scripts
Loaded Script
Yes    py-section-script.py
      full name: /tmp/py-section-script.py
No     my-foo-pretty-printers.py
```

When reading an auto-loaded file or script, GDB sets the *current objfile*. This is available via the `gdb.current_objfile` function (see Section 23.3.2.27 [Objfiles In Python], page 487). This can be useful for registering objfile-specific pretty-printers and frame-filters.

23.3.4 Python modules

GDB comes with several modules to assist writing Python code.

23.3.4.1 `gdb.printing`

This module provides a collection of utilities for working with pretty-printers.

PrettyPrinter (*name*, *subprinters*=None)

This class specifies the API that makes ‘`info pretty-printer`’, ‘`enable pretty-printer`’ and ‘`disable pretty-printer`’ work. Pretty-printers should generally inherit from this class.

SubPrettyPrinter (*name*)

For printers that handle multiple types, this class specifies the corresponding API for the subprinters.

RegexCollectionPrettyPrinter (*name*)

Utility class for handling multiple printers, all recognized via regular expressions. See Section 23.3.2.8 [Writing a Pretty-Printer], page 433, for an example.

FlagEnumerationPrinter (*name*)

A pretty-printer which handles printing of `enum` values. Unlike GDB’s built-in `enum` printing, this printer attempts to work properly when there is some overlap between the enumeration constants. The argument *name* is the name of the printer and also the name of the `enum` type to look up.

register_pretty_printer (*obj*, *printer*, *replace*=False)

Register *printer* with the pretty-printer list of *obj*. If *replace* is `True` then any existing copy of the printer is replaced. Otherwise a `RuntimeError` exception is raised if a printer with the same name already exists.

23.3.4.2 `gdb.types`

This module provides a collection of utilities for working with `gdb.Type` objects.

get_basic_type (*type*)

Return *type* with `const` and `volatile` qualifiers stripped, and with `typedefs` and `C++` references converted to the underlying type.

C++ example:

```
typedef const int const_int;
const_int foo (3);
```

```
const_int& foo_ref (foo);
int main () { return 0; }
```

Then in gdb:

```
(gdb) start
(gdb) python import gdb.types
(gdb) python foo_ref = gdb.parse_and_eval("foo_ref")
(gdb) python print(gdb.types.get_basic_type(foo_ref.type))
int
```

`has_field (type, field)`

Return True if *type*, assumed to be a type with fields (e.g., a structure or union), has field *field*.

`make_enum_dict (enum_type)`

Return a Python dictionary type produced from *enum_type*.

`deep_items (type)`

Returns a Python iterator similar to the standard `gdb.Type.iteritems` method, except that the iterator returned by `deep_items` will recursively traverse anonymous struct or union fields. For example:

```
struct A
{
    int a;
    union {
        int b0;
        int b1;
    };
};
```

Then in GDB:

```
(gdb) python import gdb.types
(gdb) python struct_a = gdb.lookup_type("struct A")
(gdb) python print(struct_a.keys ())
{'a', ''}
(gdb) python print([k for k,v in gdb.types.deep_items(struct_a)])
{'a', 'b0', 'b1'}
```

`get_type_recognizers ()`

Return a list of the enabled type recognizers for the current context. This is called by GDB during the type-printing process (see Section 23.3.2.9 [Type Printing API], page 435).

`apply_type_recognizers (recognizers, type_obj)`

Apply the type recognizers, *recognizers*, to the type object *type_obj*. If any recognizer returns a string, return that string. Otherwise, return `None`. This is called by GDB during the type-printing process (see Section 23.3.2.9 [Type Printing API], page 435).

`register_type_printer (locus, printer)`

This is a convenience function to register a type printer *printer*. The printer must implement the type printer protocol. The *locus* argument is either a `gdb.Objfile`, in which case the printer is registered with that objfile; a `gdb.Progspace`, in which case the printer is registered with that progspace; or `None`, in which case the printer is registered globally.

TypePrinter

This is a base class that implements the type printer protocol. Type printers are encouraged, but not required, to derive from this class. It defines a constructor:

```
__init__ (self, name) [Method on TypePrinter]
```

Initialize the type printer with the given name. The new printer starts in the enabled state.

23.3.4.3 gdb.prompt

This module provides a method for prompt value-substitution.

substitute_prompt (*string*)

Return *string* with escape sequences substituted by values. Some escape sequences take arguments. You can specify arguments inside “{ }” immediately following the escape sequence.

The escape sequences you can pass to this function are:

<code>\\</code>	Substitute a backslash.
<code>\e</code>	Substitute an ESC character.
<code>\f</code>	Substitute the selected frame; an argument names a frame parameter.
<code>\n</code>	Substitute a newline.
<code>\p</code>	Substitute a parameter’s value; the argument names the parameter.
<code>\r</code>	Substitute a carriage return.
<code>\t</code>	Substitute the selected thread; an argument names a thread parameter.
<code>\v</code>	Substitute the version of GDB.
<code>\w</code>	Substitute the current working directory.
<code>\[</code>	Begin a sequence of non-printing characters. These sequences are typically used with the ESC character, and are not counted in the string length. Example: “ <code>\[0;34m\](gdb)\[0m\]</code> ” will return a blue-colored “(gdb)” prompt where the length is five.
<code>\]</code>	End a sequence of non-printing characters.

For example:

```
substitute_prompt ("frame: \f, args: \p{print frame-arguments}")
```

will return the string:

```
"frame: main, args: scalars"
```

23.3.4.4 gdb.ptwrite

This module provides additional functionality for recording programs that make use of the PTWRITE instruction. PTWRITE is a x86 instruction that allows to write values into the Intel Processor Trace (see Chapter 7 [Process Record and Replay], page 105). The GCC intrinsics for it are:

```
void _ptwrite32 (unsigned int a)
void _ptwrite64 (unsigned __int64 a)
```

If an inferior uses the instruction, GDB by default inserts the raw payload value as auxiliary information into the execution history. Auxiliary information is by default printed during `record instruction-history`, `record function-call-history`, and all stepping commands, and is accessible in Python as a `RecordAuxiliary` object (see Section 23.3.2.20 [Recordings In Python], page 467).

Sample program:

```
#include <immintrin.h>

void
ptwrite64 (unsigned long long value)
{
    _ptwrite64 (value);
}

int
main (void)
{
    ptwrite64 (0x42);
    return 0; /* break here. */
}
```

GDB output after recording the sample program in pt format:

```
(gdb) record instruction-history 12,14
12      0x0040074c <ptwrite64+16>:  ptwrite %rbx
13      [0x42]
14      0x00400751 <ptwrite64+21>:  mov -0x8(%rbp),%rbx
(gdb) record function-call-history
1      main
2      ptwrite64
      [0x42]
3      main
```

The `gdb.ptwrite` module allows customizing the default output of PTWRITE auxiliary information. A custom Python function can be registered as the PTWRITE filter function. This function will be called with the PTWRITE payload and PC as arguments during trace decoding. The function can return a string, which will be printed by GDB during the aforementioned commands, or `None`, resulting in no output. To register such a filter function, the user needs to provide a filter factory function, which returns a new filter function object to be called by GDB.

`register_filter_factory` (*filter_factory*) [Function]

Used to register the PTWRITE filter factory. This filter factory can be any callable object that accepts one argument, the current thread as a `gdb.InferiorThread`. It can return `None` or a callable. This callable is the PTWRITE filter function for the specified thread. If `None` is returned by the factory function, the default auxiliary information will be printed.

`get_filter` () [Function]

Return the currently active PTWRITE filter function.

An example:

```
(gdb) python-interactive
>>> class my_filter():
...     def __init__(self):
...         self.var = 0
...     def __call__(self, payload, ip):
...         self.var += 1
...         return f"counter: {self.var}, ip: {ip:#x}"
...
>>> def my_filter_factory(thread):
...     if thread.global_num == 1:
...         return my_filter()
...     else:
...         return None
...
>>> import gdb.ptwrite
>>> gdb.ptwrite.register_filter_factory(my_filter_factory)
>>>

(gdb) record function-call-history 59,64
59     pthread_create@GLIBC_2.2.5
60     job()
61     task(void*)
62     ptwrite64(unsigned long)
        [counter: 1, ip: 0x401156]
63     task(void*)
64     ptwrite32(unsigned int)
        [counter: 2, ip: 0x40116c]

(gdb) info threads
* 1   Thread 0x7fffffd8740 (LWP 25796) "ptw_threads" task ()
    at bin/ptwrite/ptw_threads.c:45
  2   Thread 0x7ffff6eb8700 (LWP 25797) "ptw_threads" task ()
    at bin/ptwrite/ptw_threads.c:45

(gdb) thread 2
[Switching to thread 2 (Thread 0x7ffff6eb8700 (LWP 25797))]
#0  task (arg=0x0) at ptwrite_threads.c:45
45     return NULL;
```

```

(gdb) record function-call-history 10,14
10  start_thread
11  task(void*)
12  ptwrite64(unsigned long)
    [0x42]
13  task(void*)
14  ptwrite32(unsigned int)
    [0x43]

```

This GDB feature is dependent on hardware and operating system support and requires the Intel Processor Trace decoder library in version 2.0.0 or newer.

23.4 Extending GDB using Guile

You can extend GDB using the Guile implementation of the Scheme programming language (<http://www.gnu.org/software/guile/>). This feature is available only if GDB was configured using `--with-guile`.

23.4.1 Guile Introduction

Guile is an implementation of the Scheme programming language and is the GNU project's official extension language.

Guile support in GDB follows the Python support in GDB reasonably closely, so concepts there should carry over. However, some things are done differently where it makes sense.

GDB requires Guile version 3.0, 2.2.

Guile scripts used by GDB should be installed in *data-directory/guile*, where *data-directory* is the data directory as determined at GDB startup (see Section 18.8 [Data Files], page 312). This directory, known as the *guile directory*, is automatically added to the Guile Search Path in order to allow the Guile interpreter to locate all scripts installed at this location.

23.4.2 Guile Commands

GDB provides two commands for accessing the Guile interpreter:

guile-repl

gr The **guile-repl** command can be used to start an interactive Guile prompt or *repl*. To return to GDB, type **,q** or the EOF character (e.g., **Ctrl-D** on an empty prompt). These commands do not take any arguments.

guile [*scheme-expression*]

gu [*scheme-expression*]

The **guile** command can be used to evaluate a Scheme expression.

If given an argument, GDB will pass the argument to the Guile interpreter for evaluation.

```

(gdb) guile (display (+ 20 3)) (newline)
23

```

The result of the Scheme expression is displayed using normal Guile rules.

```

(gdb) guile (+ 20 3)
23

```

If you do not provide an argument to **guile**, it will act as a multi-line command, like **define**. In this case, the Guile script is made up of subsequent command

lines, given after the `guile` command. This command list is terminated using a line containing `end`. For example:

```
(gdb) guile
>(display 23)
>(newline)
>end
23
```

It is also possible to execute a Guile script from the GDB interpreter:

source script-name

The script name must end with ‘.scm’ and GDB must be configured to recognize the script language based on filename extension using the `script-extension` setting. See Chapter 23 [Extending GDB], page 397.

guile (load "script-name")

This method uses the `load` Guile function. It takes a string argument that is the name of the script to load. See the Guile documentation for a description of this function. (see Section “Loading” in *GNU Guile Reference Manual*).

23.4.3 Guile API

You can get quick online help for GDB’s Guile API by issuing the command `help guile`, or by issuing the command `,help` from an interactive Guile session. Furthermore, most Guile procedures provided by GDB have doc strings which can be obtained with `,describe procedure-name` or `,d procedure-name` from the Guile interactive prompt.

23.4.3.1 Basic Guile

At startup, GDB overrides Guile’s `current-output-port` and `current-error-port` to print using GDB’s output-paging streams. A Guile program which outputs to one of these streams may have its output interrupted by the user (see Section 22.4 [Screen Size], page 374). In this situation, a Guile `signal` exception is thrown with value `SIGINT`.

Guile’s history mechanism uses the same naming as GDB’s, namely the use of dollar-variables (e.g., `$1`, `$2`, etc.). The results of evaluations in Guile and in GDB are counted separately, `$1` in Guile is not the same value as `$1` in GDB.

GDB is not thread-safe. If your Guile program uses multiple threads, you must be careful to only call GDB-specific functions in the GDB thread.

Some care must be taken when writing Guile code to run in GDB. Two things are worth noting in particular:

- GDB installs handlers for `SIGCHLD` and `SIGINT`. Guile code must not override these, or even change the options using `sigaction`. If your program changes the handling of these signals, GDB will most likely stop working correctly. Note that it is unfortunately common for GUI toolkits to install a `SIGCHLD` handler.
- GDB takes care to mark its internal file descriptors as close-on-exec. However, this cannot be done in a thread-safe way on all platforms. Your Guile programs should be aware of this and should both create new file descriptors with the close-on-exec flag set and arrange to close unneeded file descriptors before starting a child process.

GDB introduces a new Guile module, named `gdb`. All methods and classes added by GDB are placed in this module. GDB does not automatically `import` the `gdb` module, scripts

must do this themselves. There are various options for how to import a module, so GDB leaves the choice of how the `gdb` module is imported to the user. To simplify interactive use, it is recommended to add one of the following to your `~/.gdbinit`.

```
guile (use-modules (gdb))
guile (use-modules ((gdb) #:renamer (symbol-prefix-proc 'gdb:)))
```

Which one to choose depends on your preference. The second one adds `gdb:` as a prefix to all module functions and variables.

The rest of this manual assumes the `gdb` module has been imported without any prefix. See the Guile documentation for `use-modules` for more information (see Section “Using Guile Modules” in *GNU Guile Reference Manual*).

Example:

```
(gdb) guile (value-type (make-value 1))
ERROR: Unbound variable: value-type
Error while executing Scheme code.
(gdb) guile (use-modules (gdb))
(gdb) guile (value-type (make-value 1))
int
(gdb)
```

The `(gdb)` module provides these basic Guile functions.

execute *command* [*#:from-tty* *boolean*] [Scheme Procedure]
[*#:to-string* *boolean*]

Evaluate *command*, a string, as a GDB CLI command. If a GDB exception happens while *command* runs, it is translated as described in Section 23.4.3.4 [Guile Exception Handling], page 543.

from-tty specifies whether GDB ought to consider this command as having originated from the user invoking it interactively. It must be a boolean value. If omitted, it defaults to `#f`.

By default, any output produced by *command* is sent to GDB’s standard output (and to the log output if logging is turned on). If the *to-string* parameter is `#t`, then output will be collected by **execute** and returned as a string. The default is `#f`, in which case the return value is unspecified. If *to-string* is `#t`, the GDB virtual terminal will be temporarily set to unlimited width and height, and its pagination will be disabled; see Section 22.4 [Screen Size], page 374.

history-ref *number* [Scheme Procedure]

Return a value from GDB’s value history (see Section 10.11 [Value History], page 172). The *number* argument indicates which history element to return. If *number* is negative, then GDB will take its absolute value and count backward from the last element (i.e., the most recent element) to find the value to return. If *number* is zero, then GDB will return the most recent element. If the element specified by *number* doesn’t exist in the value history, a `gdb:error` exception will be raised.

If no exception is raised, the return value is always an instance of `<gdb:value>` (see Section 23.4.3.5 [Values From Inferior In Guile], page 545).

Note: GDB’s value history is independent of Guile’s. `$1` in GDB’s value history contains the result of evaluating an expression from GDB’s command line and `$1` from Guile’s history contains the result of evaluating an expression from Guile’s command line.

history-append! *value* [Scheme Procedure]
 Append *value*, an instance of `<gdb:value>`, to GDB's value history. Return its index in the history.

Putting into history values returned by Guile extensions will allow the user convenient access to those values via CLI history facilities.

parse-and-eval *expression* [Scheme Procedure]
 Parse *expression* as an expression in the current language, evaluate it, and return the result as a `<gdb:value>`. The *expression* must be a string.

This function can be useful when implementing a new command (see Section 23.4.3.11 [Commands In Guile], page 561), as it provides a way to parse the command's arguments as an expression. It is also useful when computing values. For example, it is the only way to get the value of a convenience variable (see Section 10.12 [Convenience Vars], page 173) as a `<gdb:value>`.

23.4.3.2 Guile Configuration

GDB provides these Scheme functions to access various configuration parameters.

data-directory [Scheme Procedure]
 Return a string containing GDB's data directory. This directory contains GDB's ancillary files.

guile-data-directory [Scheme Procedure]
 Return a string containing GDB's Guile data directory. This directory contains the Guile modules provided by GDB.

gdb-version [Scheme Procedure]
 Return a string containing the GDB version.

host-config [Scheme Procedure]
 Return a string containing the host configuration. This is the string passed to `--host` when GDB was configured.

target-config [Scheme Procedure]
 Return a string containing the target configuration. This is the string passed to `--target` when GDB was configured.

23.4.3.3 GDB Scheme Data Types

The values exposed by GDB to Guile are known as GDB *objects*. There are several kinds of GDB object, and each is disjoint from all other types known to Guile.

gdb-object-kind *object* [Scheme Procedure]
 Return the kind of the GDB object, e.g., `<gdb:breakpoint>`, as a symbol.

GDB defines the following object types:

`<gdb:arch>`
 See Section 23.4.3.21 [Architectures In Guile], page 583.

- `<gdb:block>`
See Section 23.4.3.16 [Blocks In Guile], page 572.
- `<gdb:block-symbols-iterator>`
See Section 23.4.3.16 [Blocks In Guile], page 572.
- `<gdb:breakpoint>`
See Section 23.4.3.19 [Breakpoints In Guile], page 578.
- `<gdb:command>`
See Section 23.4.3.11 [Commands In Guile], page 561.
- `<gdb:exception>`
See Section 23.4.3.4 [Guile Exception Handling], page 543.
- `<gdb:frame>`
See Section 23.4.3.15 [Frames In Guile], page 569.
- `<gdb:iterator>`
See Section 23.4.3.25 [Iterators In Guile], page 587.
- `<gdb:lazy-string>`
See Section 23.4.3.20 [Lazy Strings In Guile], page 582.
- `<gdb:objfile>`
See Section 23.4.3.14 [Objfiles In Guile], page 568.
- `<gdb:parameter>`
See Section 23.4.3.12 [Parameters In Guile], page 565.
- `<gdb:pretty-printer>`
See Section 23.4.3.8 [Guile Pretty Printing API], page 556.
- `<gdb:pretty-printer-worker>`
See Section 23.4.3.8 [Guile Pretty Printing API], page 556.
- `<gdb:progspace>`
See Section 23.4.3.13 [Progspace In Guile], page 567.
- `<gdb:symbol>`
See Section 23.4.3.17 [Symbols In Guile], page 574.
- `<gdb:symtab>`
See Section 23.4.3.18 [Symbol Tables In Guile], page 577.
- `<gdb:sal>`
See Section 23.4.3.18 [Symbol Tables In Guile], page 577.
- `<gdb:type>`
See Section 23.4.3.7 [Types In Guile], page 551.
- `<gdb:field>`
See Section 23.4.3.7 [Types In Guile], page 551.
- `<gdb:value>`
See Section 23.4.3.5 [Values From Inferior In Guile], page 545.

<gdb:color>

See Section 23.4.3.26 [Colors In Guile], page 589.

The following GDB objects are managed internally so that the Scheme function `eq?` may be applied to them.

<gdb:arch>
 <gdb:block>
 <gdb:breakpoint>
 <gdb:frame>
 <gdb:objfile>
 <gdb:progspace>
 <gdb:symbol>
 <gdb:symtab>
 <gdb:type>

23.4.3.4 Guile Exception Handling

When executing the `guile` command, Guile exceptions uncaught within the Guile code are translated to calls to the GDB error-reporting mechanism. If the command that called `guile` does not handle the error, GDB will terminate it and report the error according to the setting of the `guile print-stack` parameter.

The `guile print-stack` parameter has three settings:

none Nothing is printed.

message An error message is printed containing the Guile exception name, the associated value, and the Guile call stack backtrace at the point where the exception was raised. Example:

```
(gdb) guile (display foo)
ERROR: In procedure memoize-variable-access!:
ERROR: Unbound variable: foo
Error while executing Scheme code.
```

full In addition to an error message a full backtrace is printed.

```
(gdb) set guile print-stack full
(gdb) guile (display foo)
Guile Backtrace:
In ice-9/boot-9.scm:
 157: 10 [catch #t #<catch-closure 2c76e20> ...]
In unknown file:
   ?: 9 [apply-smob/1 #<catch-closure 2c76e20>]
In ice-9/boot-9.scm:
 157: 8 [catch #t #<catch-closure 2c76d20> ...]
In unknown file:
   ?: 7 [apply-smob/1 #<catch-closure 2c76d20>]
   ?: 6 [call-with-input-string "(display foo)" ...]
In ice-9/boot-9.scm:
2320: 5 [save-module-excursion #<procedure 2c2dc30 ... ()>]
In ice-9/eval-string.scm:
  44: 4 [read-and-eval #<input: string 27cb410> #:lang ...]
  37: 3 [lp (display foo)]
In ice-9/eval.scm:
 387: 2 [eval # ()]
 393: 1 [eval #<memoized foo> ()]
```

```
In unknown file:
?: 0 [memoize-variable-access! #<memoized foo> ...]

ERROR: In procedure memoize-variable-access!:
ERROR: Unbound variable: foo
Error while executing Scheme code.
```

GDB errors that happen in GDB commands invoked by Guile code are converted to Guile exceptions. The type of the Guile exception depends on the error.

Guile procedures provided by GDB can throw the standard Guile exceptions like `wrong-type-arg` and `out-of-range`.

User interrupt (via `C-c` or by typing `q` at a pagination prompt) is translated to a Guile `signal` exception with value `SIGINT`.

GDB Guile procedures can also throw these exceptions:

`gdb:error`

This exception is a catch-all for errors generated from within GDB.

`gdb:invalid-object`

This exception is thrown when accessing Guile objects that wrap underlying GDB objects have become invalid. For example, a `<gdb:breakpoint>` object becomes invalid if the user deletes it from the command line. The object still exists in Guile, but the object it represents is gone. Further operations on this breakpoint will throw this exception.

`gdb:memory-error`

This exception is thrown when an operation tried to access invalid memory in the inferior.

`gdb:pp-type-error`

This exception is thrown when a Guile pretty-printer passes a bad object to GDB.

The following exception-related procedures are provided by the `(gdb)` module.

`make-exception` *key args* [Scheme Procedure]

Return a `<gdb:exception>` object given by its *key* and *args*, which are the standard Guile parameters of an exception. See the Guile documentation for more information (see Section “Exceptions” in *GNU Guile Reference Manual*).

`exception?` *object* [Scheme Procedure]

Return `#t` if *object* is a `<gdb:exception>` object. Otherwise return `#f`.

`exception-key` *exception* [Scheme Procedure]

Return the *args* field of a `<gdb:exception>` object.

`exception-args` *exception* [Scheme Procedure]

Return the *args* field of a `<gdb:exception>` object.

23.4.3.5 Values From Inferior In Guile

GDB provides values it obtains from the inferior program in an object of type `<gdb:value>`. GDB uses this object for its internal bookkeeping of the inferior's values, and for fetching values when necessary.

GDB does not memoize `<gdb:value>` objects. `make-value` always returns a fresh object.

```
(gdb) guile (eq? (make-value 1) (make-value 1))
$1 = #f
(gdb) guile (equal? (make-value 1) (make-value 1))
$1 = #t
```

A `<gdb:value>` that represents a function can be executed via inferior function call with `value-call`. Any arguments provided to the call must match the function's prototype, and must be provided in the order specified by that prototype.

For example, `some-val` is a `<gdb:value>` instance representing a function that takes two integers as arguments. To execute this function, call it like so:

```
(define result (value-call some-val 10 20))
```

Any values returned from a function call are `<gdb:value>` objects.

Note: Unlike Python scripting in GDB, inferior values that are simple scalars cannot be used directly in Scheme expressions that are valid for the value's data type. For example, `(+ (parse-and-eval "int_variable") 2)` does not work. And inferior values that are structures or instances of some class cannot be accessed using any special syntax, instead `value-field` must be used.

The following value-related procedures are provided by the `(gdb)` module.

value? *object* [Scheme Procedure]

Return `#t` if *object* is a `<gdb:value>` object. Otherwise return `#f`.

make-value *value* [*#:type type*] [Scheme Procedure]

Many Scheme values can be converted directly to a `<gdb:value>` with this procedure. If *type* is specified, the result is a value of this type, and if *value* can't be represented with this type an exception is thrown. Otherwise the type of the result is determined from *value* as described below.

See Section 23.4.3.21 [Architectures In Guile], page 583, for a list of the builtin types for an architecture.

Here's how Scheme values are converted when *type* argument to `make-value` is not specified:

Scheme boolean

A Scheme boolean is converted the boolean type for the current language.

Scheme integer

A Scheme integer is converted to the first of a C `int`, `unsigned int`, `long`, `unsigned long`, `long long` or `unsigned long long` type for the current architecture that can represent the value.

If the Scheme integer cannot be represented as a target integer an `out-of-range` exception is thrown.

Scheme real

A Scheme real is converted to the C `double` type for the current architecture.

Scheme string

A Scheme string is converted to a string in the current target language using the current target encoding. Characters that cannot be represented in the current target encoding are replaced with the corresponding escape sequence. This is Guile's `SCM_FAILED_CONVERSION_ESCAPE_SEQUENCE` conversion strategy (see Section “Strings” in *GNU Guile Reference Manual*). Passing *type* is not supported in this case, if it is provided a `wrong-type-arg` exception is thrown.

<gdb:lazy-string>

If *value* is a `<gdb:lazy-string>` object (see Section 23.4.3.20 [Lazy Strings In Guile], page 582), then the `lazy-string->value` procedure is called, and its result is used.

Passing *type* is not supported in this case, if it is provided a `wrong-type-arg` exception is thrown.

Scheme bytevector

If *value* is a Scheme bytevector and *type* is provided, *value* must be the same size, in bytes, of values of type *type*, and the result is essentially created by using `memcpy`.

If *value* is a Scheme bytevector and *type* is not provided, the result is an array of type `uint8` of the same length.

value-optimized-out? *value* [Scheme Procedure]

Return `#t` if the compiler optimized out *value*, thus it is not available for fetching from the inferior. Otherwise return `#f`.

value-address *value* [Scheme Procedure]

If *value* is addressable, returns a `<gdb:value>` object representing the address. Otherwise, `#f` is returned.

value-type *value* [Scheme Procedure]

Return the type of *value* as a `<gdb:type>` object (see Section 23.4.3.7 [Types In Guile], page 551).

value-dynamic-type *value* [Scheme Procedure]

Return the dynamic type of *value*. This uses C++ run-time type information (RTTI) to determine the dynamic type of the value. If the value is of class type, it will return the class in which the value is embedded, if any. If the value is of pointer or reference to a class type, it will compute the dynamic type of the referenced object, and return a pointer or reference to that type, respectively. In all other cases, it will return the value's static type.

Note that this feature will only work when debugging a C++ program that includes RTTI for the object in question. Otherwise, it will just return the static type of the value as in `ptype foo`. See Chapter 16 [Symbols], page 265.

value-cast *value type* [Scheme Procedure]

Return a new instance of `<gdb:value>` that is the result of casting *value* to the type described by *type*, which must be a `<gdb:type>` object. If the cast cannot be performed for some reason, this method throws an exception.

value-dynamic-cast *value type* [Scheme Procedure]

Like **value-cast**, but works as if the C++ `dynamic_cast` operator were used. Consult a C++ reference for details.

value-reinterpret-cast *value type* [Scheme Procedure]

Like **value-cast**, but works as if the C++ `reinterpret_cast` operator were used. Consult a C++ reference for details.

value-dereference *value* [Scheme Procedure]

For pointer data types, this method returns a new `<gdb:value>` object whose contents is the object pointed to by *value*. For example, if `foo` is a C pointer to an `int`, declared in your C program as

```
int *foo;
```

then you can use the corresponding `<gdb:value>` to access what `foo` points to like this:

```
(define bar (value-dereference foo))
```

The result `bar` will be a `<gdb:value>` object holding the value pointed to by `foo`.

A similar function **value-referenced-value** exists which also returns `<gdb:value>` objects corresponding to the values pointed to by pointer values (and additionally, values referenced by reference values). However, the behavior of **value-dereference** differs from **value-referenced-value** by the fact that the behavior of **value-dereference** is identical to applying the C unary operator `*` on a given value. For example, consider a reference to a pointer `ptrref`, declared in your C++ program as

```
typedef int *intptr;
...
int val = 10;
intptr ptr = &val;
intptr &ptrref = ptr;
```

Though `ptrref` is a reference value, one can apply the method **value-dereference** to the `<gdb:value>` object corresponding to it and obtain a `<gdb:value>` which is identical to that corresponding to `val`. However, if you apply the method **value-referenced-value**, the result would be a `<gdb:value>` object identical to that corresponding to `ptr`.

```
(define scm-ptrref (parse-and-eval "ptrref"))
(define scm-val (value-dereference scm-ptrref))
(define scm-ptr (value-referenced-value scm-ptrref))
```

The `<gdb:value>` object `scm-val` is identical to that corresponding to `val`, and `scm-ptr` is identical to that corresponding to `ptr`. In general, **value-dereference** can be applied whenever the C unary operator `*` can be applied to the corresponding C value. For those cases where applying both **value-dereference** and **value-referenced-value** is allowed, the results obtained need not be identical (as we have seen in the above example). The results are however identical when

applied on `<gdb:value>` objects corresponding to pointers (`<gdb:value>` objects with type code `TYPE_CODE_PTR`) in a C/C++ program.

value-referenced-value *value* [Scheme Procedure]

For pointer or reference data types, this method returns a new `<gdb:value>` object corresponding to the value referenced by the pointer/reference value. For pointer data types, `value-dereference` and `value-referenced-value` produce identical results. The difference between these methods is that `value-dereference` cannot get the values referenced by reference values. For example, consider a reference to an `int`, declared in your C++ program as

```
int val = 10;
int &ref = val;
```

then applying `value-dereference` to the `<gdb:value>` object corresponding to `ref` will result in an error, while applying `value-referenced-value` will result in a `<gdb:value>` object identical to that corresponding to `val`.

```
(define scm-ref (parse-and-eval "ref"))
(define err-ref (value-dereference scm-ref)) ;; error
(define scm-val (value-referenced-value scm-ref)) ;; ok
```

The `<gdb:value>` object `scm-val` is identical to that corresponding to `val`.

value-reference-value *value* [Scheme Procedure]

Return a new `<gdb:value>` object which is a reference to the value encapsulated by `<gdb:value>` object *value*.

value-rvalue-reference-value *value* [Scheme Procedure]

Return a new `<gdb:value>` object which is an rvalue reference to the value encapsulated by `<gdb:value>` object *value*.

value-const-value *value* [Scheme Procedure]

Return a new `<gdb:value>` object which is a ‘const’ version of `<gdb:value>` object *value*.

value-field *value field-name* [Scheme Procedure]

Return field *field-name* from `<gdb:value>` object *value*.

value-subscript *value index* [Scheme Procedure]

Return the value of array *value* at index *index*. The *value* argument must be a subscriptable `<gdb:value>` object.

value-call *value arg-list* [Scheme Procedure]

Perform an inferior function call, taking *value* as a pointer to the function to call. Each element of list *arg-list* must be a `<gdb:value>` object or an object that can be converted to a value. The result is the value returned by the function.

value->bool *value* [Scheme Procedure]

Return the Scheme boolean representing `<gdb:value>` *value*. The value must be “integer like”. Pointers are ok.

value->integer [Scheme Procedure]

Return the Scheme integer representing `<gdb:value>` *value*. The value must be “integer like”. Pointers are ok.

value->real [Scheme Procedure]
 Return the Scheme real number representing `<gdb:value> value`. The value must be a number.

value->bytevector [Scheme Procedure]
 Return a Scheme bytevector with the raw contents of `<gdb:value> value`. No transformation, endian or otherwise, is performed.

value->string *value* [*#:encoding encoding*] [Scheme Procedure]
 [*#:errors errors*] [*#:length length*]
 If *value* represents a string, then this method converts the contents to a Guile string. Otherwise, this method will throw an exception.

Values are interpreted as strings according to the rules of the current language. If the optional *length* argument is given, the string will be converted to that length, and will include any embedded zeroes that the string may contain. Otherwise, for languages where the string is zero-terminated, the entire string will be converted.

For example, in C-like languages, a value is a string if it is a pointer to or an array of characters or ints of type `wchar_t`, `char16_t`, or `char32_t`.

If the optional *encoding* argument is given, it must be a string naming the encoding of the string in the `<gdb:value>`, such as `"ascii"`, `"iso-8859-6"` or `"utf-8"`. It accepts the same encodings as the corresponding argument to Guile's `scm_from_stringn` function, and the Guile codec machinery will be used to convert the string. If *encoding* is not given, or if *encoding* is the empty string, then either the `target-charset` (see Section 10.21 [Character Sets], page 190) will be used, or a language-specific encoding will be used, if the current language is able to supply one.

The optional *errors* argument is one of `#f`, `error` or `substitute`. `error` and `substitute` must be symbols. If *errors* is not specified, or if its value is `#f`, then the default conversion strategy is used, which is set with the Scheme function `set-port-conversion-strategy!`. If the value is `'error` then an exception is thrown if there is any conversion error. If the value is `'substitute` then any conversion error is replaced with question marks. See Section “Strings” in *GNU Guile Reference Manual*.

If the optional *length* argument is given, the string will be fetched and converted to the given length. The length must be a Scheme integer and not a `<gdb:value>` integer.

value->lazy-string *value* [*#:encoding encoding*] [Scheme Procedure]
 [*#:length length*]

If this `<gdb:value>` represents a string, then this method converts *value* to a `<gdb:lazy-string>` (see Section 23.4.3.20 [Lazy Strings In Guile], page 582). Otherwise, this method will throw an exception.

If the optional *encoding* argument is given, it must be a string naming the encoding of the `<gdb:lazy-string>`. Some examples are: `"ascii"`, `"iso-8859-6"` or `"utf-8"`. If the *encoding* argument is an encoding that GDB does not recognize, GDB will raise an error.

When a lazy string is printed, the GDB encoding machinery is used to convert the string during printing. If the optional *encoding* argument is not provided, or is an

empty string, GDB will automatically select the encoding most suitable for the string type. For further information on encoding in GDB please see Section 10.21 [Character Sets], page 190.

If the optional *length* argument is given, the string will be fetched and encoded to the length of characters specified. If the *length* argument is not provided, the string will be fetched and encoded until a null of appropriate width is found. The length must be a Scheme integer and not a `<gdb:value>` integer.

value-lazy? *value* [Scheme Procedure]

Return `#t` if *value* has not yet been fetched from the inferior. Otherwise return `#f`. GDB does not fetch values until necessary, for efficiency. For example:

```
(define myval (parse-and-eval "somevar"))
```

The value of `somevar` is not fetched at this time. It will be fetched when the value is needed, or when the `fetch-lazy` procedure is invoked.

make-lazy-value *type address* [Scheme Procedure]

Return a `<gdb:value>` that will be lazily fetched from the target. The object of type `<gdb:type>` whose value to fetch is specified by its *type* and its target memory *address*, which is a Scheme integer.

value-fetch-lazy! *value* [Scheme Procedure]

If *value* is a lazy value (`(value-lazy? value)` is `#t`), then the value is fetched from the inferior. Any errors that occur in the process will produce a Guile exception.

If *value* is not a lazy value, this method has no effect.

The result of this function is unspecified.

value-print *value* [Scheme Procedure]

Return the string representation (print form) of `<gdb:value>` *value*.

23.4.3.6 Arithmetic In Guile

The `(gdb)` module provides several functions for performing arithmetic on `<gdb:value>` objects. The arithmetic is performed as if it were done by the target, and therefore has target semantics which are not necessarily those of Scheme. For example operations work with a fixed precision, not the arbitrary precision of Scheme.

Wherever a function takes an integer or pointer as an operand, GDB will convert appropriate Scheme values to perform the operation.

value-add *a b* [Scheme Procedure]

value-sub *a b* [Scheme Procedure]

value-mul *a b* [Scheme Procedure]

value-div *a b* [Scheme Procedure]

value-rem *a b* [Scheme Procedure]

value-mod *a b* [Scheme Procedure]

<code>value-pow a b</code>	[Scheme Procedure]
<code>value-not a</code>	[Scheme Procedure]
<code>value-neg a</code>	[Scheme Procedure]
<code>value-pos a</code>	[Scheme Procedure]
<code>value-abs a</code>	[Scheme Procedure]
<code>value-lsh a b</code>	[Scheme Procedure]
<code>value-rsh a b</code>	[Scheme Procedure]
<code>value-min a b</code>	[Scheme Procedure]
<code>value-max a b</code>	[Scheme Procedure]
<code>value-lognot a</code>	[Scheme Procedure]
<code>value-logand a b</code>	[Scheme Procedure]
<code>value-logior a b</code>	[Scheme Procedure]
<code>value-logxor a b</code>	[Scheme Procedure]
<code>value=? a b</code>	[Scheme Procedure]
<code>value<? a b</code>	[Scheme Procedure]
<code>value<=? a b</code>	[Scheme Procedure]
<code>value>? a b</code>	[Scheme Procedure]
<code>value>=? a b</code>	[Scheme Procedure]

Scheme does not provide a `not-equal` function, and thus Guile support in GDB does not either.

23.4.3.7 Types In Guile

GDB represents types from the inferior in objects of type `<gdb:type>`.

The following type-related procedures are provided by the `(gdb)` module.

`type? object` [Scheme Procedure]

Return `#t` if *object* is an object of type `<gdb:type>`. Otherwise return `#f`.

`lookup-type name [#:block block]` [Scheme Procedure]

This function looks up a type by its *name*, which must be a string.

If *block* is given, it is an object of type `<gdb:block>`, and *name* is looked up in that scope. Otherwise, it is searched for globally.

Ordinarily, this function will return an instance of `<gdb:type>`. If the named type cannot be found, it will throw an exception.

`type-code type` [Scheme Procedure]

Return the type code of *type*. The type code will be one of the `TYPE_CODE_` constants defined below.

type-tag *type* [Scheme Procedure]
 Return the tag name of *type*. The tag name is the name after **struct**, **union**, or **enum** in C and C++; not all languages have this concept. If this type has no tag name, then **#f** is returned.

type-name *type* [Scheme Procedure]
 Return the name of *type*. If this type has no name, then **#f** is returned.

type-print-name *type* [Scheme Procedure]
 Return the print name of *type*. This returns something even for anonymous types. For example, for an anonymous C struct "**struct {...}**" is returned.

type-sizeof *type* [Scheme Procedure]
 Return the size of this type, in target **char** units. Usually, a target's **char** type will be an 8-bit byte. However, on some unusual platforms, this type may have a different size.

type-strip-typedefs *type* [Scheme Procedure]
 Return a new `<gdb:type>` that represents the real type of *type*, after removing all layers of typedefs.

type-array *type* *n1* [*n2*] [Scheme Procedure]
 Return a new `<gdb:type>` object which represents an array of this type. If one argument is given, it is the inclusive upper bound of the array; in this case the lower bound is zero. If two arguments are given, the first argument is the lower bound of the array, and the second argument is the upper bound of the array. An array's length must not be negative, but the bounds can be.

type-vector *type* *n1* [*n2*] [Scheme Procedure]
 Return a new `<gdb:type>` object which represents a vector of this type. If one argument is given, it is the inclusive upper bound of the vector; in this case the lower bound is zero. If two arguments are given, the first argument is the lower bound of the vector, and the second argument is the upper bound of the vector. A vector's length must not be negative, but the bounds can be.

The difference between an **array** and a **vector** is that arrays behave like in C: when used in expressions they decay to a pointer to the first element whereas vectors are treated as first class values.

type-pointer *type* [Scheme Procedure]
 Return a new `<gdb:type>` object which represents a pointer to *type*.

type-range *type* [Scheme Procedure]
 Return a list of two elements: the low bound and high bound of *type*. If *type* does not have a range, an exception is thrown.

type-reference *type* [Scheme Procedure]
 Return a new `<gdb:type>` object which represents a reference to *type*.

type-target *type* [Scheme Procedure]

Return a new `<gdb:type>` object which represents the target type of *type*.

For a pointer type, the target type is the type of the pointed-to object. For an array type (meaning C-like arrays), the target type is the type of the elements of the array. For a function or method type, the target type is the type of the return value. For a complex type, the target type is the type of the elements. For a typedef, the target type is the aliased type.

If the type does not have a target, this method will throw an exception.

type-const *type* [Scheme Procedure]

Return a new `<gdb:type>` object which represents a `const`-qualified variant of *type*.

type-volatile *type* [Scheme Procedure]

Return a new `<gdb:type>` object which represents a `volatile`-qualified variant of *type*.

type-unqualified *type* [Scheme Procedure]

Return a new `<gdb:type>` object which represents an unqualified variant of *type*. That is, the result is neither `const` nor `volatile`.

type-num-fields [Scheme Procedure]

Return the number of fields of `<gdb:type>` *type*.

type-fields *type* [Scheme Procedure]

Return the fields of *type* as a list. For structure and union types, `fields` has the usual meaning. Range types have two fields, the minimum and maximum values. Enum types have one field per enum constant. Function and method types have one field per parameter. The base types of C++ classes are also represented as fields. If the type has no fields, or does not fit into one of these categories, an empty list will be returned. See [Fields of a type in Guile], page 555.

make-field-iterator *type* [Scheme Procedure]

Return the fields of *type* as a `<gdb:iterator>` object. See Section 23.4.3.25 [Iterators In Guile], page 587.

type-field *type field-name* [Scheme Procedure]

Return field named *field-name* in *type*. The result is an object of type `<gdb:field>`. See [Fields of a type in Guile], page 555. If the type does not have fields, or *field-name* is not a field of *type*, an exception is thrown.

For example, if `some-type` is a `<gdb:type>` instance holding a structure type, you can access its `foo` field with:

```
(define bar (type-field some-type "foo"))
```

`bar` will be a `<gdb:field>` object.

type-has-field? *type name* [Scheme Procedure]

Return `#t` if `<gdb:type>` *type* has field named *name*. Otherwise return `#f`.

Each type has a code, which indicates what category this type falls into. The available type categories are represented by constants defined in the `(gdb)` module:

`TYPE_CODE_PTR`

The type is a pointer.

`TYPE_CODE_ARRAY`

The type is an array.

`TYPE_CODE_STRUCT`

The type is a structure.

`TYPE_CODE_UNION`

The type is a union.

`TYPE_CODE_ENUM`

The type is an enum.

`TYPE_CODE_FLAGS`

A bit flags type, used for things such as status registers.

`TYPE_CODE_FUNC`

The type is a function.

`TYPE_CODE_INT`

The type is an integer type.

`TYPE_CODE_FLT`

A floating point type.

`TYPE_CODE_VOID`

The special type `void`.

`TYPE_CODE_SET`

A Pascal set type.

`TYPE_CODE_RANGE`

A range type, that is, an integer type with bounds.

`TYPE_CODE_STRING`

A string type. Note that this is only used for certain languages with language-defined string types; C strings are not represented this way.

`TYPE_CODE_BITSTRING`

A string of bits. It is deprecated.

`TYPE_CODE_ERROR`

An unknown or erroneous type.

`TYPE_CODE_METHOD`

A method type, as found in C++.

`TYPE_CODE_METHODPTR`

A pointer-to-member-function.

`TYPE_CODE_MEMBERPTR`

A pointer-to-member.

`TYPE_CODE_REF`

A reference type.

`TYPE_CODE_RVALUE_REF`

A C++11 rvalue reference type.

`TYPE_CODE_CHAR`

A character type.

`TYPE_CODE_BOOL`

A boolean type.

`TYPE_CODE_COMPLEX`

A complex float type.

`TYPE_CODE_TYPEDEF`

A typedef to some other type.

`TYPE_CODE_NAMESPACE`

A C++ namespace.

`TYPE_CODE_DECFLOAT`

A decimal floating point type.

`TYPE_CODE_INTERNAL_FUNCTION`

A function internal to GDB. This is the type used to represent convenience functions (see Section 10.13 [Convenience Funs], page 177).

`gdb.TYPE_CODE_XMETHOD`

A method internal to GDB. This is the type used to represent xmethods (see Section 23.3.2.16 [Writing an Xmethod], page 452).

`gdb.TYPE_CODE_FIXED_POINT`

A fixed-point number.

`gdb.TYPE_CODE_NAMESPACELIST`

A Fortran namelist.

Further support for types is provided in the (`gdb types`) Guile module (see Section 23.4.5.2 [Guile Types Module], page 591).

Each field is represented as an object of type `<gdb:field>`.

The following field-related procedures are provided by the (`gdb`) module:

`field? object` [Scheme Procedure]

Return `#t` if *object* is an object of type `<gdb:field>`. Otherwise return `#f`.

`field-name field` [Scheme Procedure]

Return the name of the field, or `#f` for anonymous fields.

`field-type field` [Scheme Procedure]

Return the type of the field. This is usually an instance of `<gdb:type>`, but it can be `#f` in some situations.

`field-enumval field` [Scheme Procedure]

Return the enum value represented by `<gdb:field>` *field*.

field-bitpos *field* [Scheme Procedure]
 Return the bit position of `<gdb:field> field`. This attribute is not available for `static` fields (as in C++).

field-bitsize *field* [Scheme Procedure]
 If the field is packed, or is a bitfield, return the size of `<gdb:field> field` in bits. Otherwise, zero is returned; in which case the field's size is given by its type.

field-artificial? *field* [Scheme Procedure]
 Return `#t` if the field is artificial, usually meaning that it was provided by the compiler and not the user. Otherwise return `#f`.

field-base-class? *field* [Scheme Procedure]
 Return `#t` if the field represents a base class of a C++ structure. Otherwise return `#f`.

23.4.3.8 Guile Pretty Printing API

An example output is provided (see Section 10.10 [Pretty Printing], page 170).

A pretty-printer is represented by an object of type `<gdb:pretty-printer>`. Pretty-printer objects are created with `make-pretty-printer`.

The following pretty-printer-related procedures are provided by the `(gdb)` module:

make-pretty-printer *name lookup-function* [Scheme Procedure]
 Return a `<gdb:pretty-printer>` object named *name*.
lookup-function is a function of one parameter: the value to be printed. If the value is handled by this pretty-printer, then *lookup-function* returns an object of type `<gdb:pretty-printer-worker>` to perform the actual pretty-printing. Otherwise *lookup-function* returns `#f`.

pretty-printer? *object* [Scheme Procedure]
 Return `#t` if *object* is a `<gdb:pretty-printer>` object. Otherwise return `#f`.

pretty-printer-enabled? *pretty-printer* [Scheme Procedure]
 Return `#t` if *pretty-printer* is enabled. Otherwise return `#f`.

set-pretty-printer-enabled! *pretty-printer flag* [Scheme Procedure]
 Set the enabled flag of *pretty-printer* to *flag*. The value returned is unspecified.

pretty-printers [Scheme Procedure]
 Return the list of global pretty-printers.

set-pretty-printers! *pretty-printers* [Scheme Procedure]
 Set the list of global pretty-printers to *pretty-printers*. The value returned is unspecified.

make-pretty-printer-worker *display-hint to-string children* [Scheme Procedure]
 Return an object of type `<gdb:pretty-printer-worker>`.

This function takes three parameters:

‘display-hint’

display-hint provides a hint to GDB or GDB front end via MI to change the formatting of the value being printed. The value must be a string or `#f` (meaning there is no hint). Several values for *display-hint* are predefined by GDB:

- ‘array’ Indicate that the object being printed is “array-like”. The CLI uses this to respect parameters such as `set print elements` and `set print array`.
- ‘map’ Indicate that the object being printed is “map-like”, and that the children of this value can be assumed to alternate between keys and values.
- ‘string’ Indicate that the object being printed is “string-like”. If the printer’s `to-string` function returns a Guile string of some kind, then GDB will call its internal language-specific string-printing function to format the string. For the CLI this means adding quotation marks, possibly escaping some characters, respecting `set print elements`, and the like.

‘to-string’

to-string is either a function of one parameter, the `<gdb:pretty-printer-worker>` object, or `#f`.

When printing from the CLI, if the `to-string` method exists, then GDB will prepend its result to the values returned by `children`. Exactly how this formatting is done is dependent on the display hint, and may change as more hints are added. Also, depending on the print settings (see Section 10.9 [Print Settings], page 159), the CLI may print just the result of `to-string` in a stack trace, omitting the result of `children`.

If this method returns a string, it is printed verbatim.

Otherwise, if this method returns an instance of `<gdb:value>`, then GDB prints this value. This may result in a call to another pretty-printer.

If instead the method returns a Guile value which is convertible to a `<gdb:value>`, then GDB performs the conversion and prints the resulting value. Again, this may result in a call to another pretty-printer. Guile scalars (integers, floats, and booleans) and strings are convertible to `<gdb:value>`; other types are not.

Finally, if this method returns `#f` then no further operations are performed in this method and nothing is printed.

If the result is not one of these types, an exception is raised.

to-string may also be `#f` in which case it is left to *children* to print the value.

‘children’

children is either a function of one parameter, the `<gdb:pretty-printer-worker>` object, or `#f`.

GDB will call this function on a pretty-printer to compute the children of the pretty-printer's value.

This function must return a `<gdb:iterator>` object. Each item returned by the iterator must be a tuple holding two elements. The first element is the “name” of the child; the second element is the child's value. The value can be any Guile object which is convertible to a GDB value.

If *children* is `#f`, GDB will act as though the value has no children.

Children may be hidden from display based on the value of ‘`set print max-depth`’ (see Section 10.9 [Print Settings], page 159).

GDB provides a function which can be used to look up the default pretty-printer for a `<gdb:value>`:

default-visualizer *value* [Scheme Procedure]

This function takes a `<gdb:value>` object as an argument. If a pretty-printer for this value exists, then it is returned. If no such printer exists, then this returns `#f`.

23.4.3.9 Selecting Guile Pretty-Printers

There are three sets of pretty-printers that GDB searches:

- Per-objfile list of pretty-printers (see Section 23.4.3.14 [Objfiles In Guile], page 568).
- Per-progspace list of pretty-printers (see Section 23.4.3.13 [Progspace In Guile], page 567).
- The global list of pretty-printers (see Section 23.4.3.8 [Guile Pretty Printing API], page 556). These printers are available when debugging any inferior.

Pretty-printer lookup is done by passing the value to be printed to the lookup function of each enabled object in turn. Lookup stops when a lookup function returns a non-`#f` value or when the list is exhausted. Lookup functions must return either a `<gdb:pretty-printer-worker>` object or `#f`. Otherwise an exception is thrown.

GDB first checks the result of `objfile-pretty-printers` of each `<gdb:objfile>` in the current program space and iteratively calls each enabled lookup function in the list for that `<gdb:objfile>` until a non-`#f` object is returned. If no pretty-printer is found in the objfile lists, GDB then searches the result of `progspace-pretty-printers` of the current program space, calling each enabled function until a non-`#f` object is returned. After these lists have been exhausted, it tries the global pretty-printers list, obtained with `pretty-printers`, again calling each enabled function until a non-`#f` object is returned.

The order in which the objfiles are searched is not specified. For a given list, functions are always invoked from the head of the list, and iterated over sequentially until the end of the list, or a `<gdb:pretty-printer-worker>` object is returned.

For various reasons a pretty-printer may not work. For example, the underlying data structure may have changed and the pretty-printer is out of date.

The consequences of a broken pretty-printer are severe enough that GDB provides support for enabling and disabling individual printers. For example, if `print frame-arguments` is on, a backtrace can become highly illegible if any argument is printed with a broken printer.

Pretty-printers are enabled and disabled from Scheme by calling `set-pretty-printer-enabled!`. See Section 23.4.3.8 [Guile Pretty Printing API], page 556.

23.4.3.10 Writing a Guile Pretty-Printer

A pretty-printer consists of two basic parts: a lookup function to determine if the type is supported, and the printer itself.

Here is an example showing how a `std::string` printer might be written. See Section 23.4.3.8 [Guile Pretty Printing API], page 556, for details.

```
(define (make-my-string-printer value)
  "Print a my::string string"
  (make-pretty-printer-worker
   "string"
   (lambda (printer)
     (value-field value "_data")))
  #f))
```

And here is an example showing how a lookup function for the printer example above might be written.

```
(define (str-lookup-function pretty-printer value)
  (let ((tag (type-tag (value-type value))))
    (and tag
         (string-prefix? "std::string<" tag)
         (make-my-string-printer value))))
```

Then to register this printer in the global printer list:

```
(append-pretty-printer!
 (make-pretty-printer "my-string" str-lookup-function))
```

The example lookup function extracts the value's type, and attempts to match it to a type that it can pretty-print. If it is a type the printer can pretty-print, it will return a `<gdb:pretty-printer-worker>` object. If not, it returns `#f`.

We recommend that you put your core pretty-printers into a Guile package. If your pretty-printers are for use with a library, we further recommend embedding a version number into the package name. This practice will enable GDB to load multiple versions of your pretty-printers at the same time, because they will have different names.

You should write auto-loaded code (see Section 23.4.4 [Guile Auto-loading], page 590) such that it can be evaluated multiple times without changing its meaning. An ideal auto-load file will consist solely of `imports` of your printer modules, followed by a call to a register pretty-printers with the current objfile.

Taken as a whole, this approach will scale nicely to multiple inferiors, each potentially using a different library version. Embedding a version number in the Guile package name will ensure that GDB is able to load both sets of printers simultaneously. Then, because the search for pretty-printers is done by objfile, and because your auto-loaded code took care to register your library's printers with a specific objfile, GDB will find the correct printers for the specific version of the library used by each inferior.

To continue the `my::string` example, this code might appear in `(my-project my-library v1)`:

```
(use-modules (gdb))
(define (register-printers objfile)
  (append-objfile-pretty-printer!
   (make-pretty-printer "my-string" str-lookup-function)))
```

And then the corresponding contents of the auto-load file would be:

```
(use-modules (gdb) (my-project my-library v1))
```

```
(register-printers (current-objfile))
```

The previous example illustrates a basic pretty-printer. There are a few things that can be improved on. The printer only handles one type, whereas a library typically has several types. One could install a lookup function for each desired type in the library, but one could also have a single lookup function recognize several types. The latter is the conventional way this is handled. If a pretty-printer can handle multiple data types, then its *subprinters* are the printers for the individual data types.

The (gdb printing) module provides a formal way of solving this problem (see Section 23.4.5.1 [Guile Printing Module], page 591). Here is another example that handles multiple types.

These are the types we are going to pretty-print:

```
struct foo { int a, b; };
struct bar { struct foo x, y; };
```

Here are the printers:

```
(define (make-foo-printer value)
  "Print a foo object"
  (make-pretty-printer-worker
   "foo"
   (lambda (printer)
     (format #f "a=<~a> b=<~a>"
              (value-field value "a") (value-field value "a"))
     #f)))

(define (make-bar-printer value)
  "Print a bar object"
  (make-pretty-printer-worker
   "foo"
   (lambda (printer)
     (format #f "x=<~a> y=<~a>"
              (value-field value "x") (value-field value "y"))
     #f)))
```

This example doesn't need a lookup function, that is handled by the (gdb printing) module. Instead a function is provided to build up the object that handles the lookup.

```
(use-modules (gdb printing))

(define (build-pretty-printer)
  (let ((pp (make-pretty-printer-collection "my-library")))
    (pp-collection-add-tag-printer "foo" make-foo-printer)
    (pp-collection-add-tag-printer "bar" make-bar-printer)
    pp))
```

And here is the autoload support:

```
(use-modules (gdb) (my-library))
(append-objfile-pretty-printer! (current-objfile) (build-pretty-printer))
```

Finally, when this printer is loaded into GDB, here is the corresponding output of 'info pretty-printer':

```
(gdb) info pretty-printer
my_library.so:
  my-library
    foo
    bar
```

23.4.3.11 Commands In Guile

You can implement new GDB CLI commands in Guile. A CLI command object is created with the `make-command` Guile function, and added to GDB with the `register-command!` Guile function. This two-step approach is taken to separate out the side-effect of adding the command to GDB from `make-command`.

There is no support for multi-line commands, that is commands that consist of multiple lines and are terminated with `end`.

```
make-command name [#:invoke invoke] [Scheme Procedure]
                [#:command-class command-class] [#:completer-class completer]
                [#:prefix? prefix] [#:doc doc-string]
```

The argument *name* is the name of the command. If *name* consists of multiple words, then the initial words are looked for as prefix commands. In this case, if one of the prefix commands does not exist, an exception is raised.

The result is the `<gdb:command>` object representing the command. The command is not usable until it has been registered with GDB with `register-command!`.

The rest of the arguments are optional.

The argument *invoke* is a procedure of three arguments: *self*, *args* and *from-tty*. The argument *self* is the `<gdb:command>` object representing the command. The argument *args* is a string representing the arguments passed to the command, after leading and trailing whitespace has been stripped. The argument *from-tty* is a boolean flag and specifies whether the command should consider itself to have been originated from the user invoking it interactively. If this function throws an exception, it is turned into a GDB **error** call. Otherwise, the return value is ignored.

For non-prefix commands, *invoke* is required. For prefix commands *invoke* is optional. Only prefix commands that need to handle unknown sub-commands should supply *invoke*.

For prefix commands that don't supply *invoke*, if the prefix command is used without a sub-command name then GDB will display the help text for every sub-command, unless the prefix command is a **show** sub-command, in which case GDB will list the values of all sub-commands.

The argument *command-class* is one of the `'COMMAND_'` constants defined below. This argument tells GDB how to categorize the new command in the help system. The default is `COMMAND_NONE`.

The argument *completer* is either `#f`, one of the `'COMPLETE_'` constants defined below, or a procedure, also defined below. This argument tells GDB how to perform completion for this command. If not provided or if the value is `#f`, then no completion is performed on the command.

The argument *prefix* is a boolean flag indicating whether the new command is a prefix command; sub-commands of this command may be registered.

The argument *doc-string* is help text for the new command. If no documentation string is provided, the default value "This command is not documented." is used.

```
register-command! command [Scheme Procedure]
```

Add *command*, a `<gdb:command>` object, to GDB's list of commands. It is an error to register a command more than once. The result is unspecified.

command? *object* [Scheme Procedure]
 Return **#t** if *object* is a `<gdb:command>` object. Otherwise return **#f**.

dont-repeat [Scheme Procedure]
 By default, a GDB command is repeated when the user enters a blank line at the command prompt. A command can suppress this behavior by invoking the **dont-repeat** function. This is similar to the user command **dont-repeat**, see Section 23.1.1 [Define], page 397.

string->argv *string* [Scheme Procedure]
 Convert a string to a list of strings split up according to GDB's argv parsing rules. It is recommended to use this for consistency. Arguments are separated by spaces and may be quoted. Example:

```
scheme@(guile-user)> (string->argv "1 2\\ \\\"3 '4 \"5' \"6 '7\"")
$1 = ("1" "2 \"3" "4 \"5" "6 '7")
```

throw-user-error *message* . *args* [Scheme Procedure]
 Throw a `gdb:user-error` exception. The argument *message* is the error message as a format string, like the *fmt* argument to the **format** Scheme function. See Section “Formatted Output” in *GNU Guile Reference Manual*. The argument *args* is a list of the optional arguments of *message*.

This is used when the command detects a user error of some kind, say a bad command argument.

```
(gdb) guile (use-modules (gdb))
(gdb) guile
(register-command! (make-command "test-user-error"
  #:command-class COMMAND_OBSCURE
  #:invoke (lambda (self arg from-tty)
    (throw-user-error "Bad argument ~a" arg))))
end
(gdb) test-user-error ugh
ERROR: Bad argument ugh
```

self *text word* [completer]
 If the *completer* option to **make-command** is a procedure, it takes three arguments: *self* which is the `<gdb:command>` object, and *text* and *word* which are both strings. The argument *text* holds the complete command line up to the cursor's location. The argument *word* holds the last word of the command line; this is computed using a word-breaking heuristic.

All forms of completion are handled by this function, that is, the TAB and M-? key bindings (see Section 3.3 [Completion], page 24), and the **complete** command (see Section 3.6 [Help], page 29).

This procedure can return several kinds of values:

- If the return value is a list, the contents of the list are used as the completions. It is up to *completer* to ensure that the contents actually do complete the word. An empty list is allowed, it means that there were no completions available. Only string elements of the list are used; other elements in the list are ignored.
- If the return value is a `<gdb:iterator>` object, it is iterated over to obtain the completions. It is up to *completer-procedure* to ensure that the results actually

do complete the word. Only string elements of the result are used; other elements in the sequence are ignored.

- All other results are treated as though there were no available completions.

When a new command is registered, it will have been declared as a member of some general class of commands. This is used to classify top-level commands in the on-line help system; note that prefix commands are not listed under their own category but rather that of their top-level command. The available classifications are represented by constants defined in the `gdb` module:

COMMAND_NONE

The command does not belong to any particular class. A command in this category will not be displayed in any of the help categories. This is the default.

COMMAND_RUNNING

The command is related to running the inferior. For example, `start`, `step`, and `continue` are in this category. Type *help running* at the GDB prompt to see a list of commands in this category.

COMMAND_DATA

The command is related to data or variables. For example, `call`, `find`, and `print` are in this category. Type *help data* at the GDB prompt to see a list of commands in this category.

COMMAND_STACK

The command has to do with manipulation of the stack. For example, `backtrace`, `frame`, and `return` are in this category. Type *help stack* at the GDB prompt to see a list of commands in this category.

COMMAND_FILES

This class is used for file-related commands. For example, `file`, `list` and `section` are in this category. Type *help files* at the GDB prompt to see a list of commands in this category.

COMMAND_SUPPORT

This should be used for “support facilities”, generally meaning things that are useful to the user when interacting with GDB, but not related to the state of the inferior. For example, `help`, `make`, and `shell` are in this category. Type *help support* at the GDB prompt to see a list of commands in this category.

COMMAND_STATUS

The command is an ‘info’-related command, that is, related to the state of GDB itself. For example, `info`, `macro`, and `show` are in this category. Type *help status* at the GDB prompt to see a list of commands in this category.

COMMAND_BREAKPOINTS

The command has to do with breakpoints. For example, `break`, `clear`, and `delete` are in this category. Type *help breakpoints* at the GDB prompt to see a list of commands in this category.

COMMAND_TRACEPOINTS

The command has to do with tracepoints. For example, **trace**, **actions**, and **tfind** are in this category. Type *help tracepoints* at the GDB prompt to see a list of commands in this category.

COMMAND_USER

The command is a general purpose command for the user, and typically does not fit in one of the other categories. Type *help user-defined* at the GDB prompt to see a list of commands in this category, as well as the list of gdb macros (see Section 23.1 [Sequences], page 397).

COMMAND_OBSCURE

The command is only used in unusual circumstances, or is not of general interest to users. For example, **checkpoint**, **fork**, and **stop** are in this category. Type *help obscure* at the GDB prompt to see a list of commands in this category.

COMMAND_MAINTENANCE

The command is only useful to GDB maintainers. The **maintenance** and **flushregs** commands are in this category. Type *help internals* at the GDB prompt to see a list of commands in this category.

A new command can use a predefined completion function, either by specifying it via an argument at initialization, or by returning it from the **completer** procedure. These predefined completion constants are all defined in the **gdb** module:

COMPLETE_NONE

This constant means that no completion should be done.

COMPLETE_FILENAME

This constant means that filename completion should be performed.

COMPLETE_LOCATION

This constant means that location completion should be done. See Section 9.2 [Location Specifications], page 128.

COMPLETE_COMMAND

This constant means that completion should examine GDB command names.

COMPLETE_SYMBOL

This constant means that completion should be done using symbol names as the source.

COMPLETE_EXPRESSION

This constant means that completion should be done on expressions. Often this means completing on symbol names, but some language parsers also have support for completing on field names.

The following code snippet shows how a trivial CLI command can be implemented in Guile:

```
(gdb) guile
(register-command! (make-command "hello-world"
  #:command-class COMMAND_USER
  #:doc "Greet the whole world."
  #:invoke (lambda (self args from-tty) (display "Hello, World!\n"))))
```

```

end
(gdb) hello-world
Hello, World!

```

23.4.3.12 Parameters In Guile

You can implement new GDB *parameters* using Guile³.

There are many parameters that already exist and can be set in GDB. Two examples are: **set follow-fork** and **set charset**. Setting these parameters influences certain behavior in GDB. Similarly, you can define parameters that can be used to influence behavior in custom Guile scripts and commands.

A new parameter is defined with the **make-parameter** Guile function, and added to GDB with the **register-parameter!** Guile function. This two-step approach is taken to separate out the side-effect of adding the parameter to GDB from **make-parameter**.

Parameters are exposed to the user via the **set** and **show** commands. See Section 3.6 [Help], page 29.

```

make-parameter name [#:command-class command-class] [Scheme Procedure]
                  [#:parameter-type parameter-type] [#:enum-list enum-list]
                  [#:set-func set-func] [#:show-func show-func] [#:doc doc]
                  [#:set-doc set-doc] [#:show-doc show-doc] [#:initial-value initial-value]

```

The argument *name* is the name of the new parameter. If *name* consists of multiple words, then the initial words are looked for as prefix parameters. An example of this can be illustrated with the **set print** set of parameters. If *name* is **print foo**, then **print** will be searched as the prefix parameter. In this case the parameter can subsequently be accessed in GDB as **set print foo**. If *name* consists of multiple words, and no prefix parameter group can be found, an exception is raised.

The result is the `<gdb:parameter>` object representing the parameter. The parameter is not usable until it has been registered with GDB with **register-parameter!**.

The rest of the arguments are optional.

The argument *command-class* should be one of the ‘**COMMAND_**’ constants (see Section 23.4.3.11 [Commands In Guile], page 561). This argument tells GDB how to categorize the new parameter in the help system. The default is **COMMAND_NONE**.

The argument *parameter-type* should be one of the ‘**PARAM_**’ constants defined below. This argument tells GDB the type of the new parameter; this information is used for input validation and completion. The default is **PARAM_BOOLEAN**.

If *parameter-type* is **PARAM_ENUM**, then *enum-list* must be a list of strings. These strings represent the possible values for the parameter.

If *parameter-type* is not **PARAM_ENUM**, then the presence of *enum-list* will cause an exception to be thrown.

The argument *set-func* is a function of one argument: *self* which is the `<gdb:parameter>` object representing the parameter. GDB will call this function when a *parameter*’s value has been changed via the **set** API (for example, **set foo off**). The value of the parameter has already been set to the new value. This

³ Note that GDB parameters must not be confused with Guiles parameter objects (see Section “Parameters” in *GNU Guile Reference Manual*).

function must return a string to be displayed to the user. GDB will add a trailing newline if the string is non-empty. GDB generally doesn't print anything when a parameter is set, thus typically this function should return `""`. A non-empty string result should typically be used for displaying warnings and errors.

The argument *show-func* is a function of two arguments: *self* which is the `<gdb:parameter>` object representing the parameter, and *svalue* which is the string representation of the current value. GDB will call this function when a *parameter*'s `show` API has been invoked (for example, `show foo`). This function must return a string, and will be displayed to the user. GDB will add a trailing newline.

The argument *doc* is the help text for the new parameter. If there is no documentation string, a default value is used. If the documentation string is empty, then GDB will print just the *set-doc* and *show-doc* strings (see below).

The argument *set-doc* is the help text for this parameter's `set` command.

The argument *show-doc* is the help text for this parameter's `show` command.

The argument *initial-value* specifies the initial value of the parameter. If it is a function, it takes one parameter, the `<gdb:parameter>` object and its result is used as the initial value of the parameter. The initial value must be valid for the parameter type, otherwise an exception is thrown.

register-parameter! *parameter* [Scheme Procedure]

Add *parameter*, a `<gdb:parameter>` object, to GDB's list of parameters. It is an error to register a parameter more than once. The result is unspecified.

parameter? *object* [Scheme Procedure]

Return `#t` if *object* is a `<gdb:parameter>` object. Otherwise return `#f`.

parameter-value *parameter* [Scheme Procedure]

Return the value of *parameter* which may either be a `<gdb:parameter>` object or a string naming the parameter.

set-parameter-value! *parameter new-value* [Scheme Procedure]

Assign *parameter* the value of *new-value*. The argument *parameter* must be an object of type `<gdb:parameter>`. GDB does validation when assignments are made.

When a new parameter is defined, its type must be specified. The available types are represented by constants defined in the `gdb` module:

PARAM_BOOLEAN

The value is a plain boolean. The Guile boolean values, `#t` and `#f` are the only valid values.

PARAM_AUTO_BOOLEAN

The value has three possible states: `true`, `false`, and `'auto'`. In Guile, `true` and `false` are represented using boolean constants, and `'auto'` is represented using `#:auto`.

PARAM_UINTEGER

The value is an unsigned integer. The value of `#:unlimited` should be interpreted to mean "unlimited", and the value of `'0'` is reserved and should not be used.

PARAM_ZINTEGER

The value is an integer.

PARAM_ZUINTEGER

The value is an unsigned integer.

PARAM_ZUINTEGER_UNLIMITED

The value is an integer in the range `[0, INT_MAX]`. The value of `#:unlimited` means “unlimited”, the value of `-1` is reserved and should not be used, and other negative numbers are not allowed.

PARAM_STRING

The value is a string. When the user modifies the string, any escape sequences, such as `\t`, `\f`, and octal escapes, are translated into corresponding characters and encoded into the current host charset.

PARAM_STRING_NOESCAPE

The value is a string. When the user modifies the string, escapes are passed through untranslated.

PARAM_OPTIONAL_FILENAME

The value is either a filename (a string), or `#f`.

PARAM_FILENAME

The value is a filename. This is just like **PARAM_STRING_NOESCAPE**, but uses file names for completion.

PARAM_ENUM

The value is a string, which must be one of a collection of string constants provided when the parameter is created.

PARAM_COLOR

The value is either a string or an unsigned integer. Integer from 0 to 255 means index into terminal’s color palette. String can be a hex RGB triplet in `#RRGGBB` format or one of the following color names: `‘none’` (meaning the terminal’s default color), `‘black’`, `‘red’`, `‘green’`, `‘yellow’`, `‘blue’`, `‘magenta’`, `‘cyan’`, or `‘white’`.

23.4.3.13 Program Spaces In Guile

A program space, or *progspace*, represents a symbolic view of an address space. It consists of all of the objfiles of the program. See Section 23.4.3.14 [Objfiles In Guile], page 568. See Section 4.9 [Inferiors Connections and Programs], page 43, for more details about program spaces.

Each progspace is represented by an instance of the `<gdb:progspace>` smob. See Section 23.4.3.3 [GDB Scheme Data Types], page 541.

The following progspace-related functions are available in the `(gdb)` module:

progspace? *object* [Scheme Procedure]
Return `#t` if *object* is a `<gdb:progspace>` object. Otherwise return `#f`.

progspace-valid? *progspace* [Scheme Procedure]
Return `#t` if *progspace* is valid, `#f` if not. A `<gdb:progspace>` object can become invalid if the program it refers to is not loaded in GDB any longer.

current-progspace [Scheme Procedure]

This function returns the program space of the currently selected inferior. There is always a current progspace, this never returns **#f**. See Section 4.9 [Inferiors Connections and Programs], page 43.

progspace [Scheme Procedure]

Return a list of all the progspace currently known to GDB.

progspace-filename *progspace* [Scheme Procedure]

Return the absolute file name of *progspace* as a string. This is the name of the file passed as the argument to the **file** or **symbol-file** commands. If the program space does not have an associated file name, then **#f** is returned. This occurs, for example, when GDB is started without a program to debug.

A **gdb:invalid-object-error** exception is thrown if *progspace* is invalid.

progspace-objfiles *progspace* [Scheme Procedure]

Return the list of objfiles of *progspace*. The order of objfiles in the result is arbitrary. Each element is an object of type **<gdb:objfile>**. See Section 23.4.3.14 [Objfiles In Guile], page 568.

A **gdb:invalid-object-error** exception is thrown if *progspace* is invalid.

progspace-pretty-printers *progspace* [Scheme Procedure]

Return the list of pretty-printers of *progspace*. Each element is an object of type **<gdb:pretty-printer>**. See Section 23.4.3.8 [Guile Pretty Printing API], page 556, for more information.

set-progspace-pretty-printers! *progspace printer-list* [Scheme Procedure]

Set the list of registered **<gdb:pretty-printer>** objects for *progspace* to *printer-list*. See Section 23.4.3.8 [Guile Pretty Printing API], page 556, for more information.

23.4.3.14 Objfiles In Guile

GDB loads symbols for an inferior from various symbol-containing files (see Section 18.1 [Files], page 293). These include the primary executable file, any shared libraries used by the inferior, and any separate debug info files (see Section 18.3 [Separate Debug Files], page 303). GDB calls these symbol-containing files *objfiles*.

Each objfile is represented as an object of type **<gdb:objfile>**.

The following objfile-related procedures are provided by the (**gdb**) module:

objfile? *object* [Scheme Procedure]

Return **#t** if *object* is a **<gdb:objfile>** object. Otherwise return **#f**.

objfile-valid? *objfile* [Scheme Procedure]

Return **#t** if *objfile* is valid, **#f** if not. A **<gdb:objfile>** object can become invalid if the object file it refers to is not loaded in GDB any longer. All other **<gdb:objfile>** procedures will throw an exception if it is invalid at the time the procedure is called.

objfile-filename *objfile* [Scheme Procedure]

Return the file name of *objfile* as a string, with symbolic links resolved.

objfile-progspace *objfile* [Scheme Procedure]
 Return the `<gdb:progspace>` that this object file lives in. See Section 23.4.3.13 [Progspace In Guile], page 567, for more on progspace.

objfile-pretty-printers *objfile* [Scheme Procedure]
 Return the list of registered `<gdb:pretty-printer>` objects for *objfile*. See Section 23.4.3.8 [Guile Pretty Printing API], page 556, for more information.

set-objfile-pretty-printers! *objfile printer-list* [Scheme Procedure]
 Set the list of registered `<gdb:pretty-printer>` objects for *objfile* to *printer-list*. The *printer-list* must be a list of `<gdb:pretty-printer>` objects. See Section 23.4.3.8 [Guile Pretty Printing API], page 556, for more information.

current-objfile [Scheme Procedure]
 When auto-loading a Guile script (see Section 23.4.4 [Guile Auto-loading], page 590), GDB sets the “current objfile” to the corresponding objfile. This function returns the current objfile. If there is no current objfile, this function returns `#f`.

objfiles [Scheme Procedure]
 Return a list of all the objfiles in the current program space.

23.4.3.15 Accessing inferior stack frames from Guile.

When the debugged program stops, GDB is able to analyze its call stack (see Section 8.1 [Stack frames], page 115). The `<gdb:frame>` class represents a frame in the stack. A `<gdb:frame>` object is only valid while its corresponding frame exists in the inferior’s stack. If you try to use an invalid frame object, GDB will throw a `gdb:invalid-object` exception (see Section 23.4.3.4 [Guile Exception Handling], page 543).

Two `<gdb:frame>` objects can be compared for equality with the `equal?` function, like:

```
(gdb) guile (equal? (newest-frame) (selected-frame))
#t
```

The following frame-related procedures are provided by the `(gdb)` module:

frame? *object* [Scheme Procedure]
 Return `#t` if *object* is a `<gdb:frame>` object. Otherwise return `#f`.

frame-valid? *frame* [Scheme Procedure]
 Returns `#t` if *frame* is valid, `#f` if not. A frame object can become invalid if the frame it refers to doesn’t exist anymore in the inferior. All `<gdb:frame>` procedures will throw an exception if the frame is invalid at the time the procedure is called.

frame-name *frame* [Scheme Procedure]
 Return the function name of *frame*, or `#f` if it can’t be obtained.

frame-arch *frame* [Scheme Procedure]
 Return the `<gdb:architecture>` object corresponding to *frame*’s architecture. See Section 23.4.3.21 [Architectures In Guile], page 583.

frame-type *frame* [Scheme Procedure]
 Return the type of *frame*. The value can be one of:

NORMAL_FRAME
 An ordinary stack frame.

DUMMY_FRAME

A fake stack frame that was created by GDB when performing an inferior function call.

INLINE_FRAME

A frame representing an inlined function. The function was inlined into a **NORMAL_FRAME** that is older than this one.

TAILCALL_FRAME

A frame representing a tail call. See Section 11.2 [Tail Call Frames], page 198.

SIGTRAMP_FRAME

A signal trampoline frame. This is the frame created by the OS when it calls into a signal handler.

ARCH_FRAME

A fake stack frame representing a cross-architecture call.

SENTINEL_FRAME

This is like **NORMAL_FRAME**, but it is only used for the newest frame.

frame-unwind-stop-reason *frame* [Scheme Procedure]

Return an integer representing the reason why it's not possible to find more frames toward the outermost frame. Use **unwind-stop-reason-string** to convert the value returned by this function to a string. The value can be one of:

FRAME_UNWIND_NO_REASON

No particular reason (older frames should be available).

FRAME_UNWIND_NULL_ID

The previous frame's analyzer returns an invalid result.

FRAME_UNWIND_OUTERMOST

This frame is the outermost.

FRAME_UNWIND_UNAVAILABLE

Cannot unwind further, because that would require knowing the values of registers or memory that have not been collected.

FRAME_UNWIND_INNER_ID

This frame ID looks like it ought to belong to a **NEXT** frame, but we got it for a **PREV** frame. Normally, this is a sign of unwinder failure. It could also indicate stack corruption.

FRAME_UNWIND_SAME_ID

This frame has the same ID as the previous one. That means that unwinding further would almost certainly give us another frame with exactly the same ID, so break the chain. Normally, this is a sign of unwinder failure. It could also indicate stack corruption.

FRAME_UNWIND_NO_SAVED_PC

The frame unwinder did not find any saved PC, but we needed one to unwind further.

FRAME_UNWIND_MEMORY_ERROR

The frame unwinder caused an error while trying to access memory.

FRAME_UNWIND_FIRST_ERROR

Any stop reason greater or equal to this value indicates some kind of error. This special value facilitates writing code that tests for errors in unwinding in a way that will work correctly even if the list of the other values is modified in future GDB versions. Using it, you could write:

```
(define reason (frame-unwind-stop-reason (selected-frame)))
(define reason-str (unwind-stop-reason-string reason))
(if (>= reason FRAME_UNWIND_FIRST_ERROR)
    (format #t "An error occurred: ~s\n" reason-str))
```

frame-pc *frame* [Scheme Procedure]

Return the frame's resume address.

frame-block *frame* [Scheme Procedure]

Return the frame's code block as a `<gdb:block>` object. See Section 23.4.3.16 [Blocks In Guile], page 572.

frame-function *frame* [Scheme Procedure]

Return the symbol for the function corresponding to this frame as a `<gdb:symbol>` object, or `#f` if there isn't one. See Section 23.4.3.17 [Symbols In Guile], page 574.

frame-older *frame* [Scheme Procedure]

Return the frame that called *frame*.

frame-newer *frame* [Scheme Procedure]

Return the frame called by *frame*.

frame-sal *frame* [Scheme Procedure]

Return the frame's `<gdb:sal>` (syntab and line) object. See Section 23.4.3.18 [Symbol Tables In Guile], page 577.

frame-read-register *frame register* [Scheme Procedure]

Return the value of *register* in *frame*. *register* should be a string, like 'pc'.

frame-read-var *frame variable* [`#:block block`] [Scheme Procedure]

Return the value of *variable* in *frame*. If the optional argument *block* is provided, search for the variable from that block; otherwise start at the frame's current block (which is determined by the frame's current program counter). The *variable* must be given as a string or a `<gdb:symbol>` object, and *block* must be a `<gdb:block>` object.

frame-select *frame* [Scheme Procedure]

Set *frame* to be the selected frame. See Chapter 8 [Examining the Stack], page 115.

selected-frame [Scheme Procedure]

Return the selected frame object. See Section 8.3 [Selecting a Frame], page 119.

newest-frame [Scheme Procedure]

Return the newest frame object for the selected thread.

unwind-stop-reason-string *reason* [Scheme Procedure]

Return a string explaining the reason why GDB stopped unwinding frames, as expressed by the given *reason* code (an integer, see the **frame-unwind-stop-reason** procedure above in this section).

23.4.3.16 Accessing blocks from Guile.

In GDB, symbols are stored in blocks. A block corresponds roughly to a scope in the source code. Blocks are organized hierarchically, and are represented individually in Guile as an object of type `<gdb:block>`. Blocks rely on debugging information being available.

A frame has a block. Please see Section 23.4.3.15 [Frames In Guile], page 569, for a more in-depth discussion of frames.

The outermost block is known as the *global block*. The global block typically holds public global variables and functions.

The block nested just inside the global block is the *static block*. The static block typically holds file-scoped variables and functions.

GDB provides a method to get a block's superblock, but there is currently no way to examine the sub-blocks of a block, or to iterate over all the blocks in a symbol table (see Section 23.4.3.18 [Symbol Tables In Guile], page 577).

Here is a short example that should help explain blocks:

```
/* This is in the global block. */
int global;

/* This is in the static block. */
static int file_scope;

/* 'function' is in the global block, and 'argument' is
   in a block nested inside of 'function'. */
int function (int argument)
{
  /* 'local' is in a block inside 'function'. It may or may
     not be in the same block as 'argument'. */
  int local;

  {
    /* 'inner' is in a block whose superblock is the one holding
       'local'. */
    int inner;

    /* If this call is expanded by the compiler, you may see
       a nested block here whose function is 'inline_function'
       and whose superblock is the one holding 'inner'. */
    inline_function ();
  }
}
```

The following block-related procedures are provided by the `(gdb)` module:

block? *object* [Scheme Procedure]

Return `#t` if *object* is a `<gdb:block>` object. Otherwise return `#f`.

block-valid? *block* [Scheme Procedure]

Returns `#t` if `<gdb:block>` *block* is valid, `#f` if not. A block object can become invalid if the block it refers to doesn't exist anymore in the inferior. All other `<gdb:block>`

methods will throw an exception if it is invalid at the time the procedure is called. The block's validity is also checked during iteration over symbols of the block.

block-start *block* [Scheme Procedure]

Return the start address of `<gdb:block> block`.

block-end *block* [Scheme Procedure]

Return the end address of `<gdb:block> block`.

block-function *block* [Scheme Procedure]

Return the name of `<gdb:block> block` represented as a `<gdb:symbol>` object. If the block is not named, then `#f` is returned.

For ordinary function blocks, the superblock is the static block. However, you should note that it is possible for a function block to have a superblock that is not the static block – for instance this happens for an inlined function.

block-superblock *block* [Scheme Procedure]

Return the block containing `<gdb:block> block`. If the parent block does not exist, then `#f` is returned.

block-global-block *block* [Scheme Procedure]

Return the global block associated with `<gdb:block> block`.

block-static-block *block* [Scheme Procedure]

Return the static block associated with `<gdb:block> block`.

block-global? *block* [Scheme Procedure]

Return `#t` if `<gdb:block> block` is a global block. Otherwise return `#f`.

block-static? *block* [Scheme Procedure]

Return `#t` if `<gdb:block> block` is a static block. Otherwise return `#f`.

block-symbols [Scheme Procedure]

Return a list of all symbols (as `<gdb:symbol>` objects) in `<gdb:block> block`.

make-block-symbols-iterator *block* [Scheme Procedure]

Return an object of type `<gdb:iterator>` that will iterate over all symbols of the block. Guile programs should not assume that a specific block object will always contain a given symbol, since changes in GDB features and infrastructure may cause symbols move across blocks in a symbol table. See Section 23.4.3.25 [Iterators In Guile], page 587.

block-symbols-progress? [Scheme Procedure]

Return `#t` if the object is a `<gdb:block-symbols-progress>` object. This object would be obtained from the `progress` element of the `<gdb:iterator>` object returned by `make-block-symbols-iterator`.

lookup-block *pc* [Scheme Procedure]

Return the innermost `<gdb:block>` containing the given *pc* value. If the block cannot be found for the *pc* value specified, the function will return `#f`.

23.4.3.17 Guile representation of Symbols.

GDB represents every variable, function and type as an entry in a symbol table. See Chapter 16 [Examining the Symbol Table], page 265. Guile represents these symbols in GDB with the `<gdb:symbol>` object.

The following symbol-related procedures are provided by the `(gdb)` module:

`symbol? object` [Scheme Procedure]

Return `#t` if *object* is an object of type `<gdb:symbol>`. Otherwise return `#f`.

`symbol-valid? symbol` [Scheme Procedure]

Return `#t` if the `<gdb:symbol>` object is valid, `#f` if not. A `<gdb:symbol>` object can become invalid if the symbol it refers to does not exist in GDB any longer. All other `<gdb:symbol>` procedures will throw an exception if it is invalid at the time the procedure is called.

`symbol-type symbol` [Scheme Procedure]

Return the type of *symbol* or `#f` if no type is recorded. The result is an object of type `<gdb:type>`. See Section 23.4.3.7 [Types In Guile], page 551.

`symbol-symtab symbol` [Scheme Procedure]

Return the symbol table in which *symbol* appears. The result is an object of type `<gdb:symtab>`. See Section 23.4.3.18 [Symbol Tables In Guile], page 577.

`symbol-line symbol` [Scheme Procedure]

Return the line number in the source code at which *symbol* was defined. This is an integer.

`symbol-name symbol` [Scheme Procedure]

Return the name of *symbol* as a string.

`symbol-linkage-name symbol` [Scheme Procedure]

Return the name of *symbol*, as used by the linker (i.e., may be mangled).

`symbol-print-name symbol` [Scheme Procedure]

Return the name of *symbol* in a form suitable for output. This is either `name` or `linkage_name`, depending on whether the user asked GDB to display demangled or mangled names.

`symbol-addr-class symbol` [Scheme Procedure]

Return the address class of the symbol. This classifies how to find the value of a symbol. Each address class is a constant defined in the `(gdb)` module and described later in this chapter.

`symbol-needs-frame? symbol` [Scheme Procedure]

Return `#t` if evaluating *symbol*'s value requires a frame (see Section 23.4.3.15 [Frames In Guile], page 569) and `#f` otherwise. Typically, local variables will require a frame, but other symbols will not.

`symbol-argument? symbol` [Scheme Procedure]

Return `#t` if *symbol* is an argument of a function. Otherwise return `#f`.

symbol-constant? *symbol* [Scheme Procedure]

Return **#t** if *symbol* is a constant. Otherwise return **#f**.

symbol-function? *symbol* [Scheme Procedure]

Return **#t** if *symbol* is a function or a method. Otherwise return **#f**.

symbol-variable? *symbol* [Scheme Procedure]

Return **#t** if *symbol* is a variable. Otherwise return **#f**.

symbol-value *symbol* [**#:frame** *frame*] [Scheme Procedure]

Compute the value of *symbol*, as a **<gdb:value>**. For functions, this computes the address of the function, cast to the appropriate type. If the symbol requires a frame in order to compute its value, then *frame* must be given. If *frame* is not given, or if *frame* is invalid, then an exception is thrown.

lookup-symbol *name* [**#:block** *block*] [**#:domain** *domain*] [Scheme Procedure]

This function searches for a symbol by name. The search scope can be restricted to the parameters defined in the optional domain and block arguments.

name is the name of the symbol. It must be a string. The optional *block* argument restricts the search to symbols visible in that *block*. The *block* argument must be a **<gdb:block>** object. If omitted, the block for the current frame is used. The optional *domain* argument restricts the search to the domain type. The *domain* argument must be a domain constant defined in the (gdb) module and described later in this chapter.

The result is a list of two elements. The first element is a **<gdb:symbol>** object or **#f** if the symbol is not found. If the symbol is found, the second element is **#t** if the symbol is a field of a method's object (e.g., **this** in C++), otherwise it is **#f**. If the symbol is not found, the second element is **#f**.

lookup-global-symbol *name* [**#:domain** *domain*] [Scheme Procedure]

This function searches for a global symbol by name. The search scope can be restricted by the domain argument.

name is the name of the symbol. It must be a string. The optional *domain* argument restricts the search to the domain type. The *domain* argument must be a domain constant defined in the (gdb) module and described later in this chapter.

The result is a **<gdb:symbol>** object or **#f** if the symbol is not found.

The available domain categories in **<gdb:symbol>** are represented as constants in the (gdb) module:

SYMBOL_UNDEF_DOMAIN

This is used when a domain has not been discovered or none of the following domains apply. This usually indicates an error either in the symbol information or in GDB's handling of symbols.

SYMBOL_VAR_DOMAIN

This domain contains variables, function names, typedef names and enum type values.

SYMBOL_FUNCTION_DOMAIN

This domain contains functions.

SYMBOL_TYPE_DOMAIN

This domain contains types. In a C-like language, types using a tag (the name appearing after a **struct**, **union**, or **enum** keyword) will not appear here; in other languages, all types are in this domain.

SYMBOL_STRUCT_DOMAIN

This domain holds struct, union and enum tag names. This domain is only used for C-like languages. For example, in this code:

```
struct type_one { int x; };
typedef struct type_one type_two;
```

Here **type_one** will be in **SYMBOL_STRUCT_DOMAIN**, but **type_two** will be in **SYMBOL_TYPE_DOMAIN**.

SYMBOL_LABEL_DOMAIN

This domain contains names of labels (for **gotos**).

SYMBOL_VARIABLES_DOMAIN

This domain holds a subset of the **SYMBOLS_VAR_DOMAIN**; it contains everything minus functions and types.

SYMBOL_FUNCTIONS_DOMAIN

This domain contains all functions.

SYMBOL_TYPES_DOMAIN

This domain contains all types.

The available address class categories in `<gdb:symbol>` are represented as constants in the **gdb** module:

When searching for a symbol, the desired domain constant can be passed verbatim to the lookup function.

For more complex searches, there is a corresponding set of constants, each named after one of the preceding constants, but with the **‘SEARCH’** prefix replacing the **‘SYMBOL’** prefix; for example, **SEARCH_LABEL_DOMAIN**. These may be or'd together to form a search constant.

SYMBOL_LOC_UNDEF

If this is returned by address class, it indicates an error either in the symbol information or in GDB's handling of symbols.

SYMBOL_LOC_CONST

Value is constant int.

SYMBOL_LOC_STATIC

Value is at a fixed address.

SYMBOL_LOC_REGISTER

Value is in a register.

SYMBOL_LOC_ARG

Value is an argument. This value is at the offset stored within the symbol inside the frame's argument list.

SYMBOL_LOC_REF_ARG

Value address is stored in the frame's argument list. Just like **LOC_ARG** except that the value's address is stored at the offset, not the value itself.

SYMBOL_LOC_REGPARM_ADDR

Value is a specified register. Just like **LOC_REGISTER** except the register holds the address of the argument instead of the argument itself.

SYMBOL_LOC_LOCAL

Value is a local variable.

SYMBOL_LOC_TYPEDEF

Value not used. Symbols in the domain **SYMBOL_STRUCT_DOMAIN** all have this class.

SYMBOL_LOC_BLOCK

Value is a block.

SYMBOL_LOC_CONST_BYTES

Value is a byte-sequence.

SYMBOL_LOC_UNRESOLVED

Value is at a fixed address, but the address of the variable has to be determined from the minimal symbol table whenever the variable is referenced.

SYMBOL_LOC_OPTIMIZED_OUT

The value does not actually exist in the program.

SYMBOL_LOC_COMPUTED

The value's address is a computed location.

23.4.3.18 Symbol table representation in Guile.

Access to symbol table data maintained by GDB on the inferior is exposed to Guile via two objects: **<gdb:sal>** (syntab-and-line) and **<gdb:symtab>**. Symbol table and line data for a frame is returned from the **frame-find-sal** **<gdb:frame>** procedure. See Section 23.4.3.15 [Frames In Guile], page 569.

For more information on GDB's symbol table management, see Chapter 16 [Examining the Symbol Table], page 265.

The following syntab-related procedures are provided by the (**gdb**) module:

syntab? *object* [Scheme Procedure]

Return **#t** if *object* is an object of type **<gdb:symtab>**. Otherwise return **#f**.

syntab-valid? *syntab* [Scheme Procedure]

Return **#t** if the **<gdb:symtab>** object is valid, **#f** if not. A **<gdb:symtab>** object becomes invalid when the symbol table it refers to no longer exists in GDB. All other **<gdb:symtab>** procedures will throw an exception if it is invalid at the time the procedure is called.

syntab-filename *syntab* [Scheme Procedure]

Return the symbol table's source filename.

syntab-fullname *syntab* [Scheme Procedure]

Return the symbol table's source absolute file name.

syntab-objfile *syntab* [Scheme Procedure]
 Return the symbol table's backing object file. See Section 23.4.3.14 [Objfiles In Guile], page 568.

syntab-global-block *syntab* [Scheme Procedure]
 Return the global block of the underlying symbol table. See Section 23.4.3.16 [Blocks In Guile], page 572.

syntab-static-block *syntab* [Scheme Procedure]
 Return the static block of the underlying symbol table. See Section 23.4.3.16 [Blocks In Guile], page 572.

The following syntab-and-line-related procedures are provided by the (gdb) module:

sal? *object* [Scheme Procedure]
 Return **#t** if *object* is an object of type <gdb:sal>. Otherwise return **#f**.

sal-valid? *sal* [Scheme Procedure]
 Return **#t** if *sal* is valid, **#f** if not. A <gdb:sal> object becomes invalid when the Symbol table object it refers to no longer exists in GDB. All other <gdb:sal> procedures will throw an exception if it is invalid at the time the procedure is called.

sal-syntab *sal* [Scheme Procedure]
 Return the symbol table object (<gdb:syntab>) for *sal*.

sal-line *sal* [Scheme Procedure]
 Return the line number for *sal*.

sal-pc *sal* [Scheme Procedure]
 Return the start of the address range occupied by code for *sal*.

sal-last *sal* [Scheme Procedure]
 Return the end of the address range occupied by code for *sal*.

find-pc-line *pc* [Scheme Procedure]
 Return the <gdb:sal> object corresponding to the *pc* value. If an invalid value of *pc* is passed as an argument, then the **syntab** and **line** attributes of the returned <gdb:sal> object will be **#f** and 0 respectively.

23.4.3.19 Manipulating breakpoints using Guile

Breakpoints in Guile are represented by objects of type <gdb:breakpoint>. New breakpoints can be created with the **make-breakpoint** Guile function, and then added to GDB with the **register-breakpoint!** Guile function. This two-step approach is taken to separate out the side-effect of adding the breakpoint to GDB from **make-breakpoint**.

Support is also provided to view and manipulate breakpoints created outside of Guile.

The following breakpoint-related procedures are provided by the (gdb) module:

make-breakpoint *location* [*#:type type*] [Scheme Procedure]
 [*#:wp-class wp-class*] [*#:internal internal*] [*#:temporary temporary*]

Create a new breakpoint at *location*, a string naming the location of the breakpoint, or an expression that defines a watchpoint. The contents can be any location recognized by the **break** command, or in the case of a watchpoint, by the **watch** command.

The breakpoint is initially marked as ‘invalid’. The breakpoint is not usable until it has been registered with GDB with **register-breakpoint!**, at which point it becomes ‘valid’. The result is the `<gdb:breakpoint>` object representing the breakpoint.

The optional *type* denotes the breakpoint to create. This argument can be either `BP_BREAKPOINT` or `BP_WATCHPOINT`, and defaults to `BP_BREAKPOINT`.

The optional *wp-class* argument defines the class of watchpoint to create, if *type* is `BP_WATCHPOINT`. If a watchpoint class is not provided, it is assumed to be a `WP_WRITE` class.

The optional *internal* argument allows the breakpoint to become invisible to the user. The breakpoint will neither be reported when registered, nor will it be listed in the output from **info breakpoints** (but will be listed with the **maint info breakpoints** command). If an internal flag is not provided, the breakpoint is visible (non-internal).

The optional *temporary* argument makes the breakpoint a temporary breakpoint. Temporary breakpoints are deleted after they have been hit, after which the Guile breakpoint is no longer usable (although it may be re-registered with **register-breakpoint!**).

When a watchpoint is created, GDB will try to create a hardware assisted watchpoint. If successful, the type of the watchpoint is changed from `BP_WATCHPOINT` to `BP_HARDWARE_WATCHPOINT` for `WP_WRITE`, `BP_READ_WATCHPOINT` for `WP_READ`, and `BP_ACCESS_WATCHPOINT` for `WP_ACCESS`. If not successful, the type of the watchpoint is left as `WP_WATCHPOINT`.

The available types are represented by constants defined in the `gdb` module:

`BP_BREAKPOINT`

Normal code breakpoint.

`BP_WATCHPOINT`

Watchpoint breakpoint.

`BP_HARDWARE_WATCHPOINT`

Hardware assisted watchpoint. This value cannot be specified when creating the breakpoint.

`BP_READ_WATCHPOINT`

Hardware assisted read watchpoint. This value cannot be specified when creating the breakpoint.

`BP_ACCESS_WATCHPOINT`

Hardware assisted access watchpoint. This value cannot be specified when creating the breakpoint.

`BP_CATCHPOINT`

Catchpoint. This value cannot be specified when creating the breakpoint.

The available watchpoint types are represented by constants defined in the `(gdb)` module:

`WP_READ` Read only watchpoint.

`WP_WRITE` Write only watchpoint.

`WP_ACCESS`
 Read/Write watchpoint.

register-breakpoint! *breakpoint* [Scheme Procedure]

Add *breakpoint*, a `<gdb:breakpoint>` object, to GDB's list of breakpoints. The breakpoint must have been created with `make-breakpoint`. One cannot register breakpoints that have been created outside of Guile. Once a breakpoint is registered it becomes 'valid'. It is an error to register an already registered breakpoint. The result is unspecified.

delete-breakpoint! *breakpoint* [Scheme Procedure]

Remove *breakpoint* from GDB's list of breakpoints. This also invalidates the Guile *breakpoint* object. Any further attempt to access the object will throw an exception. If *breakpoint* was created from Guile with `make-breakpoint` it may be re-registered with GDB, in which case the breakpoint becomes valid again.

breakpoints [Scheme Procedure]

Return a list of all breakpoints. Each element of the list is a `<gdb:breakpoint>` object.

breakpoint? *object* [Scheme Procedure]

Return `#t` if *object* is a `<gdb:breakpoint>` object, and `#f` otherwise.

breakpoint-valid? *breakpoint* [Scheme Procedure]

Return `#t` if *breakpoint* is valid, `#f` otherwise. Breakpoints created with `make-breakpoint` are marked as invalid until they are registered with GDB with `register-breakpoint!`. A `<gdb:breakpoint>` object can become invalid if the user deletes the breakpoint. In this case, the object still exists, but the underlying breakpoint does not. In the cases of watchpoint scope, the watchpoint remains valid even if execution of the inferior leaves the scope of that watchpoint.

breakpoint-number *breakpoint* [Scheme Procedure]

Return the breakpoint's number — the identifier used by the user to manipulate the breakpoint.

breakpoint-temporary? *breakpoint* [Scheme Procedure]

Return `#t` if the breakpoint was created as a temporary breakpoint. Temporary breakpoints are automatically deleted after they've been hit. Calling this procedure, and all other procedures other than `breakpoint-valid?` and `register-breakpoint!`, will result in an error after the breakpoint has been hit (since it has been automatically deleted).

breakpoint-type *breakpoint* [Scheme Procedure]

Return the breakpoint's type — the identifier used to determine the actual breakpoint type or use-case.

- breakpoint-visible?** *breakpoint* [Scheme Procedure]
 Return **#t** if the breakpoint is visible to the user when hit, or when the ‘**info breakpoints**’ command is run. Otherwise return **#f**.
- breakpoint-location** *breakpoint* [Scheme Procedure]
 Return the location of the breakpoint, as specified by the user. It is a string. If the breakpoint does not have a location (that is, it is a watchpoint) return **#f**.
- breakpoint-expression** *breakpoint* [Scheme Procedure]
 Return the breakpoint expression, as specified by the user. It is a string. If the breakpoint does not have an expression (the breakpoint is not a watchpoint) return **#f**.
- breakpoint-enabled?** *breakpoint* [Scheme Procedure]
 Return **#t** if the breakpoint is enabled, and **#f** otherwise.
- set-breakpoint-enabled!** *breakpoint flag* [Scheme Procedure]
 Set the enabled state of *breakpoint* to *flag*. If *flag* is **#f** it is disabled, otherwise it is enabled.
- breakpoint-silent?** *breakpoint* [Scheme Procedure]
 Return **#t** if the breakpoint is silent, and **#f** otherwise.
 Note that a breakpoint can also be silent if it has commands and the first command is **silent**. This is not reported by the **silent** attribute.
- set-breakpoint-silent!** *breakpoint flag* [Scheme Procedure]
 Set the silent state of *breakpoint* to *flag*. If *flag* is **#f** the breakpoint is made silent, otherwise it is made non-silent (or noisy).
- breakpoint-ignore-count** *breakpoint* [Scheme Procedure]
 Return the ignore count for *breakpoint*.
- set-breakpoint-ignore-count!** *breakpoint count* [Scheme Procedure]
 Set the ignore count for *breakpoint* to *count*.
- breakpoint-hit-count** *breakpoint* [Scheme Procedure]
 Return hit count of *breakpoint*.
- set-breakpoint-hit-count!** *breakpoint count* [Scheme Procedure]
 Set the hit count of *breakpoint* to *count*. At present, *count* must be zero.
- breakpoint-thread** *breakpoint* [Scheme Procedure]
 Return the global-thread-id for thread-specific breakpoint *breakpoint*. Return **#f** if *breakpoint* is not thread-specific.
- set-breakpoint-thread!** *breakpoint global-thread-id* | **#f** [Scheme Procedure]
 Set the thread-id for *breakpoint* to *global-thread-id*. If set to **#f**, the breakpoint is no longer thread-specific.
- breakpoint-task** *breakpoint* [Scheme Procedure]
 If the breakpoint is Ada task-specific, return the Ada task id. If the breakpoint is not task-specific (or the underlying language is not Ada), return **#f**.

set-breakpoint-task! *breakpoint task* [Scheme Procedure]
 Set the Ada task of *breakpoint* to *task*. If set to **#f**, the breakpoint is no longer task-specific.

breakpoint-condition *breakpoint* [Scheme Procedure]
 Return the condition of *breakpoint*, as specified by the user. It is a string. If there is no condition, return **#f**.

set-breakpoint-condition! *breakpoint condition* [Scheme Procedure]
 Set the condition of *breakpoint* to *condition*, which must be a string. If set to **#f** then the breakpoint becomes unconditional.

breakpoint-stop *breakpoint* [Scheme Procedure]
 Return the stop predicate of *breakpoint*. See **set-breakpoint-stop!** below in this section.

set-breakpoint-stop! *breakpoint procedure* | **#f** [Scheme Procedure]
 Set the stop predicate of *breakpoint*. The predicate *procedure* takes one argument: the `<gdb:breakpoint>` object. If this predicate is set to a procedure then it is invoked whenever the inferior reaches this breakpoint. If it returns **#t**, or any non-**#f** value, then the inferior is stopped, otherwise the inferior will continue.

If there are multiple breakpoints at the same location with a **stop** predicate, each one will be called regardless of the return status of the previous. This ensures that all **stop** predicates have a chance to execute at that location. In this scenario if one of the methods returns **#t** but the others return **#f**, the inferior will still be stopped. You should not alter the execution state of the inferior (i.e., step, next, etc.), alter the current frame context (i.e., change the current active frame), or alter, add or delete any breakpoint. As a general rule, you should not alter any data within GDB or the inferior at this time.

Example **stop** implementation:

```
(define (my-stop? bkpt)
  (let ((int-val (parse-and-eval "foo")))
    (value=? int-val 3)))
(define bkpt (make-breakpoint "main.c:42"))
(register-breakpoint! bkpt)
(set-breakpoint-stop! bkpt my-stop?)
```

breakpoint-commands *breakpoint* [Scheme Procedure]
 Return the commands attached to *breakpoint* as a string, or **#f** if there are none.

23.4.3.20 Guile representation of lazy strings.

A *lazy string* is a string whose contents is not retrieved or encoded until it is needed.

A `<gdb:lazy-string>` is represented in GDB as an **address** that points to a region of memory, an **encoding** that will be used to encode that region of memory, and a **length** to delimit the region of memory that represents the string. The difference between a `<gdb:lazy-string>` and a string wrapped within a `<gdb:value>` is that a `<gdb:lazy-string>` will be treated differently by GDB when printing. A `<gdb:lazy-string>` is retrieved and encoded during printing, while a `<gdb:value>` wrapping a string is immediately retrieved and encoded on creation.

The following lazy-string-related procedures are provided by the `(gdb)` module:

lazy-string? *object* [Scheme Procedure]
Return **#t** if *object* is an object of type `<gdb:lazy-string>`. Otherwise return **#f**.

lazy-string-address *lazy-string* [Scheme Procedure]
Return the address of *lazy-string*.

lazy-string-length *lazy-string* [Scheme Procedure]
Return the length of *lazy-string* in characters. If the length is -1, then the string will be fetched and encoded up to the first null of appropriate width.

lazy-string-encoding *lazy-string* [Scheme Procedure]
Return the encoding that will be applied to *lazy-string* when the string is printed by GDB. If the encoding is not set, or contains an empty string, then GDB will select the most appropriate encoding when the string is printed.

lazy-string-type *lazy-string* [Scheme Procedure]
Return the type that is represented by *lazy-string*'s type. For a lazy string this is a pointer or array type. To resolve this to the lazy string's character type, use **type-target-type**. See Section 23.4.3.7 [Types In Guile], page 551.

lazy-string->value *lazy-string* [Scheme Procedure]
Convert the `<gdb:lazy-string>` to a `<gdb:value>`. This value will point to the string in memory, but will lose all the delayed retrieval, encoding and handling that GDB applies to a `<gdb:lazy-string>`.

23.4.3.21 Guile representation of architectures

GDB uses architecture specific parameters and artifacts in a number of its various computations. An architecture is represented by an instance of the `<gdb:arch>` class.

The following architecture-related procedures are provided by the `(gdb)` module:

arch? *object* [Scheme Procedure]
Return **#t** if *object* is an object of type `<gdb:arch>`. Otherwise return **#f**.

current-arch [Scheme Procedure]
Return the current architecture as a `<gdb:arch>` object.

arch-name *arch* [Scheme Procedure]
Return the name (string value) of `<gdb:arch>` *arch*.

arch-charset *arch* [Scheme Procedure]
Return name of target character set of `<gdb:arch>` *arch*.

arch-wide-charset [Scheme Procedure]
Return name of target wide character set of `<gdb:arch>` *arch*.

Each architecture provides a set of predefined types, obtained by the following functions.

arch-void-type *arch* [Scheme Procedure]
Return the `<gdb:type>` object for a void type of architecture *arch*.

- `arch-char-type` *arch* [Scheme Procedure]
Return the `<gdb:type>` object for a `char` type of architecture *arch*.
- `arch-short-type` *arch* [Scheme Procedure]
Return the `<gdb:type>` object for a `short` type of architecture *arch*.
- `arch-int-type` *arch* [Scheme Procedure]
Return the `<gdb:type>` object for an `int` type of architecture *arch*.
- `arch-long-type` *arch* [Scheme Procedure]
Return the `<gdb:type>` object for a `long` type of architecture *arch*.
- `arch-schar-type` *arch* [Scheme Procedure]
Return the `<gdb:type>` object for a `signed char` type of architecture *arch*.
- `arch-uchar-type` *arch* [Scheme Procedure]
Return the `<gdb:type>` object for an `unsigned char` type of architecture *arch*.
- `arch-ushort-type` *arch* [Scheme Procedure]
Return the `<gdb:type>` object for an `unsigned short` type of architecture *arch*.
- `arch-uint-type` *arch* [Scheme Procedure]
Return the `<gdb:type>` object for an `unsigned int` type of architecture *arch*.
- `arch-ulong-type` *arch* [Scheme Procedure]
Return the `<gdb:type>` object for an `unsigned long` type of architecture *arch*.
- `arch-float-type` *arch* [Scheme Procedure]
Return the `<gdb:type>` object for a `float` type of architecture *arch*.
- `arch-double-type` *arch* [Scheme Procedure]
Return the `<gdb:type>` object for a `double` type of architecture *arch*.
- `arch-longdouble-type` *arch* [Scheme Procedure]
Return the `<gdb:type>` object for a `long double` type of architecture *arch*.
- `arch-bool-type` *arch* [Scheme Procedure]
Return the `<gdb:type>` object for a `bool` type of architecture *arch*.
- `arch-longlong-type` *arch* [Scheme Procedure]
Return the `<gdb:type>` object for a `long long` type of architecture *arch*.
- `arch-ulonglong-type` *arch* [Scheme Procedure]
Return the `<gdb:type>` object for an `unsigned long long` type of architecture *arch*.
- `arch-int8-type` *arch* [Scheme Procedure]
Return the `<gdb:type>` object for an `int8` type of architecture *arch*.
- `arch-uint8-type` *arch* [Scheme Procedure]
Return the `<gdb:type>` object for a `uint8` type of architecture *arch*.
- `arch-int16-type` *arch* [Scheme Procedure]
Return the `<gdb:type>` object for an `int16` type of architecture *arch*.

arch-uint16-type *arch* [Scheme Procedure]
 Return the <gdb:type> object for a uint16 type of architecture *arch*.

arch-int32-type *arch* [Scheme Procedure]
 Return the <gdb:type> object for an int32 type of architecture *arch*.

arch-uint32-type *arch* [Scheme Procedure]
 Return the <gdb:type> object for a uint32 type of architecture *arch*.

arch-int64-type *arch* [Scheme Procedure]
 Return the <gdb:type> object for an int64 type of architecture *arch*.

arch-uint64-type *arch* [Scheme Procedure]
 Return the <gdb:type> object for a uint64 type of architecture *arch*.

Example:

```
(gdb) guile (type-name (arch-uchar-type (current-arch)))
"unsigned char"
```

23.4.3.22 Disassembly In Guile

The disassembler can be invoked from Scheme code. Furthermore, the disassembler can take a Guile port as input, allowing one to disassemble from any source, and not just target memory.

arch-disassemble *arch start-pc* [#:port *port*] [#:offset *offset*] [#:size *size*] [#:count *count*] [Scheme Procedure]

Return a list of disassembled instructions starting from the memory address *start-pc*.

The optional argument *port* specifies the input port to read bytes from. If *port* is *#f* then bytes are read from target memory.

The optional argument *offset* specifies the address offset of the first byte in *port*. This is useful, for example, when *port* specifies a ‘bytevector’ and you want the bytevector to be disassembled as if it came from that address. The *start-pc* passed to the reader for *port* is offset by the same amount.

Example:

```
(gdb) guile (use-modules (rnrs io ports))
(gdb) guile (define pc (value->integer (parse-and-eval "$pc")))
(gdb) guile (define mem (open-memory #:start pc))
(gdb) guile (define bv (get-bytevector-n mem 10))
(gdb) guile (define bv-port (open-bytevector-input-port bv))
(gdb) guile (define arch (current-arch))
(gdb) guile (arch-disassemble arch pc #:port bv-port #:offset pc)
(((address . 4195516) (asm . "mov    $0x4005c8,%edi") (length . 5)))
```

The optional arguments *size* and *count* determine the number of instructions in the returned list. If either *size* or *count* is specified as zero, then no instructions are disassembled and an empty list is returned. If both the optional arguments *size* and *count* are specified, then a list of at most *count* disassembled instructions whose start address falls in the closed memory address interval from *start-pc* to (*start-pc* + *size* - 1) are returned. If *size* is not specified, but *count* is specified, then *count* number of instructions starting from the address *start-pc* are returned. If *count* is

not specified but *size* is specified, then all instructions whose start address falls in the closed memory address interval from *start-pc* to (*start-pc* + *size* - 1) are returned. If neither *size* nor *count* are specified, then a single instruction at *start-pc* is returned. Each element of the returned list is an alist (associative list) with the following keys:

address	The value corresponding to this key is a Guile integer of the memory address of the instruction.
asm	The value corresponding to this key is a string value which represents the instruction with assembly language mnemonics. The assembly language flavor used is the same as that specified by the current CLI variable <code>disassembly-flavor</code> . See Section 9.6 [Machine Code], page 137.
length	The value corresponding to this key is the length of the instruction in bytes.

23.4.3.23 I/O Ports in Guile

input-port [Scheme Procedure]
Return GDB's input port as a Guile port object.

output-port [Scheme Procedure]
Return GDB's output port as a Guile port object.

error-port [Scheme Procedure]
Return GDB's error port as a Guile port object.

stdio-port? *object* [Scheme Procedure]
Return `#t` if *object* is a GDB stdio port. Otherwise return `#f`.

23.4.3.24 Memory Ports in Guile

GDB provides a `port` interface to target memory. This allows Guile code to read/write target memory using Guile's port and bytevector functionality. The main routine is `open-memory` which returns a port object. One can then read/write memory using that object.

open-memory [*#:mode mode*] [*#:start address*] [*#:size size*] [Scheme Procedure]
Return a port object that can be used for reading and writing memory. The port will be open according to *mode*, which is the standard mode argument to Guile port open routines, except that the `"a"` and `"l"` modes are not supported. See Section "File Ports" in *GNU Guile Reference Manual*. The `"b"` (binary) character may be present, but is ignored: memory ports are binary only. If `"0"` is appended then the port is marked as unbuffered. The default is `"r"`, read-only and buffered.

The chunk of memory that can be accessed can be bounded. If both *start* and *size* are unspecified, all of memory can be accessed. If only *start* is specified, all of memory from that point on can be accessed. If only *size* is specified, all memory in the range `[0,size)` can be accessed. If both are specified, all memory in the range `[start,start+size)` can be accessed.

memory-port? [Scheme Procedure]
Return `#t` if *object* is an object of type `<gdb:memory-port>`. Otherwise return `#f`.

memory-port-range *memory-port* [Scheme Procedure]

Return the range of `<gdb:memory-port> memory-port` as a list of two elements: `(start end)`. The range is *start* to *end* inclusive.

A memory port is closed like any other port, with `close-port`.

Combined with Guile's `bytevectors`, memory ports provide a lot of utility. For example, to fill a buffer of 10 integers in memory, one can do something like the following.

```
;; In the program: int buffer[10];
(use-modules (rnrs bytevectors))
(use-modules (rnrs io ports))
(define addr (parse-and-eval "buffer"))
(define n 10)
(define byte-size (* n 4))
(define mem-port (open-memory #:mode "r+" #:start
                             (value->integer addr) #:size byte-size))
(define byte-vec (make-bytevector byte-size))
(do ((i 0 (+ i 1)))
    ((>= i n)
     (bytevector-s32-native-set! byte-vec (* i 4) (* i 42)))
    (put-bytevector mem-port byte-vec)
    (close-port mem-port))
```

23.4.3.25 Iterators In Guile

A simple iterator facility is provided to allow, for example, iterating over the set of program symbols without having to first construct a list of all of them. A useful contribution would be to add support for SRFI 41 and SRFI 45.

make-iterator *object progress next!* [Scheme Procedure]

A `<gdb:iterator>` object is constructed with the `make-iterator` procedure. It takes three arguments: the object to be iterated over, an object to record the progress of the iteration, and a procedure to return the next element in the iteration, or an implementation chosen value to denote the end of iteration.

By convention, end of iteration is marked with `(end-of-iteration)`, and may be tested with the `end-of-iteration?` predicate. The result of `(end-of-iteration)` is chosen so that it is not otherwise used by the `(gdb)` module. If you are using `<gdb:iterator>` in your own code it is your responsibility to maintain this invariant.

A trivial example for illustration's sake:

```
(use-modules (gdb iterator))
(define my-list (list 1 2 3))
(define iter
  (make-iterator my-list my-list
    (lambda (iter)
      (let ((l (iterator-progress iter)))
        (if (eq? l '())
            (end-of-iteration)
            (begin
              (set-iterator-progress! iter (cdr l))
              (car l)))))))
```

Here is a slightly more realistic example, which computes a list of all the functions in `my-global-block`.

```
(use-modules (gdb iterator))
```

```
(define this-sal (find-pc-line (frame-pc (selected-frame))))
(define this-symtab (sal-symtab this-sal))
(define this-global-block (symtab-global-block this-symtab))
(define syms-iter (make-block-symbols-iterator this-global-block))
(define functions (iterator-filter symbol-function? syms-iter))
```

iterator? *object* [Scheme Procedure]
Return #t if *object* is a <gdb:iterator> object. Otherwise return #f.

iterator-object *iterator* [Scheme Procedure]
Return the first argument that was passed to **make-iterator**. This is the object being iterated over.

iterator-progress *iterator* [Scheme Procedure]
Return the object tracking iteration progress.

set-iterator-progress! *iterator new-value* [Scheme Procedure]
Set the object tracking iteration progress.

iterator-next! *iterator* [Scheme Procedure]
Invoke the procedure that was the third argument to **make-iterator**, passing it one argument, the <gdb:iterator> object. The result is either the next element in the iteration, or an end marker as implemented by the **next!** procedure. By convention the end marker is the result of (**end-of-iteration**).

end-of-iteration [Scheme Procedure]
Return the Scheme object that denotes end of iteration.

end-of-iteration? *object* [Scheme Procedure]
Return #t if *object* is the end of iteration marker. Otherwise return #f.

These functions are provided by the (gdb iterator) module to assist in using iterators.

make-list-iterator *list* [Scheme Procedure]
Return a <gdb:iterator> object that will iterate over *list*.

iterator->list *iterator* [Scheme Procedure]
Return the elements pointed to by *iterator* as a list.

iterator-map *proc iterator* [Scheme Procedure]
Return the list of objects obtained by applying *proc* to the object pointed to by *iterator* and to each subsequent object.

iterator-for-each *proc iterator* [Scheme Procedure]
Apply *proc* to each element pointed to by *iterator*. The result is unspecified.

iterator-filter *pred iterator* [Scheme Procedure]
Return the list of elements pointed to by *iterator* that satisfy *pred*.

iterator-until *pred iterator* [Scheme Procedure]
Run *iterator* until the result of (*pred element*) is true and return that as the result. Otherwise return #f.

23.4.3.26 Colors In Guile

You can assign instance of `<gdb:color>` to the value of a `<gdb:parameter>` object created with `PARAM_COLOR`.

`<gdb:color>` may refer to an index from color palette or contain components of a color from some colorspace.

make-color [*value* [*#:color-space color-space*]] [Scheme Procedure]
value is an integer index of a color in palette, tuple with color components or a string. String can be a hex RGB triplet in `#RRGGBB` format or one of the following color names: `'none'` (meaning the terminal's default color), `'black'`, `'red'`, `'green'`, `'yellow'`, `'blue'`, `'magenta'`, `'cyan'`, or `'white'`.
color-space should be one of the `'COLORSPACE_'` constants. This argument tells GDB which color space *value* belongs.

color? *object* [Scheme Procedure]
 Return `#t` if *object* is a `<gdb:color>` object. Otherwise return `#f`.

color-none? *color* [Scheme Procedure]
 Return `#t` if *color* is terminal's default. Otherwise return `#f`.

color-indexed? *color* [Scheme Procedure]
 Return `#t` if *color* is indexed, i.e. belongs to some palette. Otherwise return `#f`.

color-direct? *color* [Scheme Procedure]
 Return `#t` if *color* is direct in the sense of ISO/IEC 8613-6. Otherwise return `#f`.

color-string *color* [Scheme Procedure]
 Return the textual representation of a `<gdb:color>` object.

color-colorspace *color* [Scheme Procedure]
 Return the color space of a `<gdb:color>` object.

color-index *color* [Scheme Procedure]
 Return index of the color of a `<gdb:color>` object in a palette.

color-components *color* [Scheme Procedure]
 Return components of the direct `<gdb:color>` object.

color-escape-sequence *color is_foreground* [Scheme Procedure]
 Return string to change terminal's color to this.
 If *is_foreground* is `#t`, then the returned sequence will change foreground color. Otherwise, the returned sequence will change background color.
 If styling is currently disabled (see Section 22.5 [`set style enabled`], page 375), then this procedure will return an empty string.

When color is initialized, its color space must be specified. The available color spaces are represented by constants defined in the `gdb` module:

`COLORSPACE_MONOCHROME`

Palette with only terminal's default color.

COLORSPACE_ANSI_8COLOR

Palette with eight standard colors of ISO/IEC 6429 "black", "red", "green", etc.

COLORSPACE_AIXTERM_16COLOR

Palette with 16 colors. First eight are standard colors of ISO/IEC 6429 "black", "red", "green", etc. Next eight are their bright version.

COLORSPACE_XTERM_256COLOR

Palette with 256 colors. First 16 are from **COLORSPACE_AIXTERM_16COLOR**. Next 216 colors are 6x6x6 RGB cube. And last 24 colors form grayscale ramp.

COLORSPACE_RGB_24BIT

Direct 24-bit RGB colors.

23.4.4 Guile Auto-loading

When a new object file is read (for example, due to the **file** command, or because the inferior has loaded a shared library), GDB will look for Guile support scripts in two ways: *objfile-gdb.scm* and the **.debug_gdb_scripts** section. See Section 23.5 [Auto-loading extensions], page 592.

The auto-loading feature is useful for supplying application-specific debugging commands and scripts.

Auto-loading can be enabled or disabled, and the list of auto-loaded scripts can be printed.

```
set auto-load guile-scripts [on|off]
```

Enable or disable the auto-loading of Guile scripts.

```
show auto-load guile-scripts
```

Show whether auto-loading of Guile scripts is enabled or disabled.

```
info auto-load guile-scripts [regex]
```

Print the list of all Guile scripts that GDB auto-loaded.

Also printed is the list of Guile scripts that were mentioned in the **.debug_gdb_scripts** section and were not found. This is useful because their names are not printed when GDB tries to load them and fails. There may be many of them, and printing an error message for each one is problematic.

If *regex* is supplied only Guile scripts with matching names are printed.

Example:

```
(gdb) info auto-load guile-scripts
Loaded Script
Yes      scm-section-script.scm
         full name: /tmp/scm-section-script.scm
No       my-foo-pretty-printers.scm
```

When reading an auto-loaded file, GDB sets the *current objfile*. This is available via the **current-objfile** procedure (see Section 23.4.3.14 [Objfiles In Guile], page 568). This can be useful for registering objfile-specific pretty-printers.

23.4.5 Guile Modules

GDB comes with several modules to assist writing Guile code.

23.4.5.1 Guile Printing Module

This module provides a collection of utilities for working with pretty-printers.

Usage:

```
(use-modules (gdb printing))
```

prepend-pretty-printer! *object printer* [Scheme Procedure]

Add *printer* to the front of the list of pretty-printers for *object*. The *object* must either be a <gdb:objfile> object, or #f in which case *printer* is added to the global list of printers.

append-pretty-printer! *object printer* [Scheme Procedure]

Add *printer* to the end of the list of pretty-printers for *object*. The *object* must either be a <gdb:objfile> object, or #f in which case *printer* is added to the global list of printers.

23.4.5.2 Guile Types Module

This module provides a collection of utilities for working with <gdb:type> objects.

Usage:

```
(use-modules (gdb types))
```

get-basic-type *type* [Scheme Procedure]

Return *type* with const and volatile qualifiers stripped, and with typedefs and C++ references converted to the underlying type.

C++ example:

```
typedef const int const_int;
const_int foo (3);
const_int& foo_ref (foo);
int main () { return 0; }
```

Then in gdb:

```
(gdb) start
(gdb) guile (use-modules (gdb) (gdb types))
(gdb) guile (define foo-ref (parse-and-eval "foo_ref"))
(gdb) guile (get-basic-type (value-type foo-ref))
#<gdb:type int>
```

type-has-field-deep? *type field* [Scheme Procedure]

Return #t if *type*, assumed to be a type with fields (e.g., a structure or union), has field *field*. Otherwise return #f. This searches baseclasses, whereas **type-has-field?** does not.

make-enum-hashtable *enum-type* [Scheme Procedure]

Return a Guile hash table produced from *enum-type*. Elements in the hash table are referenced with **hashq-ref**.

23.5 Auto-loading extensions

GDB provides two mechanisms for automatically loading extensions when a new object file is read (for example, due to the `file` command, or because the inferior has loaded a shared library): `objfile-gdb.ext` (see Section 23.5.1 [The `objfile-gdb.ext` file], page 592) and the `.debug_gdb_scripts` section of modern file formats like ELF (see Section 23.5.2 [The `.debug_gdb_scripts` section], page 593). For a discussion of the differences between these two approaches see Section 23.5.3 [Which flavor to choose?], page 595.

The auto-loading feature is useful for supplying application-specific debugging commands and features.

Auto-loading can be enabled or disabled, and the list of auto-loaded scripts can be printed. See the ‘`auto-loading`’ section of each extension language for more information. For GDB command files see Section 23.1.5 [Auto-loading sequences], page 405. For Python files see Section 23.3.3 [Python Auto-loading], page 532.

Note that loading of this script file also requires accordingly configured `auto-load safe-path` (see Section 22.8.3 [Auto-loading safe path], page 384).

23.5.1 The `objfile-gdb.ext` file

When a new object file is read, GDB looks for a file named `objfile-gdb.ext` (we call it *script-name* below), where *objfile* is the object file’s name and where *ext* is the file extension for the extension language:

```
objfile-gdb.gdb
    GDB’s own command language

objfile-gdb.py
    Python

objfile-gdb.scm
    Guile
```

script-name is formed by ensuring that the file name of *objfile* is absolute, following all symlinks, and resolving `.` and `..` components, and appending the `-gdb.ext` suffix. If this file exists and is readable, GDB will evaluate it as a script in the specified extension language.

If this file does not exist, then GDB will look for *script-name* file in all of the directories as specified below. (On MS-Windows/MS-DOS, the drive letter of the executable’s leading directories is converted to a one-letter subdirectory, i.e. `d:/usr/bin/` is converted to `/d/usr/bin/`, because Windows filesystems disallow colons in file names.)

Note that loading of these files requires an accordingly configured `auto-load safe-path` (see Section 22.8.3 [Auto-loading safe path], page 384).

For object files using `.exe` suffix GDB tries to load first the scripts normally according to its `.exe` filename. But if no scripts are found GDB also tries script filenames matching the object file without its `.exe` suffix. This `.exe` stripping is case insensitive and it is attempted on any platform. This makes the script filenames compatible between Unix and MS-Windows hosts.

set auto-load scripts-directory [*directories*]

Control GDB auto-loaded scripts location. Multiple directory entries may be delimited by the host platform path separator in use (‘:’ on Unix, ‘;’ on MS-Windows and MS-DOS).

Each entry here needs to be covered also by the security setting **set auto-load safe-path** (see [set auto-load safe-path], page 384).

This variable defaults to `$debugdir:$datadir/auto-load`. The default **set auto-load safe-path** value can be also overridden by GDB configuration option **--with-auto-load-dir**.

Any reference to `$debugdir` will get replaced by *debug-file-directory* value (see Section 18.3 [Separate Debug Files], page 303) and any reference to `$datadir` will get replaced by *data-directory* which is determined at GDB startup (see Section 18.8 [Data Files], page 312). `$debugdir` and `$datadir` must be placed as a directory component — either alone or delimited by / or \ directory separators, depending on the host platform.

The list of directories uses path separator (‘:’ on GNU and Unix systems, ‘;’ on MS-Windows and MS-DOS) to separate directories, similarly to the `PATH` environment variable.

show auto-load scripts-directory

Show GDB auto-loaded scripts location.

add-auto-load-scripts-directory [*directories...*]

Add an entry (or list of entries) to the list of auto-loaded scripts locations. Multiple entries may be delimited by the host platform path separator in use.

GDB does not track which files it has already auto-loaded this way. GDB will load the associated script every time the corresponding *objfile* is opened. So your `-gdb.ext` file should be careful to avoid errors if it is evaluated more than once.

23.5.2 The `.debug_gdb_scripts` section

For systems using file formats like ELF and COFF, when GDB loads a new object file it will look for a special section named `.debug_gdb_scripts`. If this section exists, its contents is a list of null-terminated entries specifying scripts to load. Each entry begins with a non-null prefix byte that specifies the kind of entry, typically the extension language and whether the script is in a file or inlined in `.debug_gdb_scripts`.

The following entries are supported:

```
SECTION_SCRIPT_ID_PYTHON_FILE = 1
SECTION_SCRIPT_ID_SCHEME_FILE = 3
SECTION_SCRIPT_ID_PYTHON_TEXT = 4
SECTION_SCRIPT_ID_SCHEME_TEXT = 6
```

23.5.2.1 Script File Entries

If the entry specifies a file, GDB will look for the file first in the current directory and then along the source search path (see Section 9.5 [Specifying Source Directories], page 133), except that `$cdir` is not searched, since the compilation directory is not relevant to scripts.

File entries can be placed in section `.debug_gdb_scripts` with, for example, this GCC macro for Python scripts.

```
/* Note: The "MS" section flags are to remove duplicates. */
#define DEFINE_GDB_PY_SCRIPT(script_name) \
    asm("\n\
.pushsection \".debug_gdb_scripts\", \"MS\",@progbits,1\n\
.byte 1 /* Python */\n\
.asciz \"" script_name "\"\n\
.popsection\n\
");
```

For Guile scripts, replace `.byte 1` with `.byte 3`. Then one can reference the macro in a header or source file like this:

```
DEFINE_GDB_PY_SCRIPT ("my-app-scripts.py")
```

The script name may include directories if desired.

Note that loading of this script file also requires accordingly configured `auto-load safe-path` (see Section 22.8.3 [Auto-loading safe path], page 384).

If the macro invocation is put in a header, any application or library using this header will get a reference to the specified script, and with the use of "MS" attributes on the section, the linker will remove duplicates.

23.5.2.2 Script Text Entries

Script text entries allow to put the executable script in the entry itself instead of loading it from a file. The first line of the entry, everything after the prefix byte and up to the first newline (0xa) character, is the script name, and must not contain any kind of space character, e.g., spaces or tabs. The rest of the entry, up to the trailing null byte, is the script to execute in the specified language. The name needs to be unique among all script names, as GDB executes each script only once based on its name.

Here is an example from file `py-section-script.c` in the GDB testsuite.

```
#include "symcat.h"
#include "gdb/section-scripts.h"
asm(
".pushsection \".debug_gdb_scripts\", \"MS\",@progbits,1\n"
".byte \" XSTRING (SECTION_SCRIPT_ID_PYTHON_TEXT) \"\n"
".ascii \"gdb.inlined-script\\n\\n\"
".ascii \"class test_cmd (gdb.Command):\\n\\n\"
".ascii \"    def __init__ (self):\\n\\n\"
".ascii \"        super (test_cmd, self).__init__ (\"
        \"\\\"\\\"test-cmd\\\"\\\", gdb.COMMAND_OBSCURE)\\n\\n\"
".ascii \"    def invoke (self, arg, from_tty):\\n\\n\"
".ascii \"        print (\\\"\\\"test-cmd output, arg = %s\\\" % arg)\\n\\n\"
".ascii \"test_cmd ()\\n\\n\"
".byte 0\n"
".popsection\n"
);
```


Loading of inlined scripts requires a properly configured `auto-load safe-path` (see Section 22.8.3 [Auto-loading safe path], page 384). The path to specify in `auto-load safe-path` is the path of the file containing the `.debug_gdb_scripts` section.

23.5.3 Which flavor to choose?

Given the multiple ways of auto-loading extensions, it might not always be clear which one to choose. This section provides some guidance.

Benefits of the `-gdb.ext` way:

- Can be used with file formats that don't support multiple sections.
- Ease of finding scripts for public libraries.

Scripts specified in the `.debug_gdb_scripts` section are searched for in the source search path. For publicly installed libraries, e.g., `libstdc++`, there typically isn't a source directory in which to find the script.

- Doesn't require source code additions.

Benefits of the `.debug_gdb_scripts` way:

- Works with static linking.

Scripts for libraries done the `-gdb.ext` way require an objfile to trigger their loading. When an application is statically linked the only objfile available is the executable, and it is cumbersome to attach all the scripts from all the input libraries to the executable's `-gdb.ext` script.

- Works with classes that are entirely inlined.

Some classes can be entirely inlined, and thus there may not be an associated shared library to attach a `-gdb.ext` script to.

- Scripts needn't be copied out of the source tree.

In some circumstances, apps can be built out of large collections of internal libraries, and the build infrastructure necessary to install the `-gdb.ext` scripts in a place where GDB can find them is cumbersome. It may be easier to specify the scripts in the `.debug_gdb_scripts` section as relative paths, and add a path to the top of the source tree to the source search path.

23.6 Multiple Extension Languages

The Guile and Python extension languages do not share any state, and generally do not interfere with each other. There are some things to be aware of, however.

23.6.1 Python comes first

Python was GDB's first extension language, and to avoid breaking existing behavior Python comes first. This is generally solved by the "first one wins" principle. GDB maintains a list of enabled extension languages, and when it makes a call to an extension language, (say to pretty-print a value), it tries each in turn until an extension language indicates it has performed the request (e.g., has returned the pretty-printed form of a value). This extends to errors while performing such requests: If an error happens while, for example, trying to pretty-print an object then the error is reported and any following extension languages are not tried.

24 Command Interpreters

GDB supports multiple command interpreters, and some command infrastructure to allow users or user interface writers to switch between interpreters or run commands in other interpreters.

GDB currently supports two command interpreters, the console interpreter (sometimes called the command-line interpreter or CLI) and the machine interface interpreter (or GDB/MI). This manual describes both of these interfaces in great detail.

By default, GDB will start with the console interpreter. However, the user may choose to start GDB with another interpreter by specifying the `-i` or `--interpreter` startup options. Defined interpreters include:

console	The traditional console or command-line interpreter. This is the most often used interpreter with GDB. With no interpreter specified at runtime, GDB will use this interpreter.
dap	When GDB has been built with Python support, it also supports the Debugger Adapter Protocol. This protocol can be used by a debugger GUI or an IDE to communicate with GDB. This protocol is documented at https://microsoft.github.io/debug-adapter-protocol/ . See Chapter 29 [Debugger Adapter Protocol], page 719, for information about GDB extensions to the protocol.
mi	The newest GDB/MI interface (currently mi3). Used primarily by programs wishing to use GDB as a backend for a debugger GUI or an IDE. For more information, see Chapter 27 [The GDB/MI Interface], page 609.
mi3	The GDB/MI interface introduced in GDB 9.1.
mi2	The GDB/MI interface introduced in GDB 6.0.

You may execute commands in any interpreter from the current interpreter using the appropriate command. If you are running the console interpreter, simply use the **interpreter-exec** command:

```
interpreter-exec mi "-data-list-register-names"
```

GDB/MI has a similar command, although it is only available in versions of GDB which support GDB/MI version 2 (or greater).

Note that **interpreter-exec** only changes the interpreter for the duration of the specified command. It does not change the interpreter permanently.

Although you may only choose a single interpreter at startup, it is possible to run an independent interpreter on a specified input/output device (usually a tty).

For example, consider a debugger GUI or IDE that wants to provide a GDB console view. It may do so by embedding a terminal emulator widget in its GUI, starting GDB in the traditional command-line mode with stdin/stdout/stderr redirected to that terminal, and then creating an MI interpreter running on a specified input/output device. The console interpreter created by GDB at startup handles commands the user types in the terminal widget, while the GUI controls and synchronizes state with GDB using the separate MI interpreter.

To start a new secondary *user interface* running MI, use the **new-ui** command:

```
new-ui interpreter tty
```

The *interpreter* parameter specifies the interpreter to run. This accepts the same values as the **interpreter-exec** command. For example, `'console'`, `'mi'`, `'mi2'`, etc. The *tty* parameter specifies the name of the bidirectional file the interpreter uses for input/output, usually the name of a pseudoterminal slave on Unix systems. For example:

```
(gdb) new-ui mi /dev/pts/9
```

runs an MI interpreter on `/dev/pts/9`.

25 GDB Text User Interface

The GDB Text User Interface (TUI) is a terminal interface which uses the `curses` library to show the source file, the assembly output, the program registers and GDB commands in separate text windows. The TUI mode is supported only on platforms where a suitable version of the `curses` library is available.

The TUI mode is enabled by default when you invoke GDB as ‘`gdb -tui`’. You can also switch in and out of TUI mode while GDB runs by using various TUI commands and key bindings, such as `tui enable` or `C-x C-a`. See Section 25.5 [TUI Commands], page 602, and Section 25.2 [TUI Key Bindings], page 600.

25.1 TUI Overview

In TUI mode, GDB can display several text windows:

<i>command</i>	This window is the GDB command window with the GDB prompt and the GDB output. The GDB input is still managed using readline.
<i>source</i>	The source window shows the source file of the program. The current line and active breakpoints are displayed in this window.
<i>assembly</i>	The assembly window shows the disassembly output of the program.
<i>register</i>	This window shows the processor registers. Registers are highlighted when their values change.

The source and assembly windows show the current program position by highlighting the current line and marking it with a ‘>’ marker. By default, source and assembly code styling is disabled for the highlighted text, but you can enable it with the `set style tui-current-position on` command. See Section 22.5 [Output Styling], page 375.

Breakpoints are indicated with two markers. The first marker indicates the breakpoint type:

B	Breakpoint which was hit at least once.
b	Breakpoint which was never hit.
H	Hardware breakpoint which was hit at least once.
h	Hardware breakpoint which was never hit.

The second marker indicates whether the breakpoint is enabled or not:

+	Breakpoint is enabled.
-	Breakpoint is disabled.

The source, assembly and register windows are updated when the current thread changes, when the frame changes, or when the program counter changes.

These windows are not all visible at the same time. The command window is always visible. The others can be arranged in several layouts:

- source only,
- assembly only,

- source and assembly,
- source and registers, or
- assembly and registers.

These are the standard layouts, but other layouts can be defined.

A status line above the command window shows the following information:

<i>target</i>	Indicates the current GDB target. (see Chapter 19 [Specifying a Debugging Target], page 313).
<i>process</i>	Gives the current process or thread number. When no process is being debugged, this field is set to No process .
<i>focus</i>	Shows the name of the TUI window that has the focus.
<i>function</i>	Gives the current function name for the selected frame. The name is demangled if demangling is turned on (see Section 10.9 [Print Settings], page 159). When there is no symbol corresponding to the current program counter, the string ?? is displayed.
<i>line</i>	Indicates the current line number for the selected frame. When the current line number is not known, the string ?? is displayed.
<i>pc</i>	Indicates the current program counter address.

25.2 TUI Key Bindings

The TUI installs several key bindings in the readline keymaps (see Chapter 33 [Command Line Editing], page 733). The following key bindings are installed for both TUI mode and the GDB standard mode.

<i>C-x C-a</i>	
<i>C-x a</i>	
<i>C-x A</i>	Enter or leave the TUI mode. When leaving the TUI mode, the curses window management stops and GDB operates using its standard mode, writing on the terminal directly. When reentering the TUI mode, control is given back to the curses windows. The screen is then refreshed. This key binding uses the bindable Readline function tui-switch-mode .
<i>C-x 1</i>	Use a TUI layout with only one window. The layout will either be ‘source’ or ‘assembly’. When the TUI mode is not active, it will switch to the TUI mode. Think of this key binding as the Emacs <i>C-x 1</i> binding. This key binding uses the bindable Readline function tui-delete-other-windows .
<i>C-x 2</i>	Use a TUI layout with at least two windows. When the current layout already has two windows, the next layout with two windows is used. When a new layout is chosen, one window will always be common to the previous layout and the new one. Think of it as the Emacs <i>C-x 2</i> binding. This key binding uses the bindable Readline function tui-change-windows .

- C-x o** Change the active window. The TUI associates several key bindings (like scrolling and arrow keys) with the active window. This command gives the focus to the next TUI window.
- Think of it as the Emacs **C-x o** binding.
- This key binding uses the bindable Readline function **tui-other-window**.
- C-x s** Switch in and out of the TUI SingleKey mode that binds single keys to GDB commands (see Section 25.3 [TUI Single Key Mode], page 601).
- This key binding uses the bindable Readline function **next-keymap**.

The following key bindings only work in the TUI mode:

- PgUp** Scroll the active window one page up.
- PgDn** Scroll the active window one page down.
- Up** Scroll the active window one line up.
- Down** Scroll the active window one line down.
- Left** Scroll the active window one column left.
- Right** Scroll the active window one column right.
- C-L** Refresh the screen.

Because the arrow keys scroll the active window in the TUI mode, they are not available for their normal use by readline unless the command window has the focus. When another window is active, you must use other readline key bindings such as **C-p**, **C-n**, **C-b** and **C-f** to control the command window.

25.3 TUI Single Key Mode

The TUI also provides a *SingleKey* mode, which binds several frequently used GDB commands to single keys. Type **C-x s** to switch into this mode, where the following key bindings are used:

- c** continue
- C** reverse-continue
- d** down
- f** finish
- F** reverse-finish
- n** next
- N** reverse-next
- o** nexti. The shortcut letter ‘o’ stands for “step Over”.
- O** reverse-nexti
- q** exit the SingleKey mode.
- r** run

s	step
S	reverse-step
i	stepi. The shortcut letter ‘i’ stands for “step Into”.
I	reverse-stepi
u	up
v	info locals
w	where

Other keys temporarily switch to the GDB command prompt. The key that was pressed is inserted in the editing buffer so that it is possible to type most GDB commands without interaction with the TUI SingleKey mode. Once the command is entered the TUI SingleKey mode is restored. The only way to permanently leave this mode is by typing **q** or **C-x s**.

If GDB was built with Readline 8.0 or later, the TUI SingleKey keymap will be named ‘SingleKey’. This can be used in `.inputrc` to add additional bindings to this keymap.

25.4 TUI Mouse Support

If the curses library supports the mouse, the TUI supports mouse actions.

The mouse wheel scrolls the appropriate window under the mouse cursor.

The TUI itself does not directly support copying/pasting with the mouse. However, on Unix terminals, you can typically press and hold the **SHIFT** key on your keyboard to temporarily bypass GDB’s TUI and access the terminal’s native mouse copy/paste functionality (commonly, click-drag-release or double-click to select text, middle-click to paste). This copy/paste works with the terminal’s selection buffer, as opposed to the TUI’s buffer. Alternatively, to disable mouse support in the TUI entirely and give the terminal control over mouse clicks, turn off the `tui mouse-events` setting (see [set tui mouse-events], page 606).

Python extensions can react to mouse clicks (see [Window.click], page 517).

25.5 TUI-specific Commands

The TUI has specific commands to control the text windows. These commands are always available, even when GDB is not in the TUI mode. When GDB is in the standard mode, most of these commands will automatically switch to the TUI mode.

Note that if GDB’s `stdout` is not connected to a terminal, or GDB has been started with the machine interface interpreter (see Chapter 27 [The GDB/MI Interface], page 609), most of these commands will fail with an error, because it would not be possible or desirable to enable curses window management.

`tui enable`

Activate TUI mode. The last active TUI window layout will be used if TUI mode has previously been used in the current debugging session, otherwise a default layout is used.

`tui disable`

Disable TUI mode, returning to the console interpreter.

info win List the names and sizes of all currently displayed windows.

tui new-layout *name window weight* [*window weight*...]

Create a new TUI layout. The new layout will be named *name*, and can be accessed using the **layout** command (see below).

Each *window* parameter is either the name of a window to display, or a window description. The windows will be displayed from top to bottom in the order listed.

The names of the windows are the same as the ones given to the **focus** command (see below); additionally, the **status** window can be specified. Note that, because it is of fixed height, the weight assigned to the status window is of no importance. It is conventional to use ‘0’ here.

A window description looks a bit like an invocation of **tui new-layout**, and is of the form `{[-horizontal]window weight [window weight]...}`.

This specifies a sub-layout. If **-horizontal** is given, the windows in this description will be arranged side-by-side, rather than top-to-bottom.

Each *weight* is an integer. It is the weight of this window relative to all the other windows in the layout. These numbers are used to calculate how much of the screen is given to each window.

For example:

```
(gdb) tui new-layout example src 1 regs 1 status 0 cmd 1
```

Here, the new layout is called ‘example’. It shows the source and register windows, followed by the status window, and then finally the command window. The non-status windows all have the same weight, so the terminal will be split into three roughly equal sections.

Here is a more complex example, showing a horizontal layout:

```
(gdb) tui new-layout example {-horizontal src 1 asm 1} 2 status 0 cmd 1
```

This will result in side-by-side source and assembly windows; with the status and command window being beneath these, filling the entire width of the terminal. Because they have weight 2, the source and assembly windows will be twice the height of the command window.

tui layout *name*

layout *name*

Changes which TUI windows are displayed. The *name* parameter controls which layout is shown. It can be either one of the built-in layout names, or the name of a layout defined by the user using **tui new-layout**.

The built-in layouts are as follows:

next	Display the next layout.
prev	Display the previous layout.
src	Display the source and command windows.
asm	Display the assembly and command windows.
split	Display the source, assembly, and command windows.

regs When in **src** layout display the register, source, and command windows. When in **asm** or **split** layout display the register, assembler, and command windows.

tui focus name

focus name

Changes which TUI window is currently active for scrolling. The *name* parameter can be any of the following:

next Make the next window active for scrolling.
prev Make the previous window active for scrolling.
src Make the source window active for scrolling.
asm Make the assembly window active for scrolling.
regs Make the register window active for scrolling.
cmd Make the command window active for scrolling.

tui refresh

refresh Refresh the screen. This is similar to typing *C-L*.

tui reg group

Changes the register group displayed in the tui register window to *group*. If the register window is not currently displayed this command will cause the register window to be displayed. The list of register groups, as well as their order is target specific. The following groups are available on most targets:

next Repeatedly selecting this group will cause the display to cycle through all of the available register groups.
prev Repeatedly selecting this group will cause the display to cycle through all of the available register groups in the reverse order to *next*.
general Display the general registers.
float Display the floating point registers.
system Display the system registers.
vector Display the vector registers.
all Display all registers.

update Update the source window and the current execution point.

tui window height name +count

tui window height name -count

winheight name +count

winheight name -count

Change the height of the window *name* by *count* lines. Positive counts increase the height, while negative counts decrease it. The *name* parameter can be the name of any currently visible window. The names of the currently visible windows can be discovered using *info win* (see [info win], page 602).

The set of currently visible windows must always fill the terminal, and so, it is only possible to resize on window if there are other visible windows that can either give or receive the extra terminal space.

```
tui window width name +count
tui window width name -count
winwidth name +count
winwidth name -count
```

Change the width of the window *name* by *count* columns. Positive counts increase the width, while negative counts decrease it. The *name* parameter can be the name of any currently visible window. The names of the currently visible windows can be discovered using `info win` (see [info win], page 602).

The set of currently visible windows must always fill the terminal, and so, it is only possible to resize on window if there are other visible windows that can either give or receive the extra terminal space.

25.6 TUI Configuration Variables

Several configuration variables control the appearance of TUI windows.

```
set tui border-kind kind
```

Select the border appearance for the source, assembly and register windows. The possible values are the following:

space	Use a space character to draw the border.
ascii	Use ASCII characters '+', '-' and ' ' to draw the border.
acs	Use the Alternate Character Set to draw the border. The border is drawn using character line graphics if the terminal supports them.

```
set tui border-mode mode
```

```
set tui active-border-mode mode
```

Select the display attributes for the borders of the inactive windows or the active window. The *mode* can be one of the following:

normal	Use normal attributes to display the border.
standout	Use standout mode.
reverse	Use reverse video mode.
half	Use half bright mode.
half-standout	Use half bright and standout mode.
bold	Use extra bright or bold mode.
bold-standout	Use extra bright or bold and standout mode.

```
set tui tab-width nchars
```

Set the width of tab stops to be *nchars* characters. This setting affects the display of TAB characters in the source and assembly windows.

set tui compact-source [on|off]

Set whether the TUI source window is displayed in “compact” form. The default display uses more space for line numbers; the compact display uses only as much space as is needed for the line numbers in the current file.

set tui mouse-events [on|off]

When on (default), mouse clicks control the TUI (see Section 25.4 [TUI Mouse Support], page 602). When off, mouse clicks are handled by the terminal, enabling terminal-native text selection.

set debug tui [on|off]

Turn on or off display of GDB internal debug messages relating to the TUI.

show debug tui

Show the current status of displaying GDB internal debug messages relating to the TUI.

Note that the colors of the TUI borders can be controlled using the appropriate **set style** commands. See Section 22.5 [Output Styling], page 375.

26 Using GDB under GNU Emacs

In GNU Emacs there is a special interface to GDB, which facilitates viewing the source code for the program you are debugging. There is also an IDE-like interface to GDB, with specialized buffers for breakpoints, stack frames and other aspects of the debugger state.

A detailed description of Emacs' interaction with GDB is given in the Emacs manual (see Section “Debuggers” in *The GNU Emacs Manual*).

27 The GDB/MI Interface

Function and Purpose

GDB/MI is a line based machine oriented text interface to GDB and is activated by specifying using the `--interpreter` command line option (see Section 2.1.2 [Mode Options], page 13). It is specifically intended to support the development of systems which use the debugger as just one small component of a larger system.

This chapter is a specification of the GDB/MI interface. It is written in the form of a reference manual.

Note that GDB/MI is still under construction, so some of the features described below are incomplete and subject to change (see Section 27.6 [GDB/MI Development and Front Ends], page 615).

Notation and Terminology

This chapter uses the following notation:

- `|` separates two alternatives.
- `[ something ]` indicates that *something* is optional: it may or may not be given.
- `( group )*` means that *group* inside the parentheses may repeat zero or more times.
- `( group )+` means that *group* inside the parentheses may repeat one or more times.
- `( group )` means that *group* inside the parentheses occurs exactly once.
- `"string"` means a literal *string*.

27.3 GDB/MI General Design

Interaction of a GDB/MI frontend with GDB involves three parts—commands sent to GDB, responses to those commands and notifications. Each command results in exactly one response, indicating either successful completion of the command, or an error. For the commands that do not resume the target, the response contains the requested information. For the commands that resume the target, the response only indicates whether the target was successfully resumed. Notifications is the mechanism for reporting changes in the state of the target, or in GDB state, that cannot conveniently be associated with a command and reported as part of that command response.

The important examples of notifications are:

- Exec notifications. These are used to report changes in target state—when a target is resumed, or stopped. It would not be feasible to include this information in response of resuming commands, because one resume commands can result in multiple events in different threads. Also, quite some time may pass before any event happens in the target, while a frontend needs to know whether the resuming command itself was successfully executed.
- Console output, and status notifications. Console output notifications are used to report output of CLI commands, as well as diagnostics for other commands. Status notifications are used to report the progress of a long-running operation. Naturally,

including this information in command response would mean no output is produced until the command is finished, which is undesirable.

- General notifications. Commands may have various side effects on the GDB or target state beyond their official purpose. For example, a command may change the selected thread. Although such changes can be included in command response, using notification allows for more orthogonal frontend design.

There's no guarantee that whenever an MI command reports an error, GDB or the target are in any specific state, and especially, the state is not reverted to the state before the MI command was processed. Therefore, whenever an MI command results in an error, we recommend that the frontend refreshes all the information shown in the user interface.

27.3.1 Context management

27.3.1.1 Threads and Frames

In most cases when GDB accesses the target, this access is done in context of a specific thread and frame (see Section 8.1 [Frames], page 115). Often, even when accessing global data, the target requires that a thread be specified. The CLI interface maintains the selected thread and frame, and supplies them to target on each command. This is convenient, because a command line user would not want to specify that information explicitly on each command, and because user interacts with GDB via a single terminal, so no confusion is possible as to what thread and frame are the current ones.

In the case of MI, the concept of selected thread and frame is less useful. First, a frontend can easily remember this information itself. Second, a graphical frontend can have more than one window, each one used for debugging a different thread, and the frontend might want to access additional threads for internal purposes. This increases the risk that by relying on implicitly selected thread, the frontend may be operating on a wrong one. Therefore, each MI command should explicitly specify which thread and frame to operate on. To make it possible, each MI command accepts the `--thread` and `--frame` options, the value to each is GDB global identifier for thread and frame to operate on.

Usually, each top-level window in a frontend allows the user to select a thread and a frame, and remembers the user selection for further operations. However, in some cases GDB may suggest that the current thread or frame be changed. For example, when stopping on a breakpoint it is reasonable to switch to the thread where breakpoint is hit. For another example, if the user issues the CLI `thread` or `frame` commands via the frontend, it is desirable to change the frontend's selection to the one specified by user. GDB communicates the suggestion to change current thread and frame using the `=thread-selected` notification.

Note that historically, MI shares the selected thread with CLI, so frontends used the `-thread-select` to execute commands in the right context. However, getting this to work right is cumbersome. The simplest way is for frontend to emit `-thread-select` command before every command. This doubles the number of commands that need to be sent. The alternative approach is to suppress `-thread-select` if the selected thread in GDB is supposed to be identical to the thread the frontend wants to operate on. However, getting this optimization right can be tricky. In particular, if the frontend sends several commands to GDB, and one of the commands changes the selected thread, then the behavior of subsequent commands will change. So, a frontend should either wait for response from such

problematic commands, or explicitly add `-thread-select` for all subsequent commands. No frontend is known to do this exactly right, so it is suggested to just always pass the `--thread` and `--frame` options.

27.3.1.2 Language

The execution of several commands depends on which language is selected. By default, the current language (see [show language], page 231) is used. But for commands known to be language-sensitive, it is recommended to use the `--language` option. This option takes one argument, which is the name of the language to use while executing the command. For instance:

```
-data-evaluate-expression --language c "sizeof (void*)"
^done,value="4"
(gdb)
```

The valid language names are the same names accepted by the `set language` command (see Section 15.1.2 [Manually], page 230), excluding `auto`, `local` or `unknown`.

27.3.2 Asynchronous command execution and non-stop mode

On some targets, GDB is capable of processing MI commands even while the target is running. This is called *asynchronous command execution* (see Section 5.5.3 [Background Execution], page 98). The frontend may specify a preference for asynchronous execution using the `-gdb-set mi-async 1` command, which should be emitted before either running the executable or attaching to the target. After the frontend has started the executable or attached to the target, it can find if asynchronous execution is enabled using the `-list-target-features` command.

`-gdb-set mi-async [on|off]`

Set whether MI is in asynchronous mode.

When `off`, which is the default, MI execution commands (e.g., `-exec-continue`) are foreground commands, and GDB waits for the program to stop before processing further commands.

When `on`, MI execution commands are background execution commands (e.g., `-exec-continue` becomes the equivalent of the `c&` CLI command), and so GDB is capable of processing MI commands even while the target is running.

`-gdb-show mi-async`

Show whether MI asynchronous mode is enabled.

Note: In GDB version 7.7 and earlier, this option was called `target-async` instead of `mi-async`, and it had the effect of both putting MI in asynchronous mode and making CLI background commands possible. CLI background commands are now always possible “out of the box” if the target supports them. The old spelling is kept as a deprecated alias for backwards compatibility.

Even if GDB can accept a command while target is running, many commands that access the target do not work when the target is running. Therefore, asynchronous command execution is most useful when combined with non-stop mode (see Section 5.5.2 [Non-Stop Mode], page 97). Then, it is possible to examine the state of one thread, while other threads are running.

When a given thread is running, MI commands that try to access the target in the context of that thread may not work, or may work only on some targets. In particular, commands that try to operate on thread's stack will not work, on any target. Commands that read memory, or modify breakpoints, may work or not work, depending on the target. Note that even commands that operate on global state, such as `print`, `set`, and breakpoint commands, still access the target in the context of a specific thread, so frontend should try to find a stopped thread and perform the operation on that thread (using the `--thread` option).

Which commands will work in the context of a running thread is highly target dependent. However, the two commands `-exec-interrupt`, to stop a thread, and `-thread-info`, to find the state of a thread, will always work.

27.3.3 Thread groups

GDB may be used to debug several processes at the same time. On some platforms, GDB may support debugging of several hardware systems, each one having several cores with several different processes running on each core. This section describes the MI mechanism to support such debugging scenarios.

The key observation is that regardless of the structure of the target, MI can have a global list of threads, because most commands that accept the `--thread` option do not need to know what process that thread belongs to. Therefore, it is not necessary to introduce neither additional `--process` option, nor an notion of the current process in the MI interface. The only strictly new feature that is required is the ability to find how the threads are grouped into processes.

To allow the user to discover such grouping, and to support arbitrary hierarchy of machines/cores/processes, MI introduces the concept of a *thread group*. Thread group is a collection of threads and other thread groups. A thread group always has a string identifier, a type, and may have additional attributes specific to the type. A new command, `-list-thread-groups`, returns the list of top-level thread groups, which correspond to processes that GDB is debugging at the moment. By passing an identifier of a thread group to the `-list-thread-groups` command, it is possible to obtain the members of specific thread group.

To allow the user to easily discover processes, and other objects, he wishes to debug, a concept of *available thread group* is introduced. Available thread group is a thread group that GDB is not debugging, but that can be attached to, using the `-target-attach` command. The list of available top-level thread groups can be obtained using `-list-thread-groups --available`. In general, the content of a thread group may be only retrieved only after attaching to that thread group.

Thread groups are related to inferiors (see Section 4.9 [Inferiors Connections and Programs], page 43). Each inferior corresponds to a thread group of a special type `'process'`, and some additional operations are permitted on such thread groups.

27.4 GDB/MI Command Syntax

27.4.1 GDB/MI Input Syntax

```

command  $\mapsto$ 
    cli-command | mi-command

cli-command  $\mapsto$ 
    [ token ] cli-command nl, where cli-command is any existing GDB CLI command.

mi-command  $\mapsto$ 
    [ token ] "-" operation ( " " option ) * [ " -- " ] ( " " parameter ) * nl

token  $\mapsto$  "any sequence of digits"

option  $\mapsto$  "-" parameter [ " " parameter ]

parameter  $\mapsto$ 
    non-blank-sequence | c-string

operation  $\mapsto$ 
    any of the operations described in this chapter

non-blank-sequence  $\mapsto$ 
    anything, provided it doesn't contain special characters such as "-", nl, "" and of course " "

c-string  $\mapsto$ 
    "" seven-bit-iso-c-string-content ""

nl  $\mapsto$  CR | CR-LF

```

Notes:

- The CLI commands are still handled by the MI interpreter; their output is described below.
- The `token`, when present, is passed back when the command finishes.
- Some MI commands accept optional arguments as part of the parameter list. Each option is identified by a leading ‘-’ (dash) and may be followed by an optional argument parameter. Options occur first in the parameter list and can be delimited from normal parameters using ‘--’ (this is useful when some parameters begin with a dash).

Pragmatics:

- We want easy access to the existing CLI syntax (for debugging).
- We want it to be easy to spot a MI operation.

27.4.2 GDB/MI Output Syntax

The output from GDB/MI consists of zero or more out-of-band records followed, optionally, by a single result record. This result record is for the most recent command. The sequence of output records is terminated by ‘(gdb)’.

If an input command was prefixed with a `token` then the corresponding output for that command will also be prefixed by that same `token`.

```

output  $\mapsto$  ( out-of-band-record ) * [ result-record ] "(gdb)" nl

```

```

result-record  $\mapsto$ 
    [ token ] "^" result-class ( "," result ) * nl
out-of-band-record  $\mapsto$ 
    async-record | stream-record
async-record  $\mapsto$ 
    exec-async-output | status-async-output | notify-async-output
exec-async-output  $\mapsto$ 
    [ token ] "*" async-output nl
status-async-output  $\mapsto$ 
    [ token ] "+" async-output nl
notify-async-output  $\mapsto$ 
    [ token ] "=" async-output nl
async-output  $\mapsto$ 
    async-class ( "," result ) *
result-class  $\mapsto$ 
    "done" | "running" | "connected" | "error" | "exit"
async-class  $\mapsto$ 
    "stopped" | others (where others will be added depending on the needs—this
    is still in development).
result  $\mapsto$  variable "=" value
variable  $\mapsto$ 
    string
value  $\mapsto$  const | tuple | list
const  $\mapsto$  c-string
tuple  $\mapsto$  "{" | "{" result ( "," result ) * "}"
list  $\mapsto$  "[" | "[" value ( "," value ) * "]" | "[" result ( "," result ) * "]"
stream-record  $\mapsto$ 
    console-stream-output | target-stream-output | log-stream-output
console-stream-output  $\mapsto$ 
    "~" c-string nl
target-stream-output  $\mapsto$ 
    "@" c-string nl
log-stream-output  $\mapsto$ 
    "&" c-string nl
nl  $\mapsto$  CR | CR-LF
token  $\mapsto$  any sequence of digits.

```

Notes:

- All output sequences end in a single line containing a period.

- The *token* is from the corresponding request. Note that for all async output, while the token is allowed by the grammar and may be output by future versions of GDB for select async output messages, it is generally omitted. Frontends should treat all async output as reporting general changes in the state of the target and there should be no need to associate async output to any prior command.
- *status-async-output* contains on-going status information about the progress of a slow operation. It can be discarded. All status output is prefixed by '+’.
- *exec-async-output* contains asynchronous state change on the target (stopped, started, disappeared). All async output is prefixed by '*’.
- *notify-async-output* contains supplementary information that the client should handle (e.g., a new breakpoint information). All notify output is prefixed by '='.
- *console-stream-output* is output that should be displayed as is in the console. It is the textual response to a CLI command. All the console output is prefixed by '~’.
- *target-stream-output* is the output produced by the target program. All the target output is prefixed by '@’.
- *log-stream-output* is output text coming from GDB’s internals, for instance messages that should be displayed as part of an error log. All the log output is prefixed by '&’.
- New GDB/MI commands should only output *lists* containing *values*.

See Section 27.7.2 [GDB/MI Stream Records], page 617, for more details about the various output records.

27.5 GDB/MI Compatibility with CLI

For the developers convenience CLI commands can be entered directly, but there may be some unexpected behavior. For example, commands that query the user will behave as if the user replied yes, breakpoint command lists are not executed and some CLI commands, such as `if`, `when` and `define`, prompt for further input with '>', which is not valid MI output.

This feature may be removed at some stage in the future and it is recommended that front ends use the `-interpreter-exec` command (see [-interpreter-exec], page 711).

27.6 GDB/MI Development and Front Ends

The application which takes the MI output and presents the state of the program being debugged to the user is called a *front end*.

Since GDB/MI is used by a variety of front ends to GDB, changes to the MI interface may break existing usage. This section describes how the protocol changes and how to request previous version of the protocol when it does.

Some changes in MI need not break a carefully designed front end, and for these the MI version will remain unchanged. The following is a list of changes that may occur within one level, so front ends should parse MI output in a way that can handle them:

- New MI commands may be added.
- New fields may be added to the output of any MI command.
- The range of values for fields with specified values, e.g., `in_scope` (see [-var-update], page 668) may be extended.

If the changes are likely to break front ends, the MI version level will be increased by one. The new versions of the MI protocol are not compatible with the old versions. Old versions of MI remain available, allowing front ends to keep using them until they are modified to use the latest MI version.

Since `--interpreter=mi` always points to the latest MI version, it is recommended that front ends request a specific version of MI when launching GDB (e.g. `--interpreter=mi2`) to make sure they get an interpreter with the MI version they expect.

The following table gives a summary of the released versions of the MI interface: the version number, the version of GDB in which it first appeared and the breaking changes compared to the previous version.

MI version	GDB version	Breaking changes
1	5.1	None
2	6.0	• The <code>-environment-pwd</code>, <code>-environment-directory</code> and <code>-environment-path</code> commands now returns values using the MI output syntax, rather than CLI output syntax. • <code>-var-list-children</code>'s <code>children</code> result field is now a list, rather than a tuple. • <code>-var-update</code>'s <code>changelist</code> result field is now a list, rather than a tuple.
3	9.1	• The output of information about multi-location breakpoints has changed in the responses to the <code>-break-insert</code> and <code>-break-info</code> commands, as well as in the <code>=breakpoint-created</code> and <code>=breakpoint-modified</code> events. The multiple locations are now placed in a <code>locations</code> field, whose value is a list.
4	13.1	• The syntax of the "script" field in breakpoint output has changed in the responses to the <code>-break-insert</code> and <code>-break-info</code> commands, as well as the <code>=breakpoint-created</code> and <code>=breakpoint-modified</code> events. The previous output was syntactically invalid. The new output is a list.

If your front end cannot yet migrate to a more recent version of the MI protocol, you can nevertheless selectively enable specific features available in those recent MI versions, using the following commands:

`-fix-multi-location-breakpoint-output`

Use the output for multi-location breakpoints which was introduced by MI 3, even when using MI versions below 3. This command has no effect when using MI version 3 or later.

-fix-breakpoint-script-output

Use the output for the breakpoint "script" field which was introduced by MI 4, even when using MI versions below 4. This command has no effect when using MI version 4 or later.

The best way to avoid unexpected changes in MI that might break your front end is to make your project known to GDB developers and follow development on gdb@sourceware.org and gdb-patches@sourceware.org.

27.7 GDB/MI Output Records

27.7.1 GDB/MI Result Records

In addition to a number of out-of-band notifications, the response to a GDB/MI command includes one of the following result indications:

"^done" [**"",** **results**]

The synchronous operation was successful, **results** are the return values.

"^running"

This result record is equivalent to '^done'. Historically, it was output instead of '^done' if the command has resumed the target. This behavior is maintained for backward compatibility, but all frontends should treat '^done' and '^running' identically and rely on the '^*running' output record to determine which threads are resumed.

"^connected"

GDB has connected to a remote target.

"^error" **"",** **"msg=" c-string** [**"",** **"code=" c-string**]

The operation failed. The **msg=c-string** variable contains the corresponding error message.

If present, the **code=c-string** variable provides an error code on which consumers can rely on to detect the corresponding error condition. At present, only one error code is defined:

"undefined-command"

Indicates that the command causing the error does not exist.

"^exit" GDB has terminated.

27.7.2 GDB/MI Stream Records

GDB internally maintains a number of output streams: the console, the target, and the log. The output intended for each of these streams is funneled through the GDB/MI interface using *stream records*.

Each stream record begins with a unique *prefix character* which identifies its stream (see Section 27.4.2 [GDB/MI Output Syntax], page 613). In addition to the prefix, each stream record contains a *string-output*. This is either raw text (with an implicit new line) or a quoted C string (which does not contain an implicit newline).

"~" string-output

The console output stream contains text that should be displayed in the CLI console window. It contains the textual responses to CLI commands.

"@" string-output

The target output stream contains any textual output from the running target. This is only present when GDB's event loop is truly asynchronous, which is currently only the case for remote targets.

"&" string-output

The log stream contains debugging messages being produced by GDB's internals.

27.7.3 GDB/MI Async Records

Async records are used to notify the GDB/MI client of additional changes that have occurred. Those changes can either be a consequence of GDB/MI commands (e.g., a breakpoint modified) or a result of target activity (e.g., target stopped).

The following is the list of possible async records:

***running,thread-id="thread"**

The target is now running. The *thread* field can be the global thread ID of the thread that is now running, and it can be 'all' if all threads are running. The frontend should assume that no interaction with a running thread is possible after this notification is produced. The frontend should not assume that this notification is output only once for any command. GDB may emit this notification several times, either for different threads, because it cannot resume all threads together, or even for a single thread, if the thread must be stepped through some code before letting it run freely.

***stopped,reason="reason",thread-id="id",stopped-threads="stopped",core="core"**

The target has stopped. The *reason* field can have one of the following values:

breakpoint-hit

A breakpoint was reached.

watchpoint-trigger

A watchpoint was triggered.

read-watchpoint-trigger

A read watchpoint was triggered.

access-watchpoint-trigger

An access watchpoint was triggered.

function-finished

An -exec-finish or similar CLI command was accomplished.

location-reached

An -exec-until or similar CLI command was accomplished.

watchpoint-scope

A watchpoint has gone out of scope.

end-stepping-range	An <code>-exec-next</code> , <code>-exec-next-instruction</code> , <code>-exec-step</code> , <code>-exec-step-instruction</code> or similar CLI command was accomplished.
exited-signalled	The inferior exited because of a signal.
exited	The inferior exited.
exited-normally	The inferior exited normally.
signal-received	A signal was received by the inferior.
solib-event	The inferior has stopped due to a library being loaded or unloaded. This can happen when <code>stop-on-solib-events</code> (see Section 18.1 [Files], page 293) is set or when a <code>catch load</code> or <code>catch unload</code> catchpoint is in use (see Section 5.1.3 [Set Catchpoints], page 71).
fork	The inferior has forked. This is reported when <code>catch fork</code> (see Section 5.1.3 [Set Catchpoints], page 71) has been used.
vfork	The inferior has vforked. This is reported in when <code>catch vfork</code> (see Section 5.1.3 [Set Catchpoints], page 71) has been used.
syscall-entry	The inferior entered a system call. This is reported when <code>catch syscall</code> (see Section 5.1.3 [Set Catchpoints], page 71) has been used.
syscall-return	The inferior returned from a system call. This is reported when <code>catch syscall</code> (see Section 5.1.3 [Set Catchpoints], page 71) has been used.
exec	The inferior called <code>exec</code> . This is reported when <code>catch exec</code> (see Section 5.1.3 [Set Catchpoints], page 71) has been used.
no-history	There isn't enough history recorded to continue reverse execution.

The *id* field identifies the global thread ID of the thread that directly caused the stop – for example by hitting a breakpoint. Depending on whether all-stop mode is in effect (see Section 5.5.1 [All-Stop Mode], page 96), GDB may either stop all threads, or only the thread that directly triggered the stop. If all threads are stopped, the *stopped* field will have the value of "all". Otherwise, the value of the *stopped* field will be a list of thread identifiers. Presently, this list will always include a single thread, but frontend should be prepared to see several threads in the list. The *core* field reports the processor core on which the stop event has happened. This field may be absent if such information is not available.

`=thread-group-added,id="id"`

`=thread-group-removed,id="id"`

A thread group was either added or removed. The *id* field contains the GDB identifier of the thread group. When a thread group is added, it generally might not be associated with a running process. When a thread group is removed, its *id* becomes invalid and cannot be used in any way.

`=thread-group-started,id="id",pid="pid"`

A thread group became associated with a running program, either because the program was just started or the thread group was attached to a program. The *id* field contains the GDB identifier of the thread group. The *pid* field contains process identifier, specific to the operating system.

`=thread-group-exited,id="id"[,exit-code="code"]`

A thread group is no longer associated with a running program, either because the program has exited, or because it was detached from. The *id* field contains the GDB identifier of the thread group. The *code* field is the exit code of the inferior; it exists only when the inferior exited with some code.

`=thread-created,id="id",group-id="gid"`

`=thread-exited,id="id",group-id="gid"`

A thread either was created, or has exited. The *id* field contains the global GDB identifier of the thread. The *gid* field identifies the thread group this thread belongs to.

`=thread-selected,id="id"[,frame="frame"]`

Informs that the selected thread or frame were changed. This notification is not emitted as result of the `-thread-select` or `-stack-select-frame` commands, but is emitted whenever an MI command that is not documented to change the selected thread and frame actually changes them. In particular, invoking, directly or indirectly (via user-defined command), the CLI `thread` or `frame` commands, will generate this notification. Changing the thread or frame from another user interface (see Chapter 24 [Interpreters], page 597) will also generate this notification.

The *frame* field is only present if the newly selected thread is stopped. See Section 27.7.5 [GDB/MI Frame Information], page 625, for the format of its value.

We suggest that in response to this notification, front ends highlight the selected thread and cause subsequent commands to apply to that thread.

`=library-loaded,...`

Reports that a new library file was loaded by the program. This notification has 5 fields—*id*, *target-name*, *host-name*, *symbols-loaded* and *ranges*. The *id* field is an opaque identifier of the library. For remote debugging case, *target-name* and *host-name* fields give the name of the library file on the target, and on the host respectively. For native debugging, both those fields have the same value. The *symbols-loaded* field is emitted only for backward compatibility and should not be relied on to convey any useful information. The *thread-group* field, if present, specifies the *id* of the thread group in whose context the library was

loaded. If the field is absent, it means the library was loaded in the context of all present thread groups. The *ranges* field specifies the ranges of addresses belonging to this library.

`=library-unloaded,...`

Reports that a library was unloaded by the program. This notification has the following fields—*id*, *target-name*, *host-name* and *ranges* with the same meaning as for the `=library-loaded` notification.

It is possible that a library can appear multiple times in an inferior's library list, but the library is only mapped once into the inferior's address space. When this happens, and one copy of the library is unloaded, but there are remaining copies, the *still-in-use* field will be `'true'`. In all other situations, the *still-in-use* field will have the value `'false'`.

The *thread-group* field, if present, specifies the id of the thread group in whose context the library was unloaded. If the field is absent, it means the library was unloaded in the context of all present thread groups.

`=traceframe-changed,num=tfnum,tracepoint=tpnum`

`=traceframe-changed,end`

Reports that the trace frame was changed and its new number is *tfnum*. The number of the tracepoint associated with this trace frame is *tpnum*.

`=tsv-created,name=name,initial=initial`

Reports that the new trace state variable *name* is created with initial value *initial*.

`=tsv-deleted,name=name`

`=tsv-deleted`

Reports that the trace state variable *name* is deleted or all trace state variables are deleted.

`=tsv-modified,name=name,initial=initial[,current=current]`

Reports that the trace state variable *name* is modified with the initial value *initial*. The current value *current* of trace state variable is optional and is reported if the current value of trace state variable is known.

`=breakpoint-created,bkpt={...}`

`=breakpoint-modified,bkpt={...}`

`=breakpoint-deleted,id=number`

Reports that a breakpoint was created, modified, or deleted, respectively. Only user-visible breakpoints are reported to the MI user.

The *bkpt* argument is of the same form as returned by the various breakpoint commands; See Section 27.10 [GDB/MI Breakpoint Commands], page 627. The *number* is the ordinal number of the breakpoint.

Note that if a breakpoint is emitted in the result record of a command, then it will not also be emitted in an *async* record.

`=record-started,thread-group="id",method="method"[,format="format"]`

`=record-stopped,thread-group="id"`

Execution log recording was either started or stopped on an inferior. The *id* is the GDB identifier of the thread group corresponding to the affected inferior.

The *method* field indicates the method used to record execution. If the method in use supports multiple recording formats, *format* will be present and contain the currently used format. See Chapter 7 [Process Record and Replay], page 105, for existing method and format values.

If recording was stopped for the btrace method, threads move to the end of their execution history.

`=cmd-param-changed,param=param,value=value`

Reports that a parameter of the command `set param` is changed to *value*. In the multi-word `set` command, the *param* is the whole parameter list to `set` command. For example, In command `set check type on`, *param* is `check type` and *value* is `on`.

`=memory-changed,thread-group=id,addr=addr,len=len[,type="code"]`

Reports that bytes from *addr* to *data* + *len* were written in an inferior. The *id* is the identifier of the thread group corresponding to the affected inferior. The optional `type="code"` part is reported if the memory written to holds executable code.

27.7.4 GDB/MI Breakpoint Information

When GDB reports information about a breakpoint, a tracepoint, a watchpoint, or a catchpoint, it uses a tuple with the following fields:

number	The breakpoint number.
type	The type of the breakpoint. For ordinary breakpoints this will be ‘ <code>breakpoint</code> ’, but many values are possible.
catch-type	If the type of the breakpoint is ‘ <code>catchpoint</code> ’, then this indicates the exact type of catchpoint.
disp	This is the breakpoint disposition—either ‘ <code>del</code> ’, meaning that the breakpoint will be deleted at the next stop, or ‘ <code>keep</code> ’, meaning that the breakpoint will not be deleted.
enabled	This indicates whether the breakpoint is enabled, in which case the value is ‘ <code>y</code> ’, or disabled, in which case the value is ‘ <code>n</code> ’. Note that this is not the same as the field <code>enable</code> .
addr	The address of the breakpoint. This may be a hexadecimal number, giving the address; or the string ‘ <code><PENDING></code> ’, for a pending breakpoint; or the string ‘ <code><MULTIPLE></code> ’, for a breakpoint with multiple locations. This field will not be present if no address can be determined. For example, a watchpoint does not have an address.
addr_flags	Optional field containing any flags related to the address. These flags are architecture-dependent; see Section 21.4 [Architectures], page 356, for their meaning for a particular CPU.
func	If known, the function in which the breakpoint appears. If not known, this field is not present.

filename	The name of the source file which contains this function, if known. If not known, this field is not present.
fullname	The full file name of the source file which contains this function, if known. If not known, this field is not present.
line	The line number at which this breakpoint appears, if known. If not known, this field is not present.
at	If the source file is not known, this field may be provided. If provided, this holds the address of the breakpoint, possibly followed by a symbol name.
pending	If this breakpoint is pending, this field is present and holds the text used to set the breakpoint, as entered by the user. GDB's internal breakpoints (see [maint info breakpoints], page 777) can sometimes become pending too, for these breakpoints the <i>pending</i> field will be empty as GDB automatically creates these breakpoints as shared libraries are loaded.
evaluated-by	Where this breakpoint's condition is evaluated, either 'host' or 'target'.
thread	If this is a thread-specific breakpoint, then this identifies the thread in which the breakpoint can trigger.
inferior	If this is an inferior-specific breakpoint, this this identifies the inferior in which the breakpoint can trigger.
task	If this breakpoint is restricted to a particular Ada task, then this field will hold the task identifier.
cond	If the breakpoint is conditional, this is the condition expression.
ignore	The ignore count of the breakpoint.
enable	The enable count of the breakpoint.
traceframe-usage	FIXME.
static-tracepoint-marker-string-id	For a static tracepoint, the name of the static tracepoint marker.
mask	For a masked watchpoint, this is the mask.
pass	A tracepoint's pass count.
original-location	The location of the breakpoint as originally specified by the user. This field is optional.
times	The number of times the breakpoint has been hit.
installed	This field is only given for tracepoints. This is either 'y', meaning that the tracepoint is installed, or 'n', meaning that it is not.
what	Some extra data, the exact contents of which are type-dependent.

locations

This field is present if the breakpoint has multiple locations. It is also exceptionally present if the breakpoint is enabled and has a single, disabled location.

The value is a list of locations. The format of a location is described below.

A location in a multi-location breakpoint is represented as a tuple with the following fields:

number The location number as a dotted pair, like '1.2'. The first digit is the number of the parent breakpoint. The second digit is the number of the location within that breakpoint.

enabled There are three possible values, with the following meanings:

y The location is enabled.

n The location is disabled by the user.

N The location is disabled because the breakpoint condition is invalid at this location.

addr The address of this location as an hexadecimal number.

addr_flags

Optional field containing any flags related to the address. These flags are architecture-dependent; see Section 21.4 [Architectures], page 356, for their meaning for a particular CPU.

func If known, the function in which the location appears. If not known, this field is not present.

file The name of the source file which contains this location, if known. If not known, this field is not present.

fullname The full file name of the source file which contains this location, if known. If not known, this field is not present.

line The line number at which this location appears, if known. If not known, this field is not present.

thread-groups

The thread groups this location is in.

For example, here is what the output of `-break-insert` (see Section 27.10 [GDB/MI Breakpoint Commands], page 627) might be:

```
-> -break-insert main
<- ^done,bkpt={number="1",type="breakpoint",disp="keep",
    enabled="y",addr="0x08048564",func="main",file="myprog.c",
    fullname="/home/nickrob/myprog.c",line="68",thread-groups=["i1"],
    times="0"}
<- (gdb)
```

27.7.5 GDB/MI Frame Information

Response from many MI commands includes an information about stack frame. This information is a tuple that may have the following fields:

level	The level of the stack frame. The innermost frame has the level of zero. This field is always present.
func	The name of the function corresponding to the frame. This field may be absent if GDB is unable to determine the function name.
addr	The code address for the frame. This field is always present.
addr_flags	Optional field containing any flags related to the address. These flags are architecture-dependent; see Section 21.4 [Architectures], page 356, for their meaning for a particular CPU.
file	The name of the source files that correspond to the frame's code address. This field may be absent.
line	The source line corresponding to the frames' code address. This field may be absent.
from	The name of the binary file (either executable or shared library) the corresponds to the frame's code address. This field may be absent.

27.7.6 GDB/MI Thread Information

Whenever GDB has to report an information about a thread, it uses a tuple with the following fields. The fields are always present unless stated otherwise.

id	The global numeric id assigned to the thread by GDB.
target-id	The target-specific string identifying the thread.
details	Additional information about the thread provided by the target. It is supposed to be human-readable and not interpreted by the frontend. This field is optional.
name	The name of the thread. If the user specified a name using the thread name command, then this name is given. Otherwise, if GDB can extract the thread name from the target, then that name is given. If GDB cannot find the thread name, then this field is omitted.
state	The execution state of the thread, either ' stopped ' or ' running ', depending on whether the thread is presently running.
frame	The stack frame currently executing in the thread. This field is only present if the thread is stopped. Its format is documented in Section 27.7.5 [GDB/MI Frame Information], page 625.
core	The value of this field is an integer number of the processor core the thread was last seen on. This field is optional.

27.7.7 GDB/MI Ada Exception Information

Whenever a `*stopped` record is emitted because the program stopped after hitting an exception catchpoint (see Section 5.1.3 [Set Catchpoints], page 71), GDB provides the name of the exception that was raised via the `exception-name` field. Also, for exceptions that were raised with an exception message, GDB provides that message via the `exception-message` field.

27.8 Simple Examples of GDB/MI Interaction

This subsection presents several simple examples of interaction using the GDB/MI interface. In these examples, ‘`->`’ means that the following line is passed to GDB/MI as input, while ‘`<-`’ means the output received from GDB/MI.

Note the line breaks shown in the examples are here only for readability, they don’t appear in the real output.

Setting a Breakpoint

Setting a breakpoint generates synchronous output which contains detailed information of the breakpoint.

```
-> -break-insert main
<- ^done,bkpt={number="1",type="breakpoint",disp="keep",
    enabled="y",addr="0x08048564",func="main",file="myprog.c",
    fullname="/home/nickrob/myprog.c",line="68",thread-groups=["i1"],
    times="0"}
<- (gdb)
```

Program Execution

Program execution generates asynchronous records and MI gives the reason that execution stopped.

```
-> -exec-run
<- ^running
<- (gdb)
<- *stopped,reason="breakpoint-hit",disp="keep",bkptno="1",thread-id="0",
    frame={addr="0x08048564",func="main",
    args=[{name="argc",value="1"},{name="argv",value="0xbfc4d4d4"}]},
    file="myprog.c",fullname="/home/nickrob/myprog.c",line="68",
    arch="i386:x86_64"}
<- (gdb)
-> -exec-continue
<- ^running
<- (gdb)
<- *stopped,reason="exited-normally"
<- (gdb)
```

Quitting GDB

Quitting GDB just prints the result class ‘`^exit`’.

```
-> (gdb)
<- -gdb-exit
<- ^exit
```

Please note that ‘`^exit`’ is printed immediately, but it might take some time for GDB to actually exit. During that time, GDB performs necessary cleanups, including killing

programs being debugged or disconnecting from debug hardware, so the frontend should wait till GDB exits and should only forcibly kill GDB if it fails to exit in reasonable time.

A Bad Command

Here's what happens if you pass a non-existent command:

```
-> -rubbish
<- ^error,msg="Undefined MI command: rubbish"
<- (gdb)
```

27.9 GDB/MI Command Description Format

The remaining sections describe blocks of commands. Each block of commands is laid out in a fashion similar to this section.

Motivation

The motivation for this collection of commands.

Introduction

A brief introduction to this collection of commands as a whole.

Commands

For each command in the block, the following is described:

Synopsis

```
-command args...
```

Result

GDB Command

The corresponding GDB CLI command(s), if any.

Example

Example(s) formatted for readability. Some of the described commands have not been implemented yet and these are labeled N.A. (not available).

27.10 GDB/MI Breakpoint Commands

This section documents GDB/MI commands for manipulating breakpoints.

The -break-after Command

Synopsis

```
-break-after number count
```

The breakpoint number *number* is not in effect until it has been hit *count* times. To see how this is reflected in the output of the ‘-break-list’ command, see the description of the ‘-break-list’ command below.

GDB Command

The corresponding GDB command is ‘ignore’.

Example

```
(gdb)
-break-insert main
^done,bkpt={number="1",type="breakpoint",disp="keep",
enabled="y",addr="0x000100d0",func="main",file="hello.c",
fullname="/home/foo/hello.c",line="5",thread-groups=["i1"],
times="0"}
(gdb)
-break-after 1 3
~

^done
(gdb)
-break-list
^done,BreakpointTable={nr_rows="1",nr_cols="6",
hdr=[{width="3",alignment="-1",col_name="number",colhdr="Num"},
{width="14",alignment="-1",col_name="type",colhdr="Type"},
{width="4",alignment="-1",col_name="disp",colhdr="Disp"},
{width="3",alignment="-1",col_name="enabled",colhdr="Enb"},
{width="10",alignment="-1",col_name="addr",colhdr="Address"},
{width="40",alignment="2",col_name="what",colhdr="What"}],
body=[bkpt={number="1",type="breakpoint",disp="keep",enabled="y",
addr="0x000100d0",func="main",file="hello.c",fullname="/home/foo/hello.c",
line="5",thread-groups=["i1"],times="0",ignore="3"}]}
```

The -break-commands Command

Synopsis

```
-break-commands number [ command1 ... commandN ]
```

Specifies the CLI commands that should be executed when breakpoint *number* is hit. The parameters *command1* to *commandN* are the commands. If no command is specified, any previously-set commands are cleared. See Section 5.1.7 [Break Commands], page 79. Typical use of this functionality is tracing a program, that is, printing of values of some variables whenever breakpoint is hit and then continuing.

GDB Command

The corresponding GDB command is ‘commands’.

Example

```
(gdb)
-break-insert main
^done,bkpt={number="1",type="breakpoint",disp="keep",
enabled="y",addr="0x000100d0",func="main",file="hello.c",
fullname="/home/foo/hello.c",line="5",thread-groups=["i1"],
times="0"}
(gdb)
-break-commands 1 "print v" "continue"
^done
(gdb)
```

The `-break-condition` Command

Synopsis

```
-break-condition [ --force ] number [ expr ]
```

Breakpoint *number* will stop the program only if the condition in *expr* is true. The condition becomes part of the ‘`-break-list`’ output (see the description of the ‘`-break-list`’ command below). If the ‘`--force`’ flag is passed, the condition is forcibly defined even when it is invalid for all locations of breakpoint *number*. If the *expr* argument is omitted, breakpoint *number* becomes unconditional.

GDB Command

The corresponding GDB command is ‘`condition`’.

Example

```
(gdb)
-break-condition 1 1
^done
(gdb)
-break-list
^done,BreakpointTable={nr_rows="1",nr_cols="6",
hdr=[{width="3",alignment="-1",col_name="number",colhdr="Num"},
{width="14",alignment="-1",col_name="type",colhdr="Type"},
{width="4",alignment="-1",col_name="disp",colhdr="Disp"},
{width="3",alignment="-1",col_name="enabled",colhdr="Enb"},
{width="10",alignment="-1",col_name="addr",colhdr="Address"},
{width="40",alignment="2",col_name="what",colhdr="What"}],
body=[bkpt={number="1",type="breakpoint",disp="keep",enabled="y",
addr="0x000100d0",func="main",file="hello.c",fullname="/home/foo/hello.c",
line="5",cond="1",thread-groups=["i1"],times="0",ignore="3"}]}
```

The `-break-delete` Command

Synopsis

```
-break-delete ( breakpoint )+
```

Delete the breakpoint(s) whose number(s) are specified in the argument list. This is obviously reflected in the breakpoint list.

GDB Command

The corresponding GDB command is ‘`delete`’.

Example

```
(gdb)
-break-delete 1
^done
(gdb)
-break-list
^done,BreakpointTable={nr_rows="0",nr_cols="6",
hdr=[{width="3",alignment="-1",col_name="number",colhdr="Num"},
{width="14",alignment="-1",col_name="type",colhdr="Type"},
{width="4",alignment="-1",col_name="disp",colhdr="Disp"},
```

```
{width="3",alignment="-1",col_name="enabled",colhdr="Enb"},
{width="10",alignment="-1",col_name="addr",colhdr="Address"},
{width="40",alignment="2",col_name="what",colhdr="What"}},
body=[]}
(gdb)
```

The -break-disable Command

Synopsis

```
-break-disable ( breakpoint )+
```

Disable the named *breakpoint*(s). The field ‘enabled’ in the break list is now set to ‘n’ for the named *breakpoint*(s).

GDB Command

The corresponding GDB command is ‘disable’.

Example

```
(gdb)
-break-disable 2
^done
(gdb)
-break-list
^done,BreakpointTable={nr_rows="1",nr_cols="6",
hdr=[{width="3",alignment="-1",col_name="number",colhdr="Num"},
{width="14",alignment="-1",col_name="type",colhdr="Type"},
{width="4",alignment="-1",col_name="disp",colhdr="Disp"},
{width="3",alignment="-1",col_name="enabled",colhdr="Enb"},
{width="10",alignment="-1",col_name="addr",colhdr="Address"},
{width="40",alignment="2",col_name="what",colhdr="What"}],
body=[bkpt={number="2",type="breakpoint",disp="keep",enabled="n",
addr="0x000100d0",func="main",file="hello.c",fullname="/home/foo/hello.c",
line="5",thread-groups=["i1"],times="0"}]}
(gdb)
```

The -break-enable Command

Synopsis

```
-break-enable ( breakpoint )+
```

Enable (previously disabled) *breakpoint*(s).

GDB Command

The corresponding GDB command is ‘enable’.

Example

```
(gdb)
-break-enable 2
^done
(gdb)
-break-list
^done,BreakpointTable={nr_rows="1",nr_cols="6",
hdr=[{width="3",alignment="-1",col_name="number",colhdr="Num"},
{width="14",alignment="-1",col_name="type",colhdr="Type"},
```

```
{width="4",alignment="-1",col_name="disp",colhdr="Disp"},
{width="3",alignment="-1",col_name="enabled",colhdr="Enb"},
{width="10",alignment="-1",col_name="addr",colhdr="Address"},
{width="40",alignment="2",col_name="what",colhdr="What"}],
body=[bkpt={number="2",type="breakpoint",disp="keep",enabled="y",
addr="0x000100d0",func="main",file="hello.c",fullname="/home/foo/hello.c",
line="5",thread-groups=["i1"],times="0"}]]
(gdb)
```

The `-break-info` Command

Synopsis

```
-break-info breakpoint
```

Get information about a single breakpoint.

The result is a table of breakpoints. See Section 27.7.4 [GDB/MI Breakpoint Information], page 622, for details on the format of each breakpoint in the table.

GDB Command

The corresponding GDB command is ‘`info break breakpoint`’.

Example

N.A.

The `-break-insert` Command

Synopsis

```
-break-insert [ -t ] [ -h ] [ -f ] [ -d ] [ -a ] [ --qualified ]
[ -c condition ] [ --force-condition ] [ -i ignore-count ]
[ -p thread-id ] [ -g thread-group-id ] [ locspec ]
```

If specified, *locspec*, can be one of:

linespec location

A linespec location. See Section 9.2.1 [Linespec Locations], page 129.

explicit location

An explicit location. GDB/MI explicit locations are analogous to the CLI’s explicit locations using the option names listed below. See Section 9.2.2 [Explicit Locations], page 131.

‘`--source filename`’

The source file name of the location. This option requires the use of either ‘`--function`’ or ‘`--line`’.

‘`--function function`’

The name of a function or method.

‘`--label label`’

The name of a label.

‘`--line lineoffset`’

An absolute or relative line offset from the start of the location.

address location

An address location, **address*. See Section 9.2.3 [Address Locations], page 132.

The possible optional parameters of this command are:

- ‘-t’ Insert a temporary breakpoint.
- ‘-h’ Insert a hardware breakpoint.
- ‘-f’ If *locspec* cannot be resolved (for example if it refers to unknown files or functions), create a pending breakpoint. Without this flag, GDB will report an error, and won’t create a breakpoint, if *locspec* cannot be parsed.
- ‘-d’ Create a disabled breakpoint.
- ‘-a’ Create a tracepoint. See Chapter 13 [Tracepoints], page 205. When this parameter is used together with ‘-h’, a fast tracepoint is created.
- ‘-c *condition*’
 Make the breakpoint conditional on *condition*.
- ‘--force-condition’
 Forcibly define the breakpoint even if the condition is invalid at all of the breakpoint locations.
- ‘-i *ignore-count*’
 Initialize the *ignore-count*.
- ‘-p *thread-id*’
 Restrict the breakpoint to the thread with the specified global *thread-id*. *thread-id* must be a valid thread-id at the time the breakpoint is requested. Breakpoints created with a *thread-id* will automatically be deleted when the corresponding thread exits.
- ‘-g *thread-group-id*’
 Restrict the breakpoint to the thread group with the specified *thread-group-id*.
- ‘--qualified’
 This option makes GDB interpret a function name specified as a complete fully-qualified name.

Result

See Section 27.7.4 [GDB/MI Breakpoint Information], page 622, for details on the format of the resulting breakpoint.

Note: this format is open to change.

GDB Command

The corresponding GDB commands are ‘break’, ‘tbreak’, ‘hbreak’, and ‘thbreak’.

Example

```
(gdb)
-break-insert main
^done,bkpt={number="1",addr="0x0001072c",file="recursive2.c",
fullname="/home/foo/recursive2.c,line="4",thread-groups=["i1"]},
```

```

times="0"}
(gdb)
-break-insert -t foo
^done,bkpt={number="2",addr="0x00010774",file="recursive2.c",
fullname="/home/foo/recursive2.c,line="11",thread-groups=["i1"],
times="0"}
(gdb)
-break-list
^done,BreakpointTable={nr_rows="2",nr_cols="6",
hdr=[{width="3",alignment="-1",col_name="number",colhdr="Num"},
{width="14",alignment="-1",col_name="type",colhdr="Type"},
{width="4",alignment="-1",col_name="disp",colhdr="Disp"},
{width="3",alignment="-1",col_name="enabled",colhdr="Enb"},
{width="10",alignment="-1",col_name="addr",colhdr="Address"},
{width="40",alignment="2",col_name="what",colhdr="What"}],
body=[bkpt={number="1",type="breakpoint",disp="keep",enabled="y",
addr="0x0001072c", func="main",file="recursive2.c",
fullname="/home/foo/recursive2.c,line="4",thread-groups=["i1"],
times="0"},
bkpt={number="2",type="breakpoint",disp="del",enabled="y",
addr="0x00010774",func="foo",file="recursive2.c",
fullname="/home/foo/recursive2.c,line="11",thread-groups=["i1"],
times="0"}]}
(gdb)

```

The `-dprintf-insert` Command

Synopsis

```

-dprintf-insert [ -t ] [ -f ] [ -d ] [ --qualified ]
[ -c condition ] [--force-condition] [ -i ignore-count ]
[ -p thread-id ] [ locspec ] format
[ argument... ]

```

Insert a new dynamic print breakpoint at the given location. See Section 5.1.8 [Dynamic Printf], page 81. *format* is the format to use, and any remaining arguments are passed as expressions to substitute.

If supplied, *locspec* and `--qualified` may be specified the same way as for the `-break-insert` command. See [-break-insert], page 631.

The possible optional parameters of this command are:

- ‘`-t`’ Insert a temporary breakpoint.
- ‘`-f`’ If *locspec* cannot be parsed (for example, if it refers to unknown files or functions), create a pending breakpoint. Without this flag, GDB will report an error, and won’t create a breakpoint, if *locspec* cannot be parsed.
- ‘`-d`’ Create a disabled breakpoint.
- ‘`-c condition`’ Make the breakpoint conditional on *condition*.
- ‘`--force-condition`’ Forcibly define the breakpoint even if the condition is invalid at all of the breakpoint locations.

‘-i ignore-count’

Set the ignore count of the breakpoint (see Section 5.1.6 [Conditions], page 78) to *ignore-count*.

‘-p thread-id’

Restrict the breakpoint to the thread with the specified global *thread-id*.

Result

See Section 27.7.4 [GDB/MI Breakpoint Information], page 622, for details on the format of the resulting breakpoint.

GDB Command

The corresponding GDB command is **‘dprintf’**.

Example

```
(gdb)
4-dprintf-insert foo "At foo entry\n"
4^done,bkpt={number="1",type="dprintf",disp="keep",enabled="y",
addr="0x000000000040061b",func="foo",file="mi-dprintf.c",
fullname="mi-dprintf.c",line="25",thread-groups=["i1"],
times="0",script=["printf \"At foo entry\\n\\n\",\"continue\"],
original-location="foo"}
(gdb)
5-dprintf-insert 26 "arg=%d, g=%d\n" arg g
5^done,bkpt={number="2",type="dprintf",disp="keep",enabled="y",
addr="0x000000000040062a",func="foo",file="mi-dprintf.c",
fullname="mi-dprintf.c",line="26",thread-groups=["i1"],
times="0",script=["printf \"arg=%d, g=%d\\n\\n\", arg, g\",\"continue\"],
original-location="mi-dprintf.c:26"}
(gdb)
```

The -break-list Command

Synopsis

-break-list

Displays the list of inserted breakpoints, showing the following fields:

‘Number’	number of the breakpoint
‘Type’	type of the breakpoint: ‘breakpoint’ or ‘watchpoint’
‘Disposition’	should the breakpoint be deleted or disabled when it is hit: ‘keep’ or ‘nokeep’
‘Enabled’	is the breakpoint enabled or no: ‘y’ or ‘n’
‘Address’	memory location at which the breakpoint is set
‘What’	logical location of the breakpoint, expressed by function name, file name, line number
‘Thread-groups’	list of thread groups to which this breakpoint applies
‘Times’	number of times the breakpoint has been hit

If there are no breakpoints, watchpoints, tracepoints, or catchpoints, the `BreakpointTable` body field is an empty list.

GDB Command

The corresponding GDB command is ‘`info break`’.

Example

```
(gdb)
-break-list
^done,BreakpointTable={nr_rows="2",nr_cols="6",
  hdr=[{width="3",alignment="-1",col_name="number",colhdr="Num"},
    {width="14",alignment="-1",col_name="type",colhdr="Type"},
    {width="4",alignment="-1",col_name="disp",colhdr="Disp"},
    {width="3",alignment="-1",col_name="enabled",colhdr="Enb"},
    {width="10",alignment="-1",col_name="addr",colhdr="Address"},
    {width="40",alignment="2",col_name="what",colhdr="What"}],
  body=[bkpt={number="1",type="breakpoint",disp="keep",enabled="y",
    addr="0x000100d0",func="main",file="hello.c",line="5",thread-groups=["i1"],
    times="0"},
    bkpt={number="2",type="breakpoint",disp="keep",enabled="y",
    addr="0x00010114",func="foo",file="hello.c",fullname="/home/foo/hello.c",
    line="13",thread-groups=["i1"],times="0"}]}}
```

Here’s an example of the result when there are no breakpoints:

```
(gdb)
-break-list
^done,BreakpointTable={nr_rows="0",nr_cols="6",
  hdr=[{width="3",alignment="-1",col_name="number",colhdr="Num"},
    {width="14",alignment="-1",col_name="type",colhdr="Type"},
    {width="4",alignment="-1",col_name="disp",colhdr="Disp"},
    {width="3",alignment="-1",col_name="enabled",colhdr="Enb"},
    {width="10",alignment="-1",col_name="addr",colhdr="Address"},
    {width="40",alignment="2",col_name="what",colhdr="What"}],
  body=[]}}
```

The `-break-passcount` Command

Synopsis

```
-break-passcount tracepoint-number passcount
```

Set the passcount for tracepoint *tracepoint-number* to *passcount*. If the breakpoint referred to by *tracepoint-number* is not a tracepoint, error is emitted. This corresponds to CLI command ‘`passcount`’.

The `-break-watch` Command

Synopsis

```
-break-watch [ -a | -r ]
```

Create a watchpoint. With the ‘`-a`’ option it will create an *access* watchpoint, i.e., a watchpoint that triggers either on a read from or on a write to the memory location. With the ‘`-r`’ option, the watchpoint created is a *read* watchpoint, i.e., it will trigger only when

the memory location is accessed for reading. Without either of the options, the watchpoint created is a regular watchpoint, i.e., it will trigger when the memory location is accessed for writing. See Section 5.1.2 [Setting Watchpoints], page 68.

Note that ‘`-break-list`’ will report a single list of watchpoints and breakpoints inserted.

GDB Command

The corresponding GDB commands are ‘`watch`’, ‘`awatch`’, and ‘`rwatch`’.

Example

Setting a watchpoint on a variable in the `main` function:

```
(gdb)
-break-watch x
^done,wpt={number="2",exp="x"}
(gdb)
-exec-continue
^running
(gdb)
*stopped,reason="watchpoint-trigger",wpt={number="2",exp="x"},
value={old="-268439212",new="55"},
frame={func="main",args=[],file="recursive2.c",
fullname="/home/foo/bar/recursive2.c",line="5",arch="i386:x86_64"}
(gdb)
```

Setting a watchpoint on a variable local to a function. GDB will stop the program execution twice: first for the variable changing value, then for the watchpoint going out of scope.

```
(gdb)
-break-watch C
^done,wpt={number="5",exp="C"}
(gdb)
-exec-continue
^running
(gdb)
*stopped,reason="watchpoint-trigger",
wpt={number="5",exp="C"},value={old="-276895068",new="3"},
frame={func="callee4",args=[],
file="../../../devo/gdb/testsuite/gdb.mi/basics.c",
fullname="/home/foo/bar/devo/gdb/testsuite/gdb.mi/basics.c",line="13",
arch="i386:x86_64"}
(gdb)
-exec-continue
^running
(gdb)
*stopped,reason="watchpoint-scope",wpnum="5",
frame={func="callee3",args=[{name="strarg",
value="0x11940 \\"A string argument.\\""}]},
file="../../../devo/gdb/testsuite/gdb.mi/basics.c",
fullname="/home/foo/bar/devo/gdb/testsuite/gdb.mi/basics.c",line="18",
arch="i386:x86_64"}
(gdb)
```

Listing breakpoints and watchpoints, at different points in the program execution. Note that once the watchpoint goes out of scope, it is deleted.

```
(gdb)
-break-watch C
```

```

^done,wpt={number="2",exp="C"}
(gdb)
-break-list
^done,BreakpointTable={nr_rows="2",nr_cols="6",
hdr=[{width="3",alignment="-1",col_name="number",colhdr="Num"},
{width="14",alignment="-1",col_name="type",colhdr="Type"},
{width="4",alignment="-1",col_name="disp",colhdr="Disp"},
{width="3",alignment="-1",col_name="enabled",colhdr="Enb"},
{width="10",alignment="-1",col_name="addr",colhdr="Address"},
{width="40",alignment="2",col_name="what",colhdr="What"}],
body=[bkpt={number="1",type="breakpoint",disp="keep",enabled="y",
addr="0x00010734",func="callee4",
file="../../devo/gdb/testsuite/gdb.mi/basics.c",
fullname="/home/foo/devo/gdb/testsuite/gdb.mi/basics.c",line="8",thread-groups=["i1"],
times="1"},
bkpt={number="2",type="watchpoint",disp="keep",
enabled="y",addr="",what="C",thread-groups=["i1"],times="0"}]
(gdb)
-exec-continue
^running
(gdb)
*stopped,reason="watchpoint-trigger",wpt={number="2",exp="C"},
value={old="-276895068",new="3"},
frame={func="callee4",args=[],
file="../../devo/gdb/testsuite/gdb.mi/basics.c",
fullname="/home/foo/bar/devo/gdb/testsuite/gdb.mi/basics.c",line="13",
arch="i386:x86_64"}
(gdb)
-break-list
^done,BreakpointTable={nr_rows="2",nr_cols="6",
hdr=[{width="3",alignment="-1",col_name="number",colhdr="Num"},
{width="14",alignment="-1",col_name="type",colhdr="Type"},
{width="4",alignment="-1",col_name="disp",colhdr="Disp"},
{width="3",alignment="-1",col_name="enabled",colhdr="Enb"},
{width="10",alignment="-1",col_name="addr",colhdr="Address"},
{width="40",alignment="2",col_name="what",colhdr="What"}],
body=[bkpt={number="1",type="breakpoint",disp="keep",enabled="y",
addr="0x00010734",func="callee4",
file="../../devo/gdb/testsuite/gdb.mi/basics.c",
fullname="/home/foo/devo/gdb/testsuite/gdb.mi/basics.c",line="8",thread-groups=["i1"],
times="1"},
bkpt={number="2",type="watchpoint",disp="keep",
enabled="y",addr="",what="C",thread-groups=["i1"],times="-5"}]
(gdb)
-exec-continue
^running
^done,reason="watchpoint-scope",wpnum="2",
frame={func="callee3",args=[{name="strarg",
value="0x11940 \"A string argument.\""}],
file="../../devo/gdb/testsuite/gdb.mi/basics.c",
fullname="/home/foo/bar/devo/gdb/testsuite/gdb.mi/basics.c",line="18",
arch="i386:x86_64"}
(gdb)
-break-list
^done,BreakpointTable={nr_rows="1",nr_cols="6",
hdr=[{width="3",alignment="-1",col_name="number",colhdr="Num"},
{width="14",alignment="-1",col_name="type",colhdr="Type"},
{width="4",alignment="-1",col_name="disp",colhdr="Disp"},

```

```
{width="3",alignment="-1",col_name="enabled",colhdr="Enb"},
{width="10",alignment="-1",col_name="addr",colhdr="Address"},
{width="40",alignment="2",col_name="what",colhdr="What"}},
body=[bkpt={number="1",type="breakpoint",disp="keep",enabled="y",
addr="0x00010734",func="callee4",
file="../../devo/gdb/testsuite/gdb.mi/basics.c",
fullname="/home/foo/devo/gdb/testsuite/gdb.mi/basics.c",line="8",
thread-groups=["i1"],times="1"}]}
(gdb)
```

27.11 GDB/MI Catchpoint Commands

This section documents GDB/MI commands for manipulating catchpoints.

27.11.1 Shared Library GDB/MI Catchpoints

The `-catch-load` Command

Synopsis

```
-catch-load [ -t ] [ -d ] regexp
```

Add a catchpoint for library load events. If the `-t` option is used, the catchpoint is a temporary one (see Section 5.1.1 [Setting Breakpoints], page 60). If the `-d` option is used, the catchpoint is created in a disabled state. The `'regexp'` argument is a regular expression used to match the name of the loaded library.

GDB Command

The corresponding GDB command is `'catch load'`.

Example

```
-catch-load -t foo.so
^done,bkpt={number="1",type="catchpoint",disp="del",enabled="y",
what="load of library matching foo.so",catch-type="load",times="0"}
(gdb)
```

The `-catch-unload` Command

Synopsis

```
-catch-unload [ -t ] [ -d ] regexp
```

Add a catchpoint for library unload events. If the `-t` option is used, the catchpoint is a temporary one (see Section 5.1.1 [Setting Breakpoints], page 60). If the `-d` option is used, the catchpoint is created in a disabled state. The `'regexp'` argument is a regular expression used to match the name of the unloaded library.

GDB Command

The corresponding GDB command is `'catch unload'`.

Example

```
-catch-unload -d bar.so
^done,bkpt={number="2",type="catchpoint",disp="keep",enabled="n",
what="load of library matching bar.so",catch-type="unload",times="0"}
(gdb)
```

27.11.2 Ada Exception GDB/MI Catchpoints

The following GDB/MI commands can be used to create catchpoints that stop the execution when Ada exceptions are being raised.

The `-catch-assert` Command

Synopsis

```
-catch-assert [ -c condition ] [ -d ] [ -t ]
```

Add a catchpoint for failed Ada assertions.

The possible optional parameters for this command are:

- `'-c condition'` Make the catchpoint conditional on *condition*.
- `'-d'` Create a disabled catchpoint.
- `'-t'` Create a temporary catchpoint.

GDB Command

The corresponding GDB command is `'catch assert'`.

Example

```
-catch-assert
^done,bkptno="5",bkpt={number="5",type="breakpoint",disp="keep",
enabled="y",addr="0x000000000404888",what="failed Ada assertions",
thread-groups=["i1"],times="0",
original-location="__gnat_debug_raise_assert_failure"}
(gdb)
```

The `-catch-exception` Command

Synopsis

```
-catch-exception [ -c condition ] [ -d ] [ -e exception-name ]
[ -t ] [ -u ]
```

Add a catchpoint stopping when Ada exceptions are raised. By default, the command stops the program when any Ada exception gets raised. But it is also possible, by using some of the optional parameters described below, to create more selective catchpoints.

The possible optional parameters for this command are:

- `'-c condition'` Make the catchpoint conditional on *condition*.
- `'-d'` Create a disabled catchpoint.
- `'-e exception-name'` Only stop when *exception-name* is raised. This option cannot be used combined with `'-u'`.
- `'-t'` Create a temporary catchpoint.
- `'-u'` Stop only when an unhandled exception gets raised. This option cannot be used combined with `'-e'`.

GDB Command

The corresponding GDB commands are ‘catch exception’ and ‘catch exception unhandled’.

Example

```
-catch-exception -e Program_Error
^done,bkptno="4",bkpt={number="4",type="breakpoint",disp="keep",
enabled="y",addr="0x0000000000404874",
what="‘Program_Error’ Ada exception", thread-groups=["i1"],
times="0",original-location="__gnat_debug_raise_exception"}
(gdb)
```

The -catch-handlers Command

Synopsis

```
-catch-handlers [ -c condition ] [ -d ] [ -e exception-name ]
[ -t ]
```

Add a catchpoint stopping when Ada exceptions are handled. By default, the command stops the program when any Ada exception gets handled. But it is also possible, by using some of the optional parameters described below, to create more selective catchpoints.

The possible optional parameters for this command are:

- ‘-c *condition*’
Make the catchpoint conditional on *condition*.
- ‘-d’
Create a disabled catchpoint.
- ‘-e *exception-name*’
Only stop when *exception-name* is handled.
- ‘-t’
Create a temporary catchpoint.

GDB Command

The corresponding GDB command is ‘catch handlers’.

Example

```
-catch-handlers -e Constraint_Error
^done,bkptno="4",bkpt={number="4",type="breakpoint",disp="keep",
enabled="y",addr="0x0000000000402f68",
what="‘Constraint_Error’ Ada exception handlers",thread-groups=["i1"],
times="0",original-location="__gnat_begin_handler"}
(gdb)
```

27.11.3 C++ Exception GDB/MI Catchpoints

The following GDB/MI commands can be used to create catchpoints that stop the execution when C++ exceptions are being throw, rethrown, or caught.

The -catch-throw Command

Synopsis

```
-catch-throw [ -t ] [ -r regex]
```

Stop when the debuggee throws a C++ exception. If *regex* is given, then only exceptions whose type matches the regular expression will be caught.

If ‘-t’ is given, then the catchpoint is enabled only for one stop, the catchpoint is automatically deleted after stopping once for the event.

GDB Command

The corresponding GDB commands are ‘`catch throw`’ and ‘`tcatch throw`’ (see Section 5.1.3 [Set Catchpoints], page 71).

Example

```
-catch-throw -r exception_type
^done,bkpt={number="1",type="catchpoint",disp="keep",enabled="y",
  what="exception throw",catch-type="throw",
  thread-groups=["i1"],
  regex="exception_type",times="0"}
(gdb)
-exec-run
^running
(gdb)
~"\n"
~"Catchpoint 1 (exception thrown), 0x00007ffff7ae00ed
  in __cxa_throw () from /lib64/libstdc++.so.6\n"
*stopped,bkptno="1",reason="breakpoint-hit",disp="keep",
  frame={addr="0x00007ffff7ae00ed",func="__cxa_throw",
  args=[],from="/lib64/libstdc++.so.6",arch="i386:x86-64"},
  thread-id="1",stopped-threads="all",core="6"
(gdb)
```

The -catch-rethrow Command

Synopsis

```
-catch-rethrow [ -t ] [ -r regex]
```

Stop when a C++ exception is re-thrown. If *regex* is given, then only exceptions whose type matches the regular expression will be caught.

If ‘-t’ is given, then the catchpoint is enabled only for one stop, the catchpoint is automatically deleted after the first event is caught.

GDB Command

The corresponding GDB commands are ‘`catch rethrow`’ and ‘`tcatch rethrow`’ (see Section 5.1.3 [Set Catchpoints], page 71).

Example

```
-catch-rethrow -r exception_type
^done,bkpt={number="1",type="catchpoint",disp="keep",enabled="y",
  what="exception rethrow",catch-type="rethrow",
  thread-groups=["i1"],
  regex="exception_type",times="0"}
(gdb)
```

```

-exec-run
^running
(gdb)
~"\n"
~"Catchpoint 1 (exception rethrown), 0x00007ffff7ae00ed
  in __cxa_rethrow () from /lib64/libstdc++.so.6\n"
*stopped,bkptno="1",reason="breakpoint-hit",disp="keep",
  frame={addr="0x00007ffff7ae00ed",func="__cxa_rethrow",
  args=[],from="/lib64/libstdc++.so.6",arch="i386:x86-64"},
  thread-id="1",stopped-threads="all",core="6"
(gdb)

```

The -catch-catch Command

Synopsis

```
-catch-catch [ -t ] [ -r regexp]
```

Stop when the debuggee catches a C++ exception. If *regexp* is given, then only exceptions whose type matches the regular expression will be caught.

If ‘-t’ is given, then the catchpoint is enabled only for one stop, the catchpoint is automatically deleted after the first event is caught.

GDB Command

The corresponding GDB commands are ‘catch catch’ and ‘tcatch catch’ (see Section 5.1.3 [Set Catchpoints], page 71).

Example

```

-catch-catch -r exception_type
^done,bkpt={number="1",type="catchpoint",disp="keep",enabled="y",
  what="exception catch",catch-type="catch",
  thread-groups=["i1"],
  regexp="exception_type",times="0"}
(gdb)
-exec-run
^running
(gdb)
~"\n"
~"Catchpoint 1 (exception caught), 0x00007ffff7ae00ed
  in __cxa_begin_catch () from /lib64/libstdc++.so.6\n"
*stopped,bkptno="1",reason="breakpoint-hit",disp="keep",
  frame={addr="0x00007ffff7ae00ed",func="__cxa_begin_catch",
  args=[],from="/lib64/libstdc++.so.6",arch="i386:x86-64"},
  thread-id="1",stopped-threads="all",core="6"
(gdb)

```

27.12 GDB/MI Program Context

The -exec-arguments Command

Synopsis

```
-exec-arguments args
```

Set the inferior program arguments, to be used in the next ‘-exec-run’.

GDB Command

The corresponding GDB command is ‘set args’.

Example

```
(gdb)
-exec-arguments -v word
^done
(gdb)
```

The -environment-cd Command

Synopsis

```
-environment-cd pathdir
```

Set GDB’s working directory.

GDB Command

The corresponding GDB command is ‘cd’.

Example

```
(gdb)
-environment-cd /kwikemart/marge/ezannoni/flathead-dev/devo/gdb
^done
(gdb)
```

The -environment-directory Command

Synopsis

```
-environment-directory [ -r ] [ pathdir ]+
```

Add directories *pathdir* to beginning of search path for source files. If the ‘-r’ option is used, the search path is reset to the default search path. If directories *pathdir* are supplied in addition to the ‘-r’ option, the search path is first reset and then addition occurs as normal. Multiple directories may be specified, separated by blanks. Specifying multiple directories in a single command results in the directories added to the beginning of the search path in the same order they were presented in the command. If blanks are needed as part of a directory name, double-quotes should be used around the name. In the command output, the path will show up separated by the system directory-separator character. The directory-separator character must not be used in any directory name. If no directories are specified, the current search path is displayed.

GDB Command

The corresponding GDB command is ‘dir’.

Example

```
(gdb)
-environment-directory /kwikemart/marge/ezannoni/flathead-dev/devo/gdb
^done,source-path="/kwikemart/marge/ezannoni/flathead-dev/devo/gdb:$cd:$cwd"
(gdb)
-environment-directory ""
```

```

^done,source-path="/kwikemart/marge/ezannoni/flathead-dev/devo/gdb:$cdir:$cwd"
(gdb)
-environment-directory -r /home/jjohnstn/src/gdb /usr/src
^done,source-path="/home/jjohnstn/src/gdb:/usr/src:$cdir:$cwd"
(gdb)
-environment-directory -r
^done,source-path="$cdir:$cwd"
(gdb)

```

The `-environment-path` Command

Synopsis

```
-environment-path [ -r ] [ pathdir ]+
```

Add directories *pathdir* to beginning of search path for object files. If the ‘-r’ option is used, the search path is reset to the original search path that existed at gdb start-up. If directories *pathdir* are supplied in addition to the ‘-r’ option, the search path is first reset and then addition occurs as normal. Multiple directories may be specified, separated by blanks. Specifying multiple directories in a single command results in the directories added to the beginning of the search path in the same order they were presented in the command. If blanks are needed as part of a directory name, double-quotes should be used around the name. In the command output, the path will show up separated by the system directory-separator character. The directory-separator character must not be used in any directory name. If no directories are specified, the current path is displayed.

GDB Command

The corresponding GDB command is ‘`path`’.

Example

```

(gdb)
-environment-path
^done,path="/usr/bin"
(gdb)
-environment-path /kwikemart/marge/ezannoni/flathead-dev/ppc-eabi/gdb /bin
^done,path="/kwikemart/marge/ezannoni/flathead-dev/ppc-eabi/gdb:/bin:/usr/bin"
(gdb)
-environment-path -r /usr/local/bin
^done,path="/usr/local/bin:/usr/bin"
(gdb)

```

The `-environment-pwd` Command

Synopsis

```
-environment-pwd
```

Show the current working directory.

GDB Command

The corresponding GDB command is ‘`pwd`’.

Example

```
(gdb)
```

```
-environment-pwd
^done,cwd="/kwikemart/marge/ezannoni/flathead-dev/devo/gdb"
(gdb)
```

27.13 GDB/MI Thread Commands

The `-thread-info` Command

Synopsis

```
-thread-info [ thread-id ]
```

Reports information about either a specific thread, if the *thread-id* parameter is present, or about all threads. *thread-id* is the thread's global thread ID. When printing information about all threads, also reports the global ID of the current thread.

GDB Command

The `'info thread'` command prints the same information about all threads.

Result

The result contains the following attributes:

'threads' A list of threads. The format of the elements of the list is described in Section 27.7.6 [GDB/MI Thread Information], page 625.

'current-thread-id'

The global id of the currently selected thread. This field is omitted if there is no selected thread (for example, when the selected inferior is not running, and therefore has no threads) or if a *thread-id* argument was passed to the command.

Example

```
-thread-info
^done,threads=[
{id="2",target-id="Thread 0xb7e14b90 (LWP 21257)",
  frame={level="0",addr="0xffffe410",func="__kernel_vsyscall",
    args=[]},state="running"},
{id="1",target-id="Thread 0xb7e156b0 (LWP 21254)",
  frame={level="0",addr="0x0804891f",func="foo",
    args=[{name="i",value="10"}]},
  file="/tmp/a.c",fullname="/tmp/a.c",line="158",arch="i386:x86_64"},
  state="running"}],
current-thread-id="1"
(gdb)
```

The `-thread-list-ids` Command

Synopsis

```
-thread-list-ids
```

Produces a list of the currently known global GDB thread ids. At the end of the list it also prints the total number of such threads.

This command is retained for historical reasons, the `-thread-info` command should be used instead.

GDB Command

Part of ‘info threads’ supplies the same information.

Example

```
(gdb)
-threads-list-ids
^done,thread-ids={thread-id="3",thread-id="2",thread-id="1"},
current-thread-id="1",number-of-threads="3"
(gdb)
```

The -thread-select Command

Synopsis

```
-thread-select thread-id
```

Make thread with global thread number *thread-id* the current thread. It prints the number of the new current thread, and the topmost frame for that thread.

This command is deprecated in favor of explicitly using the ‘--thread’ option to each command.

GDB Command

The corresponding GDB command is ‘thread’.

Example

```
(gdb)
-exec-next
^running
(gdb)
*stopped,reason="end-stepping-range",thread-id="2",line="187",
file="../../../../devo/gdb/testsuite/gdb.threads/linux-dp.c"
(gdb)
-threads-list-ids
^done,
thread-ids={thread-id="3",thread-id="2",thread-id="1"},
number-of-threads="3"
(gdb)
-thread-select 3
^done,new-thread-id="3",
frame={level="0",func="vprintf",
args=[{name="format",value="0x8048e9c \"%s%c %d %c\\n\\n\"},
{name="arg",value="0x2"}],file="vprintf.c",line="31",arch="i386:x86_64"}
(gdb)
```

27.14 GDB/MI Ada Tasking Commands

The -ada-task-info Command

Synopsis

```
-ada-task-info [ task-id ]
```

Reports information about either a specific Ada task, if the *task-id* parameter is present, or about all Ada tasks.

GDB Command

The ‘`info tasks`’ command prints the same information about all Ada tasks (see Section 15.4.10.7 [Ada Tasks], page 256).

Result

The result is a table of Ada tasks. The following columns are defined for each Ada task:

‘ <code>current</code> ’	This field exists only for the current thread. It has the value ‘*’.
‘ <code>id</code> ’	The identifier that GDB uses to refer to the Ada task.
‘ <code>task-id</code> ’	The identifier that the target uses to refer to the Ada task.
‘ <code>thread-id</code> ’	The global thread identifier of the thread corresponding to the Ada task. This field should always exist, as Ada tasks are always implemented on top of a thread. But if GDB cannot find this corresponding thread for any reason, the field is omitted.
‘ <code>parent-id</code> ’	This field exists only when the task was created by another task. In this case, it provides the ID of the parent task.
‘ <code>priority</code> ’	The base priority of the task.
‘ <code>state</code> ’	The current state of the task. For a detailed description of the possible states, see Section 15.4.10.7 [Ada Tasks], page 256.
‘ <code>name</code> ’	The name of the task.

Example

```
-ada-task-info
^done,tasks={nr_rows="3",nr_cols="8",
hdr=[{width="1",alignment="-1",col_name="current",colhdr=""},
{width="3",alignment="1",col_name="id",colhdr="ID"},
{width="9",alignment="1",col_name="task-id",colhdr="TID"},
{width="4",alignment="1",col_name="thread-id",colhdr=""},
{width="4",alignment="1",col_name="parent-id",colhdr="P-ID"},
{width="3",alignment="1",col_name="priority",colhdr="Pri"},
{width="22",alignment="-1",col_name="state",colhdr="State"},
{width="1",alignment="2",col_name="name",colhdr="Name"}],
body=[{current="*",id="1",task-id=" 644010",thread-id="1",priority="48",
state="Child Termination Wait",name="main_task"}]}
```

27.15 GDB/MI Program Execution

These are the asynchronous commands which generate the out-of-band record ‘`*stopped`’. Currently GDB only really executes asynchronously with remote targets and this interaction is mimicked in other cases.

The `-exec-continue` Command

Synopsis

```
-exec-continue [--reverse] [--all|--thread-group N]
```

Resumes the execution of the inferior program, which will continue to execute until it reaches a debugger stop event. If the ‘**--reverse**’ option is specified, execution resumes in reverse until it reaches a stop event. Stop events may include

- breakpoints, watchpoints, tracepoints, or catchpoints
- signals or exceptions
- the end of the process (or its beginning under ‘**--reverse**’)
- the end or beginning of a replay log if one is being used.

In all-stop mode (see Section 5.5.1 [All-Stop Mode], page 96), may resume only one thread, or all threads, depending on the value of the ‘**scheduler-locking**’ variable. If ‘**--all**’ is specified, all threads (in all inferiors) will be resumed. The ‘**--all**’ option is ignored in all-stop mode. If the ‘**--thread-group**’ options is specified, then all threads in that thread group are resumed.

GDB Command

The corresponding GDB corresponding is ‘**continue**’.

Example

```
-exec-continue
^running
(gdb)
@Hello world
*stopped,reason="breakpoint-hit",disp="keep",bkptno="2",frame={
func="foo",args=[],file="hello.c",fullname="/home/foo/bar/hello.c",
line="13",arch="i386:x86_64"}
(gdb)
```

For a ‘**breakpoint-hit**’ stopped reason, when the breakpoint encountered has multiple locations, the field ‘**bkptno**’ is followed by the field ‘**locno**’.

```
-exec-continue
^running
(gdb)
@Hello world
*stopped,reason="breakpoint-hit",disp="keep",bkptno="2",locno="3",frame={
func="foo",args=[],file="hello.c",fullname="/home/foo/bar/hello.c",
line="13",arch="i386:x86_64"}
(gdb)
```

The -exec-finish Command

Synopsis

```
-exec-finish [--reverse]
```

Resumes the execution of the inferior program until the current function is exited. Displays the results returned by the function. If the ‘**--reverse**’ option is specified, resumes the reverse execution of the inferior program until the point where current function was called.

GDB Command

The corresponding GDB command is ‘`finish`’.

Example

Function returning `void`.

```
-exec-finish
^running
(gdb)
@hello from foo
*stopped,reason="function-finished",frame={func="main",args=[],
file="hello.c",fullname="/home/foo/bar/hello.c",line="7",arch="i386:x86_64"}
(gdb)
```

Function returning other than `void`. The name of the internal GDB variable storing the result is printed, together with the value itself.

```
-exec-finish
^running
(gdb)
*stopped,reason="function-finished",frame={addr="0x000107b0",func="foo",
args=[{name="a",value="1"},{name="b",value="9"}]},
file="recursive2.c",fullname="/home/foo/bar/recursive2.c",line="14",
arch="i386:x86_64"},
gdb-result-var="$1",return-value="0"
(gdb)
```

The `-exec-interrupt` Command

Synopsis

```
-exec-interrupt [--all|--thread-group N]
```

Interrupts the background execution of the target. Note how the token associated with the stop message is the one for the execution command that has been interrupted. The token for the interrupt itself only appears in the ‘`^done`’ output. If the user is trying to interrupt a non-running program, an error message will be printed.

Note that when asynchronous execution is enabled, this command is asynchronous just like other execution commands. That is, first the ‘`^done`’ response will be printed, and the target stop will be reported after that using the ‘`*stopped`’ notification.

In non-stop mode, only the context thread is interrupted by default. All threads (in all inferiors) will be interrupted if the ‘`--all`’ option is specified. If the ‘`--thread-group`’ option is specified, all threads in that group will be interrupted.

GDB Command

The corresponding GDB command is ‘`interrupt`’.

Example

```
(gdb)
111-exec-continue
111^running

(gdb)
222-exec-interrupt
```

```

222^done
(gdb)
111*stopped,signal-name="SIGINT",signal-meaning="Interrupt",
frame={addr="0x00010140",func="foo",args=[],file="try.c",
fullname="/home/foo/bar/try.c",line="13",arch="i386:x86_64"}
(gdb)

(gdb)
~exec-interrupt
^error,msg="mi_cmd_exec_interrupt: Inferior not executing."
(gdb)

```

The `-exec-jump` Command

Synopsis

```
-exec-jump locspec
```

Resumes execution of the inferior program at the address to which *locspec* resolves. See Section 9.2 [Location Specifications], page 128, for a description of the different forms of *locspec*.

GDB Command

The corresponding GDB command is ‘`jump`’.

Example

```

~exec-jump foo.c:10
*running,thread-id="all"
^running

```

The `-exec-next` Command

Synopsis

```
-exec-next [--reverse]
```

Resumes execution of the inferior program, stopping when the beginning of the next source line is reached.

If the ‘`--reverse`’ option is specified, resumes reverse execution of the inferior program, stopping at the beginning of the previous source line. If you issue this command on the first line of a function, it will take you back to the caller of that function, to the source line where the function was called.

GDB Command

The corresponding GDB command is ‘`next`’.

Example

```

~exec-next
^running
(gdb)
*stopped,reason="end-stepping-range",line="8",file="hello.c"
(gdb)

```

The `-exec-next-instruction` Command

Synopsis

```
-exec-next-instruction [--reverse]
```

Executes one machine instruction. If the instruction is a function call, continues until the function returns. If the program stops at an instruction in the middle of a source line, the address will be printed as well.

If the ‘**--reverse**’ option is specified, resumes reverse execution of the inferior program, stopping at the previous instruction. If the previously executed instruction was a return from another function, it will continue to execute in reverse until the call to that function (from the current stack frame) is reached.

GDB Command

The corresponding GDB command is ‘**nexti**’.

Example

```
(gdb)
-exec-next-instruction
^running

(gdb)
*stopped,reason="end-stepping-range",
addr="0x000100d4",line="5",file="hello.c"
(gdb)
```

The -exec-return Command

Synopsis

```
-exec-return
```

Makes current function return immediately. Doesn’t execute the inferior. Displays the new current frame.

GDB Command

The corresponding GDB command is ‘**return**’.

Example

```
(gdb)
200-break-insert callee4
200^done,bkpt={number="1",addr="0x00010734",
file="../../../../devo/gdb/testsuite/gdb.mi/basics.c",line="8"}
(gdb)
000-exec-run
000^running
(gdb)
000*stopped,reason="breakpoint-hit",disp="keep",bkptno="1",
frame={func="callee4",args=[],
file="../../../../devo/gdb/testsuite/gdb.mi/basics.c",
fullname="/home/foo/bar/devo/gdb/testsuite/gdb.mi/basics.c",line="8",
arch="i386:x86_64"}
(gdb)
205-break-delete
205^done
(gdb)
```

```

111-exec-return
111^done,frame={level="0",func="callee3",
args=[{name="strarg",
value="0x11940 \"A string argument.\""}],
file="../../devo/gdb/testsuite/gdb.mi/basics.c",
fullname="/home/foo/bar/devo/gdb/testsuite/gdb.mi/basics.c",line="18",
arch="i386:x86_64"}
(gdb)

```

The `-exec-run` Command

Synopsis

```
-exec-run [ --all | --thread-group N ] [ --start ]
```

Starts execution of the inferior from the beginning. The inferior executes until either a breakpoint is encountered or the program exits. In the latter case the output will include an exit code, if the program has exited exceptionally.

When neither the `--all` nor the `--thread-group` option is specified, the current inferior is started. If the `--thread-group` option is specified, it should refer to a thread group of type `process`, and that thread group will be started. If the `--all` option is specified, then all inferiors will be started.

Using the `--start` option instructs the debugger to stop the execution at the start of the inferior's main subprogram, following the same behavior as the `start` command (see Section 4.2 [Starting], page 34).

GDB Command

The corresponding GDB command is `run`.

Examples

```

(gdb)
-break-insert main
^done,bkpt={number="1",addr="0x0001072c",file="recursive2.c",line="4"}
(gdb)
-exec-run
^running
(gdb)
*stopped,reason="breakpoint-hit",disp="keep",bkptno="1",
frame={func="main",args=[],file="recursive2.c",
fullname="/home/foo/bar/recursive2.c",line="4",arch="i386:x86_64"}
(gdb)

```

Program exited normally:

```

(gdb)
-exec-run
^running
(gdb)
x = 55
*stopped,reason="exited-normally"
(gdb)

```

Program exited exceptionally:

```

(gdb)
-exec-run
^running

```

```
(gdb)
x = 55
*stopped,reason="exited",exit-code="01"
(gdb)
```

Another way the program can terminate is if it receives a signal such as `SIGINT`. In this case, GDB/MI displays this:

```
(gdb)
*stopped,reason="exited-signalled",signal-name="SIGINT",
signal-meaning="Interrupt"
```

The `-exec-step` Command

Synopsis

```
-exec-step [--reverse]
```

Resumes execution of the inferior program, stopping when the beginning of the next source line is reached, if the next source line is not a function call. If it is, stop at the first instruction of the called function. If the ‘`--reverse`’ option is specified, resumes reverse execution of the inferior program, stopping at the beginning of the previously executed source line.

GDB Command

The corresponding GDB command is ‘`step`’.

Example

Stepping into a function:

```
-exec-step
^running
(gdb)
*stopped,reason="end-stepping-range",
frame={func="foo",args=[{name="a",value="10"},
{name="b",value="0"}],file="recursive2.c",
fullname="/home/foo/bar/recursive2.c",line="11",arch="i386:x86_64"}
(gdb)
```

Regular stepping:

```
-exec-step
^running
(gdb)
*stopped,reason="end-stepping-range",line="14",file="recursive2.c"
(gdb)
```

The `-exec-step-instruction` Command

Synopsis

```
-exec-step-instruction [--reverse]
```

Resumes the inferior which executes one machine instruction. If the ‘`--reverse`’ option is specified, resumes reverse execution of the inferior program, stopping at the previously executed instruction. The output, once GDB has stopped, will vary depending on whether we have stopped in the middle of a source line or not. In the former case, the address at which the program stopped will be printed as well.

GDB Command

The corresponding GDB command is ‘stepi’.

Example

```
(gdb)
-exec-step-instruction
^running

(gdb)
*stopped,reason="end-stepping-range",
frame={func="foo",args=[],file="try.c",
fullname="/home/foo/bar/try.c",line="10",arch="i386:x86_64"}
(gdb)
-exec-step-instruction
^running

(gdb)
*stopped,reason="end-stepping-range",
frame={addr="0x000100f4",func="foo",args=[],file="try.c",
fullname="/home/foo/bar/try.c",line="10",arch="i386:x86_64"}
(gdb)
```

The -exec-until Command

Synopsis

```
-exec-until [ locspec ]
```

Executes the inferior until it reaches the address to which *locspec* resolves. If there is no argument, the inferior executes until it reaches a source line greater than the current one. The reason for stopping in this case will be ‘location-reached’.

GDB Command

The corresponding GDB command is ‘until’.

Example

```
(gdb)
-exec-until recursive2.c:6
^running
(gdb)
x = 55
*stopped,reason="location-reached",frame={func="main",args=[],
file="recursive2.c",fullname="/home/foo/bar/recursive2.c",line="6",
arch="i386:x86_64"}
(gdb)
```

27.16 GDB/MI Stack Manipulation Commands

The -enable-frame-filters Command

```
-enable-frame-filters
```

GDB allows Python-based frame filters to affect the output of the MI commands relating to stack traces. As there is no way to implement this in a fully backward-compatible way, a front end must request that this functionality be enabled.

Once enabled, this feature cannot be disabled.

Note that if Python support has not been compiled into GDB, this command will still succeed (and do nothing).

The `-stack-info-frame` Command

Synopsis

```
-stack-info-frame
```

Get info on the selected frame.

GDB Command

The corresponding GDB command is ‘`info frame`’ or ‘`frame`’ (without arguments).

Example

```
(gdb)
-stack-info-frame
^done,frame={level="1",addr="0x0001076c",func="callee3",
file="../../../../devo/gdb/testsuite/gdb.mi/basics.c",
fullname="/home/foo/bar/devo/gdb/testsuite/gdb.mi/basics.c",line="17",
arch="i386:x86_64"}
```

The `-stack-info-depth` Command

Synopsis

```
-stack-info-depth [ max-depth ]
```

Return the depth of the stack. If the integer argument *max-depth* is specified, do not count beyond *max-depth* frames.

GDB Command

There’s no equivalent GDB command.

Example

For a stack with frame levels 0 through 11:

```
(gdb)
-stack-info-depth
^done,depth="12"
(gdb)
-stack-info-depth 4
^done,depth="4"
(gdb)
-stack-info-depth 12
^done,depth="12"
(gdb)
-stack-info-depth 11
^done,depth="11"
(gdb)
-stack-info-depth 13
^done,depth="12"
(gdb)
```

The `-stack-list-arguments` Command

Synopsis

```
-stack-list-arguments [ --no-frame-filters ] [ --skip-unavailable ] print-values
    [ low-frame high-frame ]
```

Display a list of the arguments for the frames between *low-frame* and *high-frame* (inclusive). If *low-frame* and *high-frame* are not provided, list the arguments for the whole call stack. If the two arguments are equal, show the single frame at the corresponding level. It is an error if *low-frame* is larger than the actual number of frames. On the other hand, *high-frame* may be larger than the actual number of frames, in which case only existing frames will be returned.

If *print-values* is 0 or `--no-values`, print only the names of the variables; if it is 1 or `--all-values`, print also their values; and if it is 2 or `--simple-values`, print the name, type and value for simple data types, and the name and type for arrays, structures and unions. If the option `--no-frame-filters` is supplied, then Python frame filters will not be executed.

If the `--skip-unavailable` option is specified, arguments that are not available are not listed. Partially available arguments are still displayed, however.

Use of this command to obtain arguments in a single frame is deprecated in favor of the `-stack-list-variables` command.

GDB Command

GDB does not have an equivalent command. `gdbtk` has a `'gdb_get_args'` command which partially overlaps with the functionality of `'-stack-list-arguments'`.

Example

```
(gdb)
-stack-list-frames
^done,
stack=[
  frame={level="0",addr="0x00010734",func="callee4",
  file="../../../../devo/gdb/testsuite/gdb.mi/basics.c",
  fullname="/home/foo/bar/devo/gdb/testsuite/gdb.mi/basics.c",line="8",
  arch="i386:x86_64"},
  frame={level="1",addr="0x0001076c",func="callee3",
  file="../../../../devo/gdb/testsuite/gdb.mi/basics.c",
  fullname="/home/foo/bar/devo/gdb/testsuite/gdb.mi/basics.c",line="17",
  arch="i386:x86_64"},
  frame={level="2",addr="0x0001078c",func="callee2",
  file="../../../../devo/gdb/testsuite/gdb.mi/basics.c",
  fullname="/home/foo/bar/devo/gdb/testsuite/gdb.mi/basics.c",line="22",
  arch="i386:x86_64"},
  frame={level="3",addr="0x000107b4",func="callee1",
  file="../../../../devo/gdb/testsuite/gdb.mi/basics.c",
  fullname="/home/foo/bar/devo/gdb/testsuite/gdb.mi/basics.c",line="27",
  arch="i386:x86_64"},
  frame={level="4",addr="0x000107e0",func="main",
  file="../../../../devo/gdb/testsuite/gdb.mi/basics.c",
  fullname="/home/foo/bar/devo/gdb/testsuite/gdb.mi/basics.c",line="32",
  arch="i386:x86_64"}]
(gdb)
```

```

-stack-list-arguments 0
^done,
stack-args=[
frame={level="0",args=[]},
frame={level="1",args=[name="strarg"]},
frame={level="2",args=[name="intarg",name="strarg"]},
frame={level="3",args=[name="intarg",name="strarg",name="fltarg"]},
frame={level="4",args=[]}]
(gdb)
-stack-list-arguments 1
^done,
stack-args=[
frame={level="0",args=[]},
frame={level="1",
  args=[{name="strarg",value="0x11940 \"A string argument.\""}]},
frame={level="2",args=[
  {name="intarg",value="2"},
  {name="strarg",value="0x11940 \"A string argument.\""}]},
frame={level="3",args=[
  {name="intarg",value="2"},
  {name="strarg",value="0x11940 \"A string argument.\""},
  {name="fltarg",value="3.5"}]},
frame={level="4",args=[]}]
(gdb)
-stack-list-arguments 0 2 2
^done,stack-args=[frame={level="2",args=[name="intarg",name="strarg"]}]]
(gdb)
-stack-list-arguments 1 2 2
^done,stack-args=[frame={level="2",
args=[{name="intarg",value="2"},
{name="strarg",value="0x11940 \"A string argument.\""}]}]]
(gdb)

```

The `-stack-list-frames` Command

Synopsis

```
-stack-list-frames [ --no-frame-filters low-frame high-frame ]
```

List the frames currently on the stack. For each frame it displays the following info:

<code>'level'</code>	The frame number, 0 being the topmost frame, i.e., the innermost function.
<code>'addr'</code>	The <code>\$pc</code> value for that frame.
<code>'func'</code>	Function name.
<code>'file'</code>	File name of the source file where the function lives.
<code>'fullname'</code>	The full file name of the source file where the function lives.
<code>'line'</code>	Line number corresponding to the <code>\$pc</code> .
<code>'from'</code>	The shared library where this function is defined. This is only given if the frame's function is not known.
<code>'arch'</code>	Frame's architecture.

If invoked without arguments, this command prints a backtrace for the whole stack. If given two integer arguments, it shows the frames whose levels are between the two arguments

(inclusive). If the two arguments are equal, it shows the single frame at the corresponding level. It is an error if *low-frame* is larger than the actual number of frames. On the other hand, *high-frame* may be larger than the actual number of frames, in which case only existing frames will be returned. If the option `--no-frame-filters` is supplied, then Python frame filters will not be executed.

GDB Command

The corresponding GDB commands are ‘`backtrace`’ and ‘`where`’.

Example

Full stack backtrace:

```
(gdb)
-stack-list-frames
^done,stack=
[frame={level="0",addr="0x0001076c",func="foo",
  file="recursive2.c",fullname="/home/foo/bar/recursive2.c",line="11",
  arch="i386:x86_64"},
frame={level="1",addr="0x000107a4",func="foo",
  file="recursive2.c",fullname="/home/foo/bar/recursive2.c",line="14",
  arch="i386:x86_64"},
frame={level="2",addr="0x000107a4",func="foo",
  file="recursive2.c",fullname="/home/foo/bar/recursive2.c",line="14",
  arch="i386:x86_64"},
frame={level="3",addr="0x000107a4",func="foo",
  file="recursive2.c",fullname="/home/foo/bar/recursive2.c",line="14",
  arch="i386:x86_64"},
frame={level="4",addr="0x000107a4",func="foo",
  file="recursive2.c",fullname="/home/foo/bar/recursive2.c",line="14",
  arch="i386:x86_64"},
frame={level="5",addr="0x000107a4",func="foo",
  file="recursive2.c",fullname="/home/foo/bar/recursive2.c",line="14",
  arch="i386:x86_64"},
frame={level="6",addr="0x000107a4",func="foo",
  file="recursive2.c",fullname="/home/foo/bar/recursive2.c",line="14",
  arch="i386:x86_64"},
frame={level="7",addr="0x000107a4",func="foo",
  file="recursive2.c",fullname="/home/foo/bar/recursive2.c",line="14",
  arch="i386:x86_64"},
frame={level="8",addr="0x000107a4",func="foo",
  file="recursive2.c",fullname="/home/foo/bar/recursive2.c",line="14",
  arch="i386:x86_64"},
frame={level="9",addr="0x000107a4",func="foo",
  file="recursive2.c",fullname="/home/foo/bar/recursive2.c",line="14",
  arch="i386:x86_64"},
frame={level="10",addr="0x000107a4",func="foo",
  file="recursive2.c",fullname="/home/foo/bar/recursive2.c",line="14",
  arch="i386:x86_64"},
frame={level="11",addr="0x00010738",func="main",
  file="recursive2.c",fullname="/home/foo/bar/recursive2.c",line="4",
  arch="i386:x86_64"}]
(gdb)
```

Show frames between *low-frame* and *high-frame*:

```
(gdb)
-stack-list-frames 3 5
^done,stack=
```



```
[frame={level="3",addr="0x000107a4",func="foo",
  file="recursive2.c",fullname="/home/foo/bar/recursive2.c",line="14",
  arch="i386:x86_64"},
frame={level="4",addr="0x000107a4",func="foo",
  file="recursive2.c",fullname="/home/foo/bar/recursive2.c",line="14",
  arch="i386:x86_64"},
frame={level="5",addr="0x000107a4",func="foo",
  file="recursive2.c",fullname="/home/foo/bar/recursive2.c",line="14",
  arch="i386:x86_64"}]
(gdb)
```

Show a single frame:

```
(gdb)
-stack-list-frames 3 3
^done,stack=
[frame={level="3",addr="0x000107a4",func="foo",
  file="recursive2.c",fullname="/home/foo/bar/recursive2.c",line="14",
  arch="i386:x86_64"}]
(gdb)
```

The `-stack-list-locals` Command

Synopsis

```
-stack-list-locals [ --no-frame-filters ] [ --skip-unavailable ] print-values
```

Display the local variable names for the selected frame. If *print-values* is 0 or `--no-values`, print only the names of the variables; if it is 1 or `--all-values`, print also their values; and if it is 2 or `--simple-values`, print the name, type and value for simple data types, and the name and type for arrays, structures and unions. In this last case, a frontend can immediately display the value of simple data types and create variable objects for other data types when the user wishes to explore their values in more detail. If the option `--no-frame-filters` is supplied, then Python frame filters will not be executed.

If the `--skip-unavailable` option is specified, local variables that are not available are not listed. Partially available local variables are still displayed, however.

This command is deprecated in favor of the `'-stack-list-variables'` command.

GDB Command

`'info locals'` in GDB, `'gdb_get_locals'` in gdbtk.

Example

```
(gdb)
-stack-list-locals 0
^done,locals=[name="A",name="B",name="C"]
(gdb)
-stack-list-locals --all-values
^done,locals=[{name="A",value="1"},{name="B",value="2"},
  {name="C",value="{1, 2, 3}"}]
-stack-list-locals --simple-values
^done,locals=[{name="A",type="int",value="1"},
  {name="B",type="int",value="2"},{name="C",type="int [3]"}]
(gdb)
```

The `-stack-list-variables` Command

Synopsis

```
-stack-list-variables [ --no-frame-filters ] [ --skip-unavailable ] print-values
```

Display the names of local variables and function arguments for the selected frame. If *print-values* is 0 or `--no-values`, print only the names of the variables; if it is 1 or `--all-values`, print also their values; and if it is 2 or `--simple-values`, print the name, type and value for simple data types, and the name and type for arrays, structures and unions. If the option `--no-frame-filters` is supplied, then Python frame filters will not be executed.

If the `--skip-unavailable` option is specified, local variables and arguments that are not available are not listed. Partially available arguments and local variables are still displayed, however.

Example

```
(gdb)
-stack-list-variables --thread 1 --frame 0 --all-values
^done,variables=[{name="x",value="11"},{name="s",value="{a = 1, b = 2}"}]
(gdb)
```

The `-stack-select-frame` Command

Synopsis

```
-stack-select-frame framenum
```

Change the selected frame. Select a different frame *framenum* on the stack.

This command is deprecated in favor of passing the `--frame` option to every command.

GDB Command

The corresponding GDB commands are `'frame'`, `'up'`, `'down'`, `'select-frame'`, `'up-silent'`, and `'down-silent'`.

Example

```
(gdb)
-stack-select-frame 2
^done
(gdb)
```

27.17 GDB/MI Variable Objects

Introduction to Variable Objects

Variable objects are "object-oriented" MI interface for examining and changing values of expressions. Unlike some other MI interfaces that work with expressions, variable objects are specifically designed for simple and efficient presentation in the frontend. A variable object is identified by string name. When a variable object is created, the frontend specifies the expression for that variable object. The expression can be a simple variable, or it can be an arbitrary complex expression, and can even involve CPU registers. After creating a variable object, the frontend can invoke other variable object operations—for example to obtain or change the value of a variable object, or to change display format.

Variable objects have hierarchical tree structure. Any variable object that corresponds to a composite type, such as structure in C, has a number of child variable objects, for example

corresponding to each element of a structure. A child variable object can itself have children, recursively. Recursion ends when we reach leaf variable objects, which always have built-in types. Child variable objects are created only by explicit request, so if a frontend is not interested in the children of a particular variable object, no child will be created.

For a leaf variable object it is possible to obtain its value as a string, or set the value from a string. String value can be also obtained for a non-leaf variable object, but it's generally a string that only indicates the type of the object, and does not list its contents. Assignment to a non-leaf variable object is not allowed.

A frontend does not need to read the values of all variable objects each time the program stops. Instead, MI provides an update command that lists all variable objects whose values has changed since the last update operation. This considerably reduces the amount of data that must be transferred to the frontend. As noted above, children variable objects are created on demand, and only leaf variable objects have a real value. As result, gdb will read target memory only for leaf variables that frontend has created.

The automatic update is not always desirable. For example, a frontend might want to keep a value of some expression for future reference, and never update it. For another example, fetching memory is relatively slow for embedded targets, so a frontend might want to disable automatic update for the variables that are either not visible on the screen, or “closed”. This is possible using so called “frozen variable objects”. Such variable objects are never implicitly updated.

Variable objects can be either *fixed* or *floating*. For the fixed variable object, the expression is parsed when the variable object is created, including associating identifiers to specific variables. The meaning of expression never changes. For a floating variable object the values of variables whose names appear in the expressions are re-evaluated every time in the context of the current frame. Consider this example:

```
void do_work(...)
{
    struct work_state state;

    if (...)
        do_work(...);
}
```

If a fixed variable object for the `state` variable is created in this function, and we enter the recursive call, the variable object will report the value of `state` in the top-level `do_work` invocation. On the other hand, a floating variable object will report the value of `state` in the current frame.

If an expression specified when creating a fixed variable object refers to a local variable, the variable object becomes bound to the thread and frame in which the variable object is created. When such variable object is updated, GDB makes sure that the thread/frame combination the variable object is bound to still exists, and re-evaluates the variable object in context of that thread/frame.

The following is the complete set of GDB/MI operations defined to access this functionality:

Operation	Description
<code>-enable-pretty-printing</code>	enable Python-based pretty-printing

<code>-var-create</code>	create a variable object
<code>-var-delete</code>	delete the variable object and/or its children
<code>-var-set-format</code>	set the display format of this variable
<code>-var-show-format</code>	show the display format of this variable
<code>-var-info-num-children</code>	tells how many children this object has
<code>-var-list-children</code>	return a list of the object's children
<code>-var-info-type</code>	show the type of this variable object
<code>-var-info-expression</code>	print parent-relative expression that this variable object represents
<code>-var-info-path-expression</code>	print full expression that this variable object represents
<code>-var-show-attributes</code>	is this variable editable? does it exist here?
<code>-var-evaluate-expression</code>	get the value of this variable
<code>-var-assign</code>	set the value of this variable
<code>-var-update</code>	update the variable and its children
<code>-var-set-frozen</code>	set frozenness attribute
<code>-var-set-update-range</code>	set range of children to display on update

In the next subsection we describe each operation in detail and suggest how it can be used.

Description And Use of Operations on Variable Objects

The `-enable-pretty-printing` Command

`-enable-pretty-printing`

GDB allows Python-based visualizers to affect the output of the MI variable object commands. However, because there was no way to implement this in a fully backward-compatible way, a front end must request that this functionality be enabled.

Once enabled, this feature cannot be disabled.

Note that if Python support has not been compiled into GDB, this command will still succeed (and do nothing).

The `-var-create` Command

Synopsis

```
-var-create {name | "-"}
           {frame-addr | "*" | "@"} expression
```

This operation creates a variable object, which allows the monitoring of a variable, the result of an expression, a memory cell or a CPU register.

The *name* parameter is the string by which the object can be referenced. It must be unique. If ‘-’ is specified, the varobj system will generate a string “varNNNNNN” automatically. It will be unique provided that one does not specify *name* of that format. The command fails if a duplicate name is found.

The frame under which the expression should be evaluated can be specified by *frame-addr*. A ‘*’ indicates that the current frame should be used. A ‘@’ indicates that a floating variable object must be created.

expression is any expression valid on the current language set (must not begin with a ‘*’), or one of the following:

- ‘**addr*’, where *addr* is the address of a memory cell
- ‘**addr-addr*’ — a memory address range (TBD)
- ‘\$*regname*’ — a CPU register name

A varobj’s contents may be provided by a Python-based pretty-printer. In this case the varobj is known as a *dynamic varobj*. Dynamic varobjs have slightly different semantics in some cases. If the `-enable-pretty-printing` command is not sent, then GDB will never create a dynamic varobj. This ensures backward compatibility for existing clients.

Result

This operation returns attributes of the newly-created varobj. These are:

‘name’	The name of the varobj.
‘numchild’	The number of children of the varobj. This number is not necessarily reliable for a dynamic varobj. Instead, you must examine the ‘has_more’ attribute.
‘value’	The varobj’s scalar value. For a varobj whose type is some sort of aggregate (e.g., a <code>struct</code>), this value will not be interesting. For a dynamic varobj, this value comes directly from the Python pretty-printer object’s <code>to_string</code> method.
‘type’	The varobj’s type. This is a string representation of the type, as would be printed by the GDB CLI. If ‘print object’ (see Section 10.9 [Print Settings], page 159) is set to <code>on</code> , the <i>actual</i> (derived) type of the object is shown rather than the <i>declared</i> one.
‘thread-id’	If a variable object is bound to a specific thread, then this is the thread’s global identifier.
‘has_more’	For a dynamic varobj, this indicates whether there appear to be any children available. For a non-dynamic varobj, this will be 0.
‘dynamic’	This attribute will be present and have the value ‘1’ if the varobj is a dynamic varobj. If the varobj is not a dynamic varobj, then this attribute will not be present.
‘displayhint’	A dynamic varobj can supply a display hint to the front end. The value comes directly from the Python pretty-printer object’s <code>display_hint</code> method. See Section 23.3.2.6 [Pretty Printing API], page 430.

Typical output will look like this:

```
name="name",numchild="N",type="type",thread-id="M",
has_more="has_more"
```

The -var-delete Command

Synopsis

```
-var-delete [ -c ] name
```

Deletes a previously created variable object and all of its children. With the ‘-c’ option, just deletes the children.

Returns an error if the object *name* is not found.

The -var-set-format Command

Synopsis

```
-var-set-format name format-spec
```

Sets the output format for the value of the object *name* to be *format-spec*.

The syntax for the *format-spec* is as follows:

```
format-spec ↦  
{binary | decimal | hexadecimal | octal | natural | zero-hexadecimal}
```

The natural format is the default format chosen automatically based on the variable type (like decimal for an `int`, hex for pointers, etc.).

The zero-hexadecimal format has a representation similar to hexadecimal but with padding zeroes to the left of the value. For example, a 32-bit hexadecimal value of 0x1234 would be represented as 0x00001234 in the zero-hexadecimal format.

For a variable with children, the format is set only on the variable itself, and the children are not affected.

The -var-show-format Command

Synopsis

```
-var-show-format name
```

Returns the format used to display the value of the object *name*.

```
format ↦  
format-spec
```

The -var-info-num-children Command

Synopsis

```
-var-info-num-children name
```

Returns the number of children of a variable object *name*:

```
numchild=n
```

Note that this number is not completely reliable for a dynamic varobj. It will return the current number of children, but more children may be available.

The -var-list-children Command

Synopsis

```
-var-list-children [print-values] name [from to]
```

Return a list of the children of the specified variable object and create variable objects for them, if they do not already exist. With a single argument or if *print-values* has a

value of 0 or `--no-values`, print only the names of the variables; if `print-values` is 1 or `--all-values`, also print their values; and if it is 2 or `--simple-values` print the name and value for simple data types and just the name for arrays, structures and unions.

`from` and `to`, if specified, indicate the range of children to report. If `from` or `to` is less than zero, the range is reset and all children will be reported. Otherwise, children starting at `from` (zero-based) and up to and excluding `to` will be reported.

If a child range is requested, it will only affect the current call to `-var-list-children`, but not future calls to `-var-update`. For this, you must instead use `-var-set-update-range`. The intent of this approach is to enable a front end to implement any update approach it likes; for example, scrolling a view may cause the front end to request more children with `-var-list-children`, and then the front end could call `-var-set-update-range` with a different range to ensure that future updates are restricted to just the visible items.

For each child the following results are returned:

<i>name</i>	Name of the variable object created for this child.
<i>exp</i>	The expression to be shown to the user by the front end to designate this child. For example this may be the name of a structure member. For a dynamic varobj, this value cannot be used to form an expression. There is no way to do this at all with a dynamic varobj. For C/C++ structures there are several pseudo children returned to designate access qualifiers. For these pseudo children <i>exp</i> is ‘public’, ‘private’, or ‘protected’. In this case the type and value are not present. A dynamic varobj will not report the access qualifying pseudo-children, regardless of the language. This information is not available at all with a dynamic varobj.
<i>numchild</i>	Number of children this child has. For a dynamic varobj, this will be 0.
<i>type</i>	The type of the child. If ‘print object’ (see Section 10.9 [Print Settings], page 159) is set to on , the <i>actual</i> (derived) type of the object is shown rather than the <i>declared</i> one.
<i>value</i>	If values were requested, this is the value.
<i>thread-id</i>	If this variable object is associated with a thread, this is the thread’s global thread id. Otherwise this result is not present.
<i>frozen</i>	If the variable object is frozen, this variable will be present with a value of 1.
<i>displayhint</i>	A dynamic varobj can supply a display hint to the front end. The value comes directly from the Python pretty-printer object’s <code>display_hint</code> method. See Section 23.3.2.6 [Pretty Printing API], page 430.
<i>dynamic</i>	This attribute will be present and have the value ‘1’ if the varobj is a dynamic varobj. If the varobj is not a dynamic varobj, then this attribute will not be present.

The result may have its own attributes:

`'displayhint'`

A dynamic varobj can supply a display hint to the front end. The value comes directly from the Python pretty-printer object's `display_hint` method. See Section 23.3.2.6 [Pretty Printing API], page 430.

`'has_more'`

This is an integer attribute which is nonzero if there are children remaining after the end of the selected range.

Example

```
(gdb)
-var-list-children n
^done,numchild=n,children=[child={name=name,exp=exp,
numchild=n,type=type},(repeats N times)]
(gdb)
-var-list-children --all-values n
^done,numchild=n,children=[child={name=name,exp=exp,
numchild=n,value=value,type=type},(repeats N times)]
```

The `-var-info-type` Command

Synopsis

```
-var-info-type name
```

Returns the type of the specified variable *name*. The type is returned as a string in the same format as it is output by the GDB CLI:

```
type=typename
```

The `-var-info-expression` Command

Synopsis

```
-var-info-expression name
```

Returns a string that is suitable for presenting this variable object in user interface. The string is generally not valid expression in the current language, and cannot be evaluated.

For example, if *a* is an array, and variable object *A* was created for *a*, then we'll get this output:

```
(gdb) -var-info-expression A.1
^done,lang="C",exp="1"
```

Here, the value of *lang* is the language name, which can be found in Section 15.4 [Supported Languages], page 233.

Note that the output of the `-var-list-children` command also includes those expressions, so the `-var-info-expression` command is of limited use.

The `-var-info-path-expression` Command

Synopsis

```
-var-info-path-expression name
```


Returns an expression that can be evaluated in the current context and will yield the same value that a variable object has. Compare this with the `-var-info-expression` command, which result can be used only for UI presentation. Typical use of the `-var-info-path-expression` command is creating a watchpoint from a variable object.

This command is currently not valid for children of a dynamic varobj, and will give an error when invoked on one.

For example, suppose `C` is a C++ class, derived from class `Base`, and that the `Base` class has a member called `m_size`. Assume a variable `c` has the type of `C` and a variable object `C` was created for variable `c`. Then, we'll get this output:

```
(gdb) -var-info-path-expression C.Base.public.m_size
^done,path_expr=((Base)c).m_size
```

The `-var-show-attributes` Command

Synopsis

```
-var-show-attributes name
```

List attributes of the specified variable object *name*:

```
status=attr [ ( ,attr )* ]
```

where *attr* is { { `editable` | `noneditable` } | TBD }.

The `-var-evaluate-expression` Command

Synopsis

```
-var-evaluate-expression [-f format-spec] name
```

Evaluates the expression that is represented by the specified variable object and returns its value as a string. The format of the string can be specified with the `-f` option. The possible values of this option are the same as for `-var-set-format` (see `[-var-set-format]`, page 664). If the `-f` option is not specified, the current display format will be used. The current display format can be changed using the `-var-set-format` command.

```
value=value
```

Note that one must invoke `-var-list-children` for a variable before the value of a child variable can be evaluated.

The `-var-assign` Command

Synopsis

```
-var-assign name expression
```

Assigns the value of *expression* to the variable object specified by *name*. The object must be `'editable'`. If the variable's value is altered by the assign, the variable will show up in any subsequent `-var-update` list.

Example

```
(gdb)
-var-assign var1 3
^done,value="3"
(gdb)
-var-update *
```

```
^done,changelist=[{name="var1",in_scope="true",type_changed="false"}]
(gdb)
```

The `-var-update` Command

Synopsis

```
-var-update [print-values] {name | "*"}
```

Reevaluate the expressions corresponding to the variable object *name* and all its direct and indirect children, and return the list of variable objects whose values have changed; *name* must be a root variable object. Here, “changed” means that the result of `-var-evaluate-expression` before and after the `-var-update` is different. If ‘*’ is used as the variable object names, all existing variable objects are updated, except for frozen ones (see `[-var-set-frozen]`, page 669). The option *print-values* determines whether both names and values, or just names are printed. The possible values of this option are the same as for `-var-list-children` (see `[-var-list-children]`, page 664). It is recommended to use the ‘`--all-values`’ option, to reduce the number of MI commands needed on each program stop.

With the ‘*’ parameter, if a variable object is bound to a currently running thread, it will not be updated, without any diagnostic.

If `-var-set-update-range` was previously used on a varobj, then only the selected range of children will be reported.

`-var-update` reports all the changed varobjs in a tuple named ‘`changelist`’.

Each item in the change list is itself a tuple holding:

‘ <code>name</code> ’	The name of the varobj.						
‘ <code>value</code> ’	If values were requested for this update, then this field will be present and will hold the value of the varobj.						
‘ <code>in_scope</code> ’	This field is a string which may take one of three values: <table> <tr> <td>“<code>true</code>”</td><td>The variable object’s current value is valid.</td></tr> <tr> <td>“<code>false</code>”</td><td>The variable object does not currently hold a valid value but it may hold one in the future if its associated expression comes back into scope.</td></tr> <tr> <td>“<code>invalid</code>”</td><td>The variable object no longer holds a valid value. This can occur when the executable file being debugged has changed, either through recompilation or by using the GDB <code>file</code> command. The front end should normally choose to delete these variable objects.</td></tr> </table>	“ <code>true</code> ”	The variable object’s current value is valid.	“ <code>false</code> ”	The variable object does not currently hold a valid value but it may hold one in the future if its associated expression comes back into scope.	“ <code>invalid</code> ”	The variable object no longer holds a valid value. This can occur when the executable file being debugged has changed, either through recompilation or by using the GDB <code>file</code> command. The front end should normally choose to delete these variable objects.
“ <code>true</code> ”	The variable object’s current value is valid.						
“ <code>false</code> ”	The variable object does not currently hold a valid value but it may hold one in the future if its associated expression comes back into scope.						
“ <code>invalid</code> ”	The variable object no longer holds a valid value. This can occur when the executable file being debugged has changed, either through recompilation or by using the GDB <code>file</code> command. The front end should normally choose to delete these variable objects.						

In the future new values may be added to this list so the front should be prepared for this possibility. See Section 27.6 [GDB/MI Development and Front Ends], page 615.

‘ <code>type_changed</code> ’	This is only present if the varobj is still valid. If the type changed, then this will be the string ‘ <code>true</code> ’; otherwise it will be ‘ <code>false</code> ’.
-------------------------------	--

When a varobj's type changes, its children are also likely to have become incorrect. Therefore, the varobj's children are automatically deleted when this attribute is `'true'`. Also, the varobj's update range, when set using the `-var-set-update-range` command, is unset.

'new_type'

If the varobj's type changed, then this field will be present and will hold the new type.

'new_num_children'

For a dynamic varobj, if the number of children changed, or if the type changed, this will be the new number of children.

The `'numchild'` field in other varobj responses is generally not valid for a dynamic varobj – it will show the number of children that GDB knows about, but because dynamic varobjs lazily instantiate their children, this will not reflect the number of children which may be available.

The `'new_num_children'` attribute only reports changes to the number of children known by GDB. This is the only way to detect whether an update has removed children (which necessarily can only happen at the end of the update range).

'displayhint'

The display hint, if any.

'has_more'

This is an integer value, which will be 1 if there are more children available outside the varobj's update range.

'dynamic' This attribute will be present and have the value `'1'` if the varobj is a dynamic varobj. If the varobj is not a dynamic varobj, then this attribute will not be present.

'new_children'

If new children were added to a dynamic varobj within the selected update range (as set by `-var-set-update-range`), then they will be listed in this attribute.

Example

```
(gdb)
-var-assign var1 3
^done,value="3"
(gdb)
-var-update --all-values var1
^done,changelist=[{name="var1",value="3",in_scope="true",
type_changed="false"}]
(gdb)
```

The `-var-set-frozen` Command

Synopsis

```
-var-set-frozen name flag
```

Set the frozenness flag on the variable object *name*. The *flag* parameter should be either `'1'` to make the variable frozen or `'0'` to make it unfrozen. If a variable object is frozen, then

neither itself, nor any of its children, are implicitly updated by `-var-update` of a parent variable or by `-var-update *`. Only `-var-update` of the variable itself will update its value and values of its children. After a variable object is unfrozen, it is implicitly updated by all subsequent `-var-update` operations. Unfreezing a variable does not update it, only subsequent `-var-update` does.

Example

```
(gdb)
-var-set-frozen V 1
^done
(gdb)
```

The `-var-set-update-range` command

Synopsis

```
-var-set-update-range name from to
```

Set the range of children to be returned by future invocations of `-var-update`.

from and *to* indicate the range of children to report. If *from* or *to* is less than zero, the range is reset and all children will be reported. Otherwise, children starting at *from* (zero-based) and up to and excluding *to* will be reported.

Example

```
(gdb)
-var-set-update-range V 1 2
^done
```

The `-var-set-visualizer` command

Synopsis

```
-var-set-visualizer name visualizer
```

Set a visualizer for the variable object *name*.

visualizer is the visualizer to use. The special value ‘None’ means to disable any visualizer in use.

If not ‘None’, *visualizer* must be a Python expression. This expression must evaluate to a callable object which accepts a single argument. GDB will call this object with the value of the varobj *name* as an argument (this is done so that the same Python pretty-printing code can be used for both the CLI and MI). When called, this object must return an object which conforms to the pretty-printing interface (see Section 23.3.2.6 [Pretty Printing API], page 430).

The pre-defined function `gdb.default_visualizer` may be used to select a visualizer by following the built-in process (see Section 23.3.2.7 [Selecting Pretty-Printers], page 432). This is done automatically when a varobj is created, and so ordinarily is not needed.

This feature is only available if Python support is enabled. The MI command `-list-features` (see Section 27.25 [GDB/MI Support Commands], page 704) can be used to check this.

Example

Resetting the visualizer:

```
(gdb)
-var-set-visualizer V None
^done
```

Reselecting the default (type-based) visualizer:

```
(gdb)
-var-set-visualizer V gdb.default_visualizer
^done
```

Suppose `SomeClass` is a visualizer class. A lambda expression can be used to instantiate this class for a varobj:

```
(gdb)
-var-set-visualizer V "lambda val: SomeClass()"
^done
```

27.18 GDB/MI Data Manipulation

This section describes the GDB/MI commands that manipulate data: examine memory and registers, evaluate expressions, etc.

For details about what an addressable memory unit is, see [addressable memory unit], page 156.

The `-data-disassemble` Command

Synopsis

```
-data-disassemble
( -s start-addr -e end-addr
| -a addr
| -f filename -l linenum [ -n lines ] )
[ --opcodes opcodes-mode ]
[ --source ]
[ -- mode ]
```

Where:

`'start-addr'`

is the beginning address (or `$pc`)

`'end-addr'`

is the end address

`'addr'`

is an address anywhere within (or the name of) the function to disassemble. If an address is specified, the whole function surrounding that address will be disassembled. If a name is specified, the whole function with that name will be disassembled.

`'filename'`

is the name of the file to disassemble

`'linenum'`

is the line number to disassemble around

`'lines'`

is the number of disassembly lines to be produced. If it is -1, the whole function will be disassembled, in case no `end-addr` is specified. If `end-addr` is specified

as a non-zero value, and *lines* is lower than the number of disassembly lines between *start-addr* and *end-addr*, only *lines* lines are displayed; if *lines* is higher than the number of lines between *start-addr* and *end-addr*, only the lines up to *end-addr* are displayed.

‘*opcodes-mode*’

can only be used with *mode* 0, and should be one of the following:

- ‘*none*’ no opcode information will be included in the result.
- ‘*bytes*’ opcodes will be included in the result, the opcodes will be formatted as for *disassemble /b*.
- ‘*display*’ opcodes will be included in the result, the opcodes will be formatted as for *disassemble /r*.

‘*mode*’

the use of *mode* is deprecated in favour of using the *--opcodes* and *--source* options. When no *mode* is given, *mode* 0 will be assumed. However, the *mode* is still available for backward compatibility. The *mode* should be one of:

- ‘0’ *disassembly only*, this is the default mode if no mode is specified.
- ‘1’ *mixed source and disassembly (deprecated)*, it is not possible to recreate this mode using *--opcodes* and *--source* options.
- ‘2’ *disassembly with raw opcodes*, this mode is equivalent to using *mode* 0 and passing *--opcodes bytes* to the command.
- ‘3’ *mixed source and disassembly with raw opcodes (deprecated)*, it is not possible to recreate this mode using *--opcodes* and *--source* options.
- ‘4’ *mixed source and disassembly*, this mode is equivalent to using *mode* 0 and passing *--source* to the command.
- ‘5’ *mixed source and disassembly with raw opcodes*, this mode is equivalent to using *mode* 0 and passing *--opcodes bytes* and *--source* to the command.

Modes 1 and 3 are deprecated. The output is “source centric” which hasn’t proved useful in practice. See Section 9.6 [Machine Code], page 137, for a discussion of the difference between */m* and */s* output of the *disassemble* command.

The *--source* can only be used with *mode* 0. Passing this option will include the source code in the disassembly result as if *mode* 4 or 5 had been used.

Result

The result of the *-data-disassemble* command will be a list named ‘*asm_insns*’, the contents of this list depend on the options used with the *-data-disassemble* command.

For modes 0 and 2, and when the *--source* option is not used, the ‘*asm_insns*’ list contains tuples with the following fields:

address The address at which this instruction was disassembled.

func-name	The name of the function this instruction is within.
offset	The decimal offset in bytes from the start of ‘func-name’.
inst	The text disassembly for this ‘address’.
opcodes	This field is only present for modes 2, 3 and 5, or when the <code>--opcodes</code> option ‘bytes’ or ‘display’ is used. This contains the raw opcode bytes for the ‘inst’ field. When the ‘--opcodes’ option is not passed to <code>-data-disassemble</code>, or the ‘bytes’ value is passed to ‘--opcodes’, then the bytes are formatted as a series of single bytes, in hex, in ascending address order, with a single space between each byte. This format is equivalent to the ‘/b’ option being used with the <i>disassemble</i> command (see [disassemble], page 138). When ‘--opcodes’ is passed the value ‘display’ then the bytes are formatted in the natural instruction display order. This means multiple bytes can be grouped together, and the bytes might be byte-swapped. This format is equivalent to the ‘/r’ option being used with the <i>disassemble</i> command.

For modes 1, 3, 4 and 5, or when the `--source` option is used, the ‘asm_insns’ list contains tuples named ‘src_and_asm_line’, each of which has the following fields:

line	The line number within ‘file’.
file	The file name from the compilation unit. This might be an absolute file name or a relative file name depending on the compile command used.
fullname	Absolute file name of ‘file’. It is converted to a canonical form using the source file search path (see Section 9.5 [Specifying Source Directories], page 133) and after resolving all the symbolic links. If the source file is not found this field will contain the path as present in the debug information.
line_asm_insn	This is a list of tuples containing the disassembly for ‘line’ in ‘file’. The fields of each tuple are the same as for <code>-data-disassemble</code> in <i>mode</i> 0 and 2, so ‘address’, ‘func-name’, ‘offset’, ‘inst’, and optionally ‘opcodes’.

Note that whatever included in the ‘inst’ field, is not manipulated directly by GDB/MI, i.e., it is not possible to adjust its format.

GDB Command

The corresponding GDB command is ‘disassemble’.

Example

Disassemble from the current value of `$pc` to `$pc + 20`:

```
(gdb)
-data-disassemble -s $pc -e "$pc + 20" -- 0
^done,
asm_insns=[
{address="0x000107c0",func-name="main",offset="4",
```

```

inst="mov 2, %o0"},
{address="0x000107c4",func-name="main",offset="8",
inst="sethi %hi(0x11800), %o2"},
{address="0x000107c8",func-name="main",offset="12",
inst="or %o2, 0x140, %o1\t! 0x11940 <_lib_version+8>"},
{address="0x000107cc",func-name="main",offset="16",
inst="sethi %hi(0x11800), %o2"},
{address="0x000107d0",func-name="main",offset="20",
inst="or %o2, 0x168, %o4\t! 0x11968 <_lib_version+48>"}]
(gdb)

```

Disassemble the whole main function. Line 32 is part of main.

```

-data-disassemble -f basics.c -l 32 -- 0
^done,asm_insns=[
{address="0x000107bc",func-name="main",offset="0",
inst="save %sp, -112, %sp"},
{address="0x000107c0",func-name="main",offset="4",
inst="mov 2, %o0"},
{address="0x000107c4",func-name="main",offset="8",
inst="sethi %hi(0x11800), %o2"},
...]
{address="0x0001081c",func-name="main",offset="96",inst="ret "},
{address="0x00010820",func-name="main",offset="100",inst="restore "}
(gdb)

```

Disassemble 3 instructions from the start of main:

```

(gdb)
-data-disassemble -f basics.c -l 32 -n 3 -- 0
^done,asm_insns=[
{address="0x000107bc",func-name="main",offset="0",
inst="save %sp, -112, %sp"},
{address="0x000107c0",func-name="main",offset="4",
inst="mov 2, %o0"},
{address="0x000107c4",func-name="main",offset="8",
inst="sethi %hi(0x11800), %o2"}]
(gdb)

```

Disassemble 3 instructions from the start of main in mixed mode:

```

(gdb)
-data-disassemble -f basics.c -l 32 -n 3 -- 1
^done,asm_insns=[
src_and_asm_line={line="31",
file="../../src/gdb/testsuite/gdb.mi/basics.c",
fullname="/absolute/path/to/src/gdb/testsuite/gdb.mi/basics.c",
line_asm_insn=[{address="0x000107bc",
func-name="main",offset="0",inst="save %sp, -112, %sp"}]},
src_and_asm_line={line="32",
file="../../src/gdb/testsuite/gdb.mi/basics.c",
fullname="/absolute/path/to/src/gdb/testsuite/gdb.mi/basics.c",
line_asm_insn=[{address="0x000107c0",
func-name="main",offset="4",inst="mov 2, %o0"},
{address="0x000107c4",func-name="main",offset="8",
inst="sethi %hi(0x11800), %o2"}]}]
(gdb)

```

The -data-evaluate-expression Command

Synopsis

```
-data-evaluate-expression expr
```


Evaluate *expr* as an expression. The expression could contain an inferior function call. The function call will execute synchronously. If the expression contains spaces, it must be enclosed in double quotes.

GDB Command

The corresponding GDB commands are ‘`print`’, ‘`output`’, and ‘`call`’. In `gdbtk` only, there’s a corresponding ‘`gdb_eval`’ command.

Example

In the following example, the numbers that precede the commands are the *tokens* described in Section 27.4 [GDB/MI Command Syntax], page 612. Notice how GDB/MI returns the same tokens in its output.

```
211-data-evaluate-expression A
211^done,value="1"
(gdb)
311-data-evaluate-expression &A
311^done,value="0xefffef7c"
(gdb)
411-data-evaluate-expression A+3
411^done,value="4"
(gdb)
511-data-evaluate-expression "A + 3"
511^done,value="4"
(gdb)
```

The `-data-list-changed-registers` Command

Synopsis

```
-data-list-changed-registers
```

Display a list of the registers that have changed.

GDB Command

GDB doesn’t have a direct analog for this command; `gdbtk` has the corresponding command ‘`gdb_changed_register_list`’.

Example

On a PPC MBX board:

```
(gdb)
-exec-continue
^running

(gdb)
*stopped,reason="breakpoint-hit",disp="keep",bkptno="1",frame={
func="main",args=[],file="try.c",fullname="/home/foo/bar/try.c",
line="5",arch="powerpc"}
(gdb)
-data-list-changed-registers
^done,changed-registers=["0","1","2","4","5","6","7","8","9",
"10","11","13","14","15","16","17","18","19","20","21","22","23",
"24","25","26","27","28","30","31","64","65","66","67","69"]
(gdb)
```

The `-data-list-register-names` Command

Synopsis

```
-data-list-register-names [ ( regno )+ ]
```

Show a list of register names for the current target. If no arguments are given, it shows a list of the names of all the registers. If integer numbers are given as arguments, it will print a list of the names of the registers corresponding to the arguments. To ensure consistency between a register name and its number, the output list may include empty register names.

GDB Command

GDB does not have a command which corresponds to ‘`-data-list-register-names`’. In `gdbtk` there is a corresponding command ‘`gdb_regnames`’.

Example

For the PPC MBX board:

```
(gdb)
-data-list-register-names
^done,register-names=["r0","r1","r2","r3","r4","r5","r6","r7",
"r8","r9","r10","r11","r12","r13","r14","r15","r16","r17","r18",
"r19","r20","r21","r22","r23","r24","r25","r26","r27","r28","r29",
"r30","r31","f0","f1","f2","f3","f4","f5","f6","f7","f8","f9",
"f10","f11","f12","f13","f14","f15","f16","f17","f18","f19","f20",
"f21","f22","f23","f24","f25","f26","f27","f28","f29","f30","f31",
"", "pc","ps","cr","lr","ctr","xer"]
(gdb)
-data-list-register-names 1 2 3
^done,register-names=["r1","r2","r3"]
(gdb)
```

The `-data-list-register-values` Command

Synopsis

```
-data-list-register-values
  [ --skip-unavailable ] fmt [ ( regno )*]
```

Display the registers’ contents. The format according to which the registers’ contents are to be returned is given by *fmt*, followed by an optional list of numbers specifying the registers to display. A missing list of numbers indicates that the contents of all the registers must be returned. The `--skip-unavailable` option indicates that only the available registers are to be returned.

Allowed formats for *fmt* are:

x	Hexadecimal
o	Octal
t	Binary
d	Decimal
r	Raw
N	Natural

GDB Command

The corresponding GDB commands are ‘info reg’, ‘info all-reg’, and (in `gdbtk`) ‘gdb_fetch_registers’.

Example

For a PPC MBX board (note: line breaks are for readability only, they don’t appear in the actual output):

```
(gdb)
~data-list-register-values r 64 65
^done,register-values=[{number="64",value="0xfe00a300"},
{number="65",value="0x00029002"}]
(gdb)
~data-list-register-values x
^done,register-values=[{number="0",value="0xfe0043c8"},
{number="1",value="0x3fff88"},{number="2",value="0xffffffff"},
{number="3",value="0x0"},{number="4",value="0xa"},
{number="5",value="0x3fff68"},{number="6",value="0x3fff58"},
{number="7",value="0xfe011e98"},{number="8",value="0x2"},
{number="9",value="0xfa202820"},{number="10",value="0xfa202808"},
{number="11",value="0x1"},{number="12",value="0x0"},
{number="13",value="0x4544"},{number="14",value="0xffdffff"},
{number="15",value="0xffffffff"},{number="16",value="0xfffffeff"},
{number="17",value="0xfffffed"},{number="18",value="0xffffffe"},
{number="19",value="0xffffffff"},{number="20",value="0xffffffff"},
{number="21",value="0xffffffff"},{number="22",value="0xffffffff7"},
{number="23",value="0xffffffff"},{number="24",value="0xffffffff"},
{number="25",value="0xffffffff"},{number="26",value="0xffffffb"},
{number="27",value="0xffffffff"},{number="28",value="0xf7bffff"},
{number="29",value="0x0"},{number="30",value="0xfe010000"},
{number="31",value="0x0"},{number="32",value="0x0"},
{number="33",value="0x0"},{number="34",value="0x0"},
{number="35",value="0x0"},{number="36",value="0x0"},
{number="37",value="0x0"},{number="38",value="0x0"},
{number="39",value="0x0"},{number="40",value="0x0"},
{number="41",value="0x0"},{number="42",value="0x0"},
{number="43",value="0x0"},{number="44",value="0x0"},
{number="45",value="0x0"},{number="46",value="0x0"},
{number="47",value="0x0"},{number="48",value="0x0"},
{number="49",value="0x0"},{number="50",value="0x0"},
{number="51",value="0x0"},{number="52",value="0x0"},
{number="53",value="0x0"},{number="54",value="0x0"},
{number="55",value="0x0"},{number="56",value="0x0"},
{number="57",value="0x0"},{number="58",value="0x0"},
{number="59",value="0x0"},{number="60",value="0x0"},
{number="61",value="0x0"},{number="62",value="0x0"},
{number="63",value="0x0"},{number="64",value="0xfe00a300"},
{number="65",value="0x29002"},{number="66",value="0x202f04b5"},
{number="67",value="0xfe0043b0"},{number="68",value="0xfe00b3e4"},
{number="69",value="0x20002b03"}]
(gdb)
```

The -data-read-memory Command

This command is deprecated, use `-data-read-memory-bytes` instead.

Synopsis

```
-data-read-memory [ -o byte-offset ]
    address word-format word-size
    nr-rows nr-cols [ aschar ]
```

where:

‘address’ An expression specifying the address of the first memory word to be read. Complex expressions containing embedded white space should be quoted using the C convention.

‘word-format’ The format to be used to print the memory words. The notation is the same as for GDB’s `print` command (see Section 10.5 [Output Formats], page 152).

‘word-size’ The size of each memory word in bytes.

‘nr-rows’ The number of rows in the output table.

‘nr-cols’ The number of columns in the output table.

‘aschar’ If present, indicates that each row should include an ASCII dump. The value of *aschar* is used as a padding character when a byte is not a member of the printable ASCII character set (printable ASCII characters are those whose code is between 32 and 126, inclusively).

‘byte-offset’ An offset to add to the *address* before fetching memory.

This command displays memory contents as a table of *nr-rows* by *nr-cols* words, each word being *word-size* bytes. In total, *nr-rows * nr-cols * word-size* bytes are read (returned as **‘total-bytes’**). Should less than the requested number of bytes be returned by the target, the missing words are identified using **‘N/A’**. The number of bytes read from the target is returned in **‘nr-bytes’** and the starting address used to read memory in **‘addr’**.

The address of the next/previous row or page is available in **‘next-row’** and **‘prev-row’**, **‘next-page’** and **‘prev-page’**.

GDB Command

The corresponding GDB command is **‘x’**. `gdbtk` has `‘gdb_get_mem’` memory read command.

Example

Read six bytes of memory starting at `bytes+6` but then offset by `-6` bytes. Format as three rows of two columns. One byte per word. Display each word in hex.

```
(gdb)
9-data-read-memory -o -6 -- bytes+6 x 1 3 2
9^done,addr="0x00001390",nr-bytes="6",total-bytes="6",
next-row="0x00001396",prev-row="0x0000138e",next-page="0x00001396",
prev-page="0x0000138a",memory=[
{addr="0x00001390",data=["0x00","0x01"]},
{addr="0x00001392",data=["0x02","0x03"]},
{addr="0x00001394",data=["0x04","0x05"]}
(gdb)
```

Read two bytes of memory starting at address `shorts + 64` and display as a single word formatted in decimal.

```
(gdb)
5-data-read-memory shorts+64 d 2 1 1
5^done,addr="0x00001510",nr-bytes="2",total-bytes="2",
next-row="0x00001512",prev-row="0x0000150e",
next-page="0x00001512",prev-page="0x0000150e",memory=[
{addr="0x00001510",data=["128"]}]
(gdb)
```

Read thirty two bytes of memory starting at `bytes+16` and format as eight rows of four columns. Include a string encoding with ‘x’ used as the non-printable character.

```
(gdb)
4-data-read-memory bytes+16 x 1 8 4 x
4^done,addr="0x000013a0",nr-bytes="32",total-bytes="32",
next-row="0x000013c0",prev-row="0x0000139c",
next-page="0x000013c0",prev-page="0x00001380",memory=[
{addr="0x000013a0",data=["0x10","0x11","0x12","0x13"],ascii="xxxx"},
{addr="0x000013a4",data=["0x14","0x15","0x16","0x17"],ascii="xxxx"},
{addr="0x000013a8",data=["0x18","0x19","0x1a","0x1b"],ascii="xxxx"},
{addr="0x000013ac",data=["0x1c","0x1d","0x1e","0x1f"],ascii="xxxx"},
{addr="0x000013b0",data=["0x20","0x21","0x22","0x23"],ascii=" !\"#"},
{addr="0x000013b4",data=["0x24","0x25","0x26","0x27"],ascii="$%&'"},
{addr="0x000013b8",data=["0x28","0x29","0x2a","0x2b"],ascii="()*+"},
{addr="0x000013bc",data=["0x2c","0x2d","0x2e","0x2f"],ascii=",-./"}]
(gdb)
```

The `-data-read-memory-bytes` Command

Synopsis

```
-data-read-memory-bytes [ -o offset ]
    address count
```

where:

- ‘*address*’ An expression specifying the address of the first addressable memory unit to be read. Complex expressions containing embedded white space should be quoted using the C convention.
- ‘*count*’ The number of addressable memory units to read. This should be an integer literal.
- ‘*offset*’ The offset relative to *address* at which to start reading. This should be an integer literal. This option is provided so that a frontend is not required to first evaluate *address* and then perform address arithmetic itself.

This command attempts to read all accessible memory regions in the specified range. First, all regions marked as unreadable in the memory map (if one is defined) will be skipped. See Section 10.18 [Memory Region Attributes], page 186. Second, GDB will attempt to read the remaining regions. For each one, if reading full region results in an errors, GDB will try to read a subset of the region.

In general, every single memory unit in the region may be readable or not, and the only way to read every readable unit is to try a read at every address, which is not practical. Therefore, GDB will attempt to read all accessible memory units at either beginning or the end of the region, using a binary division scheme. This heuristic works well for reading

across a memory map boundary. Note that if a region has a readable range that is neither at the beginning or the end, GDB will not read it.

The result record (see Section 27.7.1 [GDB/MI Result Records], page 617) that is output of the command includes a field named ‘**memory**’ whose content is a list of tuples. Each tuple represent a successfully read memory block and has the following fields:

begin The start address of the memory block, as hexadecimal literal.
end The end address of the memory block, as hexadecimal literal.
offset The offset of the memory block, as hexadecimal literal, relative to the start address passed to **-data-read-memory-bytes**.
contents The contents of the memory block, in hex.

GDB Command

The corresponding GDB command is ‘**x**’.

Example

```
(gdb)
-data-read-memory-bytes &a 10
^done,memory=[{begin="0xbffff154",offset="0x00000000",
               end="0xbffff15e",
               contents="010000000200000000300"}]
```

The -data-write-memory-bytes Command

Synopsis

```
-data-write-memory-bytes address contents
-data-write-memory-bytes address contents [count]
```

where:

‘**address**’ An expression specifying the address of the first addressable memory unit to be written. Complex expressions containing embedded white space should be quoted using the C convention.

‘**contents**’ The hex-encoded data to write. It is an error if *contents* does not represent an integral number of addressable memory units.

‘**count**’ Optional argument indicating the number of addressable memory units to be written. If *count* is greater than *contents*’ length, GDB will repeatedly write *contents* until it fills *count* memory units.

GDB Command

There’s no corresponding GDB command.

Example

```
(gdb)
-data-write-memory-bytes &a "aabbccdd"
^done
```

```
(gdb)
(gdb)
-data-write-memory-bytes &a "aabbccdd" 16e
^done
(gdb)
```

27.19 GDB/MI Tracepoint Commands

The commands defined in this section implement MI support for tracepoints. For detailed introduction, see Chapter 13 [Tracepoints], page 205.

The `-trace-find` Command

Synopsis

```
-trace-find mode [parameters...]
```

Find a trace frame using criteria defined by *mode* and *parameters*. The following table lists permissible modes and their parameters. For details of operation, see Section 13.2.1 [tfind], page 217.

- `'none'` No parameters are required. Stops examining trace frames.
- `'frame-number'`
 An integer is required as parameter. Selects tracepoint frame with that index.
- `'tracepoint-number'`
 An integer is required as parameter. Finds next trace frame that corresponds to tracepoint with the specified number.
- `'pc'` An address is required as parameter. Finds next trace frame that corresponds to any tracepoint at the specified address.
- `'pc-inside-range'`
 Two addresses are required as parameters. Finds next trace frame that corresponds to a tracepoint at an address inside the specified range. Both bounds are considered to be inside the range.
- `'pc-outside-range'`
 Two addresses are required as parameters. Finds next trace frame that corresponds to a tracepoint at an address outside the specified range. Both bounds are considered to be inside the range.
- `'line'` Location specification is required as parameter. See Section 9.2 [Location Specifications], page 128. Finds next trace frame that corresponds to a tracepoint at the specified location.

If `'none'` was passed as *mode*, the response does not have fields. Otherwise, the response may have the following fields:

- `'found'` This field has either `'0'` or `'1'` as the value, depending on whether a matching tracepoint was found.
- `'traceframe'`
 The index of the found traceframe. This field is present iff the `'found'` field has value of `'1'`.

‘tracepoint’

The index of the found tracepoint. This field is present iff the ‘found’ field has value of ‘1’.

‘frame’

The information about the frame corresponding to the found trace frame. This field is present only if a trace frame was found. See Section 27.7.5 [GDB/MI Frame Information], page 625, for description of this field.

GDB Command

The corresponding GDB command is ‘tfind’.

The -trace-define-variable Command**Synopsis**

```
-trace-define-variable name [ value ]
```

Create trace variable *name* if it does not exist. If *value* is specified, sets the initial value of the specified trace variable to that value. Note that the *name* should start with the ‘\$’ character.

GDB Command

The corresponding GDB command is ‘tvariable’.

The -trace-frame-collected Command**Synopsis**

```
-trace-frame-collected
  [--var-print-values var_pval]
  [--comp-print-values comp_pval]
  [--registers-format regformat]
  [--memory-contents]
```

This command returns the set of collected objects, register names, trace state variable names, memory ranges and computed expressions that have been collected at a particular trace frame. The optional parameters to the command affect the output format in different ways. See the output description table below for more details.

The reported names can be used in the normal manner to create varobjs and inspect the objects themselves. The items returned by this command are categorized so that it is clear which is a variable, which is a register, which is a trace state variable, which is a memory range and which is a computed expression.

For instance, if the actions were

```
collect myVar, myArray[myIndex], myObj.field, myPtr->field, myCount + 2
collect *(int*)0xaf02bef0@40
```

the object collected in its entirety would be **myVar**. The object **myArray** would be partially collected, because only the element at index **myIndex** would be collected. The remaining objects would be computed expressions.

An example output would be:

```
(gdb)
-ttrace-frame-collected
```



```

^done,
  explicit-variables=[{name="myVar",value="1"}],
  computed-expressions=[{name="myArray[myIndex]",value="0"},
                        {name="myObj.field",value="0"},
                        {name="myPtr->field",value="1"},
                        {name="myCount + 2",value="3"},
                        {name="$tvar1 + 1",value="43970027"}],
  registers=[{number="0",value="0x7fe2c6e79ec8"},
             {number="1",value="0x0"},
             {number="2",value="0x4"},
             ...
             {number="125",value="0x0"}],
  tvars=[{name="$tvar1",current="43970026"}],
  memory=[{address="0x0000000000602264",length="4"},
          {address="0x0000000000615bc0",length="4"}]
(gdb)

```

Where:

explicit-variables

The set of objects that have been collected in their entirety (as opposed to collecting just a few elements of an array or a few struct members). For each object, its name and value are printed. The `--var-print-values` option affects how or whether the value field is output. If `var_pval` is 0, then print only the names; if it is 1, print also their values; and if it is 2, print the name, type and value for simple data types, and the name and type for arrays, structures and unions.

computed-expressions

The set of computed expressions that have been collected at the current trace frame. The `--comp-print-values` option affects this set like the `--var-print-values` option affects the `explicit-variables` set. See above.

registers

The registers that have been collected at the current trace frame. For each register collected, the name and current value are returned. The value is formatted according to the `--registers-format` option. See the `-data-list-register-values` command for a list of the allowed formats. The default is 'x'.

tvars The trace state variables that have been collected at the current trace frame. For each trace state variable collected, the name and current value are returned.

memory The set of memory ranges that have been collected at the current trace frame. Its content is a list of tuples. Each tuple represents a collected memory range and has the following fields:

address	The start address of the memory range, as hexadecimal literal.
length	The length of the memory range, as decimal literal.
contents	The contents of the memory block, in hex. This field is only present if the <code>--memory-contents</code> option is specified.

GDB Command

There is no corresponding GDB command.

Example

The `-trace-list-variables` Command

Synopsis

```
-trace-list-variables
```

Return a table of all defined trace variables. Each element of the table has the following fields:

<code>'name'</code>	The name of the trace variable. This field is always present.
<code>'initial'</code>	The initial value. This is a 64-bit signed integer. This field is always present.
<code>'current'</code>	The value the trace variable has at the moment. This is a 64-bit signed integer. This field is absent iff current value is not defined, for example if the trace was never run, or is presently running.

GDB Command

The corresponding GDB command is `'tvariables'`.

Example

```
(gdb)
-trace-list-variables
^done,trace-variables={nr_rows="1",nr_cols="3",
  hdr=[{width="15",alignment="-1",col_name="name",colhdr="Name"},
        {width="11",alignment="-1",col_name="initial",colhdr="Initial"},
        {width="11",alignment="-1",col_name="current",colhdr="Current"}],
  body=[variable={name="$trace_timestamp",initial="0"}
        variable={name="$foo",initial="10",current="15"}]}
```

The `-trace-save` Command

Synopsis

```
-trace-save [ -r ] [ -ctf ] filename
```

Saves the collected trace data to *filename*. Without the `'-r'` option, the data is downloaded from the target and saved in a local file. With the `'-r'` option the target is asked to perform the save.

By default, this command will save the trace in the tfile format. You can supply the optional `'-ctf'` argument to save it the CTF format. See Section 13.4 [Trace Files], page 221, for more information about CTF.

GDB Command

The corresponding GDB command is `'tsave'`.

The `-trace-start` Command

Synopsis

```
-trace-start
```

Starts a tracing experiment. The result of this command does not have any fields.

GDB Command

The corresponding GDB command is `'tstart'`.

The `-trace-status` Command

Synopsis

`-trace-status`

Obtains the status of a tracing experiment. The result may include the following fields:

`'supported'`

May have a value of either `'0'`, when no tracing operations are supported, `'1'`, when all tracing operations are supported, or `'file'` when examining trace file. In the latter case, examining of trace frame is possible but new tracing experiment cannot be started. This field is always present.

`'running'` May have a value of either `'0'` or `'1'` depending on whether tracing experiment is in progress on target. This field is present if `'supported'` field is not `'0'`.

`'stop-reason'`

Report the reason why the tracing was stopped last time. This field may be absent iff tracing was never stopped on target yet. The value of `'request'` means the tracing was stopped as result of the `-trace-stop` command. The value of `'overflow'` means the tracing buffer is full. The value of `'disconnection'` means tracing was automatically stopped when GDB has disconnected. The value of `'passcount'` means tracing was stopped when a tracepoint was passed a maximal number of times for that tracepoint. This field is present if `'supported'` field is not `'0'`.

`'stopping-tracepoint'`

The number of tracepoint whose passcount as exceeded. This field is present iff the `'stop-reason'` field has the value of `'passcount'`.

`'frames'`

`'frames-created'`

The `'frames'` field is a count of the total number of trace frames in the trace buffer, while `'frames-created'` is the total created during the run, including ones that were discarded, such as when a circular trace buffer filled up. Both fields are optional.

`'buffer-size'`

`'buffer-free'`

These fields tell the current size of the tracing buffer and the remaining space. These fields are optional.

`'circular'`

The value of the circular trace buffer flag. `1` means that the trace buffer is circular and old trace frames will be discarded if necessary to make room, `0` means that the trace buffer is linear and may fill up.

`'disconnected'`

The value of the disconnected tracing flag. `1` means that tracing will continue after GDB disconnects, `0` means that the trace run will stop.

‘trace-file’

The filename of the trace file being examined. This field is optional, and only present when examining a trace file.

GDB Command

The corresponding GDB command is **‘tstatus’**.

The **-trace-stop** Command

Synopsis

-trace-stop

Stops a tracing experiment. The result of this command has the same fields as **-trace-status**, except that the **‘supported’** and **‘running’** fields are not output.

GDB Command

The corresponding GDB command is **‘tstop’**.

27.20 GDB/MI Symbol Query Commands

The **-symbol-info-functions** Command

Synopsis

```
-symbol-info-functions [--include-nondebug]
                        [--type type_regexp]
                        [--name name_regexp]
                        [--max-results limit]
```

Return a list containing the names and types for all global functions taken from the debug information. The functions are grouped by source file, and shown with the line number on which each function is defined.

The **--include-nondebug** option causes the output to include code symbols from the symbol table.

The options **--type** and **--name** allow the symbols returned to be filtered based on either the name of the function, or the type signature of the function.

The option **--max-results** restricts the command to return no more than *limit* results. If exactly *limit* results are returned then there might be additional results available if a higher limit is used.

GDB Command

The corresponding GDB command is **‘info functions’**.

Example

```
(gdb)
-symbol-info-functions
^done,symbols=
{debug=
  [{filename="/project/gdb/testsuite/gdb.mi/mi-sym-info-1.c",
    fullname="/project/gdb/testsuite/gdb.mi/mi-sym-info-1.c",
    symbols=[{line="36", name="f4", type="void (int *)",
      description="void f4(int *);"}],
    {line="42", name="main", type="int ()",
      description="int main();"}],
    {line="30", name="f1", type="my_int_t (int, int)",
      description="static my_int_t f1(int, int);"}]]},
  {filename="/project/gdb/testsuite/gdb.mi/mi-sym-info-2.c",
    fullname="/project/gdb/testsuite/gdb.mi/mi-sym-info-2.c",
    symbols=[{line="33", name="f2", type="float (another_float_t)",
      description="float f2(another_float_t);"},
    {line="39", name="f3", type="int (another_int_t)",
      description="int f3(another_int_t);"},
    {line="27", name="f1", type="another_float_t (int)",
      description="static another_float_t f1(int);"}]]]}

(gdb)
-symbol-info-functions --name f1
^done,symbols=
{debug=
  [{filename="/project/gdb/testsuite/gdb.mi/mi-sym-info-1.c",
    fullname="/project/gdb/testsuite/gdb.mi/mi-sym-info-1.c",
    symbols=[{line="30", name="f1", type="my_int_t (int, int)",
      description="static my_int_t f1(int, int);"}]]},
  {filename="/project/gdb/testsuite/gdb.mi/mi-sym-info-2.c",
    fullname="/project/gdb/testsuite/gdb.mi/mi-sym-info-2.c",
    symbols=[{line="27", name="f1", type="another_float_t (int)",
      description="static another_float_t f1(int);"}]]]}

(gdb)
-symbol-info-functions --type void
^done,symbols=
{debug=
  [{filename="/project/gdb/testsuite/gdb.mi/mi-sym-info-1.c",
    fullname="/project/gdb/testsuite/gdb.mi/mi-sym-info-1.c",
    symbols=[{line="36", name="f4", type="void (int *)",
      description="void f4(int *);"}]]}]}
```

```
(gdb)
-symbol-info-functions --include-nondebug
^done,symbols=
{debug=
  [{filename="/project/gdb/testsuite/gdb.mi/mi-sym-info-1.c",
    fullname="/project/gdb/testsuite/gdb.mi/mi-sym-info-1.c",
    symbols=[{line="36", name="f4", type="void (int *)",
      description="void f4(int *);"},
      {line="42", name="main", type="int ()",
        description="int main();"},
      {line="30", name="f1", type="my_int_t (int, int)",
        description="static my_int_t f1(int, int);"}]},
  {filename="/project/gdb/testsuite/gdb.mi/mi-sym-info-2.c",
    fullname="/project/gdb/testsuite/gdb.mi/mi-sym-info-2.c",
    symbols=[{line="33", name="f2", type="float (another_float_t)",
      description="float f2(another_float_t);"},
      {line="39", name="f3", type="int (another_int_t)",
        description="int f3(another_int_t);"},
      {line="27", name="f1", type="another_float_t (int)",
        description="static another_float_t f1(int);"}]}],
  nondebug=
  [{address="0x000000000400398",name="_init"},
    {address="0x0000000004003b0",name="_start"},
    ...
  ]}
```

The `-symbol-info-module-functions` Command

Synopsis

```
-symbol-info-module-functions [--module module_regexp]
                             [--name name_regexp]
                             [--type type_regexp]
```

Return a list containing the names of all known functions within all known Fortran modules. The functions are grouped by source file and containing module, and shown with the line number on which each function is defined.

The option `--module` only returns results for modules matching *module_regexp*. The option `--name` only returns functions whose name matches *name_regexp*, and `--type` only returns functions whose type matches *type_regexp*.

GDB Command

The corresponding GDB command is `'info module functions'`.

Example

```
(gdb)
-symbol-info-module-functions
^done,symbols=
  [{module="mod1",
    files=[{filename="/project/gdb/testsuite/gdb.mi/mi-fortran-modules-2.f90",
            fullname="/project/gdb/testsuite/gdb.mi/mi-fortran-modules-2.f90",
            symbols=[{line="21",name="mod1::check_all",type="void (void)",
                      description="void mod1::check_all(void);"}]}]},
    {module="mod2",
      files=[{filename="/project/gdb/testsuite/gdb.mi/mi-fortran-modules-2.f90",
              fullname="/project/gdb/testsuite/gdb.mi/mi-fortran-modules-2.f90",
              symbols=[{line="30",name="mod2::check_var_i",type="void (void)",
                        description="void mod2::check_var_i(void);"}]}]},
    {module="mod3",
      files=[{filename="/project/gdb/testsuite/gdb.mi/mi-fortran-modules.f90",
              fullname="/project/gdb/testsuite/gdb.mi/mi-fortran-modules.f90",
              symbols=[{line="21",name="mod3::check_all",type="void (void)",
                        description="void mod3::check_all(void);"},
                        {line="27",name="mod3::check_mod2",type="void (void)",
                        description="void mod3::check_mod2(void);"}]}]},
    {module="modmany",
      files=[{filename="/project/gdb/testsuite/gdb.mi/mi-fortran-modules.f90",
              fullname="/project/gdb/testsuite/gdb.mi/mi-fortran-modules.f90",
              symbols=[{line="35",name="modmany::check_some",type="void (void)",
                        description="void modmany::check_some(void);"}]}]},
    {module="moduse",
      files=[{filename="/project/gdb/testsuite/gdb.mi/mi-fortran-modules.f90",
              fullname="/project/gdb/testsuite/gdb.mi/mi-fortran-modules.f90",
              symbols=[{line="44",name="moduse::check_all",type="void (void)",
                        description="void moduse::check_all(void);"},
                        {line="49",name="moduse::check_var_x",type="void (void)",
                        description="void moduse::check_var_x(void);"}]}]}}
```

The `-symbol-info-module-variables` Command

Synopsis

```
-symbol-info-module-variables [--module module_regexp]
                             [--name name_regexp]
                             [--type type_regexp]
```

Return a list containing the names of all known variables within all known Fortran modules. The variables are grouped by source file and containing module, and shown with the line number on which each variable is defined.

The option `--module` only returns results for modules matching *module_regexp*. The option `--name` only returns variables whose name matches *name_regexp*, and `--type` only returns variables whose type matches *type_regexp*.

GDB Command

The corresponding GDB command is `'info module variables'`.

Example

```
(gdb)
-symbol-info-module-variables
^done,symbols=
[{"module="mod1",
  files=[{"filename="/project/gdb/testsuite/gdb.mi/mi-fortran-modules-2.f90",
    fullname="/project/gdb/testsuite/gdb.mi/mi-fortran-modules-2.f90",
    symbols=[{"line="18",name="mod1::var_const",type="integer(kind=4)",
      description="integer(kind=4) mod1::var_const;"},
      {"line="17",name="mod1::var_i",type="integer(kind=4)",
        description="integer(kind=4) mod1::var_i;"}]}]},
  {"module="mod2",
    files=[{"filename="/project/gdb/testsuite/gdb.mi/mi-fortran-modules-2.f90",
      fullname="/project/gdb/testsuite/gdb.mi/mi-fortran-modules-2.f90",
      symbols=[{"line="28",name="mod2::var_i",type="integer(kind=4)",
        description="integer(kind=4) mod2::var_i;"}]}]},
  {"module="mod3",
    files=[{"filename="/project/gdb/testsuite/gdb.mi/mi-fortran-modules.f90",
      fullname="/project/gdb/testsuite/gdb.mi/mi-fortran-modules.f90",
      symbols=[{"line="18",name="mod3::mod1",type="integer(kind=4)",
        description="integer(kind=4) mod3::mod1;"},
        {"line="17",name="mod3::mod2",type="integer(kind=4)",
          description="integer(kind=4) mod3::mod2;"},
        {"line="19",name="mod3::var_i",type="integer(kind=4)",
          description="integer(kind=4) mod3::var_i;"}]}]},
  {"module="modmany",
    files=[{"filename="/project/gdb/testsuite/gdb.mi/mi-fortran-modules.f90",
      fullname="/project/gdb/testsuite/gdb.mi/mi-fortran-modules.f90",
      symbols=[{"line="33",name="modmany::var_a",type="integer(kind=4)",
        description="integer(kind=4) modmany::var_a;"},
        {"line="33",name="modmany::var_b",type="integer(kind=4)",
          description="integer(kind=4) modmany::var_b;"},
        {"line="33",name="modmany::var_c",type="integer(kind=4)",
          description="integer(kind=4) modmany::var_c;"},
        {"line="33",name="modmany::var_i",type="integer(kind=4)",
          description="integer(kind=4) modmany::var_i;"}]}]},
  {"module="moduse",
    files=[{"filename="/project/gdb/testsuite/gdb.mi/mi-fortran-modules.f90",
      fullname="/project/gdb/testsuite/gdb.mi/mi-fortran-modules.f90",
      symbols=[{"line="42",name="moduse::var_x",type="integer(kind=4)",
        description="integer(kind=4) moduse::var_x;"},
        {"line="42",name="moduse::var_y",type="integer(kind=4)",
          description="integer(kind=4) moduse::var_y;"}]}]}]
```

The -symbol-info-modules Command

Synopsis

```
-symbol-info-modules [--name name_regex]
                   [--max-results limit]
```

Return a list containing the names of all known Fortran modules. The modules are grouped by source file, and shown with the line number on which each modules is defined.

The option `--name` allows the modules returned to be filtered based the name of the module.

The option `--max-results` restricts the command to return no more than *limit* results. If exactly *limit* results are returned then there might be additional results available if a higher limit is used.

GDB Command

The corresponding GDB command is ‘`info modules`’.

Example

```
(gdb)
-symbol-info-modules
^done,symbols=
  {debug=
    [{filename="/project/gdb/testsuite/gdb.mi/mi-fortran-modules-2.f90",
      fullname="/project/gdb/testsuite/gdb.mi/mi-fortran-modules-2.f90",
      symbols=[{line="16",name="mod1"},
                {line="22",name="mod2"}]},
    {filename="/project/gdb/testsuite/gdb.mi/mi-fortran-modules.f90",
      fullname="/project/gdb/testsuite/gdb.mi/mi-fortran-modules.f90",
      symbols=[{line="16",name="mod3"},
                {line="22",name="modmany"},
                {line="26",name="moduse"}]}]}

(gdb)
-symbol-info-modules --name mod[123]
^done,symbols=
  {debug=
    [{filename="/project/gdb/testsuite/gdb.mi/mi-fortran-modules-2.f90",
      fullname="/project/gdb/testsuite/gdb.mi/mi-fortran-modules-2.f90",
      symbols=[{line="16",name="mod1"},
                {line="22",name="mod2"}]},
    {filename="/project/gdb/testsuite/gdb.mi/mi-fortran-modules.f90",
      fullname="/project/gdb/testsuite/gdb.mi/mi-fortran-modules.f90",
      symbols=[{line="16",name="mod3"}]}]}
```

The `-symbol-info-types` Command

Synopsis

```
-symbol-info-types [--name name_regex]
                  [--max-results limit]
```

Return a list of all defined types. The types are grouped by source file, and shown with the line number on which each user defined type is defined. Some base types are not defined in the source code but are added to the debug information by the compiler, for example `int`, `float`, etc.; these types do not have an associated line number.

The option `--name` allows the list of types returned to be filtered by name.

The option `--max-results` restricts the command to return no more than *limit* results. If exactly *limit* results are returned then there might be additional results available if a higher limit is used.

GDB Command

The corresponding GDB command is ‘`info types`’.

Example

```
(gdb)
-symbol-info-types
^done,symbols=
{debug=
  [{filename="gdb.mi/mi-sym-info-1.c",
    fullname="/project/gdb/testsuite/gdb.mi/mi-sym-info-1.c",
    symbols=[{name="float"},
      {name="int"},
      {line="27",name="typedef int my_int_t;"}]},
  {filename="gdb.mi/mi-sym-info-2.c",
    fullname="/project/gdb/testsuite/gdb.mi/mi-sym-info-2.c",
    symbols=[{line="24",name="typedef float another_float_t;"},
      {line="23",name="typedef int another_int_t;"},
      {name="float"},
      {name="int"}]}]}

(gdb)
-symbol-info-types --name _int_
^done,symbols=
{debug=
  [{filename="gdb.mi/mi-sym-info-1.c",
    fullname="/project/gdb/testsuite/gdb.mi/mi-sym-info-1.c",
    symbols=[{line="27",name="typedef int my_int_t;"}]},
  {filename="gdb.mi/mi-sym-info-2.c",
    fullname="/project/gdb/testsuite/gdb.mi/mi-sym-info-2.c",
    symbols=[{line="23",name="typedef int another_int_t;"}]}]}
```

The `-symbol-info-variables` Command

Synopsis

```
-symbol-info-variables [--include-nondebug]
                      [--type type_regex]
                      [--name name_regex]
                      [--max-results limit]
```

Return a list containing the names and types for all global variables taken from the debug information. The variables are grouped by source file, and shown with the line number on which each variable is defined.

The `--include-nondebug` option causes the output to include data symbols from the symbol table.

The options `--type` and `--name` allow the symbols returned to be filtered based on either the name of the variable, or the type of the variable.

The option `--max-results` restricts the command to return no more than *limit* results. If exactly *limit* results are returned then there might be additional results available if a higher limit is used.

GDB Command

The corresponding GDB command is `'info variables'`.

Example

```
(gdb)
-symbol-info-variables
^done,symbols=
{debug=
  [{filename="/project/gdb/testsuite/gdb.mi/mi-sym-info-1.c",
    fullname="/project/gdb/testsuite/gdb.mi/mi-sym-info-1.c",
    symbols=[{line="25",name="global_f1",type="float",
      description="static float global_f1;"},
      {line="24",name="global_i1",type="int",
        description="static int global_i1;"}]},
  {filename="/project/gdb/testsuite/gdb.mi/mi-sym-info-2.c",
    fullname="/project/gdb/testsuite/gdb.mi/mi-sym-info-2.c",
    symbols=[{line="21",name="global_f2",type="int",
      description="int global_f2;"},
      {line="20",name="global_i2",type="int",
        description="int global_i2;"},
      {line="19",name="global_f1",type="float",
        description="static float global_f1;"},
      {line="18",name="global_i1",type="int",
        description="static int global_i1;"}]}]}

(gdb)
-symbol-info-variables --name f1
^done,symbols=
{debug=
  [{filename="/project/gdb/testsuite/gdb.mi/mi-sym-info-1.c",
    fullname="/project/gdb/testsuite/gdb.mi/mi-sym-info-1.c",
    symbols=[{line="25",name="global_f1",type="float",
      description="static float global_f1;"}]},
  {filename="/project/gdb/testsuite/gdb.mi/mi-sym-info-2.c",
    fullname="/project/gdb/testsuite/gdb.mi/mi-sym-info-2.c",
    symbols=[{line="19",name="global_f1",type="float",
      description="static float global_f1;"}]}]}

(gdb)
-symbol-info-variables --type float
^done,symbols=
{debug=
  [{filename="/project/gdb/testsuite/gdb.mi/mi-sym-info-1.c",
    fullname="/project/gdb/testsuite/gdb.mi/mi-sym-info-1.c",
    symbols=[{line="25",name="global_f1",type="float",
      description="static float global_f1;"}]},
  {filename="/project/gdb/testsuite/gdb.mi/mi-sym-info-2.c",
    fullname="/project/gdb/testsuite/gdb.mi/mi-sym-info-2.c",
    symbols=[{line="19",name="global_f1",type="float",
      description="static float global_f1;"}]}]}
```

```
(gdb)
-symbol-info-variables --include-nondebug
^done,symbols=
{debug=
  [{filename="/project/gdb/testsuite/gdb.mi/mi-sym-info-1.c",
    fullname="/project/gdb/testsuite/gdb.mi/mi-sym-info-1.c",
    symbols=[{line="25",name="global_f1",type="float",
      description="static float global_f1;"},
      {line="24",name="global_i1",type="int",
        description="static int global_i1;"}]},
  {filename="/project/gdb/testsuite/gdb.mi/mi-sym-info-2.c",
    fullname="/project/gdb/testsuite/gdb.mi/mi-sym-info-2.c",
    symbols=[{line="21",name="global_f2",type="int",
      description="int global_f2;"},
      {line="20",name="global_i2",type="int",
        description="int global_i2;"},
      {line="19",name="global_f1",type="float",
        description="static float global_f1;"},
      {line="18",name="global_i1",type="int",
        description="static int global_i1;"}]}],
  nondebug=
    [{address="0x0000000004005d0",name="_IO_stdin_used"},
      {address="0x0000000004005d8",name="__dso_handle"}
      ...
    ]}
}
```

The `-symbol-list-lines` Command

Synopsis

```
-symbol-list-lines filename
```

Print the list of lines that contain code and their associated program addresses for the given source filename. The entries are sorted in ascending PC order.

GDB Command

There is no corresponding GDB command.

Example

```
(gdb)
-symbol-list-lines basics.c
^done,lines=[{pc="0x08048554",line="7"},{pc="0x0804855a",line="8"}]
(gdb)
```

27.21 GDB/MI File Commands

This section describes the GDB/MI commands to specify executable file names and to read in and obtain symbol table information.

The `-file-exec-and-symbols` Command

Synopsis

```
-file-exec-and-symbols file
```

Specify the executable file to be debugged. This file is the one from which the symbol table is also read. If no file is specified, the command clears the executable and symbol

information. If breakpoints are set when using this command with no arguments, GDB will produce error messages. Otherwise, no output is produced, except a completion notification.

GDB Command

The corresponding GDB command is ‘file’.

Example

```
(gdb)
-file-exec-and-symbols /kwikemart/marge/ezannoni/TRUNK/mbx/hello.mbx
^done
(gdb)
```

The -file-exec-file Command

Synopsis

```
-file-exec-file file
```

Specify the executable file to be debugged. Unlike ‘-file-exec-and-symbols’, the symbol table is *not* read from this file. If used without argument, GDB clears the information about the executable file. No output is produced, except a completion notification.

GDB Command

The corresponding GDB command is ‘exec-file’.

Example

```
(gdb)
-file-exec-file /kwikemart/marge/ezannoni/TRUNK/mbx/hello.mbx
^done
(gdb)
```

The -file-list-exec-source-file Command

Synopsis

```
-file-list-exec-source-file
```

List the line number, the current source file, and the absolute path to the current source file for the current executable. The macro information field has a value of ‘1’ or ‘0’ depending on whether or not the file includes preprocessor macro information.

GDB Command

The GDB equivalent is ‘info source’

Example

```
(gdb)
123-file-list-exec-source-file
123^done,line="1",file="foo.c",fullname="/home/bar/foo.c,macro-info="1"
(gdb)
```

The -file-list-exec-source-files Command

Synopsis

```
-file-list-exec-source-files [ --group-by-objfile ]
                             [ --dirname | --basename ]
                             [ -- ]
                             [ regexp ]
```

This command returns information about the source files GDB knows about, it will output both the filename and fullname (absolute file name) of a source file, though the fullname can be elided if this information is not known to GDB.

With no arguments this command returns a list of source files. Each source file is represented by a tuple with the fields; *file*, *fullname*, and *debug-fully-read*. The *file* is the display name for the file, while *fullname* is the absolute name of the file. The *fullname* field can be elided if the absolute name of the source file can't be computed. The field *debug-fully-read* will be a string, either **true** or **false**. When **true**, this indicates the full debug information for the compilation unit describing this file has been read in. When **false**, the full debug information has not yet been read in. While reading in the full debug information it is possible that GDB could become aware of additional source files.

The optional *regexp* can be used to filter the list of source files returned. The *regexp* will be matched against the full source file name. The matching is case-sensitive, except on operating systems that have case-insensitive filesystem (e.g., MS-Windows). '--' can be used before *regexp* to prevent GDB interpreting *regexp* as a command option (e.g. if *regexp* starts with '-').

If **--dirname** is provided, then *regexp* is matched only against the directory name of each source file. If **--basename** is provided, then *regexp* is matched against the basename of each source file. Only one of **--dirname** or **--basename** may be given, and if either is given then *regexp* is required.

If **--group-by-objfile** is used then the format of the results is changed. The results will now be a list of tuples, with each tuple representing an object file (executable or shared library) loaded into GDB. The fields of these tuples are; *filename*, *debug-info*, and *sources*. The *filename* is the absolute name of the object file, *debug-info* is a string with one of the following values:

none This object file has no debug information.

partially-read

This object file has debug information, but it is not fully read in yet. When it is read in later, GDB might become aware of additional source files.

fully-read

This object file has debug information, and this information is fully read into GDB. The list of source files is complete.

The *sources* is a list or tuples, with each tuple describing a single source file with the same fields as described previously. The *sources* list can be empty for object files that have no debug information.

GDB Command

The GDB equivalent is **'info sources'**. **gdbtk** has an analogous command **'gdb_listfiles'**.

Example

```
(gdb)
-file-list-exec-source-files
^done,files=[{file="foo.c",fullname="/home/foo.c",debug-fully-read="true"},
             {file="/home/bar.c",fullname="/home/bar.c",debug-fully-read="true"},
             {file="gdb_could_not_find_fullpath.c",debug-fully-read="true"}]

(gdb)
-file-list-exec-source-files
^done,files=[{file="test.c",
             fullname="/tmp/info-sources/test.c",
             debug-fully-read="true"},
             {file="/usr/include/stdc-predef.h",
             fullname="/usr/include/stdc-predef.h",
             debug-fully-read="true"},
             {file="header.h",
             fullname="/tmp/info-sources/header.h",
             debug-fully-read="true"},
             {file="helper.c",
             fullname="/tmp/info-sources/helper.c",
             debug-fully-read="true"}]

(gdb)
-file-list-exec-source-files -- \\.c
^done,files=[{file="test.c",
             fullname="/tmp/info-sources/test.c",
             debug-fully-read="true"},
             {file="helper.c",
             fullname="/tmp/info-sources/helper.c",
             debug-fully-read="true"}]

(gdb)
-file-list-exec-source-files --group-by-objfile
^done,files=[{filename="/tmp/info-sources/test.x",
             debug-info="fully-read",
             sources=[{file="test.c",
                     fullname="/tmp/info-sources/test.c",
                     debug-fully-read="true"},
                     {file="/usr/include/stdc-predef.h",
                     fullname="/usr/include/stdc-predef.h",
                     debug-fully-read="true"},
                     {file="header.h",
                     fullname="/tmp/info-sources/header.h",
                     debug-fully-read="true"}]},
             {filename="/lib64/ld-linux-x86-64.so.2",
             debug-info="none",
             sources=[]},
             {filename="system-supplied DSO at 0x7ffff7fcf000",
             debug-info="none",
             sources=[]},
             {filename="/tmp/info-sources/libhelper.so",
             debug-info="fully-read",
             sources=[{file="helper.c",
                     fullname="/tmp/info-sources/helper.c",
                     debug-fully-read="true"},
                     {file="/usr/include/stdc-predef.h",
                     fullname="/usr/include/stdc-predef.h",
                     debug-fully-read="true"},
                     {file="header.h",
                     fullname="/tmp/info-sources/header.h",
                     debug-fully-read="true"}]}],
```

```
{filename="/lib64/libc.so.6",
 debug-info="none",
 sources=[]}]
```

The `-file-list-shared-libraries` Command

Synopsis

```
-file-list-shared-libraries [ regexp ]
```

List the shared libraries in the program. With a regular expression *regexp*, only those libraries whose names match *regexp* are listed.

GDB Command

The corresponding GDB command is ‘`info shared`’. The fields have a similar meaning to the `=library-loaded` notification. The `ranges` field specifies the multiple segments belonging to this library. Each range has the following fields:

‘`from`’ The address defining the inclusive lower bound of the segment.

‘`to`’ The address defining the exclusive upper bound of the segment.

Example

```
(gdb)
-file-list-exec-source-files
^done,shared-libraries=[
{id="/lib/libfoo.so",target-name="/lib/libfoo.so",host-name="/lib/libfoo.so",symbols-loaded="1",thread-
group="i1",ranges=[{from="0x72815989",to="0x728162c0"}]},
{id="/lib/libbar.so",target-name="/lib/libbar.so",host-name="/lib/libbar.so",symbols-loaded="1",thread-
group="i1",ranges=[{from="0x76ee48c0",to="0x76ee9160"}]}]
(gdb)
```

The `-file-symbol-file` Command

Synopsis

```
-file-symbol-file file
```

Read symbol table info from the specified *file* argument. When used without arguments, clears GDB’s symbol table info. No output is produced, except for a completion notification.

GDB Command

The corresponding GDB command is ‘`symbol-file`’.

Example

```
(gdb)
-file-symbol-file /kwikemart/marge/ezannoni/TRUNK/mbx/hello.mbx
^done
(gdb)
```

27.22 GDB/MI Target Manipulation Commands

The `-target-attach` Command

Synopsis

```
-target-attach pid | gid | file
```

Attach to a process *pid* or a file *file* outside of GDB, or a thread group *gid*. If attaching to a thread group, the id previously returned by ‘`-list-thread-groups --available`’ must be used.

GDB Command

The corresponding GDB command is ‘`attach`’.

Example

```
(gdb)
-target-attach 34
=thread-created,id="1"
*stopped,thread-id="1",frame={addr="0xb7f7e410",func="bar",args=[]}
^done
(gdb)
```

The -target-detach Command

Synopsis

```
-target-detach [ pid | gid ]
```

Detach from the remote target which normally resumes its execution. If either *pid* or *gid* is specified, detaches from either the specified process, or specified thread group. There’s no output.

GDB Command

The corresponding GDB command is ‘`detach`’.

Example

```
(gdb)
-target-detach
^done
(gdb)
```

The -target-disconnect Command

Synopsis

```
-target-disconnect
```

Disconnect from the remote target. There’s no output and the target is generally not resumed.

GDB Command

The corresponding GDB command is ‘`disconnect`’.

Example

```
(gdb)
-target-disconnect
^done
(gdb)
```

The `-target-download` Command

Synopsis

`-target-download`

Loads the executable onto the remote target. It prints out an update message every half second, which includes the fields:

‘`section`’ The name of the section.

‘`section-sent`’
The size of what has been sent so far for that section.

‘`section-size`’
The size of the section.

‘`total-sent`’
The total size of what was sent so far (the current and the previous sections).

‘`total-size`’
The size of the overall executable to download.

Each message is sent as status record (see Section 27.4.2 [GDB/MI Output Syntax], page 613).

In addition, it prints the name and size of the sections, as they are downloaded. These messages include the following fields:

‘`section`’ The name of the section.

‘`section-size`’
The size of the section.

‘`total-size`’
The size of the overall executable to download.

At the end, a summary is printed.

GDB Command

The corresponding GDB command is ‘`load`’.

Example

Note: each status message appears on a single line. Here the messages have been broken down so that they can fit onto a page.

```
(gdb)
-target-download
+download,{section=".text",section-size="6668",total-size="9880"}
+download,{section=".text",section-sent="512",section-size="6668",
total-sent="512",total-size="9880"}
+download,{section=".text",section-sent="1024",section-size="6668",
total-sent="1024",total-size="9880"}
+download,{section=".text",section-sent="1536",section-size="6668",
total-sent="1536",total-size="9880"}
+download,{section=".text",section-sent="2048",section-size="6668",
total-sent="2048",total-size="9880"}
+download,{section=".text",section-sent="2560",section-size="6668",
total-sent="2560",total-size="9880"}
```

```

+download,{section=".text",section-sent="3072",section-size="6668",
total-sent="3072",total-size="9880"}
+download,{section=".text",section-sent="3584",section-size="6668",
total-sent="3584",total-size="9880"}
+download,{section=".text",section-sent="4096",section-size="6668",
total-sent="4096",total-size="9880"}
+download,{section=".text",section-sent="4608",section-size="6668",
total-sent="4608",total-size="9880"}
+download,{section=".text",section-sent="5120",section-size="6668",
total-sent="5120",total-size="9880"}
+download,{section=".text",section-sent="5632",section-size="6668",
total-sent="5632",total-size="9880"}
+download,{section=".text",section-sent="6144",section-size="6668",
total-sent="6144",total-size="9880"}
+download,{section=".text",section-sent="6656",section-size="6668",
total-sent="6656",total-size="9880"}
+download,{section=".init",section-size="28",total-size="9880"}
+download,{section=".fini",section-size="28",total-size="9880"}
+download,{section=".data",section-size="3156",total-size="9880"}
+download,{section=".data",section-sent="512",section-size="3156",
total-sent="7236",total-size="9880"}
+download,{section=".data",section-sent="1024",section-size="3156",
total-sent="7748",total-size="9880"}
+download,{section=".data",section-sent="1536",section-size="3156",
total-sent="8260",total-size="9880"}
+download,{section=".data",section-sent="2048",section-size="3156",
total-sent="8772",total-size="9880"}
+download,{section=".data",section-sent="2560",section-size="3156",
total-sent="9284",total-size="9880"}
+download,{section=".data",section-sent="3072",section-size="3156",
total-sent="9796",total-size="9880"}
^done,address="0x10004",load-size="9880",transfer-rate="6586",
write-rate="429"
(gdb)

```

GDB Command

No equivalent.

Example

N.A.

The -target-flash-erase Command

Synopsis

```
-target-flash-erase
```

Erases all known flash memory regions on the target.

The corresponding GDB command is ‘**flash-erase**’.

The output is a list of flash regions that have been erased, with starting addresses and memory region sizes.

```

(gdb)
-target-flash-erase
^done,erased-regions={address="0x0",size="0x40000"}
(gdb)

```

The `-target-select` Command

Synopsis

```
-target-select type parameters ...
```

Connect GDB to the remote target. This command takes two args:

‘type’ The type of target, for instance **‘remote’**, etc.

‘parameters’

Device names, host names and the like. See Section 19.2 [Commands for Managing Targets], page 313, for more details.

The output is a connection notification, followed by the address at which the target program is, in the following form:

```
^connected,addr="address",func="function name",
args=[arg list]
```

GDB Command

The corresponding GDB command is **‘target’**.

Example

```
(gdb)
-target-select remote /dev/ttya
^connected,addr="0xfe00a300",func="??",args=[]
(gdb)
```

27.23 GDB/MI File Transfer Commands

The `-target-file-put` Command

Synopsis

```
-target-file-put hostfile targetfile
```

Copy file *hostfile* from the host system (the machine running GDB) to *targetfile* on the target system.

GDB Command

The corresponding GDB command is **‘remote put’**.

Example

```
(gdb)
-target-file-put localfile remotefile
^done
(gdb)
```

The `-target-file-get` Command

Synopsis

```
-target-file-get targetfile hostfile
```

Copy file *targetfile* from the target system to *hostfile* on the host system.

GDB Command

The corresponding GDB command is ‘remote get’.

Example

```
(gdb)
-target-file-get remotefile localfile
^done
(gdb)
```

The -target-file-delete Command

Synopsis

```
-target-file-delete targetfile
```

Delete *targetfile* from the target system.

GDB Command

The corresponding GDB command is ‘remote delete’.

Example

```
(gdb)
-target-file-delete remotefile
^done
(gdb)
```

27.24 Ada Exceptions GDB/MI Commands

The -info-ada-exceptions Command

Synopsis

```
-info-ada-exceptions [ regex]
```

List all Ada exceptions defined within the program being debugged. With a regular expression *regex*, only those exceptions whose names match *regex* are listed.

GDB Command

The corresponding GDB command is ‘info exceptions’.

Result

The result is a table of Ada exceptions. The following columns are defined for each exception:

‘name’ The name of the exception.
‘address’ The address of the exception.

Example

```
-info-ada-exceptions aint
^done,ada-exceptions={nr_rows="2",nr_cols="2",
hdr=[{width="1",alignment="-1",col_name="name",colhdr="Name"},
{width="1",alignment="-1",col_name="address",colhdr="Address"}],
```

```
body=[{name="constraint_error",address="0x0000000000613da0"},
{name="const.aint_global_e",address="0x0000000000613b00"}]}
```

Catching Ada Exceptions

The commands describing how to ask GDB to stop when a program raises an exception are described at Section 27.11.2 [Ada Exception GDB/MI Catchpoint Commands], page 639.

27.25 GDB/MI Support Commands

Since new commands and features get regularly added to GDB/MI, some commands are available to help front-ends query the debugger about support for these capabilities. Similarly, it is also possible to query GDB about target support of certain features.

The `-info-gdb-mi-command` Command

Synopsis

```
-info-gdb-mi-command cmd_name
```

Query support for the GDB/MI command named *cmd_name*.

Note that the dash (-) starting all GDB/MI commands is technically not part of the command name (see Section 27.4.1 [GDB/MI Input Syntax], page 613), and thus should be omitted in *cmd_name*. However, for ease of use, this command also accepts the form with the leading dash.

GDB Command

There is no corresponding GDB command.

Result

The result is a tuple. There is currently only one field:

‘exists’ This field is equal to `"true"` if the GDB/MI command exists, `"false"` otherwise.

Example

Here is an example where the GDB/MI command does not exist:

```
-info-gdb-mi-command unsupported-command
^done,command={exists="false"}
```

And here is an example where the GDB/MI command is known to the debugger:

```
-info-gdb-mi-command symbol-list-lines
^done,command={exists="true"}
```

The `-list-features` Command

Returns a list of particular features of the MI protocol that this version of gdb implements. A feature can be a command, or a new field in an output of some command, or even an important bugfix. While a frontend can sometimes detect presence of a feature at runtime, it is easier to perform detection at debugger startup.

The command returns a list of strings, with each string naming an available feature. Each returned string is just a name, it does not have any internal structure. The list of possible feature names is given below.

Example output:

```
(gdb) -list-features
^done,features=["feature1","feature2"]
```

The current list of features is:

- ‘frozen-varobjs’
Indicates support for the `-var-set-frozen` command, as well as possible presence of the `frozen` field in the output of `-varobj-create`.
- ‘pending-breakpoints’
Indicates support for the `-f` option to the `-break-insert` command.
- ‘python’
Indicates Python scripting support, Python-based pretty-printing commands, and possible presence of the ‘display_hint’ field in the output of `-var-list-children`
- ‘thread-info’
Indicates support for the `-thread-info` command.
- ‘data-read-memory-bytes’
Indicates support for the `-data-read-memory-bytes` and the `-data-write-memory-bytes` commands.
- ‘breakpoint-notifications’
Indicates that changes to breakpoints and breakpoints created via the CLI will be announced via async records.
- ‘ada-task-info’
Indicates support for the `-ada-task-info` command.
- ‘language-option’
Indicates that all GDB/MI commands accept the `--language` option (see Section 27.3.1 [Context management], page 610).
- ‘info-gdb-mi-command’
Indicates support for the `-info-gdb-mi-command` command.
- ‘undefined-command-error-code’
Indicates support for the "undefined-command" error code in error result records, produced when trying to execute an undefined GDB/MI command (see Section 27.7.1 [GDB/MI Result Records], page 617).
- ‘exec-run-start-option’
Indicates that the `-exec-run` command supports the `--start` option (see Section 27.15 [GDB/MI Program Execution], page 647).
- ‘data-disassemble-a-option’
Indicates that the `-data-disassemble` command supports the `-a` option (see Section 27.18 [GDB/MI Data Manipulation], page 671).
- ‘simple-values-ref-types’
Indicates that the `--simple-values` argument to the `-stack-list-arguments`, `-stack-list-locals`, `-stack-list-variables`, and `-var-list-children` commands takes reference types into account: that is, a value is considered simple if it is neither an array, structure, or union, nor a reference to an array, structure, or union.

The `-list-target-features` Command

Returns a list of particular features that are supported by the target. Those features affect the permitted MI commands, but unlike the features reported by the `-list-features` command, the features depend on which target GDB is using at the moment. Whenever a target can change, due to commands such as `-target-select`, `-target-attach` or `-exec-run`, the list of target features may change, and the frontend should obtain it again. Example output:

```
(gdb) -list-target-features
^done,features=["async"]
```

The current list of features is:

- ‘`async`’ Indicates that the target is capable of asynchronous command execution, which means that GDB will accept further commands while the target is running.
- ‘`reverse`’ Indicates that the target is capable of reverse execution. See Chapter 6 [Reverse Execution], page 103, for more information.

27.26 Miscellaneous GDB/MI Commands

The `-gdb-exit` Command

Synopsis

```
-gdb-exit
```

Exit GDB immediately.

GDB Command

Approximately corresponds to ‘`quit`’.

Example

```
(gdb)
-gdb-exit
^exit
```

The `-gdb-set` Command

Synopsis

```
-gdb-set
```

Set an internal GDB variable.

GDB Command

The corresponding GDB command is ‘`set`’.

Example

```
(gdb)
-gdb-set $foo=3
^done
(gdb)
```


The `-gdb-show` Command

Synopsis

```
-gdb-show
```

Show the current value of a GDB variable.

GDB Command

The corresponding GDB command is `'show'`.

Example

```
(gdb)
-gdb-show annotate
^done,value="0"
(gdb)
```

The `-gdb-version` Command

Synopsis

```
-gdb-version
```

Show version information for GDB. Used mostly in testing.

GDB Command

The GDB equivalent is `'show version'`. GDB by default shows this information when you start an interactive session.

Example

```
(gdb)
-gdb-version
^GNU gdb 5.2.1
^Copyright 2000 Free Software Foundation, Inc.
^GDB is free software, covered by the GNU General Public License, and
^you are welcome to change it and/or distribute copies of it under
~ certain conditions.
^Type "show copying" to see the conditions.
^There is absolutely no warranty for GDB. Type "show warranty" for
~ details.
^This GDB was configured as
  "--host=sparc-sun-solaris2.5.1 --target=ppc-eabi".
^done
(gdb)
```

The `-list-thread-groups` Command

Synopsis

```
-list-thread-groups [ --available ] [ --recurse 1 ] [ group ... ]
```

Lists thread groups (see Section 27.3.3 [Thread groups], page 612). When a single thread group is passed as the argument, lists the children of that group. When several thread group are passed, lists information about those thread groups. Without any parameters, lists information about all top-level thread groups.

Normally, thread groups that are being debugged are reported. With the ‘`--available`’ option, GDB reports thread groups available on the target.

The output of this command may have either a ‘`threads`’ result or a ‘`groups`’ result. The ‘`thread`’ result has a list of tuples as value, with each tuple describing a thread (see Section 27.7.6 [GDB/MI Thread Information], page 625). The ‘`groups`’ result has a list of tuples as value, each tuple describing a thread group. If top-level groups are requested (that is, no parameter is passed), or when several groups are passed, the output always has a ‘`groups`’ result. The format of the ‘`group`’ result is described below.

To reduce the number of roundtrips it’s possible to list thread groups together with their children, by passing the ‘`--recurse`’ option and the recursion depth. Presently, only recursion depth of 1 is permitted. If this option is present, then every reported thread group will also include its children, either as ‘`group`’ or ‘`threads`’ field.

In general, any combination of option and parameters is permitted, with the following caveats:

- When a single thread group is passed, the output will typically be the ‘`threads`’ result. Because threads may not contain anything, the ‘`recurse`’ option will be ignored.
- When the ‘`--available`’ option is passed, limited information may be available. In particular, the list of threads of a process might be inaccessible. Further, specifying specific thread groups might not give any performance advantage over listing all thread groups. The frontend should assume that ‘`-list-thread-groups --available`’ is always an expensive operation and cache the results.

The ‘`groups`’ result is a list of tuples, where each tuple may have the following fields:

<code>id</code>	Identifier of the thread group. This field is always present. The identifier is an opaque string; frontends should not try to convert it to an integer, even though it might look like one.
<code>type</code>	The type of the thread group. At present, only ‘ <code>process</code> ’ is a valid type.
<code>pid</code>	The target-specific process identifier. This field is only present for thread groups of type ‘ <code>process</code> ’ and only if the process exists.
<code>exit-code</code>	The exit code of this group’s last exited thread, formatted in octal. This field is only present for thread groups of type ‘ <code>process</code> ’ and only if the process is not running.
<code>num_children</code>	The number of children this thread group has. This field may be absent for an available thread group.
<code>threads</code>	This field has a list of tuples as value, each tuple describing a thread. It may be present if the ‘ <code>--recurse</code> ’ option is specified, and it’s actually possible to obtain the threads.
<code>cores</code>	This field is a list of integers, each identifying a core that one thread of the group is running on. This field may be absent if such information is not available.

executable

The name of the executable file that corresponds to this thread group. The field is only present for thread groups of type ‘process’, and only if there is a corresponding executable file.

Example

```
(gdb)
~list-thread-groups
^done,groups=[{id="17",type="process",pid="yyy",num_children="2"}]
~list-thread-groups 17
^done,threads=[{id="2",target-id="Thread 0xb7e14b90 (LWP 21257)",
  frame={level="0",addr="0xffffe410",func="__kernel_vsyscall",args=[],state="running"},
{id="1",target-id="Thread 0xb7e156b0 (LWP 21254)",
  frame={level="0",addr="0x0804891f",func="foo",args=[{name="i",value="10"}],
    file="/tmp/a.c",fullname="/tmp/a.c",line="158",arch="i386:x86_64"},state="running"}]]
~list-thread-groups --available
^done,groups=[{id="17",type="process",pid="yyy",num_children="2",cores=[1,2]}]
~list-thread-groups --available --recurse 1
^done,groups=[{id="17", types="process",pid="yyy",num_children="2",cores=[1,2],
  threads=[{id="1",target-id="Thread 0xb7e14b90",cores=[1]},
    {id="2",target-id="Thread 0xb7e14b90",cores=[2]}]},...]
~list-thread-groups --available --recurse 1 17 18
^done,groups=[{id="17", types="process",pid="yyy",num_children="2",cores=[1,2],
  threads=[{id="1",target-id="Thread 0xb7e14b90",cores=[1]},
    {id="2",target-id="Thread 0xb7e14b90",cores=[2]}]},...]
```

The -info-os Command**Synopsis**

```
-info-os [ type ]
```

If no argument is supplied, the command returns a table of available operating-system-specific information types. If one of these types is supplied as an argument *type*, then the command returns a table of data of that type.

The types of information available depend on the target operating system.

GDB Command

The corresponding GDB command is ‘info os’.

Example

When run on a GNU/Linux system, the output will look something like this:

```
(gdb)
~info-os
^done,OSDataTable={nr_rows="10",nr_cols="3",
  hdr=[{width="10",alignment="-1",col_name="col0",colhdr="Type"},
    {width="10",alignment="-1",col_name="col1",colhdr="Description"},
    {width="10",alignment="-1",col_name="col2",colhdr="Title"}],
  body=[item={col0="cpus",col1="Listing of all cpus/cores on the system",
    col2="CPUs"},
    item={col0="files",col1="Listing of all file descriptors",
    col2="File descriptors"},
    item={col0="modules",col1="Listing of all loaded kernel modules",
    col2="Kernel modules"}],
```

```

        item={col0="msg",col1="Listing of all message queues",
              col2="Message queues"},
        item={col0="processes",col1="Listing of all processes",
              col2="Processes"},
        item={col0="procgroups",col1="Listing of all process groups",
              col2="Process groups"},
        item={col0="semaphores",col1="Listing of all semaphores",
              col2="Semaphores"},
        item={col0="shm",col1="Listing of all shared-memory regions",
              col2="Shared-memory regions"},
        item={col0="sockets",col1="Listing of all internet-domain sockets",
              col2="Sockets"},
        item={col0="threads",col1="Listing of all threads",
              col2="Threads"}}]
(gdb)
~info-os processes
^done,OSDataTable={nr_rows="190",nr_cols="4",
  hdr=[{width="10",alignment="-1",col_name="col0",colhdr="pid"},
        {width="10",alignment="-1",col_name="col1",colhdr="user"},
        {width="10",alignment="-1",col_name="col2",colhdr="command"},
        {width="10",alignment="-1",col_name="col3",colhdr="cores"}],
  body=[item={col0="1",col1="root",col2="/sbin/init",col3="0"},
        item={col0="2",col1="root",col2="[kthreadd]",col3="1"},
        item={col0="3",col1="root",col2="[ksoftirqd/0]",col3="0"},
        ...
        item={col0="26446",col1="stan",col2="bash",col3="0"},
        item={col0="28152",col1="stan",col2="bash",col3="1"}]]}
(gdb)

```

(Note that the MI output here includes a "Title" column that does not appear in command-line `info os`; this column is useful for MI clients that want to enumerate the types of data, such as in a popup menu, but is needless clutter on the command line, and `info os` omits it.)

The -add-inferior Command

Synopsis

```
-add-inferior [ --no-connection ]
```

Creates a new inferior (see Section 4.9 [Inferiors Connections and Programs], page 43). The created inferior is not associated with any executable. Such association may be established with the `-file-exec-and-symbols` command (see Section 27.21 [GDB/MI File Commands], page 694).

By default, the new inferior begins connected to the same target connection as the current inferior. For example, if the current inferior was connected to `gdbserver` with `target remote`, then the new inferior will be connected to the same `gdbserver` instance. The `--no-connection` option starts the new inferior with no connection yet. You can then for example use the `-target-select remote` command to connect to some other `gdbserver` instance, use `-exec-run` to spawn a local program, etc.

If the connection of the current inferior cannot be shared, e.g. the `-target-select core` target cannot be shared between inferiors, then GDB will give a warning and create the new inferior without a connection.

The command response always has a field, *inferior*, whose value is the identifier of the thread group corresponding to the new inferior.

An additional section field, *connection*, is optional. This field will only exist if the new inferior has a target connection. If this field exists, then its value will be a tuple containing the following fields:

- ‘**number**’ The number of the connection used for the new inferior.
- ‘**name**’ The name of the connection type used for the new inferior.

GDB Command

The corresponding GDB command is ‘**add-inferior**’ (see [‘**add-inferior**’], page 45).

Example

```
(gdb)
-add-inferior
^done,inferior="i3"
```

The -remove-inferior Command

Synopsis

```
-remove-inferior inferior-id
```

Removes an inferior (see Section 4.9 [Inferiors Connections and Programs], page 43). Only inferiors that have exited can be removed. The *inferior-id* is the inferior to be removed, and should be the same id string as returned by the ‘**-add-inferior**’ command.

When an inferior is successfully removed a **=thread-group-removed** notification (see Section 27.7.3 [GDB/MI Async Records], page 618) is emitted, the *id* field of which contains the *inferior-id* for the removed inferior.

GDB Command

The corresponding GDB command is ‘**remove-inferiors**’ (see [‘**remove-inferiors**’], page 46).

Example

```
(gdb)
-remove-inferior i3
=thread-group-removed,id="i3"
^done
```

The -interpreter-exec Command

Synopsis

```
-interpreter-exec interpreter command
```

Execute the specified *command* in the given *interpreter*.

GDB Command

The corresponding GDB command is ‘**interpreter-exec**’.

Example

```
(gdb)
```

```
-interpreter-exec console "break main"
&"During symbol reading, couldn't parse type; debugger out of date?.\n"
&"During symbol reading, bad structure-type format.\n"
~"Breakpoint 1 at 0x8074fc6: file ../../src/gdb/main.c, line 743.\n"
^done
(gdb)
```

The `-inferior-tty-set` Command

Synopsis

```
-inferior-tty-set /dev/pts/1
```

Set terminal for future runs of the program being debugged.

GDB Command

The corresponding GDB command is `'set inferior-tty' /dev/pts/1`.

Example

```
(gdb)
-inferior-tty-set /dev/pts/1
^done
(gdb)
```

The `-inferior-tty-show` Command

Synopsis

```
-inferior-tty-show
```

Show terminal for future runs of program being debugged.

GDB Command

The corresponding GDB command is `'show inferior-tty'`.

Example

```
(gdb)
-inferior-tty-set /dev/pts/1
^done
(gdb)
-inferior-tty-show
^done,inferior_tty_terminal="/dev/pts/1"
(gdb)
```

The `-enable-timings` Command

Synopsis

```
-enable-timings [yes | no]
```

Toggle the printing of the wallclock, user and system times for an MI command as a field in its output. This command is to help frontend developers optimize the performance of their code. No argument is equivalent to `'yes'`.

GDB Command

No equivalent.

Example

```
(gdb)
-enable-timings
^done
(gdb)
-break-insert main
^done,bkpt={number="1",type="breakpoint",disp="keep",enabled="y",
addr="0x080484ed",func="main",file="myprog.c",
fullname="/home/nickrob/myprog.c",line="73",thread-groups=["i1"],
times="0"},
time={wallclock="0.05185",user="0.00800",system="0.00000"}
(gdb)
-enable-timings no
^done
(gdb)
-exec-run
^running
(gdb)
*stopped,reason="breakpoint-hit",disp="keep",bkptno="1",thread-id="0",
frame={addr="0x080484ed",func="main",args=[{name="argc",value="1"},
{name="argv",value="0xbfb60364"}],file="myprog.c",
fullname="/home/nickrob/myprog.c",line="73",arch="i386:x86_64"}
(gdb)
```

The `-complete` Command

Synopsis

```
-complete command
```

Show a list of completions for partially typed CLI *command*.

This command is intended for GDB/MI frontends that cannot use two separate CLI and MI channels — for example: because of lack of PTYs like on Windows or because GDB is used remotely via a SSH connection.

Result

The result consists of two or three fields:

‘completion’

This field contains the completed *command*. If *command* has no known completions, this field is omitted.

‘matches’ This field contains a (possibly empty) array of matches. It is always present.

‘max_completions_reached’

This field contains 1 if number of known completions is above `max-completions` limit (see Section 3.3 [Completion], page 24), otherwise it contains 0. It is always present.

GDB Command

The corresponding GDB command is ‘complete’.

Example

```
(gdb)
```

```
-complete br
^done,completion="break",
    matches=["break","break-range"],
    max_completions_reached="0"
(gdb)
-complete "b ma"
^done,completion="b ma",
    matches=["b madvise","b main"],max_completions_reached="0"
(gdb)
-complete "b push_b"
^done,completion="b push_back(",
    matches=[
        "b A::push_back(void*)",
        "b std::string::push_back(char)",
        "b std::vector<int, std::allocator<int> >::push_back(int&&)"],
    max_completions_reached="0"
(gdb)
-complete "nonexist"
^done,matches=[],max_completions_reached="0"
(gdb)
```


28 GDB Annotations

This chapter describes annotations in GDB. Annotations were designed to interface GDB to graphical user interfaces or other similar programs which want to interact with GDB at a relatively high level.

The annotation mechanism has largely been superseded by GDB/MI (see Chapter 27 [GDB/MI], page 609).

28.1 What is an Annotation?

Annotations start with a newline character, two ‘**control-z**’ characters, and the name of the annotation. If there is no additional information associated with this annotation, the name of the annotation is followed immediately by a newline. If there is additional information, the name of the annotation is followed by a space, the additional information, and a newline. The additional information cannot contain newline characters.

Any output not beginning with a newline and two ‘**control-z**’ characters denotes literal output from GDB. Currently there is no need for GDB to output a newline followed by two ‘**control-z**’ characters, but if there was such a need, the annotations could be extended with an ‘**escape**’ annotation which means those three characters as output.

The annotation *level*, which is specified using the `--annotate` command line option (see Section 2.1.2 [Mode Options], page 13), controls how much information GDB prints together with its prompt, values of expressions, source lines, and other types of output. Level 0 is for no annotations, level 1 is for use when GDB is run as a subprocess of GNU Emacs, level 3 is the maximum annotation suitable for programs that control GDB, and level 2 annotations have been made obsolete (see Section “Limitations of the Annotation Interface” in *GDB’s Obsolete Annotations*).

set annotate level

The GDB command `set annotate` sets the level of annotations to the specified *level*.

show annotate

Show the current annotation level.

This chapter describes level 3 annotations.

A simple example of starting up GDB with annotations is:

```
$ gdb --annotate=3
GNU gdb 6.0
Copyright 2003 Free Software Foundation, Inc.
GDB is free software, covered by the GNU General Public License,
and you are welcome to change it and/or distribute copies of it
under certain conditions.
Type "show copying" to see the conditions.
There is absolutely no warranty for GDB. Type "show warranty"
for details.
This GDB was configured as "i386-pc-linux-gnu"

^Z^Zpre-prompt
(gdb)
^Z^Zprompt
quit
```

```
^Z^Zpost-prompt
$
```

Here ‘quit’ is input to GDB; the rest is output from GDB. The three lines beginning ‘^Z^Z’ (where ‘^Z’ denotes a ‘control-z’ character) are annotations; the rest is output from GDB.

28.2 The Server Prefix

If you prefix a command with ‘**server**’ then it will not affect the command history, nor will it affect GDB’s notion of which command to repeat if **RET** is pressed on a line by itself. This means that commands can be run behind a user’s back by a front-end in a transparent manner.

The **server** prefix does not affect the recording of values into the value history; to print a value without recording it into the value history, use the **output** command instead of the **print** command.

Using this prefix also disables confirmation requests (see [confirmation requests], page 386).

28.3 Annotation for GDB Input

When GDB prompts for input, it annotates this fact so it is possible to know when to send output, when the output from a given command is over, etc.

Different kinds of input each have a different *input type*. Each input type has three annotations: a **pre-** annotation, which denotes the beginning of any prompt which is being output, a plain annotation, which denotes the end of the prompt, and then a **post-** annotation which denotes the end of any echo which may (or may not) be associated with the input. For example, the **prompt** input type features the following annotations:

```
^Z^Zpre-prompt
^Z^Zprompt
^Z^Zpost-prompt
```

The input types are

- prompt** When GDB is prompting for a command (the main GDB prompt).
- commands** When GDB prompts for a set of commands, like in the **commands** command. The annotations are repeated for each command which is input.
- overload-choice** When GDB wants the user to select between various overloaded functions.
- query** When GDB wants the user to confirm a potentially dangerous operation.
- prompt-for-continue** When GDB is asking the user to press return to continue. Note: Don’t expect this to work well; instead use **set height 0** to disable prompting. This is because the counting of lines is buggy in the presence of annotations.

28.4 Errors

`^Z^Zquit`

This annotation occurs right before GDB responds to an interrupt.

`^Z^Zerror`

This annotation occurs right before GDB responds to an error.

Quit and error annotations indicate that any annotations which GDB was in the middle of may end abruptly. For example, if a `value-history-begin` annotation is followed by a `error`, one cannot expect to receive the matching `value-history-end`. One cannot expect not to receive it either, however; an error annotation does not necessarily mean that GDB is immediately returning all the way to the top level.

A quit or error annotation may be preceded by

`^Z^Zerror-begin`

Any output between that and the quit or error annotation is the error message.

Warning messages are not yet annotated.

28.5 Invalidation Notices

The following annotations say that certain pieces of state may have changed.

`^Z^Zframes-invalid`

The frames (for example, output from the `backtrace` command) may have changed.

`^Z^Zbreakpoints-invalid`

The breakpoints may have changed. For example, the user just added or deleted a breakpoint.

28.6 Running the Program

When the program starts executing due to a GDB command such as `step` or `continue`,

`^Z^Zstarting`

is output. When the program stops,

`^Z^Zstopped`

is output. Before the `stopped` annotation, a variety of annotations describe how the program stopped.

`^Z^Zexited exit-status`

The program exited, and `exit-status` is the exit status (zero for successful exit, otherwise nonzero).

`^Z^Zsignalled`

The program exited with a signal. After the `^Z^Zsignalled`, the annotation continues:

```
intro-text
^Z^Zsignal-name
name
^Z^Zsignal-name-end
middle-text
^Z^Zsignal-string
```

```

string
^Z^Zsignal-string-end
end-text

```

where *name* is the name of the signal, such as SIGILL or SIGSEGV, and *string* is the explanation of the signal, such as **Illegal Instruction** or **Segmentation fault**. The arguments *intro-text*, *middle-text*, and *end-text* are for the user's benefit and have no particular format.

^Z^Zsignal

The syntax of this annotation is just like **signalled**, but GDB is just saying that the program received the signal, not that it was terminated with it.

^Z^Zbreakpoint *number*

The program hit breakpoint number *number*.

^Z^Zwatchpoint *number*

The program hit watchpoint number *number*.

28.7 Displaying Source

The following annotation is used instead of displaying source code:

```
^Z^Zsource filename:line:character:middle:addr
```

where *filename* is an absolute file name indicating which source file, *line* is the line number within that file (where 1 is the first line in the file), *character* is the character position within the file (where 0 is the first character in the file) (for most debug formats this will necessarily point to the beginning of a line), *middle* is 'middle' if *addr* is in the middle of the line, or 'beg' if *addr* is at the beginning of the line, and *addr* is the address in the target program associated with the source which is being displayed. The *addr* is in the form '0x' followed by one or more lowercase hex digits (note that this does not depend on the language).

29 Debugger Adapter Protocol

The Debugger Adapter Protocol is a generic API that is used by some IDEs to communicate with debuggers. It is documented at <https://microsoft.github.io/debug-adapter-protocol/>.

Generally, GDB implements the Debugger Adapter Protocol as written. However, in some cases, extensions are either needed or even expected.

In the protocol, the parameters to the **launch** and **attach** requests are defined by the adapter. GDB defines a couple parameters common to both:

adaSourceCharset

If provided, this is a string that is used to set the Ada source character set. See Section 15.4.10.10 [Ada Source Character Set], page 261.

program If provided, this is a string that specifies the program to use. This corresponds to the **file** command. See Section 18.1 [Files], page 293.

With the **attach** request, in some cases, GDB can automatically determine which program is running. However, for many remote targets, this is not the case, and so this should be supplied.

GDB defines some additional parameters that can be passed to the **launch** request:

args If provided, this should be an array of strings. These strings are provided as command-line arguments to the inferior, as if by **set args**. See Section 4.3 [Arguments], page 38.

cwd If provided, this should be a string. GDB will change its working directory to this directory, as if by the **cd** command (see Section 4.5 [Working Directory], page 40). The launched program will inherit this as its working directory. Note that change of directory happens before the **program** parameter is processed. This will affect the result if **program** is a relative filename.

env If provided, this should be an object. Each key of the object will be used as the name of an environment variable; each value must be a string and will be the value of that variable. The environment of the inferior will be set to exactly as passed in. See Section 4.4 [Environment], page 38.

program If provided, this is a string that specifies the program to use. This corresponds to the **file** command. See Section 18.1 [Files], page 293.

stopAtBeginningOfMainSubprogram

If provided, this must be a boolean. When **‘True’**, GDB will set a temporary breakpoint at the program’s main procedure, using the same approach as the **start** command. See Section 4.2 [Starting], page 34.

stopOnEntry

If provided, this must be a boolean. When **‘True’**, GDB will set a temporary breakpoint at the program’s first instruction, using the same approach as the **starti** command. See Section 4.2 [Starting], page 34.

GDB defines some additional parameters that can be passed to the **attach** request. One of **pid**, **target**, or **coreFile** must be specified. If multiple are specified, they are checked for in that order, and the first one found is used.

- pid** The process ID to which GDB should attach. See Section 4.7 [Attach], page 41.
- target** The target to which GDB should connect. This is a string and is passed to the **target remote** command. See Section 20.1 [Connecting], page 317.
- coreFile** A string that specifies a core file to use. This corresponds to the *core-file* command. See [core-file command], page 294.

In response to the **disassemble** request, DAP allows the client to return the bytes of each instruction in an implementation-defined format. GDB implements this by sending a string with the bytes encoded in hex, like "55a2b900".

When the **repl** context is used for the **evaluate** request, GDB evaluates the provided expression as a CLI command.

Evaluation in general can cause the inferior to continue execution. For example, evaluating the **continue** command could do this, as could evaluating an expression that involves an inferior function call.

repl evaluation can also cause GDB to appear to stop responding to requests, for example if a CLI script does a lengthy computation.

Evaluations like this can be interrupted using the DAP **cancel** request. (In fact, **cancel** should work for any request, but it is unlikely to be useful for most of them.)

GDB provides a couple of logging settings that can be used in DAP mode. These can be set on the command line using the **-iex** option (see Section 2.1.1 [File Options], page 12).

set debug dap-log-file *[filename]*

Enable DAP logging. Logs are written to *filename*. If no *filename* is given, logging is stopped.

set debug dap-log-level *level*

Set the DAP logging level. The default is '1', which logs the DAP protocol, whatever debug messages the developers thought were useful, and unexpected exceptions. Level '2' can be used to log all exceptions, including ones that are considered to be expected. For example, a failure to parse an expression would be considered a normal exception and not normally be logged.

30 JIT Compilation Interface

This chapter documents GDB's *just-in-time* (JIT) compilation interface. A JIT compiler is a program or library that generates native executable code at runtime and executes it, usually in order to achieve good performance while maintaining platform independence.

Programs that use JIT compilation are normally difficult to debug because portions of their code are generated at runtime, instead of being loaded from object files, which is where GDB normally finds the program's symbols and debug information. In order to debug programs that use JIT compilation, GDB has an interface that allows the program to register in-memory symbol files with GDB at runtime.

If you are using GDB to debug a program that uses this interface, then it should work transparently so long as you have not stripped the binary. If you are developing a JIT compiler, then the interface is documented in the rest of this chapter. At this time, the only known client of this interface is the LLVM JIT.

Broadly speaking, the JIT interface mirrors the dynamic loader interface. The JIT compiler communicates with GDB by writing data into a global variable and calling a function at a well-known symbol. When GDB attaches, it reads a linked list of symbol files from the global variable to find existing code, and puts a breakpoint in the function so that it can find out about additional code.

30.1 JIT Declarations

These are the relevant struct declarations that a C program should include to implement the interface:

```
typedef enum
{
    JIT_NOACTION = 0,
    JIT_REGISTER_FN,
    JIT_UNREGISTER_FN
} jit_actions_t;

struct jit_code_entry
{
    struct jit_code_entry *next_entry;
    struct jit_code_entry *prev_entry;
    const char *symfile_addr;
    uint64_t symfile_size;
};

struct jit_descriptor
{
    uint32_t version;
    /* This type should be jit_actions_t, but we use uint32_t
       to be explicit about the bitwidth. */
    uint32_t action_flag;
    struct jit_code_entry *relevant_entry;
    struct jit_code_entry *first_entry;
};

/* GDB puts a breakpoint in this function. */
void __attribute__((noinline)) __jit_debug_register_code() { };
```

```
/* Make sure to specify the version statically, because the
   debugger may check the version before we can set it. */
struct jit_descriptor __jit_debug_descriptor = { 1, 0, 0, 0 };
```

If the JIT is multi-threaded, then it is important that the JIT synchronize any modifications to this global data properly, which can easily be done by putting a global mutex around modifications to these structures.

30.2 Registering Code

To register code with GDB, the JIT should follow this protocol:

- Generate an object file in memory with symbols and other desired debug information. The file must include the virtual addresses of the sections.
- Create a code entry for the file, which gives the start and size of the symbol file.
- Add it to the linked list in the JIT descriptor.
- Point the `relevant_entry` field of the descriptor at the entry.
- Set `action_flag` to `JIT_REGISTER` and call `__jit_debug_register_code`.

When GDB is attached and the breakpoint fires, GDB uses the `relevant_entry` pointer so it doesn't have to walk the list looking for new code. However, the linked list must still be maintained in order to allow GDB to attach to a running process and still find the symbol files.

30.3 Unregistering Code

If code is freed, then the JIT should use the following protocol:

- Remove the code entry corresponding to the code from the linked list.
- Point the `relevant_entry` field of the descriptor at the code entry.
- Set `action_flag` to `JIT_UNREGISTER` and call `__jit_debug_register_code`.

If the JIT frees or recompiles code without unregistering it, then GDB and the JIT will leak the memory used for the associated symbol files.

30.4 Custom Debug Info

Generating debug information in platform-native file formats (like ELF or COFF) may be an overkill for JIT compilers; especially if all the debug info is used for is displaying a meaningful backtrace. The issue can be resolved by having the JIT writers decide on a debug info format and also provide a reader that parses the debug info generated by the JIT compiler. This section gives a brief overview on writing such a parser. More specific details can be found in the source file `gdb/jit-reader.in`, which is also installed as a header at `includedir/gdb/jit-reader.h` for easy inclusion.

The reader is implemented as a shared object (so this functionality is not available on platforms which don't allow loading shared objects at runtime). Two GDB commands, `jit-reader-load` and `jit-reader-unload` are provided, to be used to load and unload the readers from a preconfigured directory. Once loaded, the shared object is used to parse the debug information emitted by the JIT compiler.

30.4.1 Using JIT Debug Info Readers

Readers can be loaded and unloaded using the `jit-reader-load` and `jit-reader-unload` commands.

`jit-reader-load reader`

Load the JIT reader named *reader*, which is a shared object specified as either an absolute or a relative file name. In the latter case, GDB will try to load the reader from a pre-configured directory, usually *libdir/gdb/* on a UNIX system (here *libdir* is the system library directory, often */usr/local/lib*).

Only one reader can be active at a time; trying to load a second reader when one is already loaded will result in GDB reporting an error. A new JIT reader can be loaded by first unloading the current one using `jit-reader-unload` and then invoking `jit-reader-load`.

`show jit-reader-directory`

This command will show the directory that is used by `jit-reader-load` when a relative file name is specified.

`jit-reader-unload`

Unload the currently loaded JIT reader.

30.4.2 Writing JIT Debug Info Readers

As mentioned, a reader is essentially a shared object conforming to a certain ABI. This ABI is described in `jit-reader.h`.

`jit-reader.h` defines the structures, macros and functions required to write a reader. It is installed (along with GDB), in *includedir/gdb* where *includedir* is the system include directory.

Readers need to be released under a GPL compatible license. A reader can be declared as released under such a license by placing the macro `GDB_DECLARE_GPL_COMPATIBLE_READER` in a source file.

The entry point for readers is the symbol `gdb_init_reader`, which is expected to be a function with the prototype

```
extern struct gdb_reader_funcs *gdb_init_reader (void);
```

`struct gdb_reader_funcs` contains a set of pointers to callback functions. These functions are executed to read the debug info generated by the JIT compiler (`read`), to unwind stack frames (`unwind`) and to create canonical frame IDs (`get_frame_id`). It also has a callback that is called when the reader is being unloaded (`destroy`). The struct looks like this

```
struct gdb_reader_funcs
{
    /* Must be set to GDB_READER_INTERFACE_VERSION. */
    int reader_version;

    /* For use by the reader. */
    void *priv_data;

    gdb_read_debug_info *read;
    gdb_unwind_frame *unwind;
    gdb_get_frame_id *get_frame_id;
}
```

```
    gdb_destroy_reader *destroy;  
};
```

The callbacks are provided with another set of callbacks by GDB to do their job. For `read`, these callbacks are passed in a `struct gdb_symbol_callbacks` and for `unwind` and `get_frame_id`, in a `struct gdb_unwind_callbacks`. `struct gdb_symbol_callbacks` has callbacks to create new object files and new symbol tables inside those object files. `struct gdb_unwind_callbacks` has callbacks to read registers off the current frame and to write out the values of the registers in the previous frame. Both have a callback (`target_read`) to read bytes off the target's address space.

31 In-Process Agent

The traditional debugging model is conceptually low-speed, but works fine, because most bugs can be reproduced in debugging-mode execution. However, as multi-core or many-core processors are becoming mainstream, and multi-threaded programs become more and more popular, there should be more and more bugs that only manifest themselves at normal-mode execution, for example, thread races, because debugger's interference with the program's timing may conceal the bugs. On the other hand, in some applications, it is not feasible for the debugger to interrupt the program's execution long enough for the developer to learn anything helpful about its behavior. If the program's correctness depends on its real-time behavior, delays introduced by a debugger might cause the program to fail, even when the code itself is correct. It is useful to be able to observe the program's behavior without interrupting it.

Therefore, traditional debugging model is too intrusive to reproduce some bugs. In order to reduce the interference with the program, we can reduce the number of operations performed by debugger. The *In-Process Agent*, a shared library, is running within the same process with inferior, and is able to perform some debugging operations itself. As a result, debugger is only involved when necessary, and performance of debugging can be improved accordingly. Note that interference with program can be reduced but can't be removed completely, because the in-process agent will still stop or slow down the program.

The in-process agent can interpret and execute Agent Expressions (see Appendix F [Agent Expressions], page 871) during performing debugging operations. The agent expressions can be used for different purposes, such as collecting data in tracepoints, and condition evaluation in breakpoints.

You can control whether the in-process agent is used as an aid for debugging with the following commands:

set agent on

Causes the in-process agent to perform some operations on behalf of the debugger. Just which operations requested by the user will be done by the in-process agent depends on the its capabilities. For example, if you request to evaluate breakpoint conditions in the in-process agent, and the in-process agent has such capability as well, then breakpoint conditions will be evaluated in the in-process agent.

set agent off

Disables execution of debugging operations by the in-process agent. All of the operations will be performed by GDB.

show agent

Display the current setting of execution of debugging operations by the in-process agent.

31.1 In-Process Agent Protocol

The in-process agent is able to communicate with both GDB and GDBserver (see Chapter 31 [In-Process Agent], page 725). This section documents the protocol used for communications between GDB or GDBserver and the IPA. In general, GDB or GDBserver sends commands

(see Section 31.1.2 [IPA Protocol Commands], page 727) and data to in-process agent, and then in-process agent replies back with the return result of the command, or some other information. The data sent to in-process agent is composed of primitive data types, such as 4-byte or 8-byte type, and composite types, which are called objects (see Section 31.1.1 [IPA Protocol Objects], page 726).

31.1.1 IPA Protocol Objects

The commands sent to and results received from agent may contain some complex data types called *objects*.

The in-process agent is running on the same machine with GDB or GDBserver, so it doesn't have to handle as much differences between two ends as remote protocol (see Appendix E [Remote Protocol], page 795) tries to handle. However, there are still some differences of two ends in two processes:

1. word size. On some 64-bit machines, GDB or GDBserver can be compiled as a 64-bit executable, while in-process agent is a 32-bit one.
2. ABI. Some machines may have multiple types of ABI, GDB or GDBserver is compiled with one, and in-process agent is compiled with the other one.

Here are the IPA Protocol Objects:

1. agent expression object. It represents an agent expression (see Appendix F [Agent Expressions], page 871).
2. tracepoint action object. It represents a tracepoint action (see Section 13.1.6 [Tracepoint Action Lists], page 210) to collect registers, memory, static trace data and to evaluate expression.
3. tracepoint object. It represents a tracepoint (see Chapter 13 [Tracepoints], page 205).

The following table describes important attributes of each IPA protocol object:

Name	Size	Description
<i>agent expression object</i>		
length	4	length of bytes code
byte code	<i>length</i>	contents of byte code
<i>tracepoint action for collecting memory</i>		
'M'	1	type of tracepoint action
addr	8	if <i>basereg</i> is '-1', <i>addr</i> is the address of the lowest byte to collect, otherwise <i>addr</i> is the offset of <i>basereg</i> for memory collecting.
len	8	length of memory for collecting
basereg	4	the register number containing the starting memory address for collecting.
<i>tracepoint action for collecting registers</i>		
'R'	1	type of tracepoint action

tracepoint action for collecting static trace data

'L'	1	type of tracepoint action
<i>tracepoint action for expression evaluation</i>		
'X'	1	type of tracepoint action
agent expression	length of	[agent expression object], page 726,
<i>tracepoint object</i>		
number	4	number of tracepoint
address	8	address of tracepoint inserted on
type	4	type of tracepoint
enabled	1	enable or disable of tracepoint
step_count	8	step
pass_count	8	pass
numactions	4	number of tracepoint actions
hit count	8	hit count
trace frame usage	8	trace frame usage
compiled_cond	8	compiled condition
orig_size	8	orig size
condition	4 if condition is NULL otherwise length of [agent expression object], page 726,	zero if condition is NULL, otherwise is [agent expression object], page 726,
actions	variable	numactions number of [tracepoint action object], page 726,

31.1.2 IPA Protocol Commands

The spaces in each command are delimiters to ease reading this commands specification. They don't exist in real commands.

'FastTrace:tracepoint_object gdb_jump_pad_head'

Installs a new fast tracepoint described by *tracepoint_object* (see [tracepoint object], page 726). The *gdb_jump_pad_head*, 8-byte long, is the head of *jump-pad*, which is used to jump to data collection routine in IPA finally.

Replies:

'OK target_address gdb_jump_pad_head fjump_size fjump'

target_address is address of tracepoint in the inferior. The *gdb_jump_pad_head* is updated head of jumppad. Both of *target_address* and *gdb_jump_pad_head* are 8-byte long. The *fjump* contains a sequence of instructions jump to jumppad entry. The *fjump_size*, 4-byte long, is the size of *fjump*.

'close' Closes the in-process agent. This command is sent when GDB or GDBserver is about to kill inferiors.

‘qTfSTM’ See [qTfSTM], page 846.

‘qTsSTM’ See [qTsSTM], page 846.

‘qTSTMat’ See [qTSTMat], page 846.

‘probe_marker_at:address’

Asks in-process agent to probe the marker at *address*.

Replies:

‘unprobe_marker_at:address’

Asks in-process agent to unprobe the marker at *address*.

32 Reporting Bugs in GDB

Your bug reports play an essential role in making GDB reliable.

Reporting a bug may help you by bringing a solution to your problem, or it may not. But in any case the principal function of a bug report is to help the entire community by making the next version of GDB work better. Bug reports are your contribution to the maintenance of GDB.

In order for a bug report to serve its purpose, you must include the information that enables us to fix the bug.

32.1 Have You Found a Bug?

If you are not sure whether you have found a bug, here are some guidelines:

- If the debugger gets a fatal signal, for any input whatever, that is a GDB bug. Reliable debuggers never crash.
- If GDB produces an error message for valid input, that is a bug. (Note that if you're cross debugging, the problem may also be somewhere in the connection to the target.)
- If GDB does not produce an error message for invalid input, that is a bug. However, you should note that your idea of "invalid input" might be our idea of "an extension" or "support for traditional practice".
- If you are an experienced user of debugging tools, your suggestions for improvement of GDB are welcome in any case.

32.2 How to Report Bugs

A number of companies and individuals offer support for GNU products. If you obtained GDB from a support organization, we recommend you contact that organization first.

You can find contact information for many support companies and individuals in the file `etc/SERVICE` in the GNU Emacs distribution.

In any event, we also recommend that you submit bug reports for GDB to <https://www.gnu.org/software/gdb/bugs/>.

The fundamental principle of reporting bugs usefully is this: **report all the facts**. If you are not sure whether to state a fact or leave it out, state it!

Often people omit facts because they think they know what causes the problem and assume that some details do not matter. Thus, you might assume that the name of the variable you use in an example does not matter. Well, probably it does not, but one cannot be sure. Perhaps the bug is a stray memory reference which happens to fetch from the location where that name is stored in memory; perhaps, if the name were different, the contents of that location would fool the debugger into doing the right thing despite the bug. Play it safe and give a specific, complete example. That is the easiest thing for you to do, and the most helpful.

Keep in mind that the purpose of a bug report is to enable us to fix the bug. It may be that the bug has been reported previously, but neither you nor we can know that unless your bug report is complete and self-contained.

Sometimes people give a few sketchy facts and ask, “Does this ring a bell?” Those bug reports are useless, and we urge everyone to *refuse to respond to them* except to chide the sender to report bugs properly.

To enable us to fix the bug, you should include all these things:

- The version of GDB. GDB announces it if you start with no arguments; you can also print it at any time using **show version**.

Without this, we will not know whether there is any point in looking for the bug in the current version of GDB.

- The type of machine you are using, and the operating system name and version number.
- The details of the GDB build-time configuration. GDB shows these details if you invoke it with the **--configuration** command-line option, or if you type **show configuration** at GDB’s prompt.
- What compiler (and its version) was used to compile GDB—e.g. “gcc-2.8.1”.
- What compiler (and its version) was used to compile the program you are debugging—e.g. “gcc-2.8.1”, or “HP92453-01 A.10.32.03 HP C Compiler”. For GCC, you can say **gcc --version** to get this information; for other compilers, see the documentation for those compilers.
- The command arguments you gave the compiler to compile your example and observe the bug. For example, did you use ‘-O’? To guarantee you will not omit something important, list them all. A copy of the Makefile (or the output from make) is sufficient. If we were to try to guess the arguments, we would probably guess wrong and then we might not encounter the bug.
- A complete input script, and all necessary source files, that will reproduce the bug.
- A description of what behavior you observe that you believe is incorrect. For example, “It gets a fatal signal.”

Of course, if the bug is that GDB gets a fatal signal, then we will certainly notice it. But if the bug is incorrect output, we might not notice unless it is glaringly wrong. You might as well not give us a chance to make a mistake.

Even if the problem you experience is a fatal signal, you should still say so explicitly. Suppose something strange is going on, such as, your copy of GDB is out of synch, or you have encountered a bug in the C library on your system. (This has happened!) Your copy might crash and ours would not. If you told us to expect a crash, then when ours fails to crash, we would know that the bug was not happening for us. If you had not told us to expect a crash, then we would not be able to draw any conclusion from our observations.

To collect all this information, you can use a session recording program such as **script**, which is available on many Unix systems. Just run your GDB session inside **script** and then include the **typescript** file with your bug report.

Another way to record a GDB session is to run GDB inside Emacs and then save the entire buffer to a file.

- If you wish to suggest changes to the GDB source, send us context diffs. If you even discuss something in the GDB source, refer to it by context, not by line number.

The line numbers in our development sources will not match those in your sources. Your line numbers would convey no useful information to us.

Here are some things that are not necessary:

- A description of the envelope of the bug.

Often people who encounter a bug spend a lot of time investigating which changes to the input file will make the bug go away and which changes will not affect it.

This is often time consuming and not very useful, because the way we will find the bug is by running a single example under the debugger with breakpoints, not by pure deduction from a series of examples. We recommend that you save your time for something else.

Of course, if you can find a simpler example to report *instead* of the original one, that is a convenience for us. Errors in the output will be easier to spot, running under the debugger will take less time, and so on.

However, simplification is not vital; if you do not want to do this, report the bug anyway and send us the entire test case you used.

- A patch for the bug.

A patch for the bug does help us if it is a good one. But do not omit the necessary information, such as the test case, on the assumption that a patch is all we need. We might see problems with your patch and decide to fix the problem another way, or we might not understand it at all.

Sometimes with a program as complicated as GDB it is very hard to construct an example that will make the program follow a certain path through the code. If you do not send us the example, we will not be able to construct one, so we will not be able to verify that the bug is fixed.

And if we cannot understand what bug you are trying to fix, or why your patch should be an improvement, we will not install it. A test case will help us to understand.

- A guess about what the bug is or what it depends on.

Such guesses are usually wrong. Even we cannot guess right about such things without first using the debugger to find the facts.

